

General Tech Interview Prep

title: "General Tech Stack Interview Preparation"

subtitle: "Frontend, Backend, DB, Cloud, System Design, Algorithms, Security, Testing, Distributed Systems, Behavioral — Project-Independent Reference"

author: "For 4+ year experienced engineers — Uber, Amazon, Meta, NetApp, Google tier"

date: "September 2026"

General Tech Stack Interview Preparation — Project-Independent Reference

SAMPLE QUESTIONS NOTICE: This document is **completely project-independent** — it covers general tech stack questions across frontend, backend, database, cloud, system design, algorithms, security, testing, distributed systems, and behavioral that any senior engineer (4+ years) should be able to answer regardless of which project they're discussing. Treat the questions here as **SAMPLE reference questions** — a base framework. The interviewer should expand to maximum possible questions per category, including twisted variants, code-level edge cases, "what happens internally" deep dives, and whiteboard system-design exercises. The questions are arranged so you can substitute any project context (your current project, a hypothetical, or a system the interviewer names).

Target: Senior engineer interviews at Uber, Amazon, Meta, NetApp, Google, Microsoft, Atlassian, Stripe, Datadog, Snowflake. Difficulty ranges from Easy (warm-up) to Expert (whiteboard sweat).

Candidate profile: Java backend developer, 3.5 years experience, positioning as 4+ years capable. Primary stack: Java 17 + Spring Boot 3.2.x + Hibernate 6, PostgreSQL, Redis, Kafka, AWS, Docker/Kubernetes.

Use this when: The interviewer asks "tell me about a system you'd design for X" (no project context), or asks pure-knowledge questions ("explain MVCC", "what's the difference between optimistic and pessimistic locking").

Question counts: ~330 total. PART 1 (35) · PART 2 (50) · PART 3 (45) · PART 4 (35) · PART 5 (35) · PART 6 (30) · PART 7 (25) · PART 8 (25) · PART 9 (25) · PART 10 (25).

Tech Stack Coverage

Layer	Topics
Frontend	React, TypeScript, JavaScript, Web Performance, State Management, Build Tools, Browser APIs, SSR/SSG, Web Workers, Service Workers, PWA
Backend	Java 17, Spring Boot 3.2, Hibernate 6, Concurrency, JVM, GC, REST, GraphQL, gRPC, Messaging (Kafka/RabbitMQ), Microservices, Event-Driven
Database	SQL (Postgres, MySQL), NoSQL (MongoDB, DynamoDB, Cassandra), Caching (Redis, Memcached), Time-series, Search (Elasticsearch), Vector DBs
Cloud + DevOps	AWS, GCP, Azure, Kubernetes, Docker, Serverless, CDN, Edge, Multi-region, DR, CI/CD, IaC (Terraform), Networking
System Design	Distributed Systems, CAP, Consensus, Replication, Sharding, Caching Strategies, Rate Limiting, Idempotency, Event Sourcing, CQRS, Sagas
Algorithms	Big-O, Sorting, Trees, Graphs, DP, Two-Pointer, Sliding Window, Tries, Heaps, Union-Find, Concurrency problems
Security	OWASP Top 10, AuthN/AuthZ, OAuth2, JWT, OIDC, Secrets, Encryption, Hashing, TLS, Zero Trust
Testing + Observability	Unit, Integration, E2E, JUnit 5, Mockito, Testcontainers, Prometheus, Grafana, OpenTelemetry, Tracing
Distributed Systems	CAP, Consistency, Consensus (Paxos/Raft), Partitioning, Leader Election, Distributed Locks, Clocks
Behavioral	STAR, Leadership, Conflict, Failure, Teamwork, Ambiguity, Influence

PART 1 — FRONTEND QUESTIONS

Q-FE-1: Explain React's reconciliation and virtual DOM

DIFFICULTY: Medium **CATEGORY TAG:** Frontend / React

ANSWER: React's reconciliation is the process of comparing the new virtual DOM tree with the previous one and applying only the minimum necessary changes to the real DOM. The virtual DOM is an in-memory representation of the UI as a tree of React elements (plain JS objects).

When `setState` is called, React: 1. Builds a new virtual DOM tree from the updated component tree 2. Diffs it against the previous tree using a **$O(n)$ heuristic algorithm** (not the optimal $O(n^3)$ tree-diff) 3. Computes the minimum set of DOM mutations 4. Batches + applies them to the real DOM

The $O(n)$ algorithm uses these heuristics: - **Different element types** → tear down the old subtree, build the new (e.g. `<div>` → `<span>` discards everything inside) - **Same type, same key** → reuse the DOM node, update changed props - **Same type, different key** → unmount old, mount new (key is the identity)

Keys are critical for list reconciliation — using array index as key causes bugs when items reorder. Use stable unique IDs (`item.id`) as keys.

React Fiber (since React 16) is the rewrite of the reconciler: - Enables **time-slicing** — work can be split into chunks and yielded to the browser - Enables **concurrent features** (`useTransition` , `useDeferredValue` , `Suspense`) - Priority-based: high-priority updates (user input) interrupt low-priority (data fetching)

Performance tips: - `React.memo` for components that re-render unnecessarily - `useMemo` + `useCallback` for expensive computations / stable references - Virtualise long lists (`react-window` , `react-virtual`) - Avoid inline function props (new ref every render → `React.memo` break) - Use the React DevTools Profiler to find re-renders

KEY TERMS TO MENTION: - Virtual DOM (in-memory tree of React elements) - Reconciliation ($O(n)$ heuristic diff, not $O(n^3)$) - Different types → tear down subtree; same type → update props - Keys for list identity (use stable IDs, not array index) - React Fiber (since 16): time-slicing, concurrent features, priority-based - `React.memo` , `useMemo` , `useCallback` for performance - Profiler to find re-renders

FOLLOW-UP QUESTIONS: 1. Why is React's diff $O(n)$ and not $O(n^3)$? 2. What's the difference between `React.memo` and `useMemo` ? 3. How does React 18's batching work?

FOLLOW-UP ANSWERS: 1. The $O(n^3)$ tree edit distance is optimal but too expensive ($n=100$ elements → 1M operations). React uses heuristics: (a) different element types at the same position → tear down + rebuild (no recursion); (b) `key` identifies list items (no reordering consideration); (c) text node updates just replace text. These cover 99% of UI patterns in $O(n)$. The trade-off: React can make suboptimal choices in edge cases (e.g. swapping two complex subtrees), but those are rare in real UIs. 2.

`React.memo(Component)` is a HOC that memoises the whole component — re-renders only if props changed (shallow compare). `useMemo(() => computeValue(), [deps])` memoises a `value` inside a component. Use `React.memo` when a child re-renders unnecessarily because parent re-rendered; use `useMemo` when computing an expensive value inside a component. 3. React 18 batches `all` state updates (including async ones — promises, `setTimeout`, fetch callbacks), not just event-handler updates like React 17. So multiple `setState` calls in a `setTimeout` produce one re-render. You can opt out via `flushSync(() => setState(...))` (rare — for cases where you need the DOM updated synchronously, e.g. showing a tooltip before scroll).

RED FLAGS TO AVOID: - “Virtual DOM is faster than real DOM” (it’s not — it’s a `tool` for declarative UI) - Using array index as key for reorderable lists - “We use Redux because React is slow” (Redux doesn’t fix React perf)

Q-FE-2: TypeScript vs JavaScript — when would you choose TS

DIFFICULTY: Easy **CATEGORY TAG:** Frontend / TypeScript

ANSWER: TypeScript for anything > 1000 lines or > 1 developer. JavaScript for quick prototypes, scripts, or when you're size-constrained.

Why TS: 1. **Type safety** catches bugs at compile time (typos, undefined access, wrong API usage) — studies show 15-30% fewer bugs 2. **IDE support** — autocomplete, refactor, "find usages", inline docs all work better with types 3. **Self-documenting** — the types *are* the docs (no separate "API reference" that goes stale) 4. **Refactor safety** — change a type, the compiler tells you every call site that broke 5. **Onboarding** — new devs learn the codebase faster when types tell them what to expect 6. **Ecosystem** — most modern libraries ship types (DefinitelyTyped for those that don't)

Why not TS: 1. **Compile step** — adds build complexity (tsconfig, watch mode, source maps) 2. **Type definitions for libs** — sometimes lag the JS API (you fight the types) 3. **Slower initial dev** — you write types before code (the cost is small after first month) 4. **Bundle size** — types are erased at runtime, but `tsc` adds ~50ms to build 5. **Learning curve** — generics, conditional types, mapped types take months to master

Best practices: - `interface` for object shapes (extensible via declaration merging) - `type` for unions, intersections, mapped types - `enum` sparingly (we prefer union types `type Status = "open" | "closed"`) - `unknown` over `any` (forces narrowing) - Generics heavily for reusable components (`<Table<T,>>`) - Discriminated unions for state (`{ status: "loading" } | { status: "success", data: T } | { status: "error", error: E }`) - Type guards (`typeof`, `instanceof`, custom `isX` functions) - `as const` for literal types - Avoid `any` (use `unknown` + narrow). `any` opts out of type-checking — defeats the purpose - Don't fight the types — if the types are wrong, fix the types - Use `strictNullChecks` — null safety is half the value of TS - Enable `noUncheckedIndexedAccess` (`arr[0]` is `T | undefined`, not `T`) - Validate at the boundary (zod, valibot) — runtime validation for API responses

CODE/EXAMPLE SNIPPET:

```
// Discriminated union for async state
type AsyncState<T, E = Error> =
  | { status: "loading" }
  | { status: "success"; data: T }
  | { status: "error"; error: E };

// Boundary validation with zod (runtime + static)
import { z } from "zod";
const UserSchema = z.object({ id: z.string().uuid(), name: z.string() });
type User = z.infer<typeof UserSchema>; // static type inferred
const u: User = UserSchema.parse(jsonResponse); // runtime validated
```

KEY TERMS TO MENTION: - TS catches bugs at compile time, IDE support, refactor safety, self-documenting - `interface` vs `type` (interface for extension, type for unions/mapped) - `unknown` over `any`, `as const` for literals, discriminated unions for state - `strictNullChecks`, `noUncheckedIndexedAccess` - Boundary validation with zod/valibot — runtime + static types

FOLLOW-UP QUESTIONS: 1. What's the difference between `interface` and `type`? 2. How do you handle third-party libs without types? 3. How do you do runtime validation in TS?

FOLLOW-UP ANSWERS: 1. `interface` is for object shapes — supports declaration merging (you can add fields later) and `extends`. `type` is for everything else — unions, intersections, mapped types, conditionals. The TS team recommends `interface` for object shapes (slightly faster compiler, better error messages) and `type` for the rest. We use `interface` for entity shapes, `type` for unions. 2. Most modern libs ship types (`@types/...` is bundled). For untyped libs, we write a minimal `declare module "libname" { ... }` in a `.d.ts` file — type just the functions we use. We rarely use `@ts-ignore` (it suppresses real errors). 3. We use `zod` for runtime validation at API boundaries — `const UserSchema = z.object({ id: z.string().uuid(), name: z.string() }).` `UserSchema.parse(json)` throws on invalid; `UserSchema.safeParse(json)` returns `{ success: true, data } | { success: false, error }`. The TS type is inferred (`type User = z.infer<typeof UserSchema>`), so we get static + runtime validation in one. We use this for all API responses — prevents runtime crashes from unexpected backend changes.

RED FLAGS TO AVOID: - “TS is slower than JS” (the runtime perf is identical — types are erased) - Using `any` everywhere (defeats the purpose) - Skipping `strictNullChecks` (half the value) - “I don't use types, I just write JS in .ts files” (instant fail)

Q-FE-3: useEffect vs useLayoutEffect — when to use which

DIFFICULTY: Medium **CATEGORY TAG:** Frontend / React Hooks

ANSWER: Both run side effects after render, but at different phases of the commit cycle.

useEffect: - Runs **after** the browser paints (asynchronously, deferred) - Does not block visual updates - Used for: data fetching, subscriptions, logging, analytics, setting up timers - Default for 95% of side effects

useLayoutEffect: - Runs **synchronously after DOM mutations but before the browser paints** - Blocks paint (so the user never sees the intermediate “flash” state) - Used for: measuring DOM elements (getBoundingClientRect), then mutating the DOM based on the measurement (e.g. tooltips positioned relative to a trigger, scroll restoration, animations that need a calculated starting point) - Performance penalty: blocks paint → use sparingly

The commit cycle: 1. Render phase (pure) — React computes the new tree 2. DOM mutations — React applies changes to real DOM 3. **useLayoutEffect** runs synchronously → can read layout + mutate DOM 4. Browser paints 5. **useEffect** runs asynchronously → fire-and-forget side effects

Gotcha: if your `useLayoutEffect` does heavy work, the user sees a frozen frame. Avoid. If you need DOM measurements + DOM updates without a flash, that’s the use case.

SSR caveat: `useLayoutEffect` warns on the server (no DOM). Use `useIsomorphicLayoutEffect` pattern — falls back to `useEffect` on SSR.

KEY TERMS TO MENTION: - `useEffect`: after paint, async, default for side effects - `useLayoutEffect`: before paint, sync, blocks paint - Use cases: measurement + DOM mutation without flash (tooltips, scroll) - Commit cycle: render → DOM mutation → `layoutEffect` → paint → effect - SSR: `useLayoutEffect` warns → `useIsomorphicLayoutEffect`

FOLLOW-UP QUESTIONS: 1. What happens if you read layout in `useEffect` and update state? 2. How does React 18’s concurrent mode affect effects?

FOLLOW-UP ANSWERS: 1. Two renders + a visible flash: paint → `useEffect` reads layout → `setState` → re-render → re-paint. With `useLayoutEffect`: DOM mutation → `layoutEffect` reads + `setState` → re-render → re-paint happens in the same frame, no flash. The flash is the giveaway — if users see it, switch to `useLayoutEffect`. 2. Effects from suspended or interrupted renders may run multiple times. React 18 strict mode in dev double-invokes effects to surface missing cleanup. Always return a cleanup function. Concurrent features don’t change effect ordering — only commit timing.

RED FLAGS TO AVOID: - “Always use `useLayoutEffect`” (performance penalty) - “`useEffect` for measuring DOM” (flash on every measurement) - Missing cleanup in `useEffect` (memory leaks, stale closures)

Q-FE-4: useMemo, useCallback, React.memo — when, why, and the gotchas

DIFFICULTY: Medium **CATEGORY TAG:** Frontend / React Hooks

ANSWER: All three memoise — different targets. They trade CPU (compare deps) for re-render/compute savings. Don't sprinkle them everywhere: the comparison cost may exceed the savings.

React.memo(Component): HOC wrapping a component. Re-renders only when props change (shallow compare). Use when a child re-renders unnecessarily because the parent re-rendered (and props are referentially stable). Don't use when props are always new (inline functions, fresh objects) — the memo never helps.

useMemo(() => computeValue(), [deps]): memoises a [value](#) (the result of an expensive compute). Use for expensive computations (filtering 10K items, deriving charts). Don't use for cheap computations — the comparison overhead is greater than the compute.

useCallback(fn, [deps]): memoises a [function reference](#). Use when passing a callback to a memoised child (otherwise the child re-renders because the function identity changed). Don't use for functions passed to non-memoised children (wasted).

CODE/EXAMPLE SNIPPET:

```
// Good: stable callback so memoised child doesn't re-render
const handleClick = useCallback(() => save(id), [id]);
return <MemoisedButton onClick={handleClick} />;

// Good: expensive filter memoised
const filtered = useMemo(() => items.filter(expensivePredicate), [items]);

// Bad: cheap compute, no perf win, extra comparison cost
const doubled = useMemo(() => n * 2, [n]);
```

Gotchas: - Object/array deps in the dep array: new ref every render → memo busts. Memoise the object first or pass primitives. - Functions in deps: same problem — use `useCallback` for stable function refs. - Returning a new object from `useMemo` with the same deps: defeats purpose. - Memoised children with object props that are constructed inline (e.g. `style={{color: 'red'}}`) — new object every render, memo never helps.

KEY TERMS TO MENTION: - `React.memo` (component), `useMemo` (value), `useCallback` (function ref) - Trade CPU comparison for re-render savings - Use when: child re-renders unnecessarily, expensive compute, stable callbacks for memoised children - Don't use for cheap computes / unstable props - Object/function dep busting memoisation

FOLLOW-UP QUESTIONS: 1. Does `useMemo` guarantee the value won't be recomputed? 2. How do you measure if memoisation actually helps?

FOLLOW-UP ANSWERS: 1. No — React may discard the memoised value to free memory (“intentionally forget”). The guarantee is [referential equality for the same deps](#), not that the compute is skipped. So don’t rely on it for side effects; rely on it for referential stability. 2. React DevTools Profiler — record an interaction, see which components re-render and why. If a memoised child still re-renders, the props aren’t stable (find the unstable one with “Why did this render?”). If your memoised value isn’t used, the compute was wasted.

RED FLAGS TO AVOID: - “Use useMemo everywhere” (kills perf, code complexity) - Memoising with array deps that are always new - Expecting useMemo to skip compute for side effects (no)

Q-FE-5: useReducer vs useState — when does the complexity pay off

DIFFICULTY: Medium **CATEGORY TAG:** Frontend / React State

ANSWER: useState: best for independent primitive values (a string, a number, a boolean). Simple API: `const [value, setValue] = useState(0)`.

useReducer: best when state transitions are related, when next state depends on previous in non-trivial ways, or when you have many `setState` calls that always go together.

Use useReducer when: - State has multiple related fields that change together (a form's `status`, `errors`, `values`) - Next state depends on a complex transition logic (a finite state machine: `idle` → `loading` → `success` | `error`) - You test the reducer in isolation (pure function `(state, action) → state`) - Multiple sub-components dispatch actions (decoupled from state shape) - You want a single source of truth with explicit transitions (debuggable action log)

CODE/EXAMPLE SNIPPET:

```
const initial = { status: "idle", data: null, error: null };
function reducer(state, action) {
  switch (action.type) {
    case "fetch": return { ...state, status: "loading" };
    case "success": return { status: "success", data: action.data, error: null };
    case "error": return { status: "error", data: null, error: action.error };
    default: return state;
  }
}
function Component() {
  const [state, dispatch] = useReducer(reducer, initial);
  // dispatch({ type: "fetch" }); dispatch({ type: "success", data });
}
```

Rule of thumb: more than 3 `useState` calls related to one concern → `useReducer`. State transitions are conditional → `useReducer`. You want to unit test state logic → `useReducer`.

KEY TERMS TO MENTION: - `useState`: independent primitive values, simple - `useReducer`: related fields, finite state machines, complex transitions - Reducer is pure `(state, action) → state` — testable in isolation - Dispatch decouples callers from state shape - Rule: >3 related `useStates` → `useReducer`

FOLLOW-UP QUESTIONS: 1. Can you replace Redux with `useReducer` + `Context`? 2. How do you type a reducer in TypeScript?

FOLLOW-UP ANSWERS: 1. For local + medium app state, yes — `useReducer` + `Context` works well. Trade-off: every context value change re-renders all consumers (no selector subscription like Redux). For large apps with many consumers of a slice, split context or use a selector library (`use-context-selector`). Redux Toolkit still wins for: time-travel debugging,

middleware (thunk/saga), devtools ecosystem. 2. Discriminated union of actions, generic reducer signature:

```
type Action = { type: "fetch" } | { type: "success"; data: T } | { type: "error";  
  error: E };  
function reducer(state: State, action: Action): State { /* switch */ }
```

The discriminated union gives you exhaustive switch checks (`default: return state` triggers a type error if a case is missing).

RED FLAGS TO AVOID: - “useReducer is just useState with extra steps” (misses the transition model) - Using useReducer for a single toggle (over-engineering) - Mutating state in the reducer (must return new state)

Q-FE-6: Context API vs Redux Toolkit — when to use which

DIFFICULTY: Medium **CATEGORY TAG:** Frontend / State Management

ANSWER: Context API: built-in, dependency-free. Best for low-frequency updates (theme, locale, current user, feature flags) read by many components. Trade-off: any context value change re-renders **all** consumers (no selector). So don't put high-frequency state (typing, scrolling, drag) in Context.

Redux Toolkit (RTK): opinionated Redux with `createSlice` (reducers + actions), `createAsyncThunk` (async), RTK Query (data fetching + caching). Best for: large app with global state, many consumers of slices, need for time-travel debugging, middleware (logging, analytics), strict unidirectional flow, devtools. Trade-off: boilerplate (less with RTK), learning curve, not built-in.

Decision tree: - Just one or two pieces of global state, low-frequency → Context - High-frequency state (canvas, drag, scroll) → local `useState` or `useRef`, not Context - Many components need the same slice with selective subscription → Redux Toolkit + selectors - Need time-travel, middleware, or strict action log → Redux - Async data fetching + caching (the "server state" problem) → RTK Query or TanStack Query (better fit than Redux for most CRUD apps)

Most CRUD apps today: TanStack Query for server state (caching, invalidation, retries, optimistic updates) + Context or Zustand for the small UI state (theme, sidebar open). Redux is overkill unless you have complex client-side state (offline editors, games).

CODE/EXAMPLE SNIPPET:

```
// Context - simple, low-frequency
const ThemeCtx = createContext("light");
function App() { return <ThemeCtx.Provider value="dark"><Page/></ThemeCtx.Provider>; }
function Page() { const t = useContext(ThemeCtx); return <div className={t} />; }

// RTK slice
const slice = createSlice({
  name: "cart",
  initialState: [],
  reducers: {
    add(state, action) { state.push(action.payload); }, // Immer lets you "mutate"
  },
});
```

KEY TERMS TO MENTION: - Context: built-in, low-freq global state (theme, user, locale); re-renders all consumers - Redux Toolkit: `createSlice` (Immer), `createAsyncThunk`, RTK Query, devtools, middleware - TanStack Query for server state (caching, invalidation, optimistic) -

Decision: server state → TanStack/RTK Query; UI state → Context/Zustand; complex client state → Redux

FOLLOW-UP QUESTIONS: 1. How do you prevent Context from causing excess re-renders? 2. What's Immer and why does RTK use it?

FOLLOW-UP ANSWERS: 1. Split contexts by update frequency (ThemeContext vs UserContext vs ToastContext separate). Or memoise the value (`useMemo`). Or use a selector library (`use-context-selector`) that lets consumers subscribe to a slice. The naive approach — one giant AppContext — causes every consumer to re-render on any change. 2. Immer lets you write “mutating” code that produces an immutable copy (via Proxy under the hood). So `state.push(x)` in a reducer actually creates a new array. Without Immer, you'd write `{ ...state, items: [...state.items, x] }` — error-prone and verbose for nested state.

RED FLAGS TO AVOID: - “Use Redux for everything” (overkill for small apps) - Putting high-frequency state in Context (re-render storm) - Not knowing RTK (anyone saying “Redux has too much boilerplate” hasn't used RTK)

Q-FE-7: React 18 concurrent features — useTransition, useDeferredValue, Suspense

DIFFICULTY: Hard **CATEGORY TAG:** Frontend / React 18

ANSWER: Concurrent React lets the renderer pause, resume, or abandon work. Built on Fiber's time-slicing. The user-visible benefit: high-priority updates (typing, clicks) interrupt low-priority (filtering a list, rendering new route).

useTransition: marks a state update as “low priority” (transition). Returns `[isPending, startTransition]`. Wrap expensive updates in `startTransition` so they don't block the input. Use for: filtering a large list while typing, navigating to a new route with heavy render, switching tabs with a big chart.

useDeferredValue: the read-side counterpart. `const deferred = useDeferredValue(value)`. The deferred value lags the real value — used for expensive renders; when the real value changes, React keeps rendering with the old deferred value while computing the new render in the background.

Suspense: declarative loading state. Wrap a subtree in `<Suspense fallback={<Spinner/>}>` — when a child “suspends” (throws a promise, e.g. `React.lazy` or a Suspense-enabled data library), React shows the fallback. With concurrent React, Suspense works with server components, transitions, and streaming SSR.

CODE/EXAMPLE SNIPPET:

```
function Search() {
  const [query, setQuery] = useState("");
  const [isPending, start] = useTransition();
  const deferred = useDeferredValue(query);
  return (
    <>
      <input value={query} onChange={e => { setQuery(e.target.value);
        start(() => setFilter(e.target.value)); }} />
      <BigList filter={deferred} />
      { /* re-render uses deferred value, no input lag */ }
      {isPending && <span>filtering...</span>}
    </>
  );
}
```

Streaming SSR with Suspense: the server sends HTML in chunks; each Suspense boundary streams its content when ready, replacing the fallback. The user sees the page shell immediately + content filling in.

Gotchas: - `startTransition` doesn't help for CPU-bound synchronous work (no yielding mid-function) - `useDeferredValue` + memoised children: only the deferred subtree re-renders with old data - Suspense for data fetching requires a framework (Next.js, Relay) or your own throw-a-promise integration

KEY TERMS TO MENTION: - Concurrent React: pause/resume/abandon render work; priority-based - useTransition: mark updates low-priority (filtering, route change); isPending for UI - useDeferredValue: deferred input for expensive renders - Suspense: declarative loading; streaming SSR; works with React.lazy + data libs - Gotcha: doesn't help CPU-bound sync work

FOLLOW-UP QUESTIONS: 1. What's the difference between useTransition and useDeferredValue? 2. How does Suspense for data fetching work?

FOLLOW-UP ANSWERS: 1. useTransition is the **write side** — you wrap a state update to mark it low-priority. useDeferredValue is the **read side** — you get a deferred copy of a value to render with. Same intent (don't block the input), different entry points. useTransition when you control the update site; useDeferredValue when you only consume a value (e.g. a child receiving a prop). 2. A component calls a data-fetching hook that hasn't loaded yet → it throws a Promise. React catches the promise, shows the nearest Suspense fallback, and retries the component when the promise resolves. Frameworks (Next.js, Relay) wire this for you. For custom integrations, you maintain a per-resource cache that tracks **pending | resolved | error** and throw if pending.

RED FLAGS TO AVOID: - "Concurrent mode" (deprecated term — concurrent **features** now opt-in per update) - Using startTransition for cheap updates (no benefit, slight overhead) - Expecting useTransition to magically make CPU work non-blocking

Q-FE-8: React Server Components vs Client Components

DIFFICULTY: Hard **CATEGORY TAG:** Frontend / React 19 / RSC

ANSWER: Server Components (RSC): render only on the server. Output is serialised (not HTML — a special RSC payload). They: cannot use `useState`, `useEffect`, browser APIs; can directly query the DB, read files, use secrets. Zero client JS shipped. Great for: static content, data-heavy read pages, blog posts, product detail.

Client Components: the React you've always known. Run on the server (for SSR HTML) and hydrate on the client. Can use hooks, event handlers, browser APIs. The directive `"use client"` at the top of a file marks it.

The hybrid model (Next.js App Router): - Server Components by default - Add `"use client"` for interactive parts (forms, animations, state) - Server Components can import Client Components; Client Components can't import Server Components (but can receive them as props — the `children` pattern)

Why it matters: - Less JS shipped to the client (faster TTI, lower bandwidth) - Direct DB access without a separate API layer (for read-only pages) - No more "fetch in useEffect → loading spinner → render" waterfall — the server awaits data and streams HTML

CODE/EXAMPLE SNIPPET:

```
// app/page.js – Server Component (default)
import { db } from "@lib/db";
export default async function Page() {
  const users = await db.user.findMany(); // direct DB, no client fetch
  return <UserList users={users} />;      // pass to client component
}

// app/UserList.js – Client Component
"use client";
import { useState } from "react";
export function UserList({ users }) {
  const [q, setQ] = useState("");
  return <input value={q} onChange={e => setQ(e.target.value)} />;
}
```

Gotchas: - Server → Client boundary is the **network/serialisation boundary**. Props must be serialisable (no functions, no class instances, no Dates — pass ISO strings). - "use client" doesn't mean "only renders on client" — it still SSRs. It means "can hydrate + use client features." - You can't pass a function from a Server to a Client component directly. Pass a server action (Next.js) or hoist the function into the client component.

KEY TERMS TO MENTION: - RSC: server-only, no hooks/effects, zero client JS, direct DB/files/secrets - Client Components: `"use client"`, hydrate, can use hooks - Server can import Client; Client can't import Server (but can receive as `children`) - Prop serialisation boundary

(no functions/class instances/Dates) - Benefits: less JS shipped, no fetch waterfall, direct DB access

FOLLOW-UP QUESTIONS: 1. How do Server Actions fit in? 2. When must you still use Client Components?

FOLLOW-UP ANSWERS: 1. Server Actions are functions that run on the server but are invoked from the client. Marked with `"use server"`. The client gets a serialised reference, calls it like a normal async function, Next.js routes the call to the server. Replaces most hand-rolled API routes for mutations. Can be used inside a form's `action` prop (progressive enhancement — works without JS). 2. Anywhere you need interactivity: event handlers (onClick), state (`useState`), effects (`useEffect`), refs, browser APIs (`window` , `localStorage`), animations, third-party libs that depend on `window`. The pattern: keep the data-fetching shell on the server, push interactivity into leaf client components.

RED FLAGS TO AVOID: - "Server Components are SSR" (different — SSR produces HTML; RSC produces an RSC payload + zero client JS for that subtree) - Putting "use client" on every file (defeats the benefit) - Passing non-serialisable props across the boundary

Q-FE-9: Keys in React lists — why they matter and the index-key trap

DIFFICULTY: Medium **CATEGORY TAG:** Frontend / React

ANSWER: Keys are React's identity for list items during reconciliation. When the list reorders, inserts, or deletes, React uses the key to match the new element to the previous one. Wrong keys → React reuses the wrong DOM nodes → bugs (state from one item appears in another, animations jump, form inputs carry wrong values).

Index as key (the trap): - Works fine when the list is **static** (never reorders, no insert/delete in the middle) - Breaks when the list **reorders** — React reuses DOM node 0 for the new item 0 (which used to be a different entity), so any uncontrolled state (input value, transition state) carries over to the wrong item. - Also breaks performance — every re-order triggers full re-render of every item (no reuse).

Stable unique IDs as key: - `key={item.id}` — React matches by ID, reuses DOM nodes correctly, re-renders only changed items. - IDs must be **stable across renders** (same ID for same entity) and **unique within the list**.

CODE/EXAMPLE SNIPPET:

```
// BAD: index as key, list reorders → wrong state on wrong item
{items.map((item, i) => <Input key={i} value={item.text} />)}

// GOOD: stable id
{items.map(item => <Input key={item.id} value={item.text} />)}
```

Concrete bug: list of editable text inputs, index-keyed. User types in input 0, then inserts a new item at the top. React now matches `key=0` to the new item — the DOM node that had the user's text is now reused for the new (empty) item, and the user's text "moves" to input 1. Visually: the text appears to jump down. Confusing + a real bug.

KEY TERMS TO MENTION: - Key = identity for list reconciliation - Index key: OK only for static lists, breaks on reorder/insert/delete - Stable unique ID: correct reuse of DOM nodes + state - Concrete bug: uncontrolled state carrying to wrong item after reorder

FOLLOW-UP QUESTIONS: 1. What if your items have no natural ID? 2. Can two siblings share the same key if they're in different lists?

FOLLOW-UP ANSWERS: 1. Generate one on data fetch (e.g. `crypto.randomUUID()` for client-only) or use a composite of stable fields. Don't use the index, don't use `Date.now()` (new every render → re-mount every render → state lost + perf hit). 2. Yes — keys only need to be unique among siblings within the same parent. Two different lists can both have key "1". React scopes key uniqueness to siblings.

RED FLAGS TO AVOID: - “Index as key is fine” (only for static lists) - Using `Math.random()` or `Date.now()` as key (new every render) - Skipping keys entirely (React falls back to index → same trap)

Q-FE-10: React Fiber architecture — what changed and why

DIFFICULTY: Hard **CATEGORY TAG:** Frontend / React Internals

ANSWER: Before React 16, reconciliation was **recursive + synchronous** — once started, it ran to completion, blocking the main thread. Long renders froze the UI (typing lagged, animations stuttered).

Fiber (React 16+) rewrites the reconciler as an **interruptible, priority-based, linked-list of work units**. Each component is a “fiber node” with links to its parent, child, sibling — a tree stored as a linked list (not a single nested object). React can: - Pause work mid-tree and yield to the browser - Resume later (the linked list remembers where it was) - Prioritise: user input interrupts background renders - Abort stale work (e.g. an outdated fetch result)

Why linked-list instead of recursion: recursion uses the call stack — once you descend, you can’t pause. The fiber tree is a manual stack — React controls traversal, can pause/resume.

Two phases: 1. **Render phase** (pure, can be paused/aborted) — compute the new tree, diff. No side effects. 2. **Commit phase** (sync, must complete) — apply DOM mutations, run `useLayoutEffect`. Cannot be interrupted (would leave the DOM half-updated). Then `useEffect` runs after paint.

Lanes (React 18): priority model — each update is tagged with a “lane” (SyncLane, TransitionLane, IdleLane...). React processes lanes by priority. Multiple updates of different lanes don’t block each other.

KEY TERMS TO MENTION: - Pre-16: recursive + synchronous, blocked main thread - Fiber: linked-list of work units, interruptible, priority-based - Why linked-list: manual stack for pause/resume (call stack can’t pause) - Render phase (pure, pausable) vs Commit phase (sync, must complete) - Lanes: priority model in React 18 - Enables: time-slicing, concurrent features, Suspense

FOLLOW-UP QUESTIONS: 1. Why can’t the commit phase be interrupted? 2. What’s the difference between a fiber and an element?

FOLLOW-UP ANSWERS: 1. Commit applies DOM mutations. Interrupting it leaves the DOM in an inconsistent state (some nodes updated, some not) — visible bugs + broken event handlers. So commit is sync + atomic. The trade-off: very large commit phases (e.g. rendering 10K items at once) can still drop frames. Mitigation: virtualise, paginate, or split with Suspense. 2. An element (`<div/>`) is a plain object describing what to render (type + props). A fiber is the internal representation React keeps between renders — it holds the actual state, hooks queue, effects list, sibling/child pointers, alternate (previous version for diff). Elements are inputs; fibers are the working state.

RED FLAGS TO AVOID: - “Fiber makes React faster” (it makes it **interruptible** — different) - “Concurrent mode” (deprecated term — concurrent features are opt-in per update)

Q-FE-11: JavaScript event loop — microtasks vs macrotasks

DIFFICULTY: Medium **CATEGORY TAG:** Frontend / JavaScript

ANSWER: JS is single-threaded. The event loop lets it appear concurrent via a queue of jobs.

Macrotasks (task queue): `setTimeout`, `setInterval`, `setImmediate` (Node), I/O, UI events, `postMessage`. One macrotask runs per loop iteration.

Microtasks (microtask queue): Promise callbacks (`.then`, `.catch`, `.finally`), `queueMicrotask`, `MutationObserver`, `process.nextTick` (Node, highest priority). ALL microtasks drain before the next macrotask.

Order per loop iteration: 1. Run one macrotask 2. Drain the entire microtask queue (even microtasks queued by microtasks) 3. Render (browser, if needed) 4. Repeat

CODE/EXAMPLE SNIPPET:

```
console.log("1");
setTimeout(() => console.log("2")); // macrotask
Promise.resolve().then(() => console.log("3")); // microtask
console.log("4");
// Output: 1 4 3 2 – microtask 3 runs before macrotask 2
```

Practical consequences: - `await x` yields to the microtask queue — even if `x` is already resolved, the code after `await` runs in a microtask (next tick), not synchronously. - A long chain of `.then` blocks the loop (microtasks drain before render) — UI freezes. - For chunking heavy work, use `setTimeout(0)` or `scheduler.yield()` (macrotask boundary) — not promises (still microtasks, no render between).

Node.js specifics: `process.nextTick` runs before microtasks (highest priority). Node 11+ matches browser behaviour — microtasks drain between each `setTimeout` callback (previously drained between phases).

KEY TERMS TO MENTION: - Single thread, event loop, task queue - Macrotask: `setTimeout`/`setInterval`/I/O/UI events — one per loop - Microtask: Promises, `queueMicrotask`, `MutationObserver` — all drain before next macrotask - Order: macrotask → drain microtasks → render → repeat - `await` yields to microtask queue - For chunking: macrotask boundary, not microtask

FOLLOW-UP QUESTIONS: 1. Why does `await` always yield, even for an already-resolved promise? 2. How do you chunk a 10M-row processing loop without freezing the UI?

FOLLOW-UP ANSWERS: 1. Spec requirement — `await` deschedules the continuation to the microtask queue, even when the awaited value is already resolved. So `await Promise.resolve()` runs the next line in a microtask, not immediately. The reason: predictable execution semantics (you can't have async code sometimes run sync, sometimes

async — too error-prone). 2. Process in batches of N with a macrotask boundary between batches:

```
async function chunk(items) {
  for (let i = 0; i < items.length; i += 1000) {
    processBatch(items.slice(i, i + 1000));
    await new Promise(r => setTimeout(r, 0)); // macrotask → render between batches
  }
}
```

Or use `requestIdleCallback` (browser) / `setImmediate` (Node) for lower-priority chunking. Web Workers are the better solution if the work is CPU-bound — moves it off the main thread entirely.

RED FLAGS TO AVOID: - “Promise.then runs immediately” (it’s a microtask, next tick) - Using promises to chunk heavy work (they’re microtasks — no render between, still freezes UI) - Not knowing `setTimeout(0)` has a 4ms clamp (after 5 nested timers, browser enforces min 4ms)

Q-FE-12: Closures in JavaScript — and the common gotchas

DIFFICULTY: Medium **CATEGORY TAG:** Frontend / JavaScript

ANSWER: A closure is a function bundled with references to its lexical scope. The inner function retains access to outer variables even after the outer function returns.

Classic use cases: - Data privacy (module pattern) — encapsulate state without classes - Function factories — generate functions with bound parameters - Callbacks — event handlers, async callbacks that capture state

CODE/EXAMPLE SNIPPET:

```
function counter() {
  let n = 0;                // private, no external access
  return () => ++n;          // closure over n
}
const c = counter();
c(); c(); // 2
```

The classic loop gotcha (var + closure):

```
for (var i = 0; i < 3; i++) setTimeout(() => console.log(i), 0);
// Output: 3 3 3 – all three closures capture the SAME i (function-scoped)
// Fix with let (block-scoped): each iteration has its own i → 0 1 2
for (let i = 0; i < 3; i++) setTimeout(() => console.log(i), 0);
// Output: 0 1 2
```

Stale closure in React (common bug):

```
useEffect(() => {
  const id = setInterval(() => console.log(count), 1000);
  return () => clearInterval(id);
}, []); // empty deps → closure captures count=0 forever → logs 0 forever
// Fix: add count to deps, or use functional update setInterval(() => setCount(c => c+1), 1000)
```

Memory leaks: closures retain references to the entire outer scope. If a closure is long-lived (event listener on `document`) and captures a big object, the object can't be GC'd. Fix: null the reference when done, or use `WeakRef`.

KEY TERMS TO MENTION: - Closure = function + captured lexical scope - Data privacy (module pattern), factories, callbacks - `var` + closure loop gotcha → 3 3 3 (function-scoped); `let` fixes it (block-scoped) - Stale closure in `useEffect` (empty deps → captures initial state forever) - Memory leak risk: long-lived closures retain referenced objects

FOLLOW-UP QUESTIONS: 1. How do you fix a stale closure in React without re-running the effect? 2. What's the module pattern, and how do closures enable it?

FOLLOW-UP ANSWERS: 1. Use a ref to hold the latest value: `const countRef = useRef(count); useEffect(() => { countRef.current = count; });` then read `countRef.current` inside the interval. The ref is mutable and stable — no stale closure. Or use a functional update if the value is the state itself (`setCount(c => c+1)`). 2. IIFE returning an object with public methods; private state in the closure:

```
const api = (() => { let private = 0; return { inc: () => ++private, get: () => private; }; })();
api.inc(); api.get(); // 1
// 'private' is not accessible from outside – only via the returned methods.
```

Modern alternative: ES modules (each module is its own scope) or private class fields (`#private`).

RED FLAGS TO AVOID: - “Closures are a JS-specific feature” (most languages have them — Python, Java lambdas) - Not knowing the `var / let` closure loop difference - Forgetting stale closures in React effects

Q-FE-13: `this` binding in JavaScript — call/apply/bind and arrow functions

DIFFICULTY: Medium **CATEGORY TAG:** Frontend / JavaScript

ANSWER: `this` is dynamically bound at call time (not lexical). Four binding rules:

1. **Default binding:** plain function call → `this` is `undefined` (strict mode) or `globalThis` (sloppy).
2. **Implicit binding:** `obj.method()` → `this` is `obj`. Lost when you assign the method to a variable (`const m = obj.method; m();` — `this` is lost).
3. **Explicit binding:** `fn.call(thisArg, a, b)` / `fn.apply(thisArg, [a, b])` → `this` is `thisArg`.
4. **new binding:** `new Fn()` → `this` is a fresh object.

Arrow functions are special: they have **no own `this`** — they capture the lexical `this` of the enclosing scope. Cannot be changed by call/apply/bind. This makes them ideal for callbacks inside methods.

CODE/EXAMPLE SNIPPET:

```
class Timer {
  constructor() { this.count = 0; }
  start() {
    setInterval(() => this.count++, 1000); // arrow → `this` = Timer instance
    setInterval(function() { this.count++; }, 1000); // function → `this` =
      globalThis
  }
}
// Explicit bind equivalent:
setInterval(this.tick.bind(this), 1000);
```

Common pitfalls: - Event handlers: `el.addEventListener('click', obj.method)` — `this` is `el`, not `obj`. Fix: `obj.method.bind(obj)` or arrow wrapper. - Destructured methods: `const { method } = obj; method();` — `this` lost. Fix: bind or use a class with arrow fields. - Callback in map/filter: `arr.map(obj.method)` — same problem.

KEY TERMS TO MENTION: - 4 rules: default, implicit, explicit (call/apply/bind), new - Arrow functions: lexical `this`, no own binding, ideal for callbacks in methods - Pitfalls: event handlers, destructured methods, callbacks — use bind or arrow

FOLLOW-UP QUESTIONS: 1. What's the difference between call and apply? 2. Why can't you rebind `this` of an arrow function?

FOLLOW-UP ANSWERS: 1. `call(thisArg, arg1, arg2)` takes arguments individually. `apply(thisArg, [arg1, arg2])` takes an array. With spread (`fn.call(thisArg, ...args)`) they're equivalent. `bind(thisArg, ...args)` returns a new function with `this` (and any preset args) permanently bound — useful for partial application. 2. Arrow functions don't

have their own `this` slot — they reference the outer scope's `this` lexically. `call`/`apply`/`bind` set the function's own `this`, but there's nothing to set. The spec just ignores the argument. So `((() => console.log(this)).call({x: 1}))` logs the outer `this`, not `{x: 1}`.

RED FLAGS TO AVOID: - “Arrow functions are just shorthand” (lexical `this` is the real difference) - Using arrow functions as object methods that need dynamic `this` (breaks when shared/mixed-in) - Forgetting `bind` in event listeners with class methods

Q-FE-14: Prototypal inheritance vs class — how JS inheritance really works

DIFFICULTY: Medium **CATEGORY TAG:** Frontend / JavaScript

ANSWER: JS has only prototypal inheritance. `class` (ES6) is syntactic sugar over the same prototype chain.

Prototype chain: every object has an internal `[[Prototype]]` (accessed via `Object.getPrototypeOf(obj)` or `__proto__`). Property lookup walks the chain: `obj → obj.__proto__ → ... → Object.prototype → null`. If not found, `undefined`.

Constructor functions (pre-class):

```
function Animal(name) { this.name = name; }
Animal.prototype.speak = function() { return this.name; };
function Dog(name) { Animal.call(this, name); } // super constructor
Dog.prototype = Object.create(Animal.prototype); // inherit
Dog.prototype.bark = function() { return "woof"; };
```

ES6 class (same thing, cleaner):

```
class Animal {
  constructor(name) { this.name = name; }
  speak() { return this.name; }
}
class Dog extends Animal {
  bark() { return "woof"; }
}
```

Both produce the same prototype chain. `class` adds: stricter syntax (must be called with `new`), `super()` for parent constructor, `static` methods, `#private` fields.

Composition over inheritance: prefer composing small, focused objects/functions over deep inheritance trees. “Favor object composition over class inheritance” (GoF). Mixins, factories, higher-order functions.

KEY TERMS TO MENTION: - JS has only prototypal inheritance; class is sugar - Prototype chain: `obj → proto → ... → Object.prototype → null` - Constructor functions vs class (same chain, different syntax) - Composition over inheritance (mixins, factories, HOFs) - `super()`, `static`, `#private` (class-only)

FOLLOW-UP QUESTIONS: 1. What’s `Object.create(null)` useful for? 2. How do you check if x inherits from Y?

FOLLOW-UP ANSWERS: 1. Creates an object with no prototype (no `toString`, `hasOwnProperty` etc.). Useful as a pure string→value map (no risk of prototype pollution — `obj.__proto__` is undefined). Dictionary libraries (Map alternative) use it for safety. 2.

`x instanceof Y` (checks `Y.prototype` is in x's chain). `Y.prototype.isPrototypeOf(x)` (same, different phrasing). `Object.getPrototypeOf(x) === Y.prototype` (one level). For deep checks: walk the chain manually.

RED FLAGS TO AVOID: - "JS has classical inheritance" (it doesn't — only prototypes) - Mutating `Person.prototype` after subclassing (subclasses captured the old prototype) - Using `__proto__` directly (deprecated; use `Object.getPrototypeOf` / `setPrototypeOf`)

Q-FE-15: Promises, async/await, and error handling

DIFFICULTY: Medium **CATEGORY TAG:** Frontend / JavaScript Async

ANSWER: A Promise represents a value that may not be available yet. Three states: pending → fulfilled | rejected. Once settled, immutable.

Promise chaining: `.then(onFulfilled, onRejected)` returns a new promise — values flow through, errors propagate. `.catch(fn)` is sugar for `.then(null, fn)`. `.finally(fn)` runs regardless (cleanup).

async/await: syntactic sugar over promises. `async` function always returns a promise. `await` pauses until the promise settles. Errors thrown become rejected promises; use `try/catch`.

CODE/EXAMPLE SNIPPET:

```
// Promise chain
fetch(url).then(r => r.json()).then(data => render(data)).catch(e =>
  console.error(e));

// async/await (cleaner)
async function load() {
  try {
    const r = await fetch(url);
    if (!r.ok) throw new Error(`HTTP ${r.status}`);
    return await r.json();
  } catch (e) {
    console.error(e); throw e; // re-throw to caller
  }
}

// Parallel
const [a, b] = await Promise.all([fetchA(), fetchB()]); // both at once
const winner = await Promise.race([fetch(), timeout(5000)]); // first wins
```

Common pitfalls: - Sequential awaits when parallel is fine: `await a(); await b();` → 2 round trips. Use `Promise.all([a(), b()])`. - Forgetting to catch — unhandled rejections crash Node (or log to console in browser). Always add a top-level catch. - `await` in a loop (`for await`) — sequential by design. Use `Promise.all` if independent. - `return await x` adds an extra microtask vs `return x` — irrelevant except in try/catch where `await` makes the stack trace correct. - `Promise.all` fails fast (one reject rejects the whole). Use `Promise.allSettled` to wait for all + inspect failures.

KEY TERMS TO MENTION: - Promise: pending/fulfilled/rejected, immutable once settled - Chaining: then/catch/finally, values flow, errors propagate - async/await = sugar; try/catch for errors - Promise.all (parallel, fail-fast), allSettled (wait all), race (first), any (first success) - Pitfalls: sequential awaits (use Promise.all), unhandled rejections, for await

FOLLOW-UP QUESTIONS: 1. What's the difference between Promise.all, allSettled, race, and any? 2. How do you cancel an in-flight fetch?

FOLLOW-UP ANSWERS: 1. `all` — wait for all, reject on first failure (returns array of results). `allSettled` — wait for all, never reject (returns `[{status, value/reason}, ...]`). `race` — settle as soon as one settles (success or failure). `any` — resolve on first success, reject only if all fail (AggregateError). 2. AbortController:

```
const ctrl = new AbortController();
setTimeout(() => ctrl.abort(), 5000);
fetch(url, { signal: ctrl.signal }).catch(e => e.name === 'AbortError' &&
  console.log('cancelled'));
```

AbortController is now the standard cancellation primitive — used by fetch, EventTarget, and many libs. For long-running async ops without fetch, check `signal.aborted` periodically and bail.

RED FLAGS TO AVOID: - “async functions always run in parallel” (they’re sequential unless you use Promise.all) - `await` in a forEach (forEach doesn’t await — use for-of) - Not catching rejected promises

Q-FE-16: Event delegation, bubbling, and capturing

DIFFICULTY: Easy **CATEGORY TAG:** Frontend / Browser

ANSWER: DOM events propagate in three phases: **capturing** (target → root), **target** (the element), **bubbling** (target → root). Most handlers fire in the bubbling phase (the default). Use `addEventListener(type, fn, { capture: true })` to listen in the capturing phase.

Event delegation: instead of attaching a listener to every child, attach one to the parent and use `event.target` to find the actual clicked child. Benefits: fewer listeners (memory), works for dynamically added children (no re-binding), simpler code.

CODE/EXAMPLE SNIPPET:

```
// Bad: 1000 listeners
items.forEach(li => li.addEventListener('click', () => handleClick(li.id)));
// Good: 1 listener on the parent
list.addEventListener('click', e => {
  const li = e.target.closest('li');
  if (li) handleClick(li.dataset.id);
});
```

Stopping propagation: `e.stopPropagation()` stops the event from going further up. `e.stopImmediatePropagation()` also stops other listeners on the same element. `e.preventDefault()` prevents the default browser action (link navigation, form submit).

Gotchas: - `closest()` finds the nearest ancestor matching a selector — robust to nested children inside the `<li>` (e.g. a `<span>` inside). - Don't delegate `focus` / `blur` (don't bubble — use `focusin` / `focusout` instead). - Delegation breaks if a child calls `stopPropagation` — your parent never sees the event.

KEY TERMS TO MENTION: - Three phases: capture, target, bubble (default = bubble) - Delegation: 1 parent listener + `event.target.closest` — fewer listeners, dynamic children - `stopPropagation` (don't bubble further), `stopImmediatePropagation` (also same-element), `preventDefault` - Use `focusin`/`focusout` (bubble) instead of `focus`/`blur` (don't)

FOLLOW-UP QUESTIONS: 1. Why doesn't `focus` bubble? 2. When does delegation fail?

FOLLOW-UP ANSWERS: 1. `focus` / `blur` are “focus events” — by spec they don't bubble (they fire on the target only). `focusin` / `focusout` are the bubbling counterparts (added later). The reason: focus is conceptually a single-element state, not a tree event. Bubbling would mean every ancestor gets a “focus” event when any descendant focuses — noisy. So they made a separate bubbling pair. 2. When a child calls `stopPropagation` (the delegated handler never fires). When the event doesn't bubble (`focus`, `blur`, `mouseenter`, `mouseleave` — use the bubbling variants). When you need per-element state that the parent can't reconstruct (rare). When the parent's `closest()` can't disambiguate which child (use data attributes).

RED FLAGS TO AVOID: - Adding a listener per row of a 10K-row table (memory + perf) - Using `stopPropagation` carelessly (breaks delegation) - Delegating non-bubbling events

Q-FE-17: Debounce vs throttle — and when to use which

DIFFICULTY: Easy **CATEGORY TAG:** Frontend / Performance

ANSWER: Both limit the rate at which a function fires. Different strategies.

Debounce: wait until the calls `stop` for N ms, then fire once. Resets the timer on every call. Use for: search-as-you-type (fire after the user pauses typing), window resize (fire when resizing stops), autosave (save after edits stop).

Throttle: fire at most once per N ms, regardless of how many calls. Use for: scroll handlers (fire every 16ms = 60fps), mousemove, drag, button spam prevention.

CODE/EXAMPLE SNIPPET:

```
function debounce(fn, ms) {
  let t;
  return (...a) => { clearTimeout(t); t = setTimeout(() => fn(...a), ms); };
}
function throttle(fn, ms) {
  let last = 0;
  return (...a) => {
    const now = Date.now();
    if (now - last >= ms) { last = now; fn(...a); }
  };
}
// Usage
input.addEventListener('input', debounce(search, 300)); // fires 300ms after last
  keystroke
window.addEventListener('scroll', throttle(update, 16)); // fires every 16ms
```

Leading/trailing options: - Throttle `{ leading: true, trailing: true }` (default): fire immediately, then no more until window. - Debounce `{ leading: true }`: fire immediately + on trailing edge (button double-click protection).

Libraries: Lodash's `_.debounce` / `_.throttle` provide options (leading/trailing, `maxWait` for debounce). Use them — hand-rolled versions miss edge cases (this binding, args, immediate flag).

KEY TERMS TO MENTION: - Debounce: wait for pause, fire once (search, resize, autosave) - Throttle: max once per N ms (scroll, mousemove, drag) - Leading/trailing edge options - Use Lodash's — handles edge cases

FOLLOW-UP QUESTIONS: 1. How would you implement debounce with a max-wait (so it fires at least every N ms even if input continues)? 2. Why throttle scroll to 16ms?

FOLLOW-UP ANSWERS: 1. Track time of first call; if the total elapsed exceeds `maxWait`, fire immediately + reset. Lodash's `_.debounce(fn, 200, { maxWait: 1000 })` does this — useful for autosave (don't lose >1s of typing if the user keeps typing). 2. 16ms ≈ 1 frame at 60fps.

Throttling to one call per frame matches the render rate — calling more often wastes CPU (the result isn't painted). Calling less often (e.g. 32ms = 30fps) makes the effect feel laggy. `requestAnimationFrame` is even better — fires exactly before each paint, never more.

RED FLAGS TO AVOID: - "They're the same" (different strategies, different uses) - Throttling search input (debounce is right — fire after typing pauses) - Using `setTimeout(0)` for throttling (not actually throttled — fires every microtask)

Q-FE-18: CSS box model, flexbox, and grid — the layout fundamentals

DIFFICULTY: Medium **CATEGORY TAG:** Frontend / CSS

ANSWER: Box model: every element is a rectangle: content → padding → border → margin. Two sizes: `content-box` (default — width = content only) and `border-box` (width = content + padding + border). Always set `box-sizing: border-box` globally — predictable sizing.

Flexbox (1D): lay out items in a row OR column. Use for: nav bars, toolbars, card rows, centering. Properties: `justify-content` (main axis), `align-items` (cross axis), `gap`, `flex-grow/shrink/basis`. `flex: 1` = grow to fill.

Grid (2D): lay out items in rows AND columns simultaneously. Use for: page layouts, dashboards, complex grids. `grid-template-columns: repeat(12, 1fr)`, `grid-area` for placement. Implicit vs explicit tracks.

CODE/EXAMPLE SNIPPET:

```
*, *::before, *::after { box-sizing: border-box; }

/* Flexbox: center a div horizontally + vertically */
.center { display: flex; place-items: center; min-height: 100vh; }

/* Grid: 3-column responsive layout, sidebar collapses on mobile */
.layout {
  display: grid;
  grid-template-columns: 240px 1fr 320px;
  grid-template-areas: "header header header" "nav main aside" "footer footer footer";
}
@media (max-width: 768px) {
  .layout { grid-template-columns: 1fr; grid-template-areas: "header" "main" "footer"; }
}
```

Choosing: - 1D layout (row OR column of items) → flexbox - 2D layout (rows AND columns) → grid - Most real layouts: grid for the page shell, flexbox for components inside - Gap (flex/grid): replaces the old `margin-right` hack — clean, no extra on last item

KEY TERMS TO MENTION: - Box model: content/padding/border/margin; `box-sizing: border-box` - Flexbox: 1D, justify-content/align-items/gap, flex-grow/shrink/basis - Grid: 2D, template-columns/rows/areas, implicit/explicit tracks - Choose: 1D → flex, 2D → grid, real layouts use both - `gap` replaces margin hacks

FOLLOW-UP QUESTIONS: 1. What's the difference between `align-items` and `align-content`? 2. How does `flex: 1` work under the hood?

FOLLOW-UP ANSWERS: 1. `align-items` aligns items **within their grid/flex line** (cross axis). `align-content` aligns the **lines themselves** when there's extra space (only relevant with wrap)

or multi-row grid). E.g. 3 flex rows in a 600px container with `flex-wrap: align-items` centers items in their row, `align-content: space-between` distributes the 3 rows across the height. 2. `flex: 1` is shorthand for `flex: 1 1 0`. `flex-grow: 1` (grow proportionally to fill free space), `flex-shrink: 1` (shrink equally if overflow), `flex-basis: 0` (initial size = 0, then grow). The combination makes all `flex: 1` items split free space equally. `flex: 1 1 auto` would start at content size then grow — different result.

RED FLAGS TO AVOID: - “Use flexbox for everything” (grid is better for 2D) - Floating layouts (`float: left`) — obsolete, hard to maintain - Not setting `box-sizing: border-box` (sizes never add up)

Q-FE-19: CSS specificity and the cascade

DIFFICULTY: Medium **CATEGORY TAG:** Frontend / CSS

ANSWER: The cascade determines which rule wins when multiple apply. Order of precedence: 1. **Origin & importance:** `!important` > author (your CSS) > user (browser setting) > user-agent (default). Author `!important` beats user-agent `!important`. 2. **Specificity:** calculated as (a, b, c, d): - a: `!important` or inline style (`style="..."`) - b: number of IDs - c: number of classes, attributes, pseudo-classes - d: number of elements, pseudo-elements - `*` adds 0. Comparison is left-to-right at each level. 3. **Source order:** later rules win at equal specificity.

Examples: - `#nav .item` → (0, 1, 1, 0) - `.list .item:hover` → (0, 0, 3, 0) - `div ul li a` → (0, 0, 0, 4)

CODE/EXAMPLE SNIPPET:

```
.button { color: blue; } /* (0,0,1,0) */
#save .button { color: red; } /* (0,1,1,0) – wins */
.button { color: green !important; } /* !important – wins over above */
```

Practical guidance: - Keep specificity low and flat (BEM, utility-first CSS) — easier to override. - Avoid `!important` (escalating arms race). Use it only for third-party overrides. - Inline styles beat everything except `!important` — avoid them. - Specificity wars: when a developer adds `#id #id #id .class` to beat a colleague's selector — code smell.

KEY TERMS TO MENTION: - Cascade order: origin/importance → specificity → source order - Specificity tuple (a/b/c/d): `!important/inline`, IDs, classes/attrs/pseudo, elements - Keep specificity low + flat (BEM, utility-first) - Avoid `!important` (arms race)

FOLLOW-UP QUESTIONS: 1. How does the cascade layer (`@layer`) change things? 2. How do CSS modules / scoped CSS avoid specificity issues?

FOLLOW-UP ANSWERS: 1. `@layer` (Cascade Layers, broad support 2023) lets you group rules into named layers with explicit priority. `@layer base, components, utilities;` — `base` is lowest, `utilities` highest. Rules outside layers take precedence over layered rules. Lets you avoid `!important` arms races — instead of fighting specificity, you control layer order. 2. CSS Modules + various scoped-CSS solutions add a unique attribute (e.g. `class="btn_x7y2z"`) or scope via attribute selectors (`[data-v-abc]`). Same specificity tuple, but the unique suffix means your selectors don't collide with other files' selectors. Effectively gives you "component-scoped" CSS without polluting the global namespace. Doesn't change specificity calculation — just avoids collisions.

RED FLAGS TO AVOID: - Using IDs for styling (high specificity, hard to override) - `!important` everywhere (escalation) - Long descendant selectors (`#nav .menu .item .link`) — high specificity + brittle

Q-FE-20: Repaint vs reflow — and how to avoid layout thrashing

DIFFICULTY: Hard **CATEGORY TAG:** Frontend / Performance

ANSWER: Reflow (layout): the browser recomputes the geometry of elements. Triggered by: DOM mutations, size/position changes (width, height, margin, padding, position, float, display), font changes, window resize, reading layout properties (`offsetWidth`, `scrollTop`, `getBoundingClientRect`). Expensive (re-computes the whole subtree, often the whole document).

Repaint: the browser redraws pixels without changing geometry. Triggered by: color, background, visibility, box-shadow. Cheaper than reflow but still costs.

Layout thrashing (forced synchronous layout): you read a layout property, then mutate the DOM, then read again. Each read after the mutation forces a reflow **synchronously** before the read can return, because the layout is dirty.

CODE/EXAMPLE SNIPPET:

```
// BAD - thrashes: read, write, read, write forces 3 sync reflows
for (const el of elements) {
  const w = el.offsetWidth; // read → layout
  el.style.width = w + 10 + 'px'; // write → invalidates layout
}

// GOOD - batch reads, then batch writes
const widths = elements.map(el => el.offsetWidth); // all reads first
elements.forEach((el, i) => el.style.width = widths[i] + 10 + 'px'); // then all writes
```

Mitigations: - Batch DOM reads, then writes (separate loops). - Use `requestAnimationFrame` to coalesce writes before the next paint. - Use the CSS `transform` and `opacity` for animations (compositor-only — no layout, no paint). - Avoid animating `top/left/width/height` (triggers layout). Use `transform: translate/scale`. - `will-change: transform` hints the browser to promote to its own layer. - `documentFragment` for batch inserts (one reflow instead of N). - `display: none` removes from layout tree — toggle it once instead of animating each property.

KEY TERMS TO MENTION: - Reflow: recompute geometry (size/position changes, reading layout props) - Repaint: redraw pixels (color/visibility changes) - Layout thrashing: read-write-read forces sync reflow each loop iteration - Fix: batch reads, then batch writes; use rAF - Animate transform/opacity (compositor layer) not top/left/width - will-change, documentFragment, display:none toggles

FOLLOW-UP QUESTIONS: 1. Why does transform not trigger reflow? 2. How do you measure layout thrashing in DevTools?

FOLLOW-UP ANSWERS: 1. `transform` and `opacity` are composited — the browser takes a “snapshot” of the element as a texture on the GPU and applies the transform there, without re-running layout. Layout still uses the pre-transform position; the compositor only moves the texture. This is why 60fps animations are possible — the main thread isn’t involved. The trade-off: GPU memory for each promoted layer, and `will-change` overuse causes memory pressure. 2. Chrome DevTools → Performance tab → Record → look for purple “Layout” events in the flame chart. If you see them interleaved with JS reads, you’re thrashing. Also “Recalculate Style” + “Layout” warnings in the “Rendering” panel (enable “Layout Shift Regions”). The Performance Insights panel flags “Forced reflow” warnings explicitly.

RED FLAGS TO AVOID: - Animating width/height/top/left (reflow) - Reading `offsetWidth` in a loop after writing (thrash) - `will-change` on every element (memory blow-up)

Q-FE-21: Critical rendering path — from HTML to pixels

DIFFICULTY: Hard **CATEGORY TAG:** Frontend / Performance

ANSWER: The browser turns HTML/CSS/JS into pixels via:

1. **DOM construction:** HTML → tokens → nodes → DOM tree.
2. **CSSOM construction:** CSS → CSSOM tree (cascaded rules). CSS is render-blocking (browser waits for all CSS before rendering).
3. **Render tree:** DOM + CSSOM → render tree (only visible nodes — `display: none` excluded; `head`, `script` excluded).
4. **Layout (reflow):** compute geometry (position, size) per render tree node.
5. **Paint:** fill pixels for each layer.
6. **Composite:** combine layers (transform, opacity) on the GPU.

Critical rendering path optimisations: - **Critical CSS:** inline the CSS needed for above-the-fold rendering in `<style>`; load the rest async (`<link rel="preload" as="style" onload="this.rel='stylesheet'">`). - **Defer non-critical JS:** `defer` (download in parallel, run after HTML parsed) or `async` (download in parallel, run as soon as ready). `defer` preserves order; `async` doesn't. - **Preload key resources:** `<link rel="preload" href="font.woff2" as="font">` for fonts, hero images, critical JS. - **Reduce render-blocking:** fewer CSS files, smaller CSS, no `@import` (serial fetches). - **Avoid layout-blocking JS:** JS that runs before paint can delay first paint.

Waterfall gotchas: - Render-blocking CSS → browser can't paint until CSS loads + parses. - JS that touches `document.body` blocks HTML parser (`<script>` without `defer` / `async`). - Fonts loading late cause FOIT (flash of invisible text) or FOUT (flash of unstyled text) — use `font-display: swap`.

KEY TERMS TO MENTION: - Steps: DOM → CSSOM → render tree → layout → paint → composite - CSS is render-blocking; JS is parser-blocking (unless defer/async) - Critical CSS inline + async rest; preload key resources - defer (ordered, after parse) vs async (unordered, ASAP) - font-display: swap to avoid FOIT

FOLLOW-UP QUESTIONS: 1. What's the difference between defer and async? 2. Why does CSS block rendering but not parsing?

FOLLOW-UP ANSWERS: 1. Both download in parallel with HTML parsing. `defer` waits until HTML parsing completes, then runs in order before DOMContentLoaded. `async` runs as soon as it's downloaded, pausing the parser, in any order. Use `defer` for scripts that depend on the DOM or each other; `async` for independent scripts (analytics, ads). 2. The browser parses HTML fine without CSS (it builds the DOM). But it can't paint because painting needs the CSSOM (to know which styles apply). Rendering without CSS would show unstyled content (FOUC) — browsers avoid this by waiting for CSS. JS can also block — if a `<script>` runs and

could query styles, the browser must have CSSOM ready, so JS after a `<link>` to CSS waits for the CSS too.

RED FLAGS TO AVOID: - “JS in

without defer/async” (blocks parsing) - Big render-blocking CSS bundle (slow first paint) - `@import` in CSS (serial fetch)

Q-FE-22: Core Web Vitals — LCP, FID/INP, CLS

DIFFICULTY: Medium **CATEGORY TAG:** Frontend / Web Performance

ANSWER: Google's user-experience metrics (ranking signal). Three core vitals + many 辅助:

LCP (Largest Contentful Paint): time until the largest above-the-fold element renders. Target < 2.5s (good), < 4s (needs improvement), worse = poor. Largest element is usually a hero image, big heading, or video poster. Improve: preload hero image, optimize image (AVIF/WebP, sized), inline critical CSS, reduce TTFB, remove render-blocking JS.

INP (Interaction to Next Paint): replaced FID in March 2024. Measures latency of ALL interactions (click, keypress, tap) — takes the worst (or near-worst) one. Target < 200ms (good), < 500ms (needs improvement). Improve: break long tasks (yield with `setTimeout/scheduler.yield`), debounce heavy handlers, use web workers for non-UI compute, avoid layout thrashing.

CLS (Cumulative Layout Shift): sum of unexpected layout shifts. Target < 0.1 (good), < 0.25 (needs improvement). Improve: set width/height on images and ads, reserve space for late-loading content, avoid inserting content above existing content, use `font-display: swap` + `size-adjust` for fonts, avoid animating layout properties.

Field vs lab data: - Lab (Lighthouse, WebPageTest): controlled, repeatable, good for catching regressions. - Field (CrUX, RUM): real users, real devices, real network. The ranking signal uses field data.

CODE/EXAMPLE SNIPPET:

```
<!-- Reserve image space to prevent CLS -->


<!-- Async font with fallback size to prevent layout shift -->
<link rel="preload" href="font.woff2" as="font" type="font/woff2" crossorigin />
<style>
  body { font-family: 'CustomFont', system-ui, sans-serif; }
  @font-face { font-family: 'CustomFont'; src: url('font.woff2') format('woff2');
    font-display: swap; size-adjust: 100%; }
</style>
```

Other vitals: TTFB (time to first byte, < 0.8s), FCP (first contentful paint, < 1.8s), TBT (total blocking time, < 200ms).

KEY TERMS TO MENTION: - LCP (Largest Contentful Paint): <2.5s, preload hero, optimize images - INP (replaced FID in 2024): <200ms, break long tasks, web workers - CLS (Cumulative Layout Shift): <0.1, reserve space, font-display swap - Lab (Lighthouse) vs field (CrUX, RUM) — field is the ranking signal - TTFB, FCP, TBT (auxiliary)

FOLLOW-UP QUESTIONS: 1. How is INP different from FID? 2. How do you measure Core Web Vitals in production?

FOLLOW-UP ANSWERS: 1. FID measured only the [first](#) interaction's input delay (not the full processing + presentation time). Many pages passed FID despite slow later interactions. INP measures [all](#) interactions through the full P99 (well, "worst of ~50" statistically), capturing ongoing sluggishness. INP is much harder to pass — sites that were "good" on FID may be "poor" on INP. 2. RUM (Real User Monitoring): ship a small script that uses the [web-vitals](#) library to capture LCP/INP/CLS from real sessions and beacon to your analytics (Google Analytics 4 events, Datadog RUM, SpeedCurve). Aggregate by device type, page, country. CrUX (Chrome UX Report) publishes field data for public sites — see your domain in PageSpeed Insights. Combine: lab for catching regressions in CI, RUM for actual user impact.

RED FLAGS TO AVOID: - "Lighthouse score is all that matters" (field data is the ranking signal) - Optimizing only for desktop (mobile is the majority + harder) - Forgetting CLS (the silent killer — users bounce on jumpy pages)

Q-FE-23: Code splitting and lazy loading

DIFFICULTY: Medium **CATEGORY TAG:** Frontend / Performance

ANSWER: Ship less JS on initial load. Code splitting breaks the bundle into chunks loaded on demand.

Route-level splitting: each route is a separate chunk. When the user navigates, that chunk loads. Standard in Next.js (file-system routing), React Router (lazy), Vue Router.

Component-level splitting: `React.lazy` + `Suspense` for below-the-fold components (charts, modals, heavy widgets).

CODE/EXAMPLE SNIPPET:

```
import { lazy, Suspense } from "react";
const Dashboard = lazy(() => import("./Dashboard"));
function App() {
  return <Suspense fallback=<Spinner />><Dashboard /></Suspense>;
}
```

Module-level splitting: dynamic `import()` anywhere — e.g. load a heavy library (PDF renderer, monaco editor) only when the user opens that feature.

Prefetching: `<link rel="prefetch" href="dashboard.js">` — load chunks likely to be needed next (idle-time fetch). Next.js does this automatically for linked routes. Use sparingly — wastes bandwidth if not used.

Bundle analysis: `webpack-bundle-analyzer`, `rollup-plugin-visualizer`, Next.js's `@next/bundle-analyzer`. Find the biggest chunks; tree-shake; replace heavy deps.

Common wins: - Don't ship moment.js (use date-fns/dayjs, or moment's locale subset). - Don't ship lodash fully (use lodash-es + tree-shaking, or per-method imports). - Use `/* webpackChunkName: "dashboard" */` for named chunks. - Server-side render the page shell, hydrate interactive parts progressively.

KEY TERMS TO MENTION: - Route-level splitting (Next.js, React Router lazy) - Component-level: `React.lazy` + `Suspense` for below-the-fold - Module-level: dynamic `import()` for heavy libs - Prefetching (idle-time, use sparingly) - Bundle analyzer to find heavy chunks - Replace heavy deps (moment → dayjs, full lodash → lodash-es)

FOLLOW-UP QUESTIONS: 1. What's the difference between prefetch and preload? 2. How do you measure bundle size regressions in CI?

FOLLOW-UP ANSWERS: 1. `preload` (high priority) — fetch a resource the current page will need soon (e.g. the critical font, hero image). `prefetch` (low priority) — fetch a resource a future navigation may need (e.g. the next route's chunk). `preconnect` — open the TCP/TLS connection early. `dns-prefetch` — DNS lookup only. Use preload for critical-path resources; prefetch for likely-next. 2. `size-limit` (npm) or `bundlesize` — set a budget (e.g. "main bundle < 200 KB"), fail CI if exceeded. Next.js's `@next/bundle-analyzer` produces a treemap

in CI. Track bundle size as a metric over time (a Grafana panel showing JS shipped per route). Set a budget slightly above current, fail when exceeded — forces a conversation before adding bloat.

RED FLAGS TO AVOID: - Shipping the whole app as one 5MB bundle (slow first load) - Lazy-loading above-the-fold content (worse LCP) - Not tree-shaking (full lodash ships 70KB+)

Q-FE-24: Service workers, PWA, and caching strategies

DIFFICULTY: Hard **CATEGORY TAG:** Frontend / PWA

ANSWER: A service worker is a JS worker that runs in the background, separate from the page. It intercepts network requests (via the Fetch event), enabling: offline support, push notifications, background sync, caching. PWA = installable + offline + push.

Lifecycle: register → install (precache assets) → activate (cleanup old caches) → fetch (intercept requests) → push/sync.

Caching strategies (Workbox makes these easy): - **Cache-first:** check cache, fall back to network. Use for static assets (fonts, images, versioned JS). - **Network-first:** try network, fall back to cache. Use for content that updates (API responses). - **Stale-while-revalidate:** return cache immediately, fetch network in background + update cache. Best for non-critical content (images, HTML). - **Network-only:** always network (e.g. analytics). - **Cache-only:** always cache (offline-only assets).

CODE/EXAMPLE SNIPPET:

```
// sw.js
self.addEventListener("install", e => {
  e.waitUntil(caches.open("v1").then(c => c.addAll(["/", "/app.js", "/styles.css"])));
});

self.addEventListener("fetch", e => {
  e.respondWith(
    caches.match(e.request).then(cached => {
      const fetchPromise = fetch(e.request).then(resp => {
        caches.open("v1").then(c => c.put(e.request, resp.clone()));
        return resp;
      });
      return cached || fetchPromise; // stale-while-revalidate
    })
  );
});
```

Gotchas: - Service worker survives page reloads — bugs persist. Use **Update on reload** in DevTools + `self.skipWaiting()` + `clients.claim()` for instant updates in dev. - Caching the wrong thing (e.g. API response that should be fresh) → users see stale data. Be explicit about which routes are cached. - Cache storage grows unbounded — implement eviction (delete old caches on activate). - HTTPS required (except localhost) — service worker is a powerful intercept point; only HTTPS is safe. - Don't cache POST / non-GET requests (caching is for GET).

Push notifications: server sends a push to a subscription endpoint (browser-specific). Service worker receives the **push** event even when the page is closed. VAPID keys for server auth.

KEY TERMS TO MENTION: - Service worker: background worker, intercepts fetch, enables offline + push - Lifecycle: install → activate → fetch - Strategies: cache-first, network-first, stale-while-revalidate, network-only - Workbox (Google) — handles edge cases - HTTPS required; cache storage eviction; don't cache POST - Push via VAPID

FOLLOW-UP QUESTIONS: 1. How do you version a service worker and migrate users? 2. How would you implement an offline-first data app?

FOLLOW-UP ANSWERS: 1. Each SW version is keyed by its script URL + content hash. When the script changes, the browser installs the new SW in parallel (waiting), activates only when no clients use the old one (or `self.skipWaiting()` forces it). On activate, delete old caches by version. Notify the page via `controllerchange` event → reload. The pattern: `update-then-reload` — show “New version available” toast; user accepts → `skipWaiting` + reload. 2. IndexedDB for structured offline storage (larger + more queryable than cache). Background Sync API to queue mutations when offline, replay when online. Conflict resolution strategy (last-write-wins for simple cases, vector clocks / server-wins for collaboration). PouchDB + CouchDB is a battle-tested offline-first stack (auto sync + conflict resolution). For a custom stack: queue mutations in IndexedDB, POST when online, merge server response. Show the user “saved locally, syncing...” indicators.

RED FLAGS TO AVOID: - Caching API responses with stale-while-revalidate (users see stale data) - Not handling SW versioning (users stuck on old code) - Expecting SW to work without HTTPS

Q-FE-25: Web workers vs service workers vs shared workers

DIFFICULTY: Medium **CATEGORY TAG:** Frontend / Browser

ANSWER: All three are JS workers — separate threads, no DOM access, communicate via `postMessage`.

Web Worker: general-purpose worker for CPU-heavy computations (parse, hash, ML inference). Spawned by a page, dies when the page closes. Use for: don't block the main thread.

Service Worker: a Web Worker with super powers — intercepts network, runs in background (even when no page is open), powers PWA features (offline, push, sync). Persistent (until evicted). One per origin.

Shared Worker: shared across multiple pages of the same origin. Useful for multi-tab apps that need shared state (e.g. one WebSocket connection across tabs). Limited browser support (no Safari until recently).

CODE/EXAMPLE SNIPPET:

```
// main.js
const w = new Worker("./heavy.js");
w.postMessage({ data: bigArray });
w.onmessage = e => render(e.data);

// heavy.js (worker)
self.onmessage = e => {
  const result = expensiveCompute(e.data.data);
  self.postMessage(result);
};
```

Constraints: - No DOM (workers can't access `document`, `window.alert`, etc.) - No `localStorage` (use `IndexedDB` — accessible from workers) - Limited transferable types — use `Transferable` (`ArrayBuffer`, `MessagePort`) for zero-copy transfer - Same-origin policy for the worker script

When to use which: - Heavy compute → Web Worker - Offline / push / cache → Service Worker - Shared state across tabs → Shared Worker (or `BroadcastChannel` for messaging-only)

KEY TERMS TO MENTION: - Web Worker: CPU-heavy compute, dies with page - Service Worker: intercepts network, persistent, PWA features - Shared Worker: shared across tabs of same origin - No DOM, no `localStorage` (`IndexedDB`), `postMessage`, `Transferable` (zero-copy) - When: compute → web, offline → service, cross-tab → shared/`BroadcastChannel`

FOLLOW-UP QUESTIONS: 1. How do you transfer large data to a worker without copying? 2. What's `BroadcastChannel` vs Shared Worker?

FOLLOW-UP ANSWERS: 1. Transferable objects: ArrayBuffer, MessagePort, ImageBitmap, OffscreenCanvas. `worker.postMessage(buf, [buf])` — the buffer is **transferred** (zero copy), the sender's reference is detached. Saves memory + time for large data (images, arrays). The catch: you can't use it on the sender side after transfer. Structured clone (the default) copies everything else. 2. BroadcastChannel is a simple pub/sub channel across same-origin tabs/workers — no shared state, just messaging. Shared Worker is a long-running shared computation/context. BroadcastChannel is simpler — use it if you just need to send "new data available" events to other tabs. Shared Worker if you need a central processor (e.g. one WebSocket mux). BroadcastChannel has broader support now.

RED FLAGS TO AVOID: - Doing heavy compute on the main thread (UI freezes) - Sending huge objects to workers via structured clone (copies — slow) - Forgetting workers can't touch the DOM

Q-FE-26: CORS and the same-origin policy

DIFFICULTY: Medium **CATEGORY TAG:** Frontend / Web Security

ANSWER: Same-Origin Policy (SOP): a script on origin A can't read responses from origin B (different scheme, host, or port). Prevents malicious sites from reading authenticated responses from your bank etc.

CORS (Cross-Origin Resource Sharing): the server's opt-in mechanism to relax SOP for specific origins. The server sends `Access-Control-Allow-Origin` headers; the browser enforces them.

Simple requests (GET, POST with simple content-types, no custom headers): browser sends the request, checks `Access-Control-Allow-Origin` on the response. If allowed, JS can read it. If not, blocked.

Preflighted requests (PUT/DELETE, custom headers, non-simple content-types): browser sends an OPTIONS preflight first with `Access-Control-Request-Method` / `Access-Control-Request-Headers`. Server responds with `Access-Control-Allow-Methods`, `Access-Control-Allow-Headers`, `Access-Control-Max-Age`. Then the real request is sent.

CODE/EXAMPLE SNIPPET (server side, Spring Boot):

```
@CrossOrigin(origins = "https://app.example.com",
             methods = {RequestMethod.GET, RequestMethod.POST},
             allowCredentials = "true",
             maxAge = 3600)
@RestController @RequestMapping("/api")
public class ApiController { /* ... */ }
// Or global config:
@Bean
WebMvcConfigurer corsConfigurer() {
    return new WebMvcConfigurer() {
        public void addCorsMappings(CorsRegistry r) {
            r.addMapping("/api/**").allowedOrigins("https://app.example.com")
              .allowedMethods("*").allowCredentials(true).maxAge(3600);
        }
    };
}
```

Credentialed requests (cookies, Authorization): browser sends them only if `credentials: 'include'` on the fetch AND the server returns `Access-Control-Allow-Credentials: true`. Critically: when credentials are allowed, `Access-Control-Allow-Origin` cannot be `*` — must be the specific origin.

Common mistakes: - Setting `Access-Control-Allow-Origin: *` with credentials (browser blocks — must echo the origin) - Not handling OPTIONS preflight (server returns 405) - CORS is NOT a security mechanism for the server (it's enforced by the browser; a non-browser can

ignore it). Server-side authz is separate. - CORS only blocks **reading** the response, not the request being sent. CSRF attacks still work — protect with CSRF tokens.

KEY TERMS TO MENTION: - SOP: script can't read cross-origin responses - CORS: server opt-in via Access-Control-Allow-* headers, browser enforces - Simple vs preflighted (OPTIONS) requests - Credentials: allowCredentials=true, ACAO must be specific origin (not *) - CORS is browser-only — not server security; use server-side authz + CSRF tokens

FOLLOW-UP QUESTIONS: 1. Why can't you set `Access-Control-Allow-Origin: *` with credentials? 2. How does CORS relate to CSRF?

FOLLOW-UP ANSWERS: 1. Security: `*` matches any origin, so any site could make credentialed requests with the user's cookies. The spec forbids the combination — the browser blocks the response. To allow credentials, the server must echo the specific origin (e.g. `Access-Control-Allow-Origin: https://app.example.com`). This ensures only trusted origins can read credentialed responses. 2. They're complementary, not overlapping. CORS controls **reading** cross-origin responses (browser blocks reading without ACAO). CSRF is about **causing** the browser to send authenticated requests (e.g. a form POST that includes the user's cookie). CORS doesn't prevent CSRF — even a CORS-blocked request is still **sent** (just the response can't be read). Defend CSRF with: SameSite cookies, CSRF tokens, custom headers (which trigger CORS preflight, raising the bar).

RED FLAGS TO AVOID: - "CORS protects the server" (it doesn't — browsers enforce it) - `Access-Control-Allow-Origin: *` with `allowCredentials: true` - Not handling OPTIONS (preflight returns 405) - Treating CORS as CSRF protection

Q-FE-27: XSS prevention — CSP, sanitization, and Trusted Types

DIFFICULTY: Hard **CATEGORY TAG:** Frontend / Security

ANSWER: XSS (Cross-Site Scripting): attacker injects script that runs in the victim's browser, in the context of the trusted site. Three forms: - **Reflected:** payload in URL, server echoes it back (search?q=

```
```js
// Sanitize user HTML with DOMPurify
import DOMPurify from "dompurify";
element.innerHTML = DOMPurify.sanitize(userHtml, { ALLOWED_TAGS: ['b','i','a','p'],
ALLOWED_ATTR: ['href'] });
```

**Common bypasses the interviewer probes:** - `<img src=x onerror=alert(1)>` — event handlers are XSS vectors; CSP `script-src` blocks inline handlers unless `unsafe-inline` or hash. - `<a href="javascript:alert(1)">` — sanitize `href` to allowlisted protocols only. - SVG `<script>` tags — sanitize SVG too. - JSONP endpoints as script source — remove JSONP if you don't need it.

**KEY TERMS TO MENTION:** - XSS: reflected, stored, DOM-based - Output encoding (React does this by default; don't bypass) - CSP: default-src, script-src, nonces, 'strict-dynamic', report-only - Sanitize rich text with DOMPurify, allowlist tags/attrs, never regex - HttpOnly/Secure/SameSite cookies - Trusted Types (forces sanitisation at the sink)

**FOLLOW-UP QUESTIONS:** 1. What's the difference between `unsafe-inline` and a nonce in CSP? 2. How does Trusted Types prevent DOM XSS?

**FOLLOW-UP ANSWERS:** 1. `unsafe-inline` allows any inline `<script>` — basically defeats CSP for XSS (still blocks external scripts). A `nonce-$random` (per-request, unguessable) allows only inline scripts with that exact nonce — attacker can't inject a script with the right nonce (they don't know it). So nonces give you the convenience of inline scripts without the XSS hole. `'strict-dynamic'` goes further: scripts loaded by a trusted (nonce'd) loader can load other scripts — no need to enumerate every CDN. 2. By default, `element.innerHTML = "<script>..."` works — the string is parsed as HTML. Trusted Types makes the sink accept only `TrustedHTML`, not string. You create `TrustedHTML` via a policy you define

(`trustedTypes.createPolicy('sanitize', { createHTML: s => DOMPurify.sanitize(s) })`). Now any attempt to assign a raw string to `innerHTML` throws — the developer MUST go through your policy. Eliminates the entire class of "I forgot to sanitize this one sink" bugs.

**RED FLAGS TO AVOID:** - "I escape on input" (you must escape on output — context-dependent) - `dangerouslySetInnerHTML` without sanitisation - `unsafe-inline` in CSP (kills CSP) - Regex-based HTML sanitisation

## Q-FE-28: State management — Zustand, Redux Toolkit, Jotai, TanStack Query

**DIFFICULTY:** Medium **CATEGORY TAG:** Frontend / State

**ANSWER:** Four different paradigms — pick per use case.

**Zustand:** tiny (~1KB), hook-based, no boilerplate. Best for: app-wide UI state, small to medium apps. No providers, no selectors required (component subscribes via `useStore(s => s.value)`). Recoil-like “atomic” subscriptions.

**Redux Toolkit (RTK):** opinionated Redux. `createSlice` (Immer), `createAsyncThunk` (async), RTK Query (data). Best for: large apps, time-travel debugging, strict action log, many consumers of slices. Selector subscriptions (only re-renders on slice change).

**Jotai:** atomic state (smallest unit). Each atom is a piece of state; components subscribe to atoms. Best for: fine-grained dependency graphs, derived state, when you don’t want a single store. No boilerplate, scales to very large apps.

**TanStack Query (React Query):** server state — caching, invalidation, retries, optimistic updates, background refetch. Not a replacement for client state — complementary. Best for: any data that comes from an API. Most CRUD apps need [only](#) TanStack Query + a small client-state lib.

**Decision matrix:** | Need | Use | |—|—| | Server data (API) | TanStack Query (or RTK Query) | | Form state | React Hook Form / controlled state | | Global UI state (theme, sidebar) | Zustand / Context | | Complex client state (offline editor, cart with rules) | Redux Toolkit / Zustand | | Fine-grained atomic + derived | Jotai | | Large team, time-travel | Redux Toolkit |

**CODE/EXAMPLE SNIPPET (Zustand):**

```
import { create } from "zustand";
const useCart = create((set) => ({
 items: [],
 add: (item) => set((s) => ({ items: [...s.items, item] })),
 total: (s) => s.items.reduce((n, i) => n + i.price, 0),
}));
function Cart() {
 const items = useCart((s) => s.items); // subscribes only to items slice
 const add = useCart((s) => s.add);
 return <button onClick={() => add({ id: 1, price: 9.99 })}>Add</button>;
}
```

**KEY TERMS TO MENTION:** - Zustand: tiny, hook-based, atomic subscriptions, no providers - Redux Toolkit: `createSlice` (Immer), `createAsyncThunk`, RTK Query, time-travel - Jotai: atomic, fine-grained, derived state - TanStack Query: server state — caching, invalidation, retries, optimistic - Most CRUD apps: TanStack Query + small client-state lib

**FOLLOW-UP QUESTIONS:** 1. When does TanStack Query invalidate a query? 2. Why is Jotai more granular than Redux?

**FOLLOW-UP ANSWERS:** 1. Several ways: (a) explicit `queryClient.invalidateQueries({ queryKey: ['todos'] })` after a mutation; (b) on focus (`refetchOnWindowFocus`); (c) on reconnect; (d) on mount (stale data shown + refetched in background if `staleTime` passed); (e) interval (`refetchInterval`). You control staleness via `staleTime` (default 0 = always stale); `cacheTime` is how long inactive queries stay in memory. 2. Redux stores everything in one tree; every selector re-renders when its slice changes. Jotai atoms are independent — a component subscribing to `atomA` doesn't re-render when `atomB` changes. Atoms can derive from other atoms (`derivedAtom = atom(get => get(a) + get(b))`) and only re-compute when their deps change. The granularity is per-atom, not per-slice — finer subscriptions, less re-rendering for sparsely-coupled state.

**RED FLAGS TO AVOID:** - “Use Redux for everything” (overkill) - Storing server state in Redux manually (use TanStack Query) - One giant Context for all state (re-render storm)

---

## Q-FE-29: TanStack Query (React Query) — caching, invalidation, optimistic updates

**DIFFICULTY:** Medium **CATEGORY TAG:** Frontend / Data Fetching

**ANSWER:** TanStack Query is the de-facto standard for server state in React apps. Handles caching, refetching, invalidation, retries, pagination, optimistic mutations.

**Core concepts:** - **Query** — `useQuery({ queryKey, queryFn, staleTime, gcTime })`. Cached by `queryKey` (array — use `['todos', userId]` for scoped queries). - **Mutation** — `useMutation({ mutationFn, onSuccess, onMutate, onError })`. Doesn't cache by default. - **QueryClient** — singleton cache; `invalidateQueries` triggers refetch of matching queries. - **Devtools** — visual cache inspector.

**Stale vs cached:** - `staleTime` (default 0): how long data is considered fresh (no refetch on mount/focus). Set higher for slow-changing data. - `gcTime` (formerly `cacheTime`, default 5min): how long an `inactive` query stays in memory before GC.

**Optimistic updates:**

```
const m = useMutation({
 mutationFn: (newTodo) => api.post('/todos', newTodo),
 onMutate: async (newTodo) => {
 await qc.cancelQueries({ queryKey: ['todos'] });
 const prev = qc.getQueryData(['todos']);
 qc.setQueryData(['todos'], (old) => [...old, newTodo]); // optimistic
 return { prev }; // rollback context
 },
 onError: (_e, _v, ctx) => qc.setQueryData(['todos'], ctx.prev), // rollback
 onSuccess: () => qc.invalidateQueries({ queryKey: ['todos'] }), // refetch real
});
```

**Pagination / infinite scroll:** `useInfiniteQuery` with `getNextPageParam`. Cursor- or offset-based.

**Server-side rendering:** prefetch + dehydrate on the server, hydrate on the client (`HydrationBoundary`). Next.js App Router: `prefetchQuery` in a server component.

**KEY TERMS TO MENTION:** - `useQuery` (`queryKey/queryFn`), `useMutation`, `QueryClient` - `staleTime` (fresh window) vs `gcTime` (inactive cache lifetime) - `invalidateQueries` triggers refetch - Optimistic updates via `onMutate` (set cache) + `onError` (rollback) + `onSuccess` (refetch) - `useInfiniteQuery` for paging; SSR via prefetch + dehydrate/hydrate

**FOLLOW-UP QUESTIONS:** 1. How do you avoid race conditions when multiple components fetch the same query? 2. How do you handle a mutation that updates a paginated list?

**FOLLOW-UP ANSWERS:** 1. TanStack Query dedupes by `queryKey` — multiple components using `useQuery({ queryKey: ['todos'] })` share one in-flight request. If a refetch starts while another is in flight, the later wins (`useQuery` returns the latest). For mutations across

components, serialise via the QueryClient (single mutation at a time) or use the mutation's `isPending` to disable the button. 2. Two approaches: (a) invalidate the whole paginated query ( `invalidateQueries({ queryKey: ['todos'] })` ) — refetches all visible pages; simple but may refetch unnecessarily. (b) `setQueryData` to manually patch the affected page (faster, more control). For infinite scroll, the cache is `{ pages: [...], pageParams: [...] }` — you can update specific pages. Optimistic on page 0 + invalidate is the common pattern.

**RED FLAGS TO AVOID:** - `staleTime: 0` everywhere (always refetches — wastes bandwidth)  
- Not invalidating after mutations (stale data) - Replacing TanStack Query with `useEffect+useState+fetch` (you're rebuilding it badly)

-----

## Q-FE-30: Accessibility (a11y) — ARIA, semantic HTML, and keyboard nav

**DIFFICULTY:** Medium **CATEGORY TAG:** Frontend / Accessibility

**ANSWER:** Accessibility = making your app usable by people with disabilities (visual, motor, cognitive). It's a legal requirement (ADA, EU EAA) and a quality signal.

**Semantic HTML first:** use `<button>`, `<nav>`, `<main>`, `<article>`, `<header>`, `<footer>`, `<form>`, `<label>`, `<fieldset>` — they carry meaning and get keyboard + screen-reader behaviour for free. A `<div onClick>` is not a button — it's not keyboard-focusable, not announced as a button, not in the tab order.

**ARIA (Accessible Rich Internet Applications):** attributes that fill gaps semantic HTML doesn't cover. Use sparingly — first rule of ARIA: "Don't use ARIA." A `<button>` is better than `<div role="button">`.

Common ARIA: - `aria-label` / `aria-labelledby` — accessible name for icon-only buttons - `aria-hidden="true"` — hide decorative elements from screen readers - `role="dialog"`, `aria-modal="true"` — modal dialogues - `aria-expanded`, `aria-controls` — collapsible UI - `aria-live="polite"` / `"assertive"` — announce dynamic updates (toasts, errors)

**Keyboard navigation:** - All interactive elements must be reachable via Tab - Focus order matches DOM order (CSS order doesn't matter — DOM does) - Visible focus indicator (don't remove `:focus` outline; style it) - Modal: trap focus inside; restore focus on close - Keyboard shortcuts: don't conflict with browser/AT shortcuts

**WCAG (Web Content Accessibility Guidelines):** four principles — Perceivable, Operable, Understandable, Robust (POUR). Levels A, AA (legal baseline), AAA (ideal). Most laws target AA.

**Testing:** - Automated: axe-core (axe DevTools, `@axe-core/playwright`), Lighthouse a11y audit - Manual: keyboard-only test (unplug mouse), screen reader (NVDA+Firefox, VoiceOver+Safari, TalkBack) - Contrast check: WCAG AA requires 4.5:1 for normal text, 3:1 for large text

**CODE/EXAMPLE SNIPPET:**

```
// Good: semantic, labelled, keyboard-accessible
<button aria-label="Close dialog" onClick={close}>
 <XIcon aria-hidden="true" />
</button>

// Good: live region for toasts
<div role="status" aria-live="polite">{toastMessage}</div>

// Good: form with label
<label htmlFor="email">Email</label>
<input id="email" type="email" required aria-describedby="email-hint" />
<small id="email-hint">We'll never share your email.</small>
```

**KEY TERMS TO MENTION:** - Semantic HTML first (button, nav, main, label) — keyboard + AT for free - ARIA fills gaps; first rule: don't use ARIA when HTML works - aria-label, aria-hidden, role=dialog, aria-expanded, aria-live - Keyboard: Tab order, visible focus, focus trap in modals - WCAG POUR, AA is legal baseline - Testing: axe-core (auto), keyboard-only, screen reader, contrast

**FOLLOW-UP QUESTIONS:** 1. How do you make a modal accessible? 2. What's the difference between aria-label and aria-labelledby?

**FOLLOW-UP ANSWERS:** 1. Trap focus inside (Tab cycles within the modal); restore focus to the trigger on close; ESC closes; `role="dialog"` `aria-modal="true"` `aria-labelledby="<title-id>"`; prevent body scroll; click on backdrop dismisses (optional). Use `focus-trap-react` or Radix UI's Dialog which handles all this. Don't forget the focus restoration — users land where they were. 2. `aria-label="Close"` sets the accessible name directly (for icon-only buttons). `aria-labelledby="title-id"` references another element's text content as the name (use when the visible text already labels it — DRY). Use `aria-label` when there's no visible text; `aria-labelledby` when there is. For a "Save" button with visible text "Save", neither is needed — the visible text is the label. For an `<button><XIcon/></button>`, `aria-label="Close"` is needed.

**RED FLAGS TO AVOID:** - `<div onClick>` instead of `<button>` - Removing focus outline globally (keyboard users lost) - `alt=""` missing on images (decorative = `alt=""`, meaningful = real text) - Skipping manual keyboard + screen-reader tests

---

## Q-FE-31: Build tools — Vite, Webpack, Turbopack, esbuild

**DIFFICULTY:** Medium **CATEGORY TAG:** Frontend / Build

**ANSWER:** The JS build tool landscape has evolved fast.

**Webpack (2012+):** the incumbent. Module bundler — entry → graph of dependencies → bundle. Plugins for everything (CSS, images, fonts, code splitting, HMR). Powerful but slow dev cold starts (esp. large apps), complex config.

**esbuild (2020+):** Go-based JS bundler. 10-100x faster than Webpack/Rollup (parallel + native). Limited plugin API; not a full dev server. Used as the underlying minifier by Vite, and as a transformer.

**Vite (2020+):** Evan You's. Dev: uses native ESM — the browser loads modules on demand (no bundling), esbuild transforms each file. Fast dev cold start regardless of project size. Prod: bundles with Rollup (tree-shaking, code splitting). Default for new SPAs and Next's competitor framework ecosystem. ~1s dev startup for 1000-module apps.

**Turbopack (2022+):** Vercel/Next.js's Rust-based incremental bundler. Targets Next.js. Replaces Webpack for Next's dev server (faster HMR + cold start). Production bundling is rolling out.

**Rollup (2014+):** library-focused bundler. Excellent tree-shaking (used by Vite for prod, and by most npm libraries). Not great for apps (no dev server, slow at scale).

**Parcel (2017+):** zero-config bundler. Good DX, decent perf. Niche now vs Vite.

**SWC (Rust):** super-fast JS/TS compiler (replaces Babel). Used by Next.js, Vite (via plugin), Deno, Turbopack.

**Choosing today (2026):** - New SPA → Vite + React/Vue/Svelte - Next.js → Turbopack (or Webpack fallback) - Library → Rollup (or tsup which wraps esbuild) - Existing Webpack app → migrate to Vite gradually (or stay — Webpack is fine, just slower dev) - Monorepo → Turborepo / Nx for task running + caching

**KEY TERMS TO MENTION:** - Webpack: incumbent, powerful, slow cold start - esbuild: Go, 10-100x faster, limited plugin API - Vite: native ESM dev (no bundling), Rollup for prod, default for new SPAs - Turbopack: Rust, Next.js, faster HMR - Rollup: library bundler, excellent tree-shaking - SWC: Rust JS/TS compiler (replaces Babel) - 2026 default: Vite for SPA, Turbopack for Next, Rollup/tsup for libs

**FOLLOW-UP QUESTIONS:** 1. Why is Vite's dev server so fast? 2. What's tree shaking and what breaks it?

**FOLLOW-UP ANSWERS:** 1. Dev uses native ESM — the browser loads modules via `<script type="module">`, requesting each file as needed. Vite only transforms the requested file (esbuild, fast). No bundling, no waiting for the whole graph to build. Cold start is O(1) regardless of project size. HMR is per-module — only the changed module re-loads. Trade-off:

many small HTTP requests in dev (fine on localhost, less fine in some proxies); Vite's dep optimizer pre-bundles deps with esbuild to reduce request count. 2. Tree shaking = removing unused exports. Works with ESM static imports (the bundler can analyse what's used). Breaks with: (a) CommonJS `require()` (dynamic, can't analyse); (b) side-effectful modules (a module that runs code on import — bundler must keep it); (c) dynamic imports with computed paths; (d) re-exporting everything ( `export * from` ) without `"sideEffects": false` in package.json. Rollup's `--treeshake` flag + package.json `"sideEffects"` field helps. Check with bundle analyzer — if a heavy dep shows up unused, tree-shaking failed.

**RED FLAGS TO AVOID:** - Starting a new SPA with Webpack (Vite is the default now) - Bundling everything in dev (slow HMR) - Heavy deps showing up in bundle when unused (tree-shaking broken)

---

## Q-FE-32: ESM vs CommonJS — the module system split

**DIFFICULTY:** Medium **CATEGORY TAG:** Frontend / JavaScript

**ANSWER: CommonJS (CJS, 2009):** Node's original module system. `require()` / `module.exports`. Synchronous, dynamic. Allows conditional requires (`if (cond) require('x')`). Default in Node before ESM.

**ES Modules (ESM, 2015 spec, 2021 Node stable):** JS standard. `import` / `export`. Static (must be at top level), enables tree-shaking + static analysis. Async loading supported. The future.

**Key differences:** - ESM imports are hoisted + live bindings (changing the exported value reflects in importers). CJS `require()` returns a snapshot. - ESM is async (top-level `await` supported). CJS is sync. - ESM `'use strict'` by default. CJS is sloppy by default. - ESM `this` at top level is `undefined`. CJS `this` is `module.exports`. - ESM doesn't have `__dirname` / `__filename` (use `import.meta.url` + `fileURLToPath`).

**Node dual-mode:** a package can ship both (with `"type": "module"` in `package.json` defaulting to ESM, or `.mjs` / `.cjs` extensions to override). Interop: `import` from CJS works (default export = `module.exports`); `require()` of ESM is **not** allowed (use dynamic `import()`).

### CODE/EXAMPLE SNIPPET:

```
// ESM (math.mjs)
export const add = (a, b) => a + b;
export default { add };

// CJS (math.cjs)
module.exports = { add: (a, b) => a + b };

// Interop: import CJS from ESM
import pkg from "cjs-package"; // pkg = module.exports
import { thing } from "cjs-package"; // named – works if statically analyzable

// ESM-only features
import config from "./config.json" assert { type: "json" }; // JSON import
await db.connect(); // top-level await
```

**Gotchas:** - `"main"` in `package.json` is CJS; `"exports"` (modern) handles both + subpath imports. - Many old npm packages are CJS-only — using them from ESM is painful (named imports may not work if the package uses dynamic exports). - `__dirname` replacement:

```
import { fileURLToPath } from "url";
import { dirname } from "path";
const __filename = fileURLToPath(import.meta.url);
const __dirname = dirname(__filename);
```

**KEY TERMS TO MENTION:** - CJS: require/module.exports, sync, dynamic, snapshot imports  
 - ESM: import/export, static, async, live bindings, top-level await - Node dual-mode: "type": "module", .mjs/.cjs, "exports" field - require() of ESM fails (use dynamic import()) - \_\_dirname → import.meta.url + fileURLToPath

**FOLLOW-UP QUESTIONS:** 1. Why can't you `require()` an ESM module? 2. How do you make a library that works for both CJS and ESM consumers?

**FOLLOW-UP ANSWERS:** 1. ESM is async (top-level await, asynchronous dependency loading). CJS `require()` is synchronous by design — it returns immediately. You can't synchronously return the result of an async operation. So `require()` of ESM throws. The workaround is dynamic `import()` (async, returns a promise). Node may add sync `require(esm)` for modules without top-level await in the future — still experimental. 2. Ship both: build ESM to `dist/esm` + CJS to `dist/cjs`, set "exports" to map:

```
{
 "main": "./dist/cjs/index.js",
 "module": "./dist/esm/index.js",
 "exports": {
 "import": "./dist/esm/index.js",
 "require": "./dist/cjs/index.js"
 }
}
```

Tools like `tsup`, `microbundle`, `unbuild` handle this. Or use a single ESM build + tell CJS consumers to use dynamic `import()` (modern approach, drops CJS support).

**RED FLAGS TO AVOID:** - "Use CJS, it's simpler" (ESM is the future — Node 22+ defaults shift) - Mixed `.js` files with both `import` and `require` (will throw without "type": "module") - Named imports from a CJS package that uses `Object.defineProperty` getters (works only if statically analyzable)

---

## Q-FE-33: HTTP/2, HTTP/3, and caching headers

**DIFFICULTY:** Medium **CATEGORY TAG:** Frontend / HTTP

**ANSWER: HTTP/1.1 (1997):** text protocol, one request per TCP connection (or pipelining, rarely used). Head-of-line blocking. Workarounds: domain sharding, concatenation, spriting. Still widely supported.

**HTTP/2 (2015):** binary, multiplexed (multiple requests over one connection), header compression (HPACK), server push (deprecated in practice). Solves head-of-line blocking at the HTTP layer. Same origin only. No more need for sharding/concatenation — multiplexing handles it.

**HTTP/3 (2022):** over QUIC (UDP-based). No TCP head-of-line blocking (a lost packet blocks only its stream, not the connection). Faster connection setup (TLS 1.3 merged with QUIC handshake). Better on lossy networks (mobile). ~30% of web traffic today.

**Caching headers:** - **Cache-Control** (HTTP/1.1, modern): **max-age=31536000** (1 year), **public** / **private**, **no-cache** (must revalidate — counterintuitive!), **no-store** (never cache), **must-revalidate**, **stale-while-revalidate**, **immutable**. - **Expires** (HTTP/1.0, legacy): absolute date. Use Cache-Control instead. - **ETag** + **If-None-Match**: content hash; server returns 304 if unchanged. - **Last-Modified** + **If-Modified-Since**: timestamp-based 304.

**The fingerprinted-asset pattern:** - Hash-named assets (**app.abc123.js**) → **Cache-Control: public, max-age=31536000, immutable** — cache forever; URL changes when content does. - **index.html** → **Cache-Control: no-cache** (must revalidate) — always fetches latest, which references the latest hashed assets. - HTML revalidate + assets immutable = perfect cache + instant updates.

**CODE/EXAMPLE SNIPPET (Spring Boot):**

```
@GetMapping(value = "/app.js", produces = "application/javascript")
public ResponseEntity<String> app() {
 return ResponseEntity.ok()
 .cacheControl(CacheControl.maxAge(365, TimeUnit.DAYS).cachePublic().immutable())
 .eTag("\"v1.2.3\"")
 .body(js);
}
// For HTML — must revalidate
@GetMapping("/")
public ResponseEntity<String> html() {
 return
 ResponseEntity.ok().cacheControl(CacheControl.noCache()).eTag(htmlHash).body(html);
}
```

**Vary header:** **Vary: Accept-Encoding** — tells caches to key on encoding (gzip vs br vs identity). Without it, a br-capable client might get the gzip version from cache.

**KEY TERMS TO MENTION:** - HTTP/1.1: head-of-line blocking, domain sharding - HTTP/2: multiplexed, HPACK headers, server push (deprecated) - HTTP/3: QUIC/UDP, no TCP HOL blocking, faster handshake, better mobile - Cache-Control (modern) vs Expires (legacy) - no-cache = must revalidate (counterintuitive); no-store = never - Fingerprinted assets: max-age=1year+immutable; HTML: no-cache - ETag + 304; Vary: Accept-Encoding

**FOLLOW-UP QUESTIONS:** 1. What does `immutable` do in Cache-Control? 2. How do you bust a cache when you can't rename files?

**FOLLOW-UP ANSWERS:** 1. `immutable` tells the browser the resource will `never` change — skip revalidation entirely, even on refresh. Without it, a refresh sends `If-None-Match / If-Modified-Since` (a network round trip). With it, no request at all. Only safe for fingerprinted assets (URL changes on content change). Don't use on `index.html` or any URL that might update without a name change. 2. You can't fully — that's the point of caching. Workarounds: (a) cache-busting query string ( `app.js?v=2` ) — works but proxies sometimes ignore query strings; (b) shorter `max-age` (e.g. 1 hour) so it re-validates often; (c) `must-revalidate` + `proxy-revalidate` so stale is never served; (d) `Service-Worker` to control caching in code (you can purge by URL). Best practice: build with content hashes so you never need to bust.

**RED FLAGS TO AVOID:** - `Cache-Control: no-cache` thinking it means “don't cache” (it means revalidate) - Long max-age on `index.html` (users stuck on old app) - Forgetting `Vary: Accept-Encoding` (br clients get gzip)

---

## Q-FE-34: IntersectionObserver, ResizeObserver, MutationObserver

**DIFFICULTY:** Easy **CATEGORY TAG:** Frontend / Browser APIs

**ANSWER:** Three observer APIs let you react to changes without polling (no more `setInterval` + `getBoundingClientRect`).

**IntersectionObserver:** fires when an element enters/leaves a viewport (or another root). Use for: lazy loading images, infinite scroll, “active section” nav highlighting, autoplay video on view, analytics (“element seen”).

```
const io = new IntersectionObserver((entries) => {
 entries.forEach(e => { if (e.isIntersecting) e.target.play(); else
 e.target.pause(); });
}, { rootMargin: "100px", threshold: 0.5 });
document.querySelectorAll("video").forEach(v => io.observe(v));
// Disconnect when done: io.disconnect();
```

**ResizeObserver:** fires when an element’s size changes. Use for: responsive components (render different layout based on width), chart re-renders, dynamic font sizing. Better than `window.resize` (works for any element, not just window).

```
const ro = new ResizeObserver(entries => {
 for (const e of entries) render(e.target, e.contentRect.width);
});
ro.observe(document.querySelector(".chart"));
```

**MutationObserver:** fires when the DOM changes (child added/removed, attribute changed, text content). Use for: third-party widget integration (react to injected content), virtual DOM diffing, autosizing textareas (when content is added programmatically).

```
const mo = new MutationObserver(muts => {
 for (const m of muts) console.log(m.type, m.addedNodes);
});
mo.observe(document.body, { childList: true, subtree: true, attributes: true });
```

**Gotchas:** - Each observer callback runs as a microtask batch (after DOM change, before paint) — efficient. - Disconnect when done (memory leak otherwise — observer holds the target). - Avoid triggering mutations inside a MutationObserver callback (infinite loop). - ResizeObserver loops (“ResizeObserver loop limit exceeded”) — happens when your callback resizes the element it’s observing; usually harmless (browser cancels the recursion), but log it.

**KEY TERMS TO MENTION:** - IntersectionObserver: enter/leave viewport (lazy load, infinite scroll) - ResizeObserver: element size change (responsive, charts) - MutationObserver: DOM

changes (childList/attributes/subtree) - Callbacks are microtask batches (efficient) - Disconnect to avoid leaks; avoid mutation loops

**FOLLOW-UP QUESTIONS:** 1. How is IntersectionObserver better than scroll listeners? 2. What's the ResizeObserver loop error?

**FOLLOW-UP ANSWERS:** 1. Scroll listeners fire on every scroll event (potentially 60/s), each triggering `getBoundingClientRect` (forced reflow). IntersectionObserver is event-driven — the browser only calls you when the intersection actually changes, and batches notifications. Less CPU, less reflow, less battery. You also get `isIntersecting` + the intersection ratio for free. 2. If your ResizeObserver callback modifies the observed element's size, the browser fires again next frame, infinitely. Browsers detect this and break the loop after a few iterations, logging "ResizeObserver loop limit exceeded" (a non-blocking warning). Fix: don't resize the observed element in the callback (use `requestAnimationFrame` to defer, or observe a different element). The error is harmless for users but noisy in logs.

**RED FLAGS TO AVOID:** - Polling with `setInterval` + `getBoundingClientRect` (use IntersectionObserver) - Forgetting to disconnect observers (leak) - Triggering mutations inside MutationObserver (infinite loop)

---

## Q-FE-35: Micro-frontends and module federation

**DIFFICULTY:** Hard **CATEGORY TAG:** Frontend / Architecture

**ANSWER: Micro-frontends:** split a monolithic frontend into independently deployable pieces, each owned by a team. Each piece can use a different framework (or different version). Trade-offs: more infrastructure, potential UI inconsistency, harder end-to-end testing.

**Integration patterns:** 1. **Build-time composition** (single-spa, Webpack Module Federation): one host loads remote modules. Shared deps (React) can be deduplicated. 2. **Server-side composition** (Next.js multi-zone, Tailor, Pirai): the server stitches HTML from multiple apps. 3. **Run-time iframe / Web Components:** each MFE is a custom element or iframe; host composes them. Strongest isolation, weakest UX (iframes: slow, no shared state).

**Webpack Module Federation (2020+):** a host app imports a “remote” app’s exposed modules at runtime. Shared dependencies (React) are deduplicated (only one React instance). Each team deploys their remote independently.

**CODE/EXAMPLE SNIPPET:**

```
// Remote (Team B's webpack config)
new ModuleFederationPlugin({
 name: "dashboard",
 filename: "remoteEntry.js",
 exposes: { ". /Widget": ". /src/Widget" },
 shared: { react: { singleton: true, requiredVersion: "^18" }, "react-dom": {
 singleton: true } },
});

// Host (Team A)
const Widget = React.lazy(() => import("dashboard/Widget"));
<Suspense fallback=<Spinner/>><Widget/></Suspense>;
```

**When to use micro-frontends:** - Multiple teams (>5) owning distinct parts of a large app - Need to deploy independently (different release cadences) - Acquisition: integrating a purchased product’s UI - Migration: incrementally rewrite a legacy app

**When NOT to:** - Small team (one team can own the whole frontend) - Strong design system needs (consistency harder across MFEs) - Need shared state (cross-MFE state is painful — use events, query cache, or shared store with namespacing)

**Trade-offs:** - Independent deploys ↔ potential version drift (React 18 + React 19 across MFEs) - Tech flexibility ↔ inconsistent UX - Team autonomy ↔ harder end-to-end tests + cross-MFE features - Bundle size: shared deps + remote entries add overhead

**Real-world:** Spotify (home page composed of MFEs), IKEA, Atlassian (many MFEs), Tesla’s internal tools.

**KEY TERMS TO MENTION:** - Micro-frontends: independently deployable UI pieces, multi-team - Patterns: build-time (single-spa, Module Federation), server-side (Next multi-zone), run-time (iframe, Web Components) - Module Federation: host imports remote exposes, shared deps deduped (singleton React) - Use when: many teams, independent deploys, acquisitions, legacy migration - Don't use when: small team, strong design system, shared state - Trade-offs: version drift, UX inconsistency, harder E2E

**FOLLOW-UP QUESTIONS:** 1. How do you share state across micro-frontends? 2. How do you prevent two React instances from conflicting?

**FOLLOW-UP ANSWERS:** 1. Options: (a) Custom Events / pub-sub bus — each MFE publishes/consumes events; (b) shared query cache (TanStack Query with a global QueryClient + namespaced keys); (c) URL as state (router params shared); (d) a thin global store (Zustand) for cross-cutting UI state (theme, current user); (e) server-side state via API + each MFE re-fetches. Avoid heavy shared state — it couples MFEs (defeats the point). Prefer event-driven for actions, URL for navigation, API for shared data. 2. Module Federation's `shared: { react: { singleton: true, requiredVersion: "^18" } }` ensures only one React loads — the first to register wins; later ones use the registered instance if version-compatible. If incompatible, the build fails (forces alignment). Without singleton: each MFE loads its own React → context providers don't share, hooks throw ("invalid hook call"), events don't match. If you can't use MF, load each MFE in an iframe (true isolation) or use import maps to pin React's URL across all MFEs.

**RED FLAGS TO AVOID:** - "Micro-frontends are the new default" (only justified at scale with many teams) - Multiple React instances (context + hook breakage) - No cross-MFE design system (UX chaos) - Sharing too much state (defeats autonomy)

## PART 2 — BACKEND QUESTIONS

## Q-BE-1: Explain thread pools and how to size them

**DIFFICULTY:** Hard **CATEGORY TAG:** Backend / Concurrency

**ANSWER:** A thread pool is a fixed set of worker threads that process submitted tasks. Sizing depends on the workload type: CPU-bound or IO-bound.

**CPU-bound (computation, hashing, parsing):** thread count = number of CPU cores. More threads than cores → context-switching overhead, no parallelism gain. Formula: `threads = N_cpu`.

**IO-bound (DB, HTTP, file):** thread count = much higher than CPU cores — threads spend most time waiting on IO. Formula (Little's Law): `threads = throughput × latency`. For 100 req/s × 100ms each = 10 threads. For 1000 req/s × 1s each = 1000 threads.

**Hybrid (CPU + IO):** use the formula from “Java Concurrency in Practice”: `threads = N_cpu × U_cpu × (1 + W / C) - N_cpu` = cores - `U_cpu` = target CPU utilisation (0.0-1.0) - `W / C` = wait time / compute time ratio (e.g. 9 for 90% IO, 10% compute)

For 8 cores, target 80% CPU, 90% IO workload: `threads = 8 × 0.8 × (1 + 9) = 64 threads`.

**Spring Boot Tomcat defaults:** `server.tomcat.threads.max = 200`, `min-spare = 10`. For most CRUD apps this is fine. For IO-heavy with high latency, you'd tune up. For CPU-heavy, you'd tune down.

**Why not unlimited threads?** 1. Memory: each Java thread has a stack (default 1MB) — 1000 threads = 1GB 2. Context switching: kernel scheduler switches threads — costs CPU cycles 3. Lock contention: more threads = more contention on shared resources (DB connection pool) 4. DB connection pool: each thread needs a connection — HikariCP at 30 + 200 threads = 170 threads waiting for connections

**Virtual threads (Java 21):** solve the IO-bound case — virtual threads are cheap (no kernel thread, no 1MB stack), can have 100K+ in one JVM. They block on IO without consuming a kernel thread. For CRUD apps, virtual threads + a single connection pool of 30 = handle 1000s of concurrent requests. Trade-off: not great for CPU-bound work; not great for code that holds locks across blocking calls.

**CODE/EXAMPLE SNIPPET:**

```

@Configuration
public class ThreadPoolConfig {
 @Bean("notifyExecutor")
 public ExecutorService notifyExecutor() {
 return new ThreadPoolExecutor(
 8, 16, 60L, TimeUnit.SECONDS,
 new LinkedBlockingQueue<>(500), // bounded queue
 new ThreadFactoryBuilder().setNameFormat("notify-%d").build(),
 new ThreadPoolExecutor.CallerRunsPolicy() // backpressure to submitter
);
 }
}

```

**KEY TERMS TO MENTION:** - CPU-bound: threads = cores; IO-bound: Little's Law  $\text{threads} = \text{throughput} \times \text{latency}$  - Hybrid formula:  $\text{threads} = N_{\text{cpu}} \times U_{\text{cpu}} \times (1 + W/C)$  - Spring Boot Tomcat defaults: 200 max, 10 min-spares - Memory cost (1MB stack/thread), context switching, lock contention - `ThreadPoolExecutor` with bounded queue + `CallerRunsPolicy` backpressure - Timeouts via `Future.get(timeout)` or `CompletableFuture.orTimeout()` - Micrometer metrics: `active`, `queued`, `pool.size`, `completed` - Virtual threads (Java 21): cheap, IO-bound, ~100K threads

**FOLLOW-UP QUESTIONS:** 1. What's the difference between `submit()` and `execute()`? 2. How do you handle a task that takes too long? 3. When would you NOT use a thread pool?

**FOLLOW-UP ANSWERS:** 1. `execute(Runnable)` returns void and throws on rejection. `submit(Callable)` returns a `Future` you can cancel + get the result. We use `submit` for everything (we want cancellation + result). 2. `Future.get(timeout)` — throws `TimeoutException`, then `future.cancel(true)` interrupts the worker. Or `CompletableFuture.orTimeout(5, SECONDS)`. For long-running batches, checkpoint progress so a restart resumes from the last checkpoint, not from zero. 3. For very short tasks (microseconds) — the pool overhead exceeds the work. Use direct execution. For one-off tasks (server startup), use a single-thread executor or `new Thread()`. For tasks requiring strict ordering, use a single-thread executor (queue serialises them).

**RED FLAGS TO AVOID:** - "More threads = more throughput" (context switch overhead) - Unbounded queue (`new LinkedBlockingQueue<>()`) → OOM under load - No `RejectedExecutionHandler` → `RejectedExecutionException` in production

---

## Q-BE-2: JVM memory model — heap, stack, metaspace, direct buffers

**DIFFICULTY:** Hard **CATEGORY TAG:** Backend / JVM

**ANSWER:** JVM memory has several regions, each with different lifetime + GC characteristics.

**Heap (young + old generation):** where object instances live. Young gen = Eden + 2 Survivor (S0, S1); new objects start in Eden, surviving minor GCs promote to survivors, then to old gen. Old gen holds long-lived objects, collected by major GC.

**Stacks (per thread):** each thread has its own stack of frames. A frame per method call holds local variables, operand stack, return address. Stack frames are not shared (thread-safe by construction). Stack Overflow if recursion is too deep (default stack 1MB; `-Xss` to change). Not GC'd (frame is freed when method returns).

**Metaspace (Java 8+, replaced PermGen):** where class metadata, method bytecode, static fields live. Native memory (off-heap), grows automatically unless `-XX:MaxMetaspaceSize` caps it. Memory leak: dynamic class generation (proxies, CGLib) without unloading → grows until OOM. Spring AOP / Hibernate proxies can cause this.

**Code cache:** JIT-compiled native code. Capped by `-XX:ReservedCodeCacheSize` (default 240MB). Rarely an issue unless you generate many classes.

**Direct buffers (NIO `ByteBuffer.allocateDirect`):** off-heap, bypass JVM heap GC. Used for IO (Netty, gRPC). Tracked by `-XX:MaxDirectMemorySize`. Cleaner (PhantomReference + Cleaner) frees them; if you forget, you leak native memory.

**String pool / interned strings:** in the heap (since Java 7). `String.intern()` puts a string in the pool; same content → same reference. Used by the JVM to deduplicate identical strings (compile-time string constants auto-pooled). Overuse causes heap pressure.

**CODE/EXAMPLE SNIPPET (common sizes):**

```
-Xms2g -Xmx2g # initial + max heap (set equal to avoid resize)
-Xss512k # per-thread stack
-XX:MaxMetaspaceSize=256m
-XX:MaxDirectMemorySize=512m
-XX:+UseG1GC # GC algorithm
-XX:MaxGCPauseMillis=200 # G1 target
```

**Common memory issues:** - `OutOfMemoryError: Java heap space` — too many objects. Heap dump + MAT. - `OutOfMemoryError: Metaspace` — class leak (dynamic proxies). Cap Metaspace + find what's loading classes. - `OutOfMemoryError: Direct buffer memory` — NIO leak. Check Netty buffer release. - `OutOfMemoryError: unable to create new native thread` — too many threads (ulimit + memory). Reduce thread count or raise `ulimit -u`.

**KEY TERMS TO MENTION:** - Heap: young (Eden + S0/S1) + old gen - Stack: per-thread, frames, no GC, StackOverflow - Metaspace (replaces PermGen): class metadata, native, dynamic proxy leak - Code cache, direct buffers (off-heap, NIO), string pool - OOM types: heap, metaspace, direct, native thread

**FOLLOW-UP QUESTIONS:** 1. Why was PermGen replaced with Metaspace? 2. How do you size the heap for a service?

**FOLLOW-UP ANSWERS:** 1. PermGen was fixed-size + inside the JVM heap — `OutOfMemoryError: PermGen` was common (dynamic class loading, JSP redeploys). Metaspace is off-heap native memory + auto-grows (capped by `MaxMetaspaceSize`), so class metadata rarely causes OOM. The class unloading also improved (G1 + concurrent class unloading). 2. Measure with load: heap should fit working set + headroom for GC. Rule of thumb: working set  $\leq$  50% of heap (so GC has room to breath). For a service with 5GB live data: 10GB heap, G1 GC, `-Xms10g -Xmx10g` (equal to avoid resize jitter). Watch GC log for time-in-GC; if >5%, add heap or tune GC. Don't over-size — huge heaps make full GC catastrophic (seconds of pause).

**RED FLAGS TO AVOID:** - “Bigger heap is always better” (longer full-GC pauses) - Not setting `-Xms = -Xmx` (heap resize jitter in prod) - Ignoring Metaspace/direct memory (separate OOM class)

-----

## Q-BE-3: Garbage collection — G1, ZGC, Shenandoah, Serial

**DIFFICULTY:** Hard **CATEGORY TAG:** Backend / JVM

**ANSWER:** GC finds unreachable objects and reclaims their memory. The choice of collector depends on pause time vs throughput trade-off.

**Serial GC ( `-XX:+UseSerialGC` ):** single-threaded. For small heaps (<100MB), client apps, or dev. Not for servers.

**Parallel GC ( `-XX:+UseParallelGC` ):** multi-threaded young + old collection. Maximises throughput, long pauses (full GC pauses all app threads). Default in Java 8. Bad for latency-sensitive services.

**G1 GC ( `-XX:+UseG1GC` ):** default since Java 9. Splits heap into regions, collects subsets (“garbage first”). Predictable pause times ( `-XX:MaxGCPauseMillis=200` ). Good general-purpose for heaps up to ~32GB. Mixed GC (young + parts of old).

**ZGC ( `-XX:+UseZGC` ):** low-latency, concurrent, colocation. Pause times <10ms even on multi-TB heaps. Uses coloured pointers + load barriers. Production-ready since Java 15 (experimental earlier). Trade-off: slight throughput hit. Best for very large heaps + low-latency needs.

**Shenandoah ( `-XX:+UseShenandoahGC` ):** Red Hat’s low-latency collector. Concurrent compaction via Brooks pointers (older) / pass-through barriers (newer). Similar goals to ZGC. Production-ready since Java 15.

**Generational ZGC (JDK 21):** adds young/old generations to ZGC (was generational-less before). Better throughput + still <1ms pauses.

### CODE/EXAMPLE SNIPPET:

```
Production service flags
-Xms8g -Xmx8g \
-XX:+UseG1GC \
-XX:MaxGCPauseMillis=200 \
-XX:+ParallelRefProcEnabled \
-XX:+UnlockExperimentalVMOptions -XX:+UseZGC \
-Xlog:gc*:file=gc.log:time,level,tags
```

**Choosing (2026):** | Workload | Collector | |—|—| | Small heap, latency not critical | G1 | | Large heap (32GB+), low latency | ZGC (generational) | | Throughput > latency (batch) | Parallel | | Red Hat / want options | Shenandoah | | Single-core / tiny | Serial |

**Diagnosing GC issues:** - `jstat -gcutil <pid>` — Eden/S0/S1/Old/Metaspace usage + GC counts - GC log ( `-Xlog:gc*` ) — parse with GCEasy / GCViewer - Frequent young GC with big pause → too small young gen or too many allocations - Frequent full GC → memory leak or too small heap

**KEY TERMS TO MENTION:** - Serial, Parallel (throughput, long pauses), G1 (regions, predictable pauses), ZGC (<10ms, large heaps), Shenandoah (Red Hat, low-latency) - Generational ZGC (JDK 21) - Choosing: G1 default, ZGC for large + low latency, Parallel for throughput - Diagnosing: jstat, GC log, GCEasy - Common issues: frequent young GC (small gen, many allocs), frequent full GC (leak/small heap)

**FOLLOW-UP QUESTIONS:** 1. Why does ZGC have such low pause times? 2. How do you tune G1?

**FOLLOW-UP ANSWERS:** 1. ZGC is **concurrent** — most GC work runs alongside app threads. It uses coloured pointers (metadata bits in the pointer's high bits) + a load barrier (fixes pointers when loaded). Mutations happen while app runs; only short sync points for marking start/end + relocation start/end (sub-millisecond). Trade-off: the load barrier has a throughput cost (typically 5-15% slower than G1). 2. Key flags: `-XX:MaxGCPauseMillis=200` (target — G1 sizes regions to meet it), `-XX:G1HeapRegionSize=4m` (or let G1 choose; 1-32MB based on heap), `-XX:InitiatingHeapOccupancyPercent=45` (trigger concurrent marking when old gen 45% full), `-XX:G1ReservePercent=10` (reserve for evacuation). Tune by watching GC log: if you miss pause target, increase heap or reduce IHOP; if old gen fills, your working set is too big for the heap.

**RED FLAGS TO AVOID:** - “Just use Parallel for max throughput” (long pauses kill latency) - Using ZGC on a small heap (overkill, throughput hit) - Not enabling GC logging in production

---

## Q-BE-4: OutOfMemoryError — diagnose and fix in production

**DIFFICULTY:** Hard **CATEGORY TAG:** Backend / JVM

**ANSWER:** OOM types and their diagnoses:

- Java heap space** — too many live objects. Most common. - Capture: `-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/tmp/heap.hprof` - Analyse: Eclipse MAT — “Dominator Tree” shows biggest retainers; “Leak Suspects Report” auto-finds leaks. - Common causes: cache without eviction, unbounded list/queue, static collections growing, listener not unregistered, large query results ( `SELECT *` ).
- Metaspace** — class metadata leak. Dynamic class generation (CGLib proxies, dynamic languages) without unloading. Find: `jcmd <pid> GC.class_histogram` shows class counts; spikes indicate leak. Cap Metaspace + fix the generator (use a single class loader, avoid generating per-request).
- Direct buffer memory** — NIO `ByteBuffer.allocateDirect` leak (Netty, gRPC). The cleaner frees when GC'd; if you hold references, no GC → no free. Find: `jcmd <pid> VM.native_memory` (with `-XX:NativeMemoryTracking=summary` ).
- unable to create new native thread** — thread count exhausted (`ulimit`) or memory for thread stacks. Reduce thread pools or raise `ulimit -u` + `ulimit -n` .
- GC overhead limit exceeded** — GC running >98% of time + recovering <2% heap. Effectively a leak or undersized heap. Heap dump + investigate.

### Investigation playbook:

```
1. Capture heap dump (auto on OOM via flags above; or manually)
jcmd <pid> GC.heap_dump /tmp/heap.hprof
2. Live object histogram (quick)
jcmd <pid> GC.class_histogram | head -20
3. Thread dump (deadlock? stuck threads?)
jstack <pid> > threads.txt
4. Analyse with MAT: Dominator Tree → biggest retainers
5. Find the leak path (MAT "Shortest Path to GC Root")
```

**Common fixes:** - Cache eviction (Caffeine with `maximumSize` + `expireAfterWrite` ) - Bounded queues ( `new LinkedBlockingQueue<>(1000)` ) - Pagination on queries ( `PageRequest.of(0, 100)` ) - Try-with-resources / explicit close for IO - Static collection → replace with bounded cache or weak refs

**KEY TERMS TO MENTION:** - 5 OOM types: heap space, Metaspace, direct buffer, native thread, GC overhead - `HeapDumpOnOutOfMemoryError` + `HeapDumpPath` (auto-capture) - Eclipse MAT: Dominator Tree, Leak Suspects, shortest path to GC root - `jcmd`: `GC.heap_dump`, `GC.class_histogram` - `jstack` for thread state - Fixes: cache eviction (Caffeine), bounded queues, pagination, try-with-resources

**FOLLOW-UP QUESTIONS:** 1. How do you take a heap dump without restarting? 2. What's a "GC root" and why does MAT use shortest-path?

**FOLLOW-UP ANSWERS:** 1. `jcmd <pid> GC.heap_dump /tmp/heap.hprof` (live, no restart). Or `jmap -dump:format=b,file=heap.hprof <pid>` (legacy). The dump pauses the JVM briefly (a few seconds for big heaps) — acceptable in most prod. For very large heaps, capture during a deploy or use a tool like `jhsdb jmap --heap --exe java --pid <pid>` for sampling. Compress with gzip. 2. GC roots are things the GC never reclaims: static fields, active thread stack locals, JNI global refs, synchronized monitors. An object is reachable if there's a path of references from a GC root to it. MAT's "shortest path to GC root" finds the most direct retention path — usually pinpoints the leak (e.g. "this HashMap in this static field in this class holds 4GB of entries"). If you remove the reference from the root, the whole subtree becomes garbage.

**RED FLAGS TO AVOID:** - Restarting the JVM without capturing a heap dump (you've lost the leak) - "Just bump -Xmx" (postpones the OOM; the leak still grows) - Not setting `-XX:+HeapDumpOnOutOfMemoryError` in prod

---

## Q-BE-5: Spring Bean lifecycle and scopes

**DIFFICULTY:** Medium **CATEGORY TAG:** Backend / Spring

**ANSWER:** A bean's lifecycle: instantiation → populate properties → BeanNameAware/BeanFactoryAware/ApplicationContextHolder → BeanPostProcessor (before init) → @PostConstruct / InitializingBean.afterPropertiesSet / init-method → BeanPostProcessor (after init) → use → @PreDestroy / DisposableBean.destroy / destroy-method.

**Scopes:** - **singleton** (default): one instance per Spring container. Most beans. - **prototype**: new instance per injection. For stateful beans. - **request**: one per HTTP request (web only). - **session**: one per HTTP session (web only). - **application**: one per ServletContext (web). - **websocket**: one per WebSocket session.

**The singleton trap:** a singleton bean injecting a prototype bean — the prototype is created once at the singleton's construction. From then on, the singleton always uses the same prototype. To get a fresh prototype each time: use `@Lookup` (Spring injects a method that looks up the prototype each call) or `ObjectFactory<MyPrototype>` / `Provider<MyPrototype>`.

**Lifecycle hooks:**

```
@Component
public class MyBean implements InitializingBean, DisposableBean {
 @PostConstruct
 public void init() { /* after construction + injection */ }
 @PreDestroy
 public void cleanup() { /* before destruction */ }
 @Override
 public void afterPropertiesSet() { /* alternative to @PostConstruct */ }
 @Override
 public void destroy() { /* alternative to @PreDestroy */ }
}
```

**BeanPostProcessor:** runs code before/after every bean's init. Used by Spring internally ( `AutowiredAnnotationBeanPostProcessor` processes `@Autowired` ). Custom ones let you wrap beans (proxy creation, metric instrumentation).

**Destruction order:** on container shutdown, singleton beans are destroyed in reverse creation order (LIFO). `@PreDestroy` runs first, then `DisposableBean.destroy()`, then `destroy-method`.

**KEY TERMS TO MENTION:** - Lifecycle: instantiate → populate → aware → postProcess-before → init (@PostConstruct) → postProcess-after → use → destroy (@PreDestroy) - Scopes: singleton (default), prototype (per-injection), request, session, application, websocket - Singleton injects prototype → only one prototype (use `@Lookup` or `ObjectFactory`) - BeanPostProcessor for cross-cutting wrapping (AOP, metrics) - Destruction in LIFO order

**FOLLOW-UP QUESTIONS:** 1. What's the difference between `@PostConstruct` and `@Bean(initMethod=...)`? 2. How do you handle async destruction (e.g. close a connection pool)?

**FOLLOW-UP ANSWERS:** 1. They run in order: `@PostConstruct` → `InitializingBean.afterPropertiesSet()` → `@Bean(initMethod=...)`. `@PostConstruct` is on the bean class (annotation); `initMethod` is on the `@Bean` definition (in a `@Configuration` class — useful when you can't modify the bean class, e.g. third-party). In practice, they all run before the bean is put in service — order rarely matters; just pick one (avoid mixing). 2. The bean's `@PreDestroy` (or `destroyMethod`) runs on container shutdown. For a connection pool (HikariCP), calling `close()` on the `DataSource` triggers pool shutdown — Spring auto-detects `close()` via `@Bean(destroyMethod = "close")` (it infers this if the bean has a `close()` or `shutdown()` method). For async cleanup (e.g. graceful drain), use Spring's `SmartLifecycle` interface with `getPhase()` to control order, or implement `ApplicationListener<ContextClosedEvent>`.

**RED FLAGS TO AVOID:** - Putting state in a singleton (race condition across threads) - Expecting prototype injection in a singleton to give new instances each call - Heavy work in `@PostConstruct` (delays startup; use `ApplicationReadyEvent`)

---

## Q-BE-6: Spring Boot auto-configuration — how it works under the hood

**DIFFICULTY:** Hard **CATEGORY TAG:** Backend / Spring Boot

**ANSWER:** `@SpringBootApplication` = `@SpringBootConfiguration` + `@EnableAutoConfiguration` + `@ComponentScan`. Auto-configuration is the magic.

`@EnableAutoConfiguration` triggers loading of auto-config classes via `spring.factories` (Spring Boot 2.x) or `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` (3.x). Each auto-config class has `@Conditional` annotations that decide whether to apply.

**Conditional annotations:** - `@ConditionalOnClass(SomeClass.class)` — only if class is on classpath (e.g. enable `DataSource` auto-config only if `HikariCP` is present) - `@ConditionalOnMissingBean(SomeBean.class)` — only if no user-defined bean of this type (user's bean wins) - `@ConditionalOnProperty(prefix="spring.datasource", name="url")` — only if property set - `@ConditionalOnWebApplication` — only in web apps - `@ConditionalOnBean(DataSource.class)` — only if another bean exists

**Example (DataSourceAutoConfiguration simplified):**

```
@AutoConfiguration
@ConditionalOnClass({ DataSource.class, EmbeddedDatabaseType.class })
@EnableConfigurationProperties(DataSourceProperties.class)
public class DataSourceAutoConfiguration {
 @Bean
 @ConditionalOnMissingBean
 public DataSource dataSource(DataSourceProperties props) {
 return props.initializeDataSourceBuilder().build();
 }
}
```

If you define your own `DataSource` bean, this is skipped (`@ConditionalOnMissingBean`).

`@ConfigurationProperties` binds prefixed properties to a typed object (`spring.datasource.url` → `DataSourceProperties.url`). Loose binding (`camelCase`, `snake_case`, `kebab-case` all match).

**Excluding** **auto-configs:**  
`@SpringBootApplication(exclude = {DataSourceAutoConfiguration.class})` when you don't want the default.

**Debugging:** `--debug` flag logs which auto-configs matched / didn't + why. `spring-boot-actuator`'s `/actuator/conditions` endpoint shows the same at runtime.

**KEY TERMS TO MENTION:** - `@SpringBootApplication` = config + autoconfig + component scan - Auto-config classes loaded via `spring.factories` (2.x) / `AutoConfiguration.imports` (3.x) - Conditional annotations: `OnClass`, `OnMissingBean` (user wins), `OnProperty`,

OnWebApplication, OnBean - @ConfigurationProperties for typed binding - Exclude via @SpringBootApplication(exclude=...) - Debug: -debug flag, /actuator/conditions

**FOLLOW-UP QUESTIONS:** 1. What happens if two auto-configs define the same bean? 2. How do you write your own auto-configuration?

**FOLLOW-UP ANSWERS:** 1. @ConditionalOnMissingBean ensures only one applies — the first to register wins; later ones see the bean exists and skip. Order is controlled by @AutoConfigureBefore, @AutoConfigureAfter, @AutoConfigureOrder. Conflicts that aren't guarded by @ConditionalOnMissingBean throw BeanDefinitionStoreException. Spring's auto-configs are carefully ordered to avoid this. 2. Create a class with @AutoConfiguration, annotate with @ConditionalOnClass etc., add to META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports (a list of fully-qualified class names). Use @ConditionalOnProperty for a feature flag. Provide a @ConfigurationProperties class for user config. Test with ApplicationRunner (loads just your auto-config in a test context). Ship as a starter (spring-boot-starter-foo) so users just add the dependency.

**RED FLAGS TO AVOID:** - "Auto-config just works by magic" (you must know conditions to debug) - Defining a bean that conflicts with auto-config without @ConditionalOnMissingBean (causes startup failure) - Not excluding auto-configs you don't need (extra startup time)

---

## Q-BE-7: Dependency injection — constructor vs field vs setter

**DIFFICULTY:** Medium **CATEGORY TAG:** Backend / Spring

**ANSWER:** Three ways to inject dependencies:

**Constructor injection (recommended):**

```
@Service
public class OrderService {
 private final OrderRepository repo; // final = required
 private final PaymentGateway gateway;
 public OrderService(OrderRepository repo, PaymentGateway gateway) {
 this.repo = repo; this.gateway = gateway;
 }
}
```

Pros: immutable (final fields), testable (pass mocks in constructor), fails fast (missing dep → startup error), no Spring-required annotation on the field. Cons: many deps → long constructor (use Lombok `@RequiredArgsConstructor` or Spring 4.3+ implicit single-constructor injection).

**Field injection ( `@Autowired` on field):**

```
@Service
public class OrderService {
 @Autowired private OrderRepository repo; // can't be final
 @Autowired private PaymentGateway gateway;
}
```

Pros: concise. Cons: not immutable, hides dependencies, hard to test (need reflection to inject mocks), allows circular dependencies to slip (Spring can construct with field injection even when constructors would fail). Deprecated in modern Spring.

**Setter injection ( `@Autowired` on setter):**

```
@Service
public class OrderService {
 private OrderRepository repo;
 @Autowired
 public void setRepo(OrderRepository repo) { this.repo = repo; }
}
```

Pros: optional deps, reconfigurable. Cons: mutable, allows partially-constructed beans. Use for genuinely optional deps.

**Why constructor injection is best practice:** 1. Immutability (final fields) → thread-safe by construction 2. Testability — instantiate with mocks without Spring 3. Fails fast — missing dep caught at startup, not at first use 4. No hidden dependencies — the constructor signature shows what's needed 5. Prevents circular dependencies (Spring would catch them at construction)

**Circular dependencies:** A depends on B, B depends on A. With constructor injection, Spring throws `BeanCurrentlyInCreationException` — you must refactor (extract a third bean, use events, or use `@Lazy`). With field injection, Spring allows it (instantiates both, then injects fields) — masks a design smell.

**KEY TERMS TO MENTION:** - Constructor: immutable, testable, fails fast, prevents circular deps (recommended) - Field (`@Autowired`): concise but mutable, hard to test, hides deps, allows circular - Setter: optional/reconfigurable deps - Spring 4.3+ single-constructor auto-injection (no `@Autowired` needed) - Lombok `@RequiredArgsConstructor` for concise constructor injection

**FOLLOW-UP QUESTIONS:** 1. How do you break a circular dependency? 2. What's `@Lazy` and when to use it?

**FOLLOW-UP ANSWERS:** 1. Refactor — the circular dependency is usually a design smell. Options: (a) extract the shared logic into a third bean (C) that both A and B depend on; (b) use events (A publishes, B listens — no direct dependency); (c) if truly needed, `@Lazy` on one side (Spring injects a proxy that resolves on first call); (d) convert one side to setter/field injection (not recommended — masks the smell). 2. `@Lazy` injects a proxy that defers creating the bean until first method call. Use for: breaking circular deps (the proxy lets both beans construct first), expensive beans that may not be used (lazy-init on first call), or breaking startup ordering. Trade-off: first call has a small lookup overhead; debug is harder (the bean you inject isn't the bean you get — it's a proxy). Use sparingly — usually a refactor is better.

**RED FLAGS TO AVOID:** - Field injection as the default (mutable, untestable) - Allowing circular deps via field injection (masks design smell) - Not making injected fields final (mutability bugs)

---

## Q-BE-8: @Transactional — propagation and isolation levels

**DIFFICULTY:** Hard **CATEGORY TAG:** Backend / Spring / Transactions

**ANSWER:** `@Transactional` wraps a method in a DB transaction: open transaction → execute → commit on success / rollback on exception. But propagation + isolation are the gotchas.

**Propagation (how the method participates in an existing tx):** - **REQUIRED** (default): use existing tx, or start a new one if none. - **REQUIRES\_NEW**: always start a new tx, suspend the outer. - **NESTED**: use existing tx, but create a savepoint — partial rollback possible (DB must support savepoints). - **SUPPORTS**: use existing tx if present, else run non-transactional. - **NOT\_SUPPORTED**: run non-transactional, suspend any existing tx. - **NEVER**: throw if a tx exists. - **MANDATORY**: throw if no tx exists.

**Isolation (concurrency visibility):** - **DEFAULT**: DB default (usually `READ_COMMITTED`). - **READ\_UNCOMMITTED**: see uncommitted changes (dirty reads). Rarely used. - **READ\_COMMITTED**: see only committed changes (no dirty reads). Default in PG/Oracle. - **REPEATABLE\_READ**: same query returns same results within a tx (no non-repeatable reads). MySQL InnoDB default; also prevents phantom reads via gap locks. - **SERIALIZABLE**: full locking, no concurrency anomalies; slowest.

**Common pitfalls:** 1. **Self-invocation:** `@Transactional` is proxy-based. Calling `methodA()` → `methodB()` within the same class bypasses the proxy → `@Transactional` on `methodB` is ignored. Use `AopContext.currentProxy()` or split into two beans. 2. **Rollback only on unchecked exceptions:** `@Transactional` rolls back on `RuntimeException` and `Error` by default. To roll back on checked exceptions: `@Transactional(rollbackFor = BusinessException.class)`. 3. **Checked exception swallowed in try/catch:** if you catch the exception inside the method, Spring doesn't see it → no rollback. Re-throw or use `TransactionAspectSupport.currentTransactionStatus().setRollbackOnly()`. 4. **Long transactions:** holding a tx open while calling an external API (HTTP, email) holds DB connections + locks. Move external calls outside the tx. 5. **Read-only mismatch:** `@Transactional(readOnly = true)` on a write operation throws on commit (some DBs enforce). 6. **Connection pool exhaustion:** long txs hold HikariCP connections. Watch `hikari.connections.active` metric.

**CODE/EXAMPLE SNIPPET:**

```

@Service
public class OrderService {
 @Transactional
 public void placeOrder(Order o) {
 repo.save(o); // tx REQUIRED
 audit.log("ordered"); // same tx
 try { gateway.charge(o); }
 catch (PaymentException e) {
 // re-throw to roll back; or setRollbackOnly()
 throw e;
 }
 }

 @Transactional(propagation = Propagation.REQUIRES_NEW)
 public void sendReceipt(Long orderId) { // new tx even if outer rolls back
 email.send(orderId);
 }
}

```

**KEY TERMS TO MENTION:** - 7 propagations: REQUIRED (default), REQUIRES\_NEW, NESTED, SUPPORTS, NOT\_SUPPORTED, NEVER, MANDATORY - Isolations: READ\_COMMITTED (default), REPEATABLE\_READ (MySQL), SERIALIZABLE - Self-invocation bypasses proxy → @Transactional ignored - Rollback on unchecked by default; use rollbackFor for checked - Long txs hold connections + locks — external calls outside tx - readOnly = true for read paths (Hibernate skips dirty checks, faster)

**FOLLOW-UP QUESTIONS:** 1. What's the difference between REQUIRED and NESTED? 2. Why doesn't @Transactional work on private methods?

**FOLLOW-UP ANSWERS:** 1. REQUIRED joins the outer tx — if the inner throws, the whole tx rolls back (outer can't recover). NESTED creates a savepoint — if the inner throws, you can catch + roll back to the savepoint, leaving the outer tx intact (partial rollback). Requires DB savepoint support (PG/Oracle yes; some don't). Use NESTED when you want to retry the inner operation without killing the outer. 2. Spring creates a CGLib (or JDK dynamic) proxy around the bean. Proxies can only intercept public methods (Java visibility rules). Private methods are not visible to the proxy — even if it tried, the proxy subclass can't override private methods. Same for protected/package-private in some cases. Workaround: extract the private method into a separate bean (whose public method is transactional) or use `TransactionTemplate` programmatically.

**RED FLAGS TO AVOID:** - Self-invocation (proxy bypass) - Catching the exception inside the tx (no rollback) - Long-running external calls inside a tx (holds connection + lock) - @Transactional on private methods (silently ignored)

---

## Q-BE-9: Hibernate N+1 problem — detection and fixes

**DIFFICULTY:** Hard **CATEGORY TAG:** Backend / Hibernate

**ANSWER:** N+1: fetching N parent entities triggers 1 query for parents + N queries for children (lazy load each). E.g.

`List<Order> orders = repo.findAll(); orders.forEach(o -> o.getCustomer().getName());` → 1 query for orders + 1 query per order for customer = 101 queries for 100 orders.

**Detection:** - Hibernate statistics ( `hibernate.generate_statistics=true` ) — logs query counts. - Log SQL ( `spring.jpa.show-sql=true` or `hibernate.logging=DEBUG` ). - DataSourceProxy (helpful in dev — logs query counts + slow queries). - Look at query patterns in logs: 1 + N identical “select \* from customer where id = ?” → N+1.

**Fixes:**

### 1. EntityGraph (explicit fetch plan):

```
@EntityGraph(attributePaths = "customer")
List<Order> findAllByStatus(String status);
```

Generates a single query with JOIN to fetch customers in one shot.

### 2. JOIN FETCH in JPQL:

```
@Query("SELECT o FROM Order o JOIN FETCH o.customer WHERE o.status = :status")
List<Order> findWithCustomer(@Param("status") String status);
```

### 3. Batch fetching (lazy, but in batches):

```
spring.jpa.properties.hibernate.default_batch_fetch_size: 50
```

Instead of N queries, Hibernate loads 50 at a time → 1 + N/50 queries. Less ideal than JOIN FETCH but no code change.

### 4. DTO projection (avoid entity hydration):

```
@Query("SELECT new com.example.OrderDTO(o.id, o.total, c.name) FROM Order o JOIN o.customer c")
List<OrderDTO> findAllDTO();
```

Single query, no lazy load needed, no entity-graph cost.

**5. Fetch.LAZY on collections (default) + explicit eager loading only where needed:** avoid `FetchType.EAGER` (the trap that causes N+1 every time you load the parent). Make associations lazy by default; fetch eagerly only where you need them.

**Gotchas:** - `JOIN FETCH` + pagination on a many-to-many: Hibernate warns “firstResult / maxResults specified with collection fetch; applying in memory” — fetches **all** then paginates in memory (OOM risk). Use `EntityGraph` + `@BatchSize` or two queries (fetch IDs paginated, then fetch by IDs). - `@OneToMany` lazy loading inside a Stream/lambda that runs after the session closes → `LazyInitializationException`. Use `@Transactional` on the service or fetch eagerly.

**KEY TERMS TO MENTION:** - N+1: 1 query for parents + N lazy loads for children - Detection: `show-sql`, `statistics`, `DataSourceProxy` - Fixes: `EntityGraph`, `JOIN FETCH`, `batch_fetch_size`, DTO projection - Avoid `FetchType.EAGER` (causes N+1 always) - `JOIN FETCH` + pagination on `@OneToMany` → in-memory pagination (OOM) → use IDs + batch

**FOLLOW-UP QUESTIONS:** 1. What’s the difference between `JOIN` and `JOIN FETCH`? 2. How do you fix N+1 on a `@OneToMany` that’s paginated?

**FOLLOW-UP ANSWERS:** 1. `JOIN` filters rows but doesn’t fetch the joined association — the association stays lazy, you still hit N+1. `JOIN FETCH` both filters AND fetches (the association is initialised in the same query). Always use `FETCH` when you want to eager-load. For pure filtering without fetching, plain `JOIN` is fine. 2. Fetch the paginated IDs first (one query, with limit/offset), then fetch the parent + children in a second query by ID list:

```
List<Long> ids = repo.findIdsByStatus(status, pageable); // paginated IDs
List<Order> orders = repo.findByIdInWithItems(ids); // JOIN FETCH items
```

Two queries, no in-memory pagination, no OOM. Hibernate 6 has improved this; you can also use `EntityGraph` + `@BatchSize(size=50)` to lazy-load in batches instead of `JOIN FETCH`.

**RED FLAGS TO AVOID:** - `FetchType.EAGER` on `@OneToMany` (instant N+1) - `spring.jpa.open-in-view: true` masking N+1 (lazy load works in the view layer; prod kills you) - Ignoring query count logs (don’t ship N+1 to prod)

---

## Q-BE-10: JPA first-level vs second-level cache

**DIFFICULTY:** Medium **CATEGORY TAG:** Backend / Hibernate

**ANSWER: First-level cache (L1):** the `PersistenceContext` — per-session, mandatory. Each `EntityManager` (or Hibernate Session) holds loaded entities by ID. Repeated `find(Order.class, 1L)` returns the same instance (no second DB hit). Cleared on tx commit/rollback + on `em.clear()`. Not shared across sessions. Always on.

**Second-level cache (L2):** optional, shared across sessions + EMs, keyed by entity class + ID. Backed by a cache provider (Ehcache, Caffeine, Infinispan). Stores entity state (not instances — each session reconstructs its own instance from the cached state).

**Enable L2:**

```
spring.jpa.properties.hibernate.cache.use_second_level_cache: true
spring.jpa.properties.hibernate.javax.cache.provider:
 org.ehcache.jsr107.EhcacheCachingProvider
spring.jpa.properties.hibernate.cache.region.factory_class: jcache
```

```
@Entity @Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
public class Order { ... }

@Cacheable
@Entity
public class Product { ... } // JPA 2 standard annotation
```

**Query cache:** caches query results (not just entities). Key: query + params + flush mode. Returns IDs, then L2 resolves entities. Useful for stable, frequently-run queries. Disabled by default (overhead). Enable with `hibernate.cache.use_query_cache: true` + `@QueryHints` or programmatic. Invalidate on any write to the affected table → careful (can cause more invalidation traffic than hits).

**When to use L2:** - Read-heavy entities that change rarely (reference data, lookup tables, configuration) - Need cross-session consistency on cached reads - Don't use for: high-write entities (constant invalidation), user-specific data (per-user cache poisoning), or where Redis is already used (use Redis as the L2 instead)

**Concurrency strategies:** - **READ\_ONLY:** entity never changes. Fastest. - **NONSTRICT\_READ\_WRITE:** occasional writes, slight staleness OK. - **READ\_WRITE:** strong consistency via locks. Slower. - **TRANSACTIONAL:** JTA-backed (XA). Rare.

**Pitfalls:** - L2 stores across all EMs → memory pressure on huge datasets. Cache only reference data. - Stale data if a non-Hibernate process updates the DB (L2 doesn't know). Use `em.refresh` or evict. - Query cache invalidates on any write to the entity's table — can be slower than no cache for write-heavy entities.

**Modern alternative:** use Redis as a manual cache layer ( `@Cacheable` on service methods, with `@CacheEvict` on writes). More control, decoupled from Hibernate, works for non-entity data.

**KEY TERMS TO MENTION:** - L1 (PersistenceContext): per-session, mandatory, cleared on tx end, always on - L2: optional, shared across sessions, keyed by class+ID, Ehcache/Caffeine/Infinispan - `@Cache` / `@Cacheable`, `CacheConcurrencyStrategy` (READ\_ONLY fastest, READ\_WRITE strongest) - Query cache: caches query+params→IDs, invalidates on table write (often more harm than good) - Use L2 for read-heavy reference data; use Redis for app-level cache (more control)

**FOLLOW-UP QUESTIONS:** 1. When does L1 cache cause stale reads? 2. Why is the query cache often a bad idea?

**FOLLOW-UP ANSWERS:** 1. Within one tx/session, L1 returns the same instance even if another tx modified it — that's the point (repeatable read at the session level). Across sessions, L1 is fresh (new session → no cache). So L1 doesn't cause stale reads **across** sessions. Within a session, if you need a fresh read, call `em.refresh(entity)` or open a new session. The risk is if you hold a session open too long + expect to see other tx's writes — that's not how it works. 2. Query cache keys include the query + parameter values. On **any** write to any entity of the affected table, Hibernate invalidates the **entire region** for that table. So even unrelated writes blow the cache. For write-heavy tables, the invalidation rate exceeds the hit rate. Plus, the cache returns IDs → L2 lookup → entity hydration; the round trip is slower than a DB hit for many cases. Use it only for queries that re-run frequently on data that barely changes.

**RED FLAGS TO AVOID:** - L2 on write-heavy entities (invalidation storm) - Open session in view masking L2 staleness - Query cache without measuring hit rate

---

## Q-BE-11: Optimistic vs pessimistic locking

**DIFFICULTY:** Hard **CATEGORY TAG:** Backend / Concurrency

**ANSWER:** Two strategies for concurrent updates to the same row.

**Optimistic locking (no DB locks, version check on commit):** - Add a `@Version` column (Long/Timestamp) to the entity. - On read, fetch the version. On update, `UPDATE ... SET ... version = version+1 WHERE id = ? AND version = ?`. - If another tx updated the row in between, the WHERE clause matches 0 rows → `OptimisticLockException` (Hibernate) / retry. - Best for: low contention (most updates don't conflict), long transactions (DB locks would block), distributed systems (no DB-level coordination needed).

```
@Entity
public class Account {
 @Id Long id;
 @Version Long version; // managed by JPA
 BigDecimal balance;
}
```

**Pessimistic locking (DB row locks):** - `SELECT ... FOR UPDATE` (or `FOR SHARE`) at read time — locks the row until tx commits. - Other txs wait (or get a `LockTimeoutException` if timeout hits). - Best for: high contention (conflicts common), short txs (lock held briefly), when you must serialize access.

```
Account a = em.find(Account.class, 1L, LockModeType.PESSIMISTIC_WRITE);
// SELECT ... FOR UPDATE
```

**Trade-offs:** | | Optimistic | Pessimistic | |—|—|—| | Locks | None | DB row locks | | Contention handling | Retry on conflict | Wait (block) | | Best when | Low contention, long txs | High contention, short txs | | Deadlock risk | None (no locks) | Yes (lock ordering matters) | | Distributed | Works across instances | Requires DB-level (single DB) | | Failure mode | `OptimisticLockException` (retry) | `LockTimeout` / deadlock |

**Retry on optimistic lock failure:**

```
@Retryable(value = OptimisticLockException.class, maxAttempts = 3, backoff =
 @Backoff(delay = 100))
@Transactional
public void transfer(Long from, Long to, BigDecimal amt) { ... }
```

**Pessimistic modes:** - `PESSIMISTIC_WRITE` (FOR UPDATE): exclusive lock, no other tx can read/write (until commit). - `PESSIMISTIC_READ` (FOR SHARE): shared lock, others can read but not write. - `PESSIMISTIC_FORCE_INCREMENT`: write-lock + bump version immediately (for forced ordering).

**Real-world:** banking transfers — pessimistic on the account row (short tx, contention). Shopping cart updates — optimistic (long session, low contention per user). Distributed multi-region — optimistic (no global lock).

**KEY TERMS TO MENTION:** - Optimistic: @Version, no locks, retry on OptimisticLockException, low contention - Pessimistic: SELECT FOR UPDATE, blocks others, high contention, short txs - PESSIMISTIC\_WRITE (exclusive) vs PESSIMISTIC\_READ (shared) - @Retryable for optimistic retry - Distributed → optimistic (no global lock) - Real-world: banking=pessimistic, cart=optimistic

**FOLLOW-UP QUESTIONS:** 1. How do you avoid deadlocks with pessimistic locking? 2. What if optimistic retry keeps failing (live lock)?

**FOLLOW-UP ANSWERS:** 1. Always lock in a consistent order across all transactions (e.g. sort account IDs, lock lowest first). Use short transactions. Set `javax.persistence.lock.timeout` (PG supports `SET lock_timeout`) so a stuck lock fails fast instead of waiting forever. Catch `LockTimeoutException` and retry. Avoid holding locks across user input or external calls. 2. After N retries, give up — return a 409 Conflict to the client with a “try again” message. Use exponential backoff with jitter (randomised delay) to avoid synchronised retries. If the same key always conflicts (a hot row), consider: (a) redesign to reduce contention (batching, queue), (b) switch to pessimistic for that row, (c) use a Redis-based distributed lock for serialization. Live lock is rare; usually indicates a true hot spot.

**RED FLAGS TO AVOID:** - Optimistic on a high-contention row (constant retries → throughput collapse) - Pessimistic on long txs with external calls (locks held for seconds → gridlock) - Not retrying optimistic failures (user sees a 500)

---

## Q-BE-12: Java concurrency — volatile, synchronized, Lock, atomic

**DIFFICULTY:** Hard **CATEGORY TAG:** Backend / Java Concurrency

**ANSWER:** Concurrency primitives in Java, from low to high level:

**volatile**: guarantees visibility (writes by one thread are seen by others) + happens-before ordering. Does NOT provide atomicity for compound ops ( `count++` is read-modify-write — not atomic). Use for: flags, single reference publication. Cheaper than synchronized.

**synchronized (method or block)**: mutex on a single object's monitor. Reentrant (same thread can re-enter). Provides both visibility + atomicity. Coarse but simple. Every `synchronized` access to a field uses the same lock — contention.

**Lock (ReentrantLock)**: explicit lock. More flexible than synchronized: `tryLock()` (non-blocking), `tryLock(timeout)` (timed), `lockInterruptibly()` (interruptible), fair ordering ( `new ReentrantLock(true)` ). Must `finally { lock.unlock(); }` — easy to forget. Use when you need features synchronized lacks.

**ReadWriteLock (ReentrantReadWriteLock)**: separate read + write locks. Multiple readers OR one writer. Speeds up read-heavy code. Watch out for write starvation if you don't use fair.

**StampedLock (Java 8+)**: optimistic read (no lock — check a stamp), pessimistic read, write. Faster than ReentrantReadWriteLock for read-heavy if you can tolerate retry on optimistic failure. Not reentrant — careful.

**Atomic variables (AtomicInteger, AtomicLong, AtomicReference)**: lock-free via CAS. `incrementAndGet`, `compareAndSet`. Use for counters, flags, single references. Faster than synchronized for low contention.

**LongAdder (Java 8+)**: better than `AtomicLong` for high-contention counters — stripes across cells, sums on read. Use for metrics counters.

**Concurrent collections**: - `ConcurrentHashMap`: lock per-segment (Java 7) / CAS + synchronized on bins (Java 8+). Atomic `computeIfAbsent`, `merge`. - `CopyOnWriteArrayList`: copy on write — best for read-heavy, write-rare lists (listeners). - `ConcurrentLinkedQueue` (lock-free) vs `LinkedBlockingQueue` (lock-based, blocking). - `ArrayBlockingQueue` (bounded, blocking) — for producer/consumer.

**CODE/EXAMPLE SNIPPET:**

```

class Counter {
 private final AtomicLong count = new AtomicLong();
 public void inc() { count.incrementAndGet(); }
 public long get() { return count.longValue(); }
}

// ReentrantLock with timeout
private final Lock lock = new ReentrantLock();
public boolean tryOp() {
 try {
 if (lock.tryLock(100, TimeUnit.MILLISECONDS)) {
 try { /* critical section */ } finally { lock.unlock(); }
 return true;
 }
 return false; // timed out
 } catch (InterruptedException e) { Thread.currentThread().interrupt(); return false; }
}

```

**KEY TERMS TO MENTION:** - volatile: visibility + happens-before, NOT atomicity - synchronized: mutex, reentrant, simple, contention - Lock (ReentrantLock): tryLock, timeout, interruptible, fair — must unlock in finally - ReadWriteLock, StampedLock (optimistic read) - Atomic\* (CAS), LongAdder (high-contention counters) - ConcurrentHashMap, CopyOnWriteArrayList, BlockingQueue

**FOLLOW-UP QUESTIONS:** 1. When is volatile not enough? 2. Why is ConcurrentHashMap.get() not lock-free in Java 8+?

**FOLLOW-UP ANSWERS:** 1. For any compound operation (**check-then-act**): `if (flag) { flag = false; }` — volatile gives visibility but not atomicity; two threads can both see `flag=true` and both set it false. Use `AtomicBoolean.compareAndSet(true, false)` or synchronized. Volatile is enough for pure publication of a single value (writer sets, reader reads). 2. `get()` is mostly lock-free (reads a volatile `Node[]` and the bucket head). But on a hash collision (multiple keys same bucket), Java 8 uses a synchronized block on the bin head for `put` (to safely transform a list into a tree). `get` doesn't need locks (volatile reads + final fields). `computeIfAbsent` / `merge` do lock the bin. So `get` is essentially lock-free; writes lock the bin, not the whole map.

**RED FLAGS TO AVOID:** - `volatile` for compound ops (no atomicity) - Forgetting `unlock()` in finally (deadlock) - Using `Collections.synchronizedMap` (locks whole map; use `ConcurrentHashMap`)

---

## Q-BE-13: CompletableFuture — composition, error handling, and gotchas

**DIFFICULTY:** Hard **CATEGORY TAG:** Backend / Java Async

**ANSWER:** `CompletableFuture` is Java's promise/future. Compose async work, handle errors, run in parallel.

**Creation:**

```
CompletableFuture.supplyAsync(() -> fetch(), executor); // runs on executor
CompletableFuture.completedFuture(value); // already done
```

**Composition:** - `thenApply(fn)` — transform value (sync, runs on caller or completion thread). - `thenApplyAsync(fn, executor)` — runs on executor. - `thenCompose(fn)` — flatMap (fn returns a `CompletableFuture` — avoids `CompletableFuture<CompletableFuture<T>>`). - `thenCombine(other, biFn)` — combine two futures' results. - `allOf(f1, f2, ...)` — wait for all (returns CF). - `anyOf(f1, f2, ...)` — wait for first (returns Object). - `exceptionally(fn)` — handle error, return fallback. - `handle((value, error) -> ...)` — handle both. - `whenComplete((value, error) -> ...)` — side effect, doesn't transform. - `orTimeout(5, SECONDS)` — Java 9+, complete exceptionally on timeout. - `completeOnTimeout(value, 5, SECONDS)` — complete with value on timeout.

**CODE/EXAMPLE SNIPPET:**

```
ExecutorService pool = Executors.newFixedThreadPool(8);

CompletableFuture<User> userF = CompletableFuture.supplyAsync(() -> userSvc.get(id),
 pool);
CompletableFuture<List<Order>> ordersF = CompletableFuture.supplyAsync(() ->
 orderSvc.byUser(id), pool);

CompletableFuture<Profile> profile = userF.thenCombine(ordersF, Profile::new)
 .orTimeout(2, TimeUnit.SECONDS)
 .exceptionally(ex -> { log.error("profile fail", ex); return Profile.empty(); });
Profile p = profile.join(); // blocking — only at the end
```

**Pitfalls:** 1. **Default ForkJoinPool:** `supplyAsync(fn)` without an executor runs on the common pool (shared across the JVM, low parallelism = cores-1). Saturate it → all async ops block. Always pass an explicit executor. 2. **Blocking on `join()` / `get()` in a request thread** — defeats async; only block at the very end. 3. **Exception handling:** `thenApply` doesn't catch exceptions in downstream — use `exceptionally` or `handle`. Uncaught exceptions in a `CompletableFuture` are silently swallowed (no log) unless you `whenComplete` log them. 4. **Cancellation:** `cancel(true)` only prevents downstream from running; it doesn't interrupt the upstream task. To actually interrupt, the supplier must check `Thread.currentThread().isInterrupted()`. 5. **Stack traces:** async composition loses caller

stack — debug is harder. Use `thenApply(fn, executor)` and log the correlation ID at each step.

**KEY TERMS TO MENTION:** - `supplyAsync` (with `executor!`), `thenApply` (`map`), `thenCompose` (`flatMap`), `thenCombine` - `allOf` (`wait all`), `anyOf` (`first`), `exceptionally/handle/whenComplete` (errors) - `orTimeout` / `completeOnTimeout` (Java 9+) - Default `ForkJoinPool` is shared — pass explicit `executor` - Don't block mid-chain; block only at the end - Cancellation doesn't interrupt upstream; stack traces lost in `async`

**FOLLOW-UP QUESTIONS:** 1. What's the difference between `thenApply` and `thenCompose`? 2. How do you cancel an in-flight `CompletableFuture` chain?

**FOLLOW-UP ANSWERS:** 1. `thenApply(Function<T, R>)` transforms  $T \rightarrow R$  synchronously (returns `CF<R>`). `thenCompose(Function<T, CF<R>>)` flatMaps  $T \rightarrow CF<R>$  (returns `CF<R>`, not `CF<CF<R>>`). Use `thenCompose` when the transformation itself is `async` (returns a `CompletableFuture`). Otherwise use `thenApply`. Same distinction as `map` vs `flatMap` in `Optional/Stream`. 2. `cf.cancel(true)` completes the future exceptionally with `CancellationException`. Downstream stages see the cancellation + skip. But the `upstream` task (already running on the executor) is NOT interrupted — `cancel` only affects the future's state. To interrupt the upstream, the supplier must cooperatively check `Thread.currentThread().isInterrupted()` or use a `Future` from `executor.submit()` whose `cancel(true)` interrupts the thread. For real cancellation of HTTP calls, use `AbortController`-like patterns (Java's `HttpClient.sendAsync` supports `cancel`).

**RED FLAGS TO AVOID:** - Default executor (shared common pool, low parallelism) - Blocking `join()` mid-chain - No error handler (silently swallowed)

---

## Q-BE-14: ExecutorService vs ForkJoinPool

**DIFFICULTY:** Medium **CATEGORY TAG:** Backend / Java Concurrency

**ANSWER:** **ExecutorService** is the interface; **ForkJoinPool** is a specific implementation optimised for divide-and-conquer workloads.

**ThreadPoolExecutor (typical ExecutorService):** external task submission (`submit` / `execute`). Tasks queued + run by N workers. Each task is opaque — pool doesn't know about sub-tasks. Best for: independent work (HTTP requests, async ops), where each task is a unit.

**ForkJoinPool:** designed for work that splits into smaller sub-tasks (`RecursiveTask` / `RecursiveAction`). Features: - **Work stealing:** idle workers steal from other workers' queues (no global queue bottleneck). - **Async mode** (`new ForkJoinPool(parallelism, ..., AsyncMode=true)`): for event-style tasks (`CompletableFuture`) — tasks never block + are independent. - Each task can `fork()` (submit a sub-task) + `join()` (wait for it). The current thread helps other tasks while waiting (no idle).

**CODE/EXAMPLE SNIPPET:**

```
class SumTask extends RecursiveTask<Long> {
 private final long[] arr; private final int lo, hi;
 protected Long compute() {
 if (hi - lo < THRESHOLD) return sumDirect();
 int mid = (lo + hi) >>> 1;
 SumTask left = new SumTask(arr, lo, mid), right = new SumTask(arr, mid, hi);
 left.fork(); // async
 return right.compute() + left.join(); // work on right while left runs
 }
}
long total = pool.invoke(new SumTask(arr, 0, arr.length));
```

**When to use which:** - Independent tasks (HTTP, IO) → `ThreadPoolExecutor` (or virtual threads). - Recursive divide-and-conquer (sort, sum, graph traversal) → `ForkJoinPool`. - Many short async tasks (`CompletableFuture` without an explicit executor) → `ForkJoinPool.commonPool()` (the default).

**The common pool (`ForkJoinPool.commonPool()`):** shared across the JVM, `parallelism = cores-1`. Used by `parallelStream()`, `CompletableFuture.supplyAsync(fn)` without executor, `Arrays.parallelSort`. Don't block on it — saturate it and every async op in the JVM slows.

**parallelStream gotcha:** `list.parallelStream().map(...)` uses the common pool. If you map to a blocking IO call, you block the whole JVM's async backbone. Use a dedicated `ForkJoinPool` or `ThreadPoolExecutor` for blocking work.

**KEY TERMS TO MENTION:** - `ExecutorService`: interface; `ThreadPoolExecutor` for independent tasks - `ForkJoinPool`: work stealing, `RecursiveTask/Action`, `fork/join` - Common pool: shared, `parallelism=cores-1`, used by `parallelStream` + default CF executor - Don't

block the common pool (saturates JVM-wide async) - Use ForkJoinPool for divide-and-conquer, ThreadPoolExecutor for independent IO

**FOLLOW-UP QUESTIONS:** 1. What's work stealing? 2. Why does parallelStream have contention issues?

**FOLLOW-UP ANSWERS:** 1. Each worker has its own deque of tasks. Workers push + pop from their own deque (LIFO, cache-friendly). When a worker's deque is empty, it steals from the [back](#) of another worker's deque (FIFO from the victim's view). This balances load without a central queue bottleneck. Tasks are mostly local (no contention); only stealing touches other workers' deques. Result: with N workers, throughput scales near-linearly even with uneven task sizes. 2. `parallelStream` uses the common pool. If your pipeline does: (a) blocking IO — blocks common pool workers, slowing the whole JVM; (b) CPU work — competes with other common-pool users (CompletableFuture without explicit executor). If two parallelStreams run simultaneously, they share the same pool → each gets half the parallelism. Solution: use a dedicated pool for CPU-bound parallelStream work (wrap in `pool.submit(() -> list.parallelStream())`); don't do IO in parallelStream.

**RED FLAGS TO AVOID:** - parallelStream for IO (blocks common pool) - Default ExecutorService with unbounded queue (OOM) - ForkJoinPool for independent IO tasks (work-stealing doesn't help)

---

## Q-BE-15: Virtual threads (Java 21) — when, how, and the gotchas

**DIFFICULTY:** Hard **CATEGORY TAG:** Backend / Java 21

**ANSWER:** Virtual threads are lightweight threads scheduled by the JVM, not the OS. Hundreds of thousands can run in one JVM. They're cheap (no kernel thread, ~few KB stack that grows/shrinks). When they block on IO/JNI/synchronized wait, the JVM parks them and frees the carrier (OS) thread for another virtual thread.

**Why they matter:** the IO-bound thread-per-request model finally scales. Instead of 200 OS threads handling 200 concurrent requests, you have 200K virtual threads, each in its own logical request — no thread pool tuning, no reactive programming complexity for IO.

**Usage:**

```
// Java 21+: per-request virtual threads in Tomcat
server.tomcat.threads.virtual: true // Spring Boot 3.2+

// Or manually
Thread.startVirtualThread(() -> handle(req));
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
 executor.submit(() -> fetch(url));
}
```

**Carrier threads:** virtual threads run on carrier threads (a small ForkJoinPool, parallelism = cores). Many virtual threads share few carriers. Blocking a virtual thread doesn't block a carrier (except when it can't unmount — see pitfalls).

**When virtual threads shine:** - IO-bound request handling (HTTP, DB, downstream calls) — 100K concurrent requests on a small JVM - Code that's already written in synchronous style (no CompletableFuture/Reactor rewrite) - Per-request isolation + cheap context switching

**When they DON'T help:** - CPU-bound work (still limited by cores; virtual threads don't add parallelism) - Code that holds OS-thread resources across blocking calls ( `synchronized` blocks holding a monitor + blocking inside; native frames — JNI)

**Pitfalls (the `synchronized` pinning issue):** - If a virtual thread blocks `inside` a `synchronized` block, the JVM can't unmount it → the carrier thread is pinned → blocks another virtual thread from using that carrier. Fix: replace `synchronized` with `ReentrantLock` in code paths used by virtual threads (Java 21); Java 24 fixes the pinning. - Stack traces are deeper (carrier frame + virtual frames). - `ThreadLocal` per virtual thread = a lot of memory if you have 100K virtual threads. Use `ScopedValue` (Java 21 preview) instead.

**Don't pool virtual threads** — they're cheap to create. Each task = a new virtual thread. Pooling defeats the purpose.

**Spring Boot 3.2+ virtual threads:** set `spring.threads.virtual.enabled=true` — Tomcat uses virtual threads per request, `@Async` executors use virtual threads, `@Scheduled` too. Massive concurrency win for IO-heavy apps.

**KEY TERMS TO MENTION:** - Virtual threads: JVM-scheduled, lightweight, ~100K per JVM, cheap stack - Carriers: small ForkJoinPool, parallelism=cores - Block on IO → unmount, free carrier for another VT - Great for IO-bound request handling (no reactive rewrite needed) - Don't pool VTs; use `ReentrantLock` not `synchronized` (pinning); use `ScopedValue` not `ThreadLocal` - Spring Boot 3.2: `spring.threads.virtual.enabled=true`

**FOLLOW-UP QUESTIONS:** 1. What is “thread pinning” and how do you detect it? 2. Should virtual threads replace reactive (Reactor/WebFlux)?

**FOLLOW-UP ANSWERS:** 1. Pinning = a virtual thread can't unmount from its carrier (because it holds something the carrier “owns”). Causes: (a) `synchronized` block + blocking inside (the monitor is owned by the carrier); (b) JNI native frames; (c) JVM internals. Fix: use `ReentrantLock` (Java 24 fixes `synchronized`). Detect: `-Djdk.tracePinnedThreads=full` logs when a virtual thread pins a carrier — run with load + see which paths pin. 2. For most IO-bound apps, yes — virtual threads give you the concurrency of reactive without the programming model complexity (no Mono/Flux, no backpressure operators, no `block()` caveats). Reactive is still better for: streaming pipelines with complex composition (Kafka Streams-style), when you need backpressure (rate matching between producer and consumer), or you've already invested heavily. New code: start with virtual threads; add reactive only if you hit a backpressure/composition ceiling.

**RED FLAGS TO AVOID:** - Pooling virtual threads (they're meant to be created per-task) - `synchronized` in VT paths (pinning) - Heavy `ThreadLocal` use per VT (memory) - Expecting VTs to speed up CPU-bound work (they don't)

---

## Q-BE-16: ConcurrentHashMap internals — how it's thread-safe and fast

**DIFFICULTY:** Hard **CATEGORY TAG:** Backend / Java Internals

**ANSWER:** `ConcurrentHashMap` (CHM) is the lock-free-ish concurrent hash map. Iteration: Java 7 used segments (16 `ReentrantLocks`, each a mini-map); Java 8+ uses CAS + per-bin synchronized.

**Java 8+ structure:** - `volatile Node[] table` — the bucket array. - Each bucket head is a `Node` (key, value, hash, next). - On `put`: hash the key, find the bin, CAS-insert into the bin head or `synchronized (head)` if the bin has nodes (for safety when transforming a list into a tree). - On hash collisions beyond threshold (8): list → red-black tree for  $O(\log n)$  worst case.

**Reads are lock-free:** `get` reads volatile fields + uses final fields (safe publication). No locks.

**Writes:** - Empty bin: CAS the bin head (single op, no lock). - Non-empty bin: `synchronized (bin head)` for insertion (one bin at a time — minimal contention). - Resize: thread helping + transfer (each thread transfers a stride of bins).

**Atomic operations:**

```
map.computeIfAbsent(key, k -> expensive(k)); // atomic — only one thread computes
map.putIfAbsent(key, val); // atomic
map.replace(key, oldVal, newVal); // CAS-like
```

These are atomic across the map. No external lock needed. Beware: `computeIfAbsent` holds the bin lock during the lambda — if the lambda blocks (e.g. calls back into the map → deadlock) or runs long, you block other writers on that bin.

**Iteration:** weakly consistent — sees entries that existed at iterator creation, may or may not see concurrent modifications, never throws `ConcurrentModificationException`. Safe for use during concurrent updates.

**Sizing:** `size()` is approximate — sums a `counterCells` array (avoids contention on a single counter). Not exact under concurrency; if you need exact, lock the map.

**Load factor + capacity:** `0.75f` default, initial capacity 16. Pre-size to avoid early resizing: `new ConcurrentHashMap<>(expectedSize / 0.75 + 1)`.

**Null keys/values:** NOT allowed (unlike `HashMap`). NPE on `put(null, ...)`. Reason: ambiguity in `get` returning null (could mean “no key” or “value is null”) in a concurrent setting where the state could change between `containsKey` and `get`.

**KEY TERMS TO MENTION:** - Java 7: 16 segments (`ReentrantLock` per segment) - Java 8+: CAS for empty bins, synchronized on bin head for non-empty - Lock-free reads (volatile + final fields) - Atomic ops: `computeIfAbsent` (holds bin lock during lambda — keep it short) -

Weakly consistent iteration (no CME) - size() approximate (counterCells) - No null keys/values (concurrent ambiguity)

**FOLLOW-UP QUESTIONS:** 1. Why does computeIfAbsent deadlock if the lambda calls back into the same map? 2. When does the bin become a tree?

**FOLLOW-UP ANSWERS:** 1. `computeIfAbsent` holds the bin's `synchronized` lock. If the lambda does `map.put(someKey, ...)` where `someKey` hashes to the same bin, you try to re-acquire the same monitor. ReentrantLock is reentrant for the `same thread`, so the put succeeds — but if the lambda does it on a `different` key colliding with another bin's locked-by-another-thread, you wait. Worse: if the lambda triggers a resize, the resize needs locks on all bins → deadlock. Rule: keep the lambda fast + side-effect-free, no calls back into the same map. 2. When a bin has  $\geq 8$  nodes (`TREEIFY_THRESHOLD`) AND the table is  $\geq 64$  (`MIN_TREEIFY_CAPACITY`). If the table is small, it grows instead (a small table doesn't benefit from tree overhead). Once a tree, it untreeifies back to a list when  $\leq 6$  nodes (`UNTREEIFY_THRESHOLD`). Tree gives  $O(\log n)$  worst case for pathological hash collisions (including deliberate DoS).

**RED FLAGS TO AVOID:** - `computeIfAbsent` with a slow/blocking lambda (bin lock held) - Expecting exact size() under concurrency - Using HashMap under concurrency (subtle corruption; use CHM)

---

## Q-BE-17: Deadlocks — detection, prevention, and recovery

**DIFFICULTY:** Hard **CATEGORY TAG:** Backend / Concurrency

**ANSWER:** A deadlock: T1 holds Lock A, waits for Lock B; T2 holds Lock B, waits for Lock A. Neither progresses. Four conditions (Coffman): mutual exclusion, hold-and-wait, no preemption, circular wait. Break any to prevent.

**Detection:** - `jstack <pid>` — prints “Found one Java-level deadlock” with the cycle + stack traces. - `jcmd <pid> Thread.print` - Java 10+: `ManagementFactory.getThreadMXBean().findDeadlockedThreads()` (programmatic — can expose via actuator). - APM tools (Datadog) auto-detect thread-blocked cycles.

**Prevention:** 1. **Lock ordering** — always acquire locks in the same order across all threads (e.g. by ID). Breaks “circular wait.” 2. **Lock timeout** — `tryLock(timeout)` instead of `lock()`. If timeout, release + retry (backoff). Breaks “no preemption.” 3. **Single lock** — coarsen to one lock for the whole operation (kills concurrency but eliminates deadlock). 4. **Lock-free data structures** — `AtomicReference`, `ConcurrentHashMap` (no locks in the user’s sense). 5. **Avoid holding locks across calls** — don’t call external code (callbacks, IO) while holding a lock.

**Recovery:** - Throw a `LockTimeoutException` to the caller (don’t retry forever — backoff + give up after N). - Log + metric (deadlock count → page). - Some systems forcibly kill one transaction (database does this — picks a “victim” to abort).

**Live lock:** threads keep changing state but make no progress (e.g. both back off, then both retry simultaneously). Use randomised backoff (jitter) to break synchrony.

### CODE/EXAMPLE SNIPPET:

```
// Bad: T1 locks A then B; T2 locks B then A → deadlock
public void transfer1(Account a, Account b) { synchronized (a) { synchronized (b)
 { ... } } }
public void transfer2(Account a, Account b) { synchronized (b) { synchronized (a)
 { ... } } }

// Good: consistent ordering by ID
public void transfer(Account a, Account b) {
 Account first = a.id < b.id ? a : b, second = a.id < b.id ? b : a;
 synchronized (first) { synchronized (second) { ... } }
}
```

**Database deadlocks:** detected by the DB (PG: `ERROR: deadlock detected`). The DB picks a victim (one tx aborted with `SQLSTATE 40P01`). Catch + retry. Common in row-by-row updates with non-deterministic order (batch update → sort first).

**KEY TERMS TO MENTION:** - Coffman 4 conditions: mutual exclusion, hold-and-wait, no preemption, circular wait - Detect: `jstack` / `jcmd Thread.print` / `ThreadMXBean` / APM -

Prevent: lock ordering (by ID), tryLock + timeout, lock-free, avoid holding locks across calls - Recover: timeout exception, retry with backoff, abort victim (DB does this) - Live lock: randomised backoff - DB deadlocks: SQLSTATE 40P01, catch + retry

**FOLLOW-UP QUESTIONS:** 1. How do you detect a deadlock in production? 2. What if lock ordering is impossible (the locks are dynamic)?

**FOLLOW-UP ANSWERS:** 1. `jstack <pid>` shows the deadlock + cycle. In a long-running service, expose `ThreadMXBean.findDeadlockedThreads()` via Spring Actuator (`/actuator/threaddump` includes deadlock info). APM tools (Datadog, New Relic) detect blocked threads. Alert on “deadlock detected” events. For DB deadlocks, log the SQLSTATE + count metric + page if rate spikes. 2. Use `tryLock(timeout)` on all locks + retry with backoff if you can’t acquire all within the timeout. Or use a “back-off and retry” pattern: acquire first lock with `tryLock`; if second `tryLock` fails, release first, sleep random, retry. Or use a single coordinator (Redis Redlock, Zookeeper) for distributed cross-process locks. Or restructure to avoid the multi-lock pattern entirely (use a single transactional outbox, or a saga).

**RED FLAGS TO AVOID:** - Nested `synchronized` blocks without ordering - Holding a lock across an external call (DB, HTTP) - Not catching SQLSTATE 40P01 + retrying (deadlocks happen; plan for them)

---

## Q-BE-18: Java Memory Model — happens-before and visibility

**DIFFICULTY:** Expert **CATEGORY TAG:** Backend / JVM Internals

**ANSWER:** The JMM (JSR 133) defines when one thread's writes are visible to another. Without it, compilers + CPUs can reorder instructions for performance, breaking naive multi-threaded assumptions.

**Happens-before:** a partial order on actions. If action A happens-before action B, then A's effects (writes, locks) are visible to B. Establishes happens-before via: 1. **Program order** — within a single thread, statements happen-before later statements (intra-thread). 2. **Monitor lock** — `synchronized` unlock happens-before subsequent lock of the same monitor (cross-thread). 3. **volatile** — a write to a volatile field happens-before subsequent reads of that field. 4. **Thread.start** — `start()` happens-before any action in the started thread. 5. **Thread.join** — actions in the joined thread happen-before `join()` returns. 6. **Final field** — writes to final fields in a constructor happen-before any other thread sees the object reference (safe publication). 7. **Atomic + Concurrent collections** — internal happens-before for atomic ops.

**What it prevents:** - Reordering: `x = 1; ready = true;` could be reordered to `ready = true; x = 1;` by the compiler/CPU if no happens-before relation. Mark `ready` volatile → write to `x` happens-before write to `ready` (the JMM links volatile to all prior writes). - Visibility: without volatile/synchronized, thread A's write to `x` may sit in A's cache and never be flushed to main memory; thread B reads stale.

**Practical implications:** - Use `volatile` for flags (visibility + ordering, no atomicity). - Use `synchronized` or `Atomic*` for compound ops. - Don't use `final` only for immutability — it also provides safe publication (a half-constructed object reference can leak without `final`). - Double-checked locking without volatile is broken (the classic Java bug pre-2004):

```
// BROKEN without volatile
private static Singleton instance;
public static Singleton get() {
 if (instance == null) { // read 1
 synchronized (Singleton.class) {
 if (instance == null) instance = new Singleton(); // write could reorder
 }
 }
 return instance;
}
// FIX: volatile
private static volatile Singleton instance;
```

**CODE/EXAMPLE SNIPPET:**

```

class VolatileExample {
 private volatile boolean ready = false; // volatile – visible + ordered
 private int data; // non-volatile – but transitively
 visible via ready

 public void producer() { data = 42; ready = true; } // write to data HB write to
 ready
 public void consumer() { if (ready) use(data); } // read of ready HB read of
 data
}

```

**KEY TERMS TO MENTION:** - JMM defines visibility + ordering across threads - Happens-before: program order, monitor lock, volatile, Thread.start/join, final, atomic - volatile: visibility + HB ordering, no atomicity - final fields = safe publication (prevents half-constructed leaks) - Double-checked locking requires volatile (classic bug) - Atomic + Concurrent collections: internal HB guarantees

**FOLLOW-UP QUESTIONS:** 1. Why does final give safe publication? 2. What's the difference between volatile and synchronized?

**FOLLOW-UP ANSWERS:** 1. The JMM guarantees that writes to final fields in a constructor happen-before the constructor completes + the reference is published. So any thread that sees the reference via a data race (no synchronization) is still guaranteed to see the final field's value (not the default 0/null). Without final, the compiler/CPU could publish the reference before the constructor finishes writing fields → another thread sees partially-constructed state. Final fixes this for immutable parts of objects. 2. `volatile` provides visibility + ordering (happens-before) for a single variable, no atomicity for compound ops, no mutual exclusion, very cheap. `synchronized` provides visibility + atomicity (mutex) for a block of code, much heavier (lock acquisition). Use volatile for flags/single-value publication; synchronized (or Lock) for multi-step critical sections. For counters, `AtomicInteger` (CAS — has HB + atomicity) is best.

**RED FLAGS TO AVOID:** - `volatile` for compound ops (no atomicity) - DCL without volatile (broken) - Believing `final` is only for const-correctness (it's also for thread safety)

---

## Q-BE-19: Spring Boot Actuator + health checks

**DIFFICULTY:** Medium **CATEGORY TAG:** Backend / Spring Boot

**ANSWER:** Spring Boot Actuator adds production-grade endpoints: health, metrics, info, env, loggers, threaddump, heapdump, prometheus. Most are disabled or restricted by default.

**Key endpoints:** - `/actuator/health` — aggregated health (DB, Redis, disk, custom). Used by load balancers + k8s probes. - `/actuator/info` — app metadata (git commit, build info, custom). - `/actuator/metrics` — Micrometer metrics (List → metric → tags → datapoints). - `/actuator/prometheus` — Prometheus scrape format. - `/actuator/loggers` — view + change log levels at runtime. - `/actuator/threaddump` — thread state (great for deadlocks). - `/actuator/heapdump` — full heap dump (heavy — restrict). - `/actuator/env` — env vars + config (sensitive — mask secrets). - `/actuator/conditions` — auto-config conditions (debugging).

**Configuration:**

```
management:
 endpoints:
 web:
 exposure:
 include: health, info, metrics, prometheus, loggers, threaddump
 exclude: heapdump
 endpoint:
 health:
 show-details: when-authorized # don't leak DB host to public
 probes:
 enabled: true # k8s liveness/readiness
 health:
 defaults:
 enabled: true
```

**Custom health indicator:**

```
@Component
public class DownstreamHealthIndicator implements HealthIndicator {
 @Override
 public Health health() {
 if (downstream.ping()) return Health.up().withDetail("latency", 30).build();
 return Health.down().withDetail("error", "downstream timeout").build();
 }
}
```

Aggregated into `/actuator/health` — overall DOWN if any indicator is down.

**K8s probes:** - `liveness` — `/actuator/health/liveness` — is the app alive (restart if down)? Don't include downstream checks (a downstream blip shouldn't restart you). - `readiness` —

`/actuator/health/readiness` — can the app take traffic (route away if down)? OK to include downstream + DB.

**Security:** actuator endpoints should NEVER be public. Restrict via Spring Security (`/actuator/**` requires `ACTUATOR` role), bind to a separate management port (`management.server.port=8081`), or only on localhost.

**KEY TERMS TO MENTION:** - Endpoints: health, info, metrics, prometheus, loggers, threaddump, heapdump, env, conditions - Custom HealthIndicator → aggregated into `/health` - K8s: liveness (restart?) vs readiness (route traffic?) — different concerns - Security: never public; separate management port or restrict role - show-details: when-authorized (don't leak DB host to public)

**FOLLOW-UP QUESTIONS:** 1. Why separate liveness from readiness? 2. How do you avoid cascade failures from health checks?

**FOLLOW-UP ANSWERS:** 1. Liveness = “is the JVM running + can I recover?” Restarting fixes stuck threads, OOMs, deadlocks. If a downstream is down, restarting doesn't help → don't include downstream in liveness. Readiness = “can I serve traffic?” If downstream is down and I depend on it for every request, I can't serve → readiness down, route away (but don't restart). Mixing them (using `/health` for both) causes cascade restarts during a downstream outage — exactly wrong. 2. (a) Health checks should be fast (timeout < 5s) + cached (don't re-check downstream every poll — check every 10s, cache the result). (b) Don't make health checks depend on downstream (or include them only in readiness with a short timeout). (c) Use circuit breakers in your real request path — if downstream is down, your circuit is open and your readiness should reflect that, not via a synchronous downstream ping. (d) Avoid thundering herd: add jitter to k8s probe interval.

**RED FLAGS TO AVOID:** - Public `/actuator/env` (secrets leak) - `/health` including downstream for liveness (cascade restarts) - Slow health checks (k8s kills the pod)

## Q-BE-20: Spring Security filter chain + JWT

**DIFFICULTY:** Hard **CATEGORY TAG:** Backend / Spring Security

**ANSWER:** Spring Security is built on a chain of servlet `Filter`s. Each request passes through them in order: security headers → CSRF → CORS → authentication → authorization → exception handling → controller.

**Filter chain (typical order):** 1. `SecurityContextHolderFilter` — sets up the `SecurityContext` (per-request). 2. `HeaderWriterFilter` — security headers (CSP, HSTS, X-Frame-Options). 3. `CorsFilter` — CORS. 4. `CsrfFilter` — CSRF token validation. 5. `UsernamePasswordAuthenticationFilter` — form login (if matched). 6. `JWT authentication filter` (custom, before `UsernamePasswordAuthenticationFilter`). 7. `AuthorizationFilter` (Spring Security 6+, replaces `FilterSecurityInterceptor`) — enforces authorization rules. 8. `ExceptionTranslationFilter` — translates `AccessDenied`/`AuthenticationException` to 401/403.

**JWT filter (custom):**

```
@Component
public class JwtAuthFilter extends OncePerRequestFilter {
 @Override
 protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,
 FilterChain chain)
 throws ServletException, IOException {
 String token = resolveToken(req);
 if (token != null && jwtService.validate(token)) {
 var auth = jwtService.toAuthentication(token); // UsernamePasswordAuthToken
 SecurityContextHolder.getContext().setAuthentication(auth);
 }
 chain.doFilter(req, res);
 }
}

// SecurityConfig
@Bean
SecurityFilterChain api(HttpSecurity http) throws Exception {
 http.csrf(csrf -> csrf.disable())
 .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
 .authorizeHttpRequests(a -> a
 .requestMatchers("/api/auth/**", "/actuator/health").permitAll()
 .requestMatchers("/api/admin/**").hasRole("ADMIN")
 .anyRequest().authenticated())
 .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class)
 .exceptionHandling(e -> e
 .authenticationEntryPoint((req, res, ex) -> res.sendError(401))
 .accessDeniedHandler((req, res, ex) -> res.sendError(403)));
 return http.build();
}
```

**Key concepts:** - `SecurityContext` — per-request auth info (held in `SecurityContextHolder` via `ThreadLocal`). Cleared after the request. - `Authentication` — principal (user), credentials, authorities. - `GrantedAuthority` — role/permission (`ROLE_ADMIN`, `read:write`). - `UserDetailsService` — load user by username (your DB lookup). - `PasswordEncoder` — BCrypt for password hashing. - Stateless (no session) for JWT APIs — `SessionCreationPolicy.STATELESS`.

**Common pitfalls:** - Forgetting `SecurityContextHolder.clearContext()` after async — leaks auth across threads. Use `DelegatingSecurityContextCallable` for async. - CSRF disabled for non-browser APIs (correct) — but don't disable for cookie-based session APIs (need CSRF). - Putting `permitAll()` on a broad pattern that accidentally allows `/api/admin/**`.

**KEY TERMS TO MENTION:** - Filter chain (security headers → CSRF → CORS → auth → authz → controller) - `SecurityContext` (`ThreadLocal`, cleared per request) - `Authentication`, `GrantedAuthority`, `UserDetailsService`, `PasswordEncoder` (BCrypt) - Custom `JwtAuthFilter`, added before `UsernamePasswordAuthenticationFilter` - `SessionCreationPolicy.STATELESS` for JWT - Spring Security 6 lambda DSL, `AuthorizationFilter`

**FOLLOW-UP QUESTIONS:** 1. How do you handle method-level security? 2. Why disable CSRF for JWT APIs?

**FOLLOW-UP ANSWERS:** 1. `@PreAuthorize("hasRole('ADMIN')")` / `@PreAuthorize("@authz.canAccess(principal, #id)")` on methods. Backed by AOP (`@EnableMethodSecurity`). SpEL expressions: `hasAuthority(...)`, `hasAnyRole(...)`, custom beans. Use for fine-grained checks (per-resource ownership); use the filter chain for coarse URL-pattern rules. Both are common — URL patterns for endpoints, method-level for object-level rules. 2. CSRF protects cookie-based auth from cross-site POSTs (browser auto-sends the cookie). JWT in the `Authorization` header isn't auto-sent by the browser (no cookie) — cross-site attackers can't forge the header. So CSRF doesn't apply. If you put the JWT in a cookie (e.g. for SSR-friendly auth), CSRF applies and you need a CSRF token or `SameSite=Strict` cookie.

**RED FLAGS TO AVOID:** - Putting JWT in `localStorage` (XSS exfiltration) — use `httpOnly` cookie - `permitAll()` on broad patterns - Enabling sessions with JWT (stateless intent + stateful impl mismatch)

---

## Q-BE-21: REST API design — versioning, pagination, errors

**DIFFICULTY:** Medium **CATEGORY TAG:** Backend / REST

**ANSWER:** REST API design has de-facto conventions. There's no single right answer; consistency + clarity matter most.

**Versioning:** - **URI versioning:** `/v1/users`, `/v2/users`. Easy, visible, caches per-version. Most common. Breaks if clients hard-code URLs. - **Header versioning:** `Accept: application/vnd.example.v1+json`. Clean URLs, no version in path. Hard to test in browser. - **Query param:** `?version=1`. Easy but loses cache benefits. - **No versioning:** backward-compatible only (add fields, never remove). Works for internal APIs with disciplined consumers.

**Pagination:** - **Offset/limit:** `?offset=100&limit=20`. Simple. Bad for deep pagination (OFFSET 1M scans 1M rows). Use for shallow lists. - **Cursor:** `?cursor=base64(timestamp,id)`. Stable under inserts. Good for "load more" UIs. Hard to jump to page 50. - **Page/pageSize:** `?page=5&size=20`. Common but same deep-pagination problem.

```
GET /api/orders?cursor=eyJ0cyI6MTYwfQ&limit=20
200 OK
Link: <...?cursor=eyJ0cyI6MTgwfwQ&limit=20>; rel="next"
```

**Errors:** consistent envelope.

```
{
 "error": {
 "code": "VALIDATION_ERROR",
 "message": "email must be a valid email address",
 "field": "email",
 "traceId": "abc-123"
 }
}
```

- Use correct HTTP status (400 for client, 401 unauth, 403 forbidden, 404 not found, 409 conflict, 422 unprocessable, 429 rate limit, 500 server).
- Don't use 200 OK with `{ "error": ... }` (violates HTTP semantics; clients can't distinguish).
- Include a `traceId` for debugging (log correlation).

**Other conventions:** - Nouns, not verbs: `/users` not `/getUsers`. HTTP method is the verb (GET/POST/PUT/DELETE/PATCH). - Plural nouns: `/users`, `/orders`. - Sub-resources: `/users/123/orders` (orders for user 123). - Filter via query: `?status=active&createdAfter=2024-01-01`. - HTTP methods: GET (read, idempotent, cacheable), POST (create, not idempotent), PUT (full replace, idempotent), PATCH (partial

update), DELETE (idempotent). - Idempotency-Key header on POST for safe retries. - Rate limit headers: `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `Retry-After`.

**KEY TERMS TO MENTION:** - Versioning: URI (/v1), header, query — URI most common - Pagination: offset/limit (deep paginate bad), cursor (stable, no jump), page (bad deep) - Errors: envelope + correct status + traceId; not 200+error - Nouns not verbs; plural; sub-resources; HTTP method is the verb - Idempotency-Key, RateLimit-\* headers

**FOLLOW-UP QUESTIONS:** 1. When would you use POST to read (instead of GET)? 2. What's idempotency and which methods are idempotent?

**FOLLOW-UP ANSWERS:** 1. When the query is too complex for URL + query params (long filters, complex JSON, multi-layer criteria that exceed URL length limits or escape encoding). The pattern: `POST /api/orders/search { filters: {...}, sort: {...}, pagination: {...} }`. Common in Elasticsearch-style search APIs. Trade-off: not cacheable by default (need explicit Cache-Control + a body hash). GraphQL also solves this (one POST endpoint with arbitrary queries). 2. Idempotent = calling N times has the same effect as calling once. GET, PUT, DELETE are idempotent. POST is not (each call creates a new resource). PATCH is sometimes (depends on operation — `{"op": "set", "v": 5}` is; `{"op": "increment"}` isn't). Idempotency matters for retries — if a network blip causes the client to retry, an idempotent endpoint is safe. For non-idempotent POST (e.g. payment), use an `Idempotency-Key` header so the server can dedupe retries.

**RED FLAGS TO AVOID:** - 200 OK + error body (broken status) - POST for reads everywhere (defeats caching) - Verbs in URL ( `/getUsers`, `/createOrder` ) - No pagination on list endpoints (returning 100K rows)

---

## Q-BE-22: Idempotency in REST APIs — implementation

**DIFFICULTY:** Hard **CATEGORY TAG:** Backend / API Design

**ANSWER:** Idempotency: calling the API N times produces the same result as once. Critical for safe retries (network blips, double-clicks, payment processors retrying webhooks).

**Idempotency-Key header:** client sends a unique key per logical operation ( `Idempotency-Key: uuid-1234` ). Server stores `key → result` (response body, status) for 24h. On retry with the same key, return the stored result (don't re-execute).

**Stripe's pattern (the industry standard):**

```
POST /v1/charges
Idempotency-Key: ABC123
{ "amount": 1000, "currency": "usd", "source": "tok_visa" }

Retry (network blip)
POST /v1/charges
Idempotency-Key: ABC123
{ "amount": 1000, "currency": "usd", "source": "tok_visa" }

Server: same key + same params → return stored charge, don't create a second
```

**Implementation:**

```
@PostMapping("/charges")
public ResponseEntity<?> charge(@RequestBody ChargeReq req,
 @RequestHeader("Idempotency-Key") String key) {
 Optional<CachedResult> cached = idempotencyStore.get(key);
 if (cached.isPresent()) {
 if (cached.get().paramsMatch(req)) return cached.get().response();
 else return ResponseEntity.status(409).body("idempotency key reused with
 different params");
 }
 ResponseEntity<?> result = doCharge(req); // execute
 idempotencyStore.put(key, req, result, Duration.ofHours(24));
 return result;
}
```

**Storage:** - Redis (key: TTL 24h, value: hash of params + response). Distributed, fast. - DB table (idempotency\_key PK, params\_hash, response, expires\_at). Persistent but slower.

**Edge cases:** - Same key, different params → 409 (idempotency key reused with different body) — protects against key collision attacks. - Same key, in-flight request (started, not done) → return 409 ("request in flight") or hold the connection + return when done. Stripe returns 409; some return a wait. - Key TTL too short → retries after TTL execute twice. Match

TTL to your client's max retry window (24h is standard). - Don't reuse keys across different operations (key is scoped to the endpoint).

**What's idempotent by HTTP method:** - GET, PUT, DELETE: idempotent (re-send-safe). PUT (full replace) + DELETE (delete by ID) — same result every time. - POST: not idempotent (creates new). Use idempotency key for safe retries. - PATCH: depends — replacing a field is idempotent; incrementing isn't.

**Real-world:** Stripe, Square, Twilio, Adyen all use Idempotency-Key for payment + mutation APIs. It's table stakes for any payment integration.

**KEY TERMS TO MENTION:** - Idempotency = N calls = 1 call effect - Idempotency-Key header, server stores key→result for 24h - Stripe's pattern (industry standard) - Storage: Redis (TTL) or DB table - Same key + different params → 409 - POST needs it; GET/PUT/DELETE are idempotent by HTTP method - Real-world: Stripe, Square, Twilio all require it for payments

**FOLLOW-UP QUESTIONS:** 1. How do you handle a retry that arrives during the original request's processing? 2. What if the client reuses an idempotency key for different operations?

**FOLLOW-UP ANSWERS:** 1. Two strategies: (a) return 409 Conflict ("request in flight") — client must retry later. Simple, works for short ops. (b) Block the retry until the original completes — hold the connection, return the original's result. More user-friendly (client sees the right answer) but holds a thread longer. Stripe uses (a). For your own APIs, (b) is friendlier if you have headroom. Implementation: store "in-flight" status under the key on first sight; second request waits on a latch released by the first. 2. Reject with 409. The key is scoped to one logical operation. Reusing it for a different operation (different endpoint, or same endpoint with different body) is a client bug or an attack (key squatting). Compare the params hash; if it differs from the stored one, 409. Don't silently overwrite — that hides a bug. Document the scoping in your API docs.

**RED FLAGS TO AVOID:** - Accepting idempotency key without checking params (key squatting attacks) - TTL too short (retries after TTL execute twice) - Storing the result forever (storage grows unbounded)

---

## Q-BE-23: REST vs GraphQL vs gRPC — when to use which

**DIFFICULTY:** Medium **CATEGORY TAG:** Backend / API

**ANSWER:** Three API styles, different trade-offs.

**REST:** - Resource-oriented, HTTP verbs, JSON usually. - Cacheable (HTTP caching, CDNs). - Browser-friendly, well-tooled, easy to learn. - Over-fetching (client gets fields it doesn't need) + under-fetching (client makes N calls to assemble a view). - Schema optional (OpenAPI documents it). - Best for: CRUD, public APIs, browser apps, anything where caching + simplicity matter.

**GraphQL:** - Single endpoint, client specifies the exact fields. - No over/under-fetching — client gets exactly what it asks for. - Schema + types required; introspection. - Harder to cache (POST body; need persisted queries / cache layer). - N+1 problem at the server (DataLoader to batch). - Best for: mobile clients (low bandwidth), aggregated dashboards (one call), rapidly-evolving client needs. - Bad for: simple CRUD, file uploads (multipart awkward), binary streaming.

**gRPC:** - HTTP/2, Protocol Buffers (binary, schema-first). - Strict schema, codegen for many languages. - Bidirectional streaming (server, client, both). - ~7-10x smaller payloads than JSON; faster parsing. - Not browser-friendly (needs grpc-web proxy). Not human-readable. - Best for: service-to-service internal comms, low-latency, streaming, polyglot microservices. - Bad for: public APIs (browser), human debugging, ad-hoc curl.

**Decision matrix:** | Need | Pick | |—|—| | Public API, browser, simple CRUD | REST | | Mobile clients, dashboard with many views | GraphQL | | Internal microservice comms | gRPC | | Real-time streaming | gRPC (bidirectional) or WebSocket (browser) | | File uploads | REST (multipart) or GraphQL multipart | | Need strict schema + codegen | gRPC or GraphQL | | Backward compat critical | REST (add fields) or GraphQL (deprecated fields) or protobuf (optional fields) |

**Hybrid patterns:** REST for external + gRPC for internal (most common at scale). “Backend for Frontend” (BFF) — GraphQL gateway aggregating REST/gRPC services for a specific client.

**CODE/EXAMPLE SNIPPET (gRPC proto):**

```
syntax = "proto3";
service OrderService {
 rpc CreateOrder(CreateOrderReq) returns (Order);
 rpc StreamOrders(StreamReq) returns (stream Order);
}
message Order { int64 id = 1; string status = 2; }
```

**KEY TERMS TO MENTION:** - REST: resource + verbs + JSON, cacheable, over/under-fetch, OpenAPI optional - GraphQL: single endpoint, exact fields, schema required, DataLoader for

N+1, hard to cache - gRPC: HTTP/2 + protobuf, strict schema + codegen, streaming, internal  
 S2S - Hybrid: REST external + gRPC internal; BFF pattern - Browser + caching → REST;  
 Mobile + aggregated → GraphQL; S2S + low latency → gRPC

**FOLLOW-UP QUESTIONS:** 1. How do you version a GraphQL schema? 2. Why doesn't gRPC work directly in browsers?

**FOLLOW-UP ANSWERS:** 1. Additive changes only — add fields/enum values/types (optional, default-valued). Never remove (breaks clients). Deprecate via `@deprecated` directive; remove after all clients migrate. For breaking changes: a new “schema version” via different endpoint or a custom header. GraphQL's design favors additive evolution — the schema is the contract, and clients only ask for what they need, so adding fields never breaks them. 2. Browsers can't open raw HTTP/2 with trailers (gRPC uses trailers for status). Also, browsers restrict custom Content-Type + require CORS. `grpc-web` is a proxy (Envoy) that translates between `grpc-web` (HTTP/1.1 + base64) and real gRPC. Adds latency + complexity. Newer alternatives: Connect protocol (Buf) that works directly over HTTP/1.1 + JSON or protobuf, browser-native. For browser → server, REST or GraphQL is simpler.

**RED FLAGS TO AVOID:** - “Use GraphQL for everything” (over-fetching fix doesn't justify the complexity for simple APIs) - gRPC for public browser-facing APIs (won't work without proxy) - REST without OpenAPI (no contract)

---

## Q-BE-24: Kafka — partitions, consumer groups, offsets

**DIFFICULTY:** Hard **CATEGORY TAG:** Backend / Messaging

**ANSWER: Topic** — a stream of events (like a log). **Partition** — the unit of parallelism (ordered sub-log). **Producer** writes to a partition. **Consumer** reads from a partition. **Consumer group** — group of consumers that share partitions (each partition assigned to exactly one consumer in the group).

**Partitions:** - Each partition is an append-only log with an offset (per partition, monotonic). - Order guaranteed **within** a partition, not across. - Partition count = max parallelism for the topic (one consumer per partition in a group). - More partitions → more parallelism + more broker overhead. Plan: enough for peak consumer parallelism × 2 (room to grow). Don't overdo it (10K+ partitions on one broker is trouble). - Partition key: producer sets a key → all messages with the same key go to the same partition. Use for: per-entity ordering (all "user-123" events in order).

**Consumer groups:** - Each group has its own offset per partition (independent progress). - Adding consumers to a group → rebalance (partitions reassigned). Sticky assignor minimises movement. - One consumer per partition in a group (others idle if more consumers than partitions).

**Offsets:** - Consumer tracks its position per partition. Stored in `__consumer_offsets` topic (Kafka internal) by default. - `enable.auto.commit=true` (default) — auto-commit every 5s (at-most-once-ish: if process crashes after consuming but before commit, you reprocess; if after commit but before processing, you lose). - Manual commit: `commitSync()` (after processing — at-least-once), `commitAsync()` (faster, may lose on crash before commit). - At-least-once is the safe default — design consumers to be idempotent (dedupe by event ID).

**Delivery semantics:** - **At-most-once:** commit before processing. May lose messages on crash. - **At-least-once:** commit after processing. May reprocess on crash. Most common; require idempotent consumers. - **Exactly-once (EOS):** Kafka transactions (producer) + read-process-write pattern (consume → process → produce to another topic, all in one tx). Heavy. Use for: financial pipelines.

**CODE/EXAMPLE SNIPPET (Spring Kafka):**

```

@KafkaListener(topics = "orders", groupId = "order-service")
public void handle(OrderEvent event, Acknowledgment ack) {
 try {
 orderService.process(event); // idempotent (dedupe by event.id)
 ack.acknowledge(); // manual commit
 } catch (Exception e) {
 // don't ack - Kafka will redeliver
 log.error("processing failed for {}", event.id(), e);
 }
}

```

**KEY TERMS TO MENTION:** - Topic, partition (ordered sub-log), offset (per-partition) - Order within partition, not across - Partition key → same key same partition (per-entity ordering) - Consumer group: shares partitions, one consumer per partition - Manual commit for at-least-once; auto-commit for at-most-once-ish - EOS via Kafka transactions (heavy) - Idempotent consumers required for at-least-once

**FOLLOW-UP QUESTIONS:** 1. How do you choose the number of partitions? 2. What happens during a consumer rebalance?

**FOLLOW-UP ANSWERS:** 1. Peak consumer parallelism × 2 (room to grow). One consumer per partition → if you want 20 consumers in parallel, you need  $\geq 20$  partitions. Also consider broker count (partition leader per broker; spread load). Don't exceed ~4K partitions per broker (overhead grows). Future partition growth is painful (existing messages don't move; new keys may hash differently → breaks per-key ordering). Pick high enough from the start. 2. Consumers coordinate via a group coordinator (a broker). On join/leave/crash, the coordinator triggers a rebalance: pause consumption, assign partitions to consumers (CooperativeStickyAssignor minimises movement), resume. During the rebalance, no consumption happens (stop-the-world). Mitigations: CooperativeSticky (incremental rebalance, no stop-the-world for unaffected partitions); StaticGroupMembership (consumers leave + rejoin without rebalance if they come back quickly).

**RED FLAGS TO AVOID:** - Too few partitions (limits parallelism) - Auto-commit expecting exactly-once (it's at-most-once-ish) - Non-idempotent consumer (duplicates on redelivery)

---

## Q-BE-25: Kafka delivery semantics — at-most, at-least, exactly-once

**DIFFICULTY:** Hard **CATEGORY TAG:** Backend / Messaging

**ANSWER:** Three delivery guarantees, increasing difficulty + cost.

**At-most-once:** a message may be delivered 0 or 1 times (may be lost). - Producer: `acks=0` (fire and forget) or `acks=1` (leader only, no ISR replication wait). May lose on leader crash before replication. - Consumer: commit offset **before** processing. If consumer crashes after commit, message lost. - Use when: telemetry, metrics, clickstream — loss acceptable, throughput critical.

**At-least-once:** a message may be delivered 1+ times (no loss, may duplicate). - Producer: `acks=all` (wait for all in-sync replicas). `enable.idempotence=true` (dedupe producer retries). - Consumer: commit offset **after** processing. Crash after processing + before commit → reprocess. - Use when: most business events. Requires idempotent consumers (dedupe by event ID). - This is the default + most common.

**Exactly-once (EOS):** a message is delivered exactly once. Hard. - Kafka transactions (`transactional.id`): producer writes are atomic across partitions. Consumer can read **committed** messages only (`isolation.level=read_committed`). - **Read-process-write pattern:** consume from topic A, process, produce to topic B — all in one Kafka transaction. The consumer's offset commit + the producer's writes are atomic. - Use when: financial pipelines (debit + credit must both happen or neither). - Limitations: only works for Kafka→Kafka pipelines. For side effects to external systems (DB, HTTP), you need outbox pattern or transactional outbox + CDC.

**CODE/EXAMPLE SNIPPET (Kafka EOS):**

```
// Producer
props.put("transactional.id", "order-tx-1");
props.put("enable.idempotence", "true");
props.put("acks", "all");
KafkaProducer p = new KafkaProducer(props);
p.initTransactions();

// Read-process-write in a single transaction
p.beginTransaction();
try {
 for (ConsumerRecord<String, String> rec : consumer.poll(Duration.ofMillis(100))) {
 String processed = process(rec.value());
 p.send(new ProducerRecord<>("output", rec.key(), processed));
 }
 consumer.commitSync(); // offset commit is part of the tx
 p.commitTransaction();
} catch (Exception e) {
 p.abortTransaction();
}
```

**Why EOS is hard for side effects:** - If you write to Postgres **and** commit the Kafka offset, you have two systems → can't atomic. If Kafka commits but Postgres write fails → lost. If Postgres writes but Kafka commit fails → reprocess + duplicate Postgres row. - Solution: **transactional outbox** — write to Postgres + an **outbox** table in one DB transaction. A separate process (Debezium/CDC) reads the outbox table + publishes to Kafka. The DB write + outbox write are atomic. Kafka publish is best-effort (CDC retries on failure; outbox row marked "published" when done). At-least-once to Kafka, exactly-once to the DB.

**KEY TERMS TO MENTION:** - At-most-once: acks=0/1, commit-before-process. Loss OK (telemetry). - At-least-once: acks=all + idempotence, commit-after-process. Default. Needs idempotent consumers. - Exactly-once: Kafka transactions, read-process-write, read\_committed. Kafka→Kafka only. - Side effects to external systems → transactional outbox + CDC. - Idempotent consumers required regardless (Kafka redelivers on rebalance/crash).

**FOLLOW-UP QUESTIONS:** 1. How do you make a consumer idempotent? 2. Why isn't at-least-once enough for payments?

**FOLLOW-UP ANSWERS:** 1. Dedupe by event ID: store processed event IDs in a DB / Redis (TTL = your max replay window). On receive, check if already processed → skip. Or use the event ID as a unique constraint in your write (insert-ignore / upsert). For aggregates with versioning (event sourcing), apply events in order + skip already-applied versions. The pattern: `if (alreadyProcessed(event.id)) return; doWork(); markProcessed(event.id);` — must be atomic (transaction or conditional write). 2. At-least-once means a redelivery may charge a customer twice. If your consumer debits a card on every message + Kafka redelivers (rebalance, network blip), you charge twice. Idempotency fixes this IF the consumer dedupes by event ID — but you have to implement it carefully (no race conditions, atomic check-and-write). For payments specifically, use a payment ID as the idempotency key at the payment processor (Stripe) so even if you send the same charge twice, they dedupe. Combine: idempotent consumer + idempotent payment.

**RED FLAGS TO AVOID:** - Expecting Kafka to be exactly-once by default (it's at-least-once) - Auto-commit expecting no duplicates - Side effects to DB + commit offset non-atomically (lost or duplicated)

-----

-----

## Q-BE-26: RabbitMQ vs Kafka — when to use which

**DIFFICULTY:** Medium **CATEGORY TAG:** Backend / Messaging

**ANSWER:** Both are message brokers but designed for different use cases.

**Kafka:** distributed log. High throughput (millions/sec per broker), messages retained on disk (hours to days), consumers pull at their own pace, replay supported. Best for: event streaming, log aggregation, CDC, real-time analytics, fan-out to many consumers.

**RabbitMQ:** traditional AMQP broker. Smart routing (exchanges, queues, bindings), per-message ACK, lower throughput (~50K-200K msgs/sec), messages deleted on ACK. Best for: work queues, RPC, request/reply, complex routing (topic exchanges), task distribution.

**Feature comparison:** | | Kafka | RabbitMQ | |—|—|—| | Model | Append-only log, pull-based | Queue, push-based | | Message retention | Days/weeks (configurable) | Deleted on ACK | | Throughput | Millions/sec | ~100K/sec | | Latency | ~ms (low) | sub-ms (lower) | | Routing | Topic + partition key | Rich (direct, topic, fanout, headers) | | Replay | Yes (re-read from offset) | No (consumed = gone) | | Ordering | Per-partition | Per-queue | | Backpressure | Consumer-controlled pull | Broker pushes (can OOM consumer) | | Scale | Horizontal (partitions) | Vertical (queues per node) | | Ops | Heavier (ZooKeeper/KRaft + brokers) | Lighter (Erlang cluster) |

**Choose Kafka when:** - Event streaming (CDC, audit log, analytics pipeline) - Multiple consumers reprocess the same events (replay) - Very high throughput - Need to retain messages (e.g. regulatory)

**Choose RabbitMQ when:** - Work queue (distribute jobs to workers) - Complex routing (topic exchanges with wildcards) - Request/reply (RPC) - Low latency, smaller scale - AMQP semantics (per-message ACK, dead-letter exchanges)

**Other brokers (worth knowing):** - **Redis Streams / Redis Pub-Sub:** simpler, less durable, good for ephemeral. - **NATS / Pulsar:** Pulsar = Kafka-like + multi-tenant; NATS = ultra-low latency, simple. - **SQS / SNS:** AWS-managed. SQS = work queue (FIFO + standard); SNS = pub-sub fan-out.

**CODE/EXAMPLE SNIPPET (Spring AMQP — RabbitMQ):**

```
@RabbitListener(queues = "orders")
public void handle(OrderEvent e) {
 orderService.process(e); // throws → re-queue or DLQ
}
// Config: TTL, dead-letter-exchange
@Bean Queue orders() {
 return QueueBuilder.durable("orders")
 .withArgument("x-message-ttl", 60000)
 .withArgument("x-dead-letter-exchange", "orders.dlx")
 .build();
}
```

**KEY TERMS TO MENTION:** - Kafka: append-only log, retained, pull-based, replay, high throughput, partition ordering - RabbitMQ: AMQP, smart routing (exchanges/bindings), per-message ACK, deleted on consume - Kafka for streaming/replay/CDC; RabbitMQ for work queue/routing/RPC - Alternatives: Redis Streams, Pulsar, NATS, SQS/SNS

**FOLLOW-UP QUESTIONS:** 1. How do you choose partition count for Kafka? 2. How do you handle poison messages in RabbitMQ?

**FOLLOW-UP ANSWERS:** 1. Enough for max consumer parallelism  $\times 2$  (room to scale). One consumer per partition in a group  $\rightarrow$  20 parallel consumers needs  $\geq 20$  partitions. Don't exceed  $\sim 4K$  partitions/broker (overhead). Future partition growth is painful (new partitions don't move existing keys  $\rightarrow$  ordering breaks). Pick high from start. 2. Dead-letter exchange (DLX): configure the queue with `x-dead-letter-exchange`. On NACK/reject (requeue=false), the message goes to the DLX  $\rightarrow$  a dead-letter queue for inspection. After M retries, move to DLQ. Pattern: retry queue with TTL  $\rightarrow$  bounces back to main  $\rightarrow$  on max-retries header, route to DLQ. Don't infinite-loop poison messages.

**RED FLAGS TO AVOID:** - Kafka as a queue (it's a log; messages don't delete on read) - RabbitMQ for high-throughput streaming (throughput-limited) - Not configuring DLQ (poison messages loop forever)

-----

## Q-BE-27: Dead letter queue and retry strategies

**DIFFICULTY:** Hard **CATEGORY TAG:** Backend / Messaging

**ANSWER:** Messages that can't be processed (poison messages) need explicit handling — retry with backoff, then DLQ for inspection. Infinite retries → loop forever; no DLQ → messages vanish.

**Retry strategies:** 1. **Immediate retry (3x):** fast, catches transient errors (network blip). Don't retry forever. 2. **Exponential backoff:** delay doubles each retry (100ms → 200 → 400 → ...). Caps at minutes. 3. **Jitter:** add randomness to avoid synchronised retry storms (thundering herd). 4. **Scheduled retry (delay queues):** message goes to a delay queue → bounces back after N seconds. RabbitMQ: TTL + DLX; Kafka: no native delay (use external scheduler or Kafka's DelayedMessage via plugin); SQS: DelaySeconds.

**After max retries → DLQ.** - DLQ = a queue holding failed messages for inspection. - Header metadata: original queue, failure reason, retry count, stack trace, original timestamp. - Manual inspection + replay (after fixing the consumer bug or the bad message).

**CODE/EXAMPLE SNIPPET (Resilience4j for Spring):**

```
resilience4j.retry:
 instances:
 order-processing:
 max-attempts: 3
 wait-duration: 1s
 exponential-multiplier: 2
 exponential-max-wait: 30s
 retry-exceptions: [TransientException]
 ignore-exceptions: [IllegalArgumentException]
```

```
@Retry(name = "order-processing", fallbackMethod = "fallback")
public void process(OrderEvent e) { /* may throw */ }
public void fallback(OrderEvent e, Exception ex) { dlqPublisher.send(e, ex); }
```

**DLQ pattern (RabbitMQ with TTL + DLX):** - Main queue: `x-dead-letter-exchange: retry-exchange` - Retry queue: `x-message-ttl: 5s`, `x-dead-letter-exchange: main-exchange` - On failure: reject to retry-exchange → 5s later bounces back to main → if still fails N times (header `x-retry-count`), reject to final DLQ.

**Kafka:** no native retry/delay. Options: - External retry topic with a scheduler that re-publishes after delay. - Spring Kafka `DeadLetterPublishingRecoverer` + `DefaultErrorHandler` with `ExponentialBackOff`. - For non-recoverable: park in a `dead-letter-topic` for manual replay.

**Why not infinite retry:** poison messages (corrupt payload, missing reference) never succeed; infinite retry wastes resources + blocks the queue. After N retries, move to DLQ and let a human fix the message (or the consumer code).

**KEY TERMS TO MENTION:** - Poison messages → retry with exponential backoff + jitter → DLQ after max retries - DLQ: hold failed messages for inspection + replay - RabbitMQ: TTL + DLX for delay; Kafka: external retry topic or Spring's `DeadLetterPublishingRecoverer` - Resilience4j `@Retry` + `fallbackMethod` for service-level retry - Never infinite retry (poison message loops forever, blocks queue)

**FOLLOW-UP QUESTIONS:** 1. How do you prevent a poison message from blocking the whole queue? 2. When should you NOT retry?

**FOLLOW-UP ANSWERS:** 1. (a) Bounded retries (max 3-5) + DLQ. (b) Use a separate retry/delay queue so the main queue keeps draining. (c) For ordered queues (one partition), retry + DLQ means later messages wait — consider a “skip + DLQ + continue” policy: if a message fails N times, move to DLQ + commit the offset (don't block the partition). (d) Rate-limit retries to avoid hammering a downstream that's down. 2. For non-recoverable errors: malformed message (can't parse), business validation failure (invalid input), missing reference (a deleted user). These won't succeed on retry — go straight to DLQ. For errors that *will* succeed on retry: transient network blip, downstream 503, lock contention. Differentiate by exception type. Resilience4j lets you `ignore-exceptions` (no retry) vs `retry-exceptions`.

**RED FLAGS TO AVOID:** - Infinite retry (poison messages block the queue) - No DLQ (failed messages disappear) - Retry everything (some errors are permanent — straight to DLQ)

---

## Q-BE-28: Circuit breaker pattern (Resilience4j)

**DIFFICULTY:** Hard **CATEGORY TAG:** Backend / Resilience

**ANSWER:** A circuit breaker stops calling a failing downstream service, letting it recover. Without it, every request to a down service wastes a thread + connection (cascade failure).

**States:** - **CLOSED:** normal — calls go through. Track failures. - **OPEN:** failing — calls fail fast (no downstream call). After a wait, move to HALF\_OPEN. - **HALF\_OPEN:** trial — let N test calls through. If they succeed → CLOSE. If they fail → OPEN again.

**Config (Resilience4j):**

```
resilience4j.circuitbreaker:
 instances:
 payment:
 sliding-window-type: COUNT_BASED
 sliding-window-size: 100 # last 100 calls
 failure-rate-threshold: 50 # 50% failures → open
 wait-duration-in-open-state: 30s
 permitted-number-of-calls-in-half-open-state: 5
 automatic-transition-from-open-to-half-open-enabled: true
 register-health-indicator: true
```

```
@CircuitBreaker(name = "payment", fallbackMethod = "fallback")
public Payment pay(Order o) { return paymentClient.charge(o); }
public Payment fallback(Order o, Exception e) { return Payment.declined("service
down"); }
```

**When to trip:** a downstream is failing (50% errors, slow p99, timeouts). The circuit protects your service + the downstream (less load on it).

**Other resilience patterns (use together):** - **Retry:** retry transient failures (network blip). - **Rate limiter:** limit calls/sec to protect downstream (and yourself). - **Bulkhead:** isolate resources (a slow service shouldn't starve others). - **Timeout:** bound waiting — don't hold threads indefinitely.

**Fall back gracefully:** - Cached/stale data (show last-known-good). - Default response ("service temporarily unavailable"). - Async-retry later (queue the request, process when service recovers). - Degrade features (hide the affected UI section instead of crashing the page).

**Don't:** - Trip the circuit on business errors (4xx — those are expected, not failures). - Forget to test the fallback (it must actually work). - Use a circuit for every call (only for downstream services that may fail).

**KEY TERMS TO MENTION:** - 3 states: CLOSED, OPEN, HALF\_OPEN - Sliding window (count or time-based), failure rate threshold, wait duration - Resilience4j: @CircuitBreaker +

fallbackMethod - Combine with Retry, RateLimiter, Bulkhead, TimeLimiter - Don't trip on business errors (4xx expected) - Fallback: cache, default, queue, degrade

**FOLLOW-UP QUESTIONS:** 1. How do you choose the failure threshold? 2. What's the difference between circuit breaker and retry?

**FOLLOW-UP ANSWERS:** 1. Low enough to trip on real outages, high enough not to trip on noise. Defaults: 50% failures over a 100-call sliding window. For low-traffic services, time-based window (e.g. 30s) — count-based is too sparse. For a 1000 req/s service, count-based (1000 calls = 1s of data) trips fast. Tune to your SLO: if 5% error rate violates your SLO, set threshold at 5-10% (don't wait for 50%). Add a `slow-call-rate-threshold` (e.g. p99 > 2s counts as a "slow call") — a service that's "up" but slow can also trip the circuit. 2. Retry handles transient failures (one-off, will succeed on retry). Circuit breaker handles sustained failures (downstream is down — retry won't help, it just makes it worse). Use both: retry with backoff *inside* the circuit breaker. If retries keep failing, the circuit opens. Retry without circuit = hammering a down service = cascade failure.

**RED FLAGS TO AVOID:** - Circuit on every call (overhead; only for downstream that may fail)  
- Tripping on 4xx (business errors) - Fallback that doesn't work (untested)

---

## Q-BE-29: Rate limiting in Spring (Bucket4j, distributed)

**DIFFICULTY:** Hard **CATEGORY TAG:** Backend / API

**ANSWER:** Rate limiting protects your service from abuse + ensures fair use. Two main algorithms:

**Token bucket:** a bucket holds N tokens (capacity). Each request consumes 1 token. Tokens refill at a rate (R per second). If empty → 429. Allows bursts (up to capacity) but average rate = refill rate. Flexible + widely used.

**Leaky bucket:** requests enter a queue, processed at fixed rate. Overflow → 429. Smooths bursts (no burst absorption). Used for traffic shaping.

**Other algorithms:** - **Fixed window:** count per minute; resets at minute boundary. Easy but allows 2x at boundary (60 at end of minute 1 + 60 at start of minute 2 = 120 in a second). - **Sliding window log:** store each request timestamp; count in window. Accurate but memory-heavy. - **Sliding window counter:** split into sub-windows + interpolate. Approximates sliding window with less memory.

**Bucket4j (Java library):**

```
@Bean
Bucket bucket() {
 return Bucket.builder()
 .addLimit(limit -> limit.capacity(100).refillGreedy(100, Duration.ofMinutes(1)))
 .build();
}

@GetMapping("/api")
public ResponseEntity<?> api() {
 if (bucket.tryConsume(1)) return ResponseEntity.ok("done");
 return ResponseEntity.status(429).header("Retry-After", "60").build();
}
```

**Per-user rate limit:** one bucket per user (ConcurrentHashMap<String, Bucket>). For distributed: store buckets in Redis (Bucket4j has a Redis integration) or use a Redis-based limiter (Lua script for atomic check-and-decrement).

**Distributed rate limiting:** - In-memory per-instance doesn't work across instances (each instance has its own bucket → 5 instances × 100 = 500/min, not 100/min). - Redis: atomic INCR + EXPIRE per key (window) or Lua script for token bucket. - Dedicated limiter service (a microservice that handles all limits). - API gateway (Kong, AWS API Gateway, Cloudflare) — handles limits at the edge before reaching your app.

**Response conventions:** - Status: 429 Too Many Requests. - Headers: **Retry-After** (seconds to wait), **X-RateLimit-Limit**, **X-RateLimit-Remaining**, **X-RateLimit-Reset**.

**Hierarchical limits:** - Per-IP (DDoS): 1000 req/min. - Per-user (abuse): 100 req/min. - Per-tier (free vs pro): free 10/min, pro 100/min.

**Twist — sticky session distributed rate limit:** “Design a rate limiter that works across a cluster with sticky sessions.” Sticky sessions route a user to the same instance — in-memory per-instance works. If sticky not available, use Redis (atomic). For very high throughput (millions/sec), use a sharded counter (e.g. sharded Redis by user ID) or approximate via sliding window counters.

**KEY TERMS TO MENTION:** - Token bucket (bursts OK, average = refill), leaky bucket (smooths bursts) - Fixed window (2x at boundary), sliding window log, sliding window counter - Bucket4j: capacity + refillGreedy; per-user buckets in Map - Distributed: Redis (atomic INCR or Lua), dedicated limiter service, API gateway - 429 + Retry-After + X-RateLimit-\* headers - Hierarchical: per-IP (DDoS), per-user (abuse), per-tier (free/pro) - Sticky session distributed limiter: in-memory if sticky, Redis if not

**FOLLOW-UP QUESTIONS:** 1. How do you implement an atomic token bucket in Redis? 2. What if a user exceeds the limit — do you charge them or block them?

**FOLLOW-UP ANSWERS:** 1. Lua script for atomicity (Redis executes Lua atomically):

```
-- KEYS[1]=bucket key, ARGV[1]=capacity, ARGV[2]=refill_rate, ARGV[3]=now,
 ARGV[4]=cost
local tokens = tonumber(redis.call("hget", KEYS[1], "tokens")) or ARGV[1]
local last = tonumber(redis.call("hget", KEYS[1], "last")) or ARGV[3]
local refill = math.min(ARGV[1], tokens + (ARGV[3] - last) * ARGV[2])
if refill >= ARGV[4] then
 redis.call("hmset", KEYS[1], "tokens", refill - ARGV[4], "last", ARGV[3])
 redis.call("expire", KEYS[1], 3600)
 return 1
else
 return 0
end
```

The Lua script reads + writes atomically (no race between get + set). Race condition without atomicity: two requests both see 1 token, both decrement, both succeed → 2 tokens consumed from a bucket with 1. 2. Depends on business. Most apps block (429 + Retry-After) — keeps cost predictable. Some allow overage at a price (e.g. API marketplaces) — but you must communicate it. For free tiers: hard block (no overage — they need to upgrade). For paid: soft limit (warn at 80%, charge at 100%, hard block at 200%). For enterprise: negotiated limits + alerts. The choice is business, not technical. Document it.

**RED FLAGS TO AVOID:** - In-memory limiter across instances (5 instances = 5x limit) - Non-atomic check-and-decrement in Redis (race → over-limit) - Forgetting Retry-After (client retries immediately, amplifying load)

## Q-BE-30: Spring Batch — chunk processing

**DIFFICULTY:** Hard **CATEGORY TAG:** Backend / Spring Batch

**ANSWER:** Spring Batch is a framework for robust batch processing: ETL, nightly reports, data migration. Built for restartability (resume on failure), partitioning, and large datasets.

**Core concepts:** - **Job** — a sequence of steps. - **Step** — a unit of work. Chunk-oriented (read → process → write in batches) or tasklet (single action). - **ItemReader** — reads items (DB, file, Kafka). Paged + transactional. - **ItemProcessor** — transforms items, filters. - **ItemWriter** — writes items (DB, file, Kafka). - **JobRepository** — stores job/step execution metadata (DB tables). - **JobLauncher** — starts a job.

**Chunk processing:** read N items → process → write N items in one transaction. Commit interval = chunk size (e.g. 100). On failure, only the current chunk rolls back; the job can restart from the last successful chunk.

**CODE/EXAMPLE SNIPPET:**

```
@Bean
Step importStep(JobRepository repo, PlatformTransactionManager tx) {
 return new StepBuilder("importStep", repo)
 .chunk(100, tx) // 100 items per tx
 .reader(itemReader()) // JpaPagingItemReader
 .processor(item -> item.status() == ACTIVE ? item : null) // filter
 .writer(items -> repo.saveAll(items)) // batch insert
 .faultTolerant()
 .retryLimit(3)
 .retry(TransientException.class)
 .skipLimit(10)
 .skip(MalformedRowException.class)
 .listener(new StepExecutionListener() { ... })
 .build();
}

@Bean
Job importJob(JobRepository repo, Step importStep) {
 return new JobBuilder("importJob", repo)
 .start(importStep)
 .incrementer(new RunIdIncrementer()) // unique params per run
 .build();
}
```

**Scaling options:** - **Multi-threaded step:** chunk processing with multiple threads (writer must be thread-safe). - **Partitioning:** split input across partitions, each runs as a separate step (parallel or remote). - **Remote chunking:** reader on master, processors/writers on workers (via messaging). - **Parallel steps:** multiple independent steps run in parallel.

**Restartability:** JobRepository stores the last successful read position. On restart, the reader resumes from there (not from start). Requires a restartable reader (JdbcPagingItemReader with `setSaveState(true)` ).

**Skip vs retry:** - Retry: transient errors — try again (e.g. deadlocks, network blip). - Skip: bad data — log + move on (e.g. malformed rows). Skip limit prevents skipping everything.

**Best practices:** - Use chunk size 100-1000 (too small = per-tx overhead; too large = memory + rollback cost). - Use paging readers (don't load the whole table). - Idempotent writes (in case of retry — don't duplicate). - Use the JobRepository DB (not in-memory) for restartability. - Don't hold DB connections across chunk boundaries (release after write).

**KEY TERMS TO MENTION:** - Job → Step (chunk or tasklet) → Reader/Processor/Writer - Chunk: read N → process → write N in one tx; commit interval = chunk size - JobRepository for restartability (resume from last chunk) - Scaling: multi-threaded step, partitioning, remote chunking, parallel steps - Retry (transient) vs skip (bad data), both with limits - Chunk size 100-1000; paging readers; idempotent writes

**FOLLOW-UP QUESTIONS:** 1. How do you restart a failed batch job? 2. How do you process 1B rows without OOM?

**FOLLOW-UP ANSWERS:** 1. The JobRepository stores step execution state (last successful read position, commit count). On restart, use the same job parameters + `jobOperator.restart(executionId)` (or `JobLauncher.run(job, params)` with the same params — Spring detects it's a restart). The reader resumes from the last commit point. Requires a restartable reader + the JobRepository DB intact. If the chunk size is 100 and you've committed 1000 chunks, restart processes chunk 1001 onward. 2. Use a paging/cursor reader (JdbcPagingItemReader with `pageSize=1000` — fetches 1000 rows at a time, not all 1B). Process in chunks of 1000, commit, release to GC. Don't accumulate in memory. Partition the input (e.g. by date range or ID range) + run partitions in parallel (multi-threaded step or remote workers). Use a remote chunking topology if a single JVM can't keep up: master reads, workers process + write. Use a database cursor (ResultSet streaming) for very large reads — but most readers (JpaPagingItemReader) need paging because Hibernate holds the persistence context.

**RED FLAGS TO AVOID:** - Loading the whole table into memory (OOM) - Chunk size too small (per-tx overhead) or too large (rollback cost) - Not storing JobRepository (no restart on failure)

---

## Q-BE-31: Connection pooling — HikariCP tuning

**DIFFICULTY:** Hard **CATEGORY TAG:** Backend / Database

**ANSWER:** HikariCP is the default connection pool in Spring Boot. It's fast (microsecond overhead) + well-tuned, but the pool size matters.

**Pool sizing:** - Too small → requests wait for connections (latency spike). - Too large → DB CPU thrashes (each connection = a backend process in PG; too many → context switching + cache thrashing). The DB can usually handle fewer than you think.

**Formula:**  $\text{pool\_size} = (\text{peak\_connections}) \times (1 + \text{wait\_time} / \text{active\_time})$ . For most CRUD apps: 10-30 connections is plenty. Don't match your thread pool size — IO-bound threads wait on the DB; the DB is the bottleneck.

**PostgreSQL Universal Connection Pooler (PgBouncer):** if you have many app instances × 30 connections each = thousands → PG struggles. PgBouncer multiplexes (transaction mode: thousands of client connections → ~100 server connections). Use for high-fan-out topologies.

**Key Hikari settings:**

```
spring.datasource.hikari:
 maximum-pool-size: 20 # default 10 — tune up for IO-heavy
 minimum-idle: 20 # keep all open (avoid creation latency)
 connection-timeout: 30000 # wait 30s for a connection (else
 SQLTransientConnectionException)
 idle-timeout: 600000 # close idle after 10 min
 max-lifetime: 1800000 # close after 30 min (prevents DB-side idle cleanup issues)
 keepalive-time: 300000 # keep connections alive every 5 min
 leak-detection-threshold: 60000 # warn if a connection held >60s (leak)
 connection-test-query: SELECT 1 # or use JDBC4 isValid (driver handles)
```

**Monitoring (Micrometer):** - `hikari.connections.active` — current in use. - `hikari.connections.pending` — waiting for a connection (bad if >0). - `hikari.connections.idle` — idle. - `hikari.connections.creation` — time to create a new connection. - `hikari.connections.acquisition` — time to acquire from pool (should be sub-ms).

**Common issues:** - **Connection leak:** a thread holds a connection without closing (forgot `try-with-resources` or `@Transactional`). Hikari's `leak-detection-threshold` logs the stack trace. - **Stale connections:** DB or network firewall kills idle connections. Hikari's `keepalive-time` + `max-lifetime` recycle before they go stale. - **Pool exhausted:** all connections in use, requests wait → `connection-timeout` → 500. Either raise pool size, reduce query time, or add a circuit breaker. - **Idle connections killed by AWS RDS proxy / firewall:** use `keepalive-time` (Hikari 4.0.3+) to send keepalive queries.

**Don't:** - Set pool size = thread count (over-provisions DB). - Forget to close connections (leaks). - Use auto-commit=false with manual commit (bugs); let the pool handle defaults.

**KEY TERMS TO MENTION:** - HikariCP: default in Spring Boot, microsecond overhead - Pool size 10-30 typical; formula based on wait/active ratio - PgBouncer for high-fan-out (thousands of clients → ~100 server conns) - Settings: max-pool-size, connection-timeout, max-lifetime, keepalive-time, leak-detection-threshold - Micrometer: active, pending, idle, acquisition time - Issues: leaks (try-with-resources), stale (keepalive), exhaustion (raise pool or circuit break)

**FOLLOW-UP QUESTIONS:** 1. How do you detect a connection leak? 2. Why does pool size > DB capacity hurt?

**FOLLOW-UP ANSWERS:** 1. `leak-detection-threshold: 60000` — Hikari logs a stack trace if a connection is checked out >60s. The stack points to the code that acquired it (didn't release). Also watch `hikari.connections.active` approaching `max-pool-size` with low throughput — strong leak signal. Fix: ensure every acquisition is in try-with-resources or wrapped in `@Transactional` (Spring auto-releases). 2. Each PG connection = a backend process. 100 connections = 100 processes, context-switching + cache thrash. PG's default `max_connections=100` but practical optimum is ~CPU cores × 2-3 (active). Beyond that, throughput drops (CPU spent on switching, not queries). Use a pooler (PgBouncer) to multiplex many app connections to fewer DB connections. Or read replicas for reads. Or shard. The mistake: matching pool size to max thread count "for safety" — actually hurts the DB.

**RED FLAGS TO AVOID:** - Pool size = thread count (DB thrash) - Forgetting to close connections (leaks) - No keepalive on long-idle pools (stale connections behind firewall)

---

## Q-BE-32: Open Session in View (OSIV) — and why you should disable it

**DIFFICULTY:** Medium **CATEGORY TAG:** Backend / Hibernate

**ANSWER:** Open Session in View (OSIV): Spring Boot's default ( `spring.jpa.open-in-view=true` ) keeps the Hibernate session open for the entire HTTP request, not just the service layer. Lazy loading works in the view layer (controller, Jackson serialization).

**Why it's bad:** 1. **Hidden N+1:** lazy loads happen during Jackson serialization (in the controller), after the service returns. N+1 hidden in the view layer. 2. **Slow queries in the request thread:** lazy loads extend the DB session lifetime to the whole request. Each lazy load is a separate DB round trip in the request thread. 3. **Connection held longer:** the DB connection is held during serialization (Jackson runs in the request thread). Pool exhausted faster. 4. **Coupling:** view layer depends on entity lazy-loaded associations. Refactoring the entity breaks the view. 5. **Unpredictable errors:** `LazyInitializationException` disappears (everything works in dev) until you disable OSIV — then prod breaks.

**The right pattern:** - Disable OSIV: `spring.jpa.open-in-view: false`. - Service layer returns DTOs (or fully-fetched entities via EntityGraph/JOIN FETCH). - Controller serializes the DTO (no lazy load needed). - DB session closes at service end → connection released.

**When OSIV might be OK:** - Prototyping / dev (lazy loads work, no DTO boilerplate). - Read-only endpoints with light lazy loads (low traffic). - If you accept the trade-offs explicitly.

<b>Spring</b>	<b>Boot</b>	<b>warns:</b>
<code>spring.jpa.open-in-view</code> is enabled by default... it could have performance implications.		

Take the warning seriously.

**Modern Spring Boot 3.x:** OSIV still default but the warning is louder. The community consensus: disable + use DTOs.

**KEY TERMS TO MENTION:** - OSIV: session open for whole HTTP request, lazy loads in view work - Bad: hidden N+1, slow DB in request thread, connection held longer, view-entity coupling, hidden LazyInitException - Disable: `spring.jpa.open-in-view: false` - Right: service returns DTOs / fully-fetched entities; controller serializes - OK only for prototyping/read-only/light traffic

**FOLLOW-UP QUESTIONS:** 1. What if a DTO needs a lazily-loaded association? 2. How do you detect OSIV-induced N+1 in production?

**FOLLOW-UP ANSWERS:** 1. Fetch eagerly at the service layer (EntityGraph or JOIN FETCH), then map to the DTO. The DTO has the value already; no lazy load. Or use a custom JPQL query that selects exactly the columns needed into a projection (constructor expression). The point is the service layer does all the DB work; the controller/serializer does no DB. 2. (a) Disable OSIV in a staging env — if endpoints start throwing `LazyInitializationException` during serialization, those are the OSIV-hidden N+1s. Fix each by fetching eagerly in the

service. (b) Hibernate statistics in prod ( `hibernate.generate_statistics` ) — query count per request > expected indicates N+1. (c) p99 latency correlated with serialization time (Jackson) — DB queries inside serialization show as “extra” latency after the service returns. (d) APM tools (Datadog DB tracing) show query calls during the response phase.

**RED FLAGS TO AVOID:** - OSIV on in production (hidden N+1) - Returning entities from controllers (coupling) - Not measuring query count per request

-----

## Q-BE-33: JPA entity states and persistence context

**DIFFICULTY:** Medium **CATEGORY TAG:** Backend / JPA

**ANSWER:** Four entity states in JPA:

1. **New (transient):** instantiated with `new`, not associated with a persistence context. No DB row, no ID.
2. **Managed (persistent):** associated with a persistence context (within a `@Transactional` method). Changes are tracked; on flush/commit, Hibernate generates UPDATE.
3. **Detached:** was managed, but the persistence context closed (or `em.detach`). No longer tracked; changes don't propagate.
4. **Removed:** marked for deletion (`em.remove`); flushed on commit.

**Transitions:** - `new` → `em.persist(e)` → managed. - `em.find(Order.class, 1)` or query → managed. - managed → `em.detach(e)` or tx end → detached. - managed → `em.remove(e)` → removed → flushed → row deleted. - detached → `em.merge(e)` → managed (returns the managed instance — note: the argument stays detached!). - detached → `em.refresh(e)` throws (refresh only on managed).

**Dirty checking:** Hibernate tracks the original state of managed entities; on flush, it compares current vs original, generating UPDATE only for changed columns (or all columns if `dynamicUpdate=false`).

**Flush modes:** - **AUTO** (default): flush before queries that may be affected by pending changes. - **COMMIT**: flush only on commit. - **MANUAL** (Hibernate-specific): explicit `em.flush()`. - Don't flush mid-tx unnecessarily — it's a round trip + invalidates the L1 cache.

**Common gotchas:** - `em.merge(detached)` returns a `new` managed instance. The original argument is NOT tracked. Common bug: `em.merge(detached); detached.setX(5);` — the change is on the detached instance, no update happens. Use `detached = em.merge(detached);` then modify. - `equals/hashCode` with lazy associations: comparing entities by ID is safer than by all fields (lazy fields not loaded → `LazyInitException`). - `@Version` for optimistic locking — bumped on each update. - `@GeneratedValue` IDENTITY delays the INSERT until flush (the DB generates the ID); SEQUENCE/TABLE pre-allocate.

**CODE/EXAMPLE SNIPPET:**

```

@Transactional
public Order update(Long id, Status s) {
 Order o = em.find(Order.class, id); // managed
 o.setStatus(s); // dirty tracked – UPDATE on commit
 return o; // caller gets managed entity (no DTO
 // conversion here)
}

@Transactional
public Order merge(Order detached) {
 Order managed = em.merge(detached); // returns managed copy
 managed.setStatus(Status.SHIPPED); // tracked
 return managed; // don't modify 'detached' – won't persist
}

```

**KEY TERMS TO MENTION:** - 4 states: New, Managed, Detached, Removed - Transitions: persist, find/query (managed), detach/tx end (detached), remove (removed) - em.merge returns new managed; argument stays detached (common bug) - Dirty checking: Hibernate compares original vs current, UPDATE on commit - Flush modes: AUTO (default), COMMIT, MANUAL

**FOLLOW-UP QUESTIONS:** 1. What's the difference between persist() and merge()? 2. How do you avoid LazyInitializationException after tx end?

**FOLLOW-UP ANSWERS:** 1. `persist(newEntity)` : makes a NEW entity managed (no DB row yet → INSERT on flush). Throws if the entity already has an ID (or is detached). `merge(detachedEntity)` : takes a detached (or new) entity, returns a managed copy (UPDATE if exists, INSERT if new). Use persist for new entities; merge for re-attaching detached (e.g. from a web form). For updates of fetched entities, don't merge — modify the managed entity directly (dirty checking handles the UPDATE). 2. (a) Fetch all needed associations eagerly in the tx (EntityGraph/JOIN FETCH). (b) Convert to DTO in the tx (the DTO has all data, no lazy load needed). (c) Disable OSIV so you find the bug at dev time, then fix with (a) or (b). (d) For read-only endpoints, use a JPA projection (constructor expression) — no entity, no lazy load. Don't "keep the session open" longer — that's the OSIV antipattern.

**RED FLAGS TO AVOID:** - Modifying the detached argument after merge (no update) - Lazy loading after tx end (LazyInitException) - Flushing mid-tx unnecessarily

---

## Q-BE-34: Spring profiles + configuration externalization

**DIFFICULTY:** Medium **CATEGORY TAG:** Backend / Spring Boot

**ANSWER:** Spring Boot config: layered + overridable. Order (lowest to highest precedence):  
 1. Default ( `application.yml` inside jar). 2. Profile-specific ( `application-{profile}.yml` ). 3. External `application.yml` (outside jar). 4. Environment variables ( `SPRING_DATASOURCE_URL=...` ). 5. Command-line args ( `--spring.datasource.url=...` ). 6. `@TestPropertySource` / Spring Test (test scope).

**Profiles:** activate via `spring.profiles.active=prod` or env var `SPRING_PROFILES_ACTIVE`. Beans annotated `@Profile("prod")` only load when prod active.

```
spring:
 profiles:
 active: dev
 datasource:
 url: ${DB_URL:jdbc:postgresql://localhost/app} # default if env not set

spring:
 config:
 activate:
 on-profile: prod
 datasource:
 url: ${DB_URL}
 hikari.maximum-pool-size: 50
```

**Profile-specific beans:**

```
@Profile("dev")
@Bean
public DataSource devDataSource() { /* in-memory */ }
@Profile("prod")
@Bean
public DataSource prodDataSource() { /* AWS RDS */ }
```

**Spring Cloud Config:** externalize config to a server (Git-backed). Apps fetch config at startup. Allows runtime config refresh ( `@RefreshScope` + Spring Cloud Bus). Useful for many microservices with shared config.

**Externalized secrets:** never put secrets in `application.yml` (committed). Use: - Env vars (12-factor). - Spring Cloud Config with encryption. - Vault, AWS Secrets Manager, AWS Parameter Store. - Kubernetes secrets (mounted as env or files).

`@ConfigurationProperties` for typed binding:

```
@ConfigurationProperties(prefix = "app.cache")
public record CacheProps(Duration ttl, int maxSize, boolean enabled) {}
// usage: app.cache.ttl=10m, app.cache.max-size=1000
```

**@Value vs @ConfigurationProperties** : @Value for single values, @ConfigurationProperties for groups (typed + validated + IDE support).

**Validation:** @Validated + JSR-380 on @ConfigurationProperties (e.g. @NotBlank, @Min) — fails fast on startup if config invalid.

**KEY TERMS TO MENTION:** - Config precedence: default < profile < external < env < cmdline - spring.profiles.active (or SPRING\_PROFILES\_ACTIVE env) - @Profile on beans — conditional loading - application-{profile}.yml + multi-document YAML (—) - Spring Cloud Config for externalized/refreshable - Secrets: env, Vault, AWS SM, k8s secrets (never in committed YAML) - @ConfigurationProperties (typed + validated) vs @Value

**FOLLOW-UP QUESTIONS:** 1. How do you reload config without restart? 2. How do you test profile-specific code?

**FOLLOW-UP ANSWERS:** 1. Spring Cloud Config + @RefreshScope : config server reads new value on /actuator/refresh (or /busrefresh via Cloud Bus to broadcast). Beans marked @RefreshScope are recreated on the next access. Trade-off: only beans annotated @RefreshScope see the new value; you must annotate every bean that uses the config. For non-trivial cases, use a config library with a watching client (Archaius, custom). Some properties (like datasource URL) can't be refreshed — you'd need a pool restart (which the framework doesn't do automatically). 2. @SpringBootTest with @ActiveProfiles("test") to load profile-specific config. Or @TestPropertySource(properties = "app.feature=true") to override. Or ApplicationRunner for testing auto-configs with specific profiles. Use Testcontainers for the DB so the test profile matches prod (no in-memory DB vs real PG mismatch).

**RED FLAGS TO AVOID:** - Secrets in committed YAML (leak) - @Value everywhere (no validation, no grouping) - Profile mismatch between dev and prod (works on dev, breaks on prod)

---

## Q-BE-35: Spring AOP — aspects, pointcuts, advice

**DIFFICULTY:** Hard **CATEGORY TAG:** Backend / Spring AOP

**ANSWER:** AOP (Aspect-Oriented Programming) modularizes cross-cutting concerns (logging, transactions, security, metrics). Spring AOP uses proxies (runtime weaving); AspectJ uses bytecode weaving (compile-time or load-time) for more power.

**Key terms:** - **Aspect:** a module of cross-cutting concern ( `@Aspect` class). - **Join point:** a point in execution (method call, field access). Spring AOP only supports method join points. - **Pointcut:** an expression matching join points ( `execution(* com.example..*.*(..))` ). - **Advice:** what to do at matched join points ( `@Before` , `@After` , `@Around` ). - **Weaving:** linking aspects to targets (Spring: proxy at runtime; AspectJ: compile-time).

**Advice types:** - `@Before` — runs before the method. - `@AfterReturning` — runs after successful return (can access the return value). - `@AfterThrowing` — runs after an exception (can access the exception). - `@After` — runs after (finally). - `@Around` — wraps the method; you call `proceed()` to invoke it. Most powerful (can short-circuit, change args, change return value).

**CODE/EXAMPLE SNIPPET:**

```
@Aspect
@Component
public class MetricsAspect {
 @Around("@annotation(Timed)")
 public Object timed(ProceedingJoinPoint pjp) throws Throwable {
 long start = System.nanoTime();
 try {
 return pjp.proceed();
 } finally {
 long ms = (System.nanoTime() - start) / 1_000_000;
 metrics.timer(pjp.getSignature().toShortString()).record(ms, MILLISECONDS);
 }
 }

 @Before("execution(* com.example.service.*.*(..)) && args(req,..)")
 public void logRequest(Object req) { log.info("req {}", req); }
}
```

**Pointcut designators:** - `execution(...)` — method execution. - `within(com.example..*)` — within a type. - `@annotation(Timed)` — method has the annotation. - `@within(Service)` — class has the annotation. - `bean(orderService)` — by bean name. - `args(String, ..)` — by argument types. - `target(SomeInterface)` — runtime type.

**@Transactional is an AOP aspect:** Spring wraps `@Transactional` methods with an `Around` advice that opens the tx, calls `proceed`, commits/rolls back. This is why self-invocation (calling the method via `this`) bypasses the proxy — the proxy isn't involved.

**Limitations of Spring AOP (proxy-based):** - Only public methods on Spring beans (proxy can't intercept private, final, or non-Spring objects). - Self-invocation bypasses the proxy. - Can't intercept fields (only methods). - Use AspectJ (compile-time weaving) for the rest.

**Performance:** proxies add a small overhead (~microseconds). Don't use AOP for hot paths in tight loops. AspectJ compile-time weaving is faster (no proxy).

**KEY TERMS TO MENTION:** - Aspect, Join point (method only in Spring), Pointcut, Advice, Weaving - Advice: @Before, @AfterReturning, @AfterThrowing, @After, @Around (most powerful) - Pointcut designators: execution, within, @annotation, @within, bean, args, target - @Transactional = AOP aspect (proxy-based) - Limitations: public methods on Spring beans, self-invocation bypass, no fields - AspectJ compile-time for more power

**FOLLOW-UP QUESTIONS:** 1. Why does self-invocation bypass AOP? 2. How does Spring choose between JDK dynamic proxy and CGLib?

**FOLLOW-UP ANSWERS:** 1. Spring AOP wraps the bean in a proxy. External calls go through the proxy (advice runs). Inside the class, calling `this.method()` invokes the actual method directly (the proxy is not involved) — the proxy is the wrapper, the original is the wrapped. So `methodA()` calling `methodB()` inside the same class bypasses any aspect on `methodB`. Fix: (a) extract `methodB` into another bean; (b) `AopContext.currentProxy()` (requires `@EnableAspectJAutoProxy(exposeProxy = true)`); (c) self-inject the bean and call via the reference. 2. JDK dynamic proxy: requires the target to implement an interface — proxies the interface. CGLib: subclasses the target — works for any class (but can't proxy final classes/methods). Spring Boot 2.x+ defaults to CGLib ( `spring.aop.proxy-target-class=true` ), so all beans get CGLib proxies (works without interfaces). Pre-2.x defaulted to JDK proxy for interface beans. Trade-off: CGLib slightly slower to create, faster to call; JDK proxy opposite.

**RED FLAGS TO AVOID:** - Self-invocation expecting aspects to fire - AOP on private/final methods (silently ignored) - Heavy advice on hot paths (overhead)

---

## Q-BE-36: Circular dependencies and how Spring resolves them

**DIFFICULTY:** Hard **CATEGORY TAG:** Backend / Spring

**ANSWER:** A circular dependency: A's constructor needs B, B's constructor needs A. With constructor injection, Spring throws `BeanCurrentlyInCreationException` — it can't construct either first.

**How Spring handles (only for field/setter injection):** Spring creates a **half-constructed** instance (allocate the object, no props set), registers it, then injects. So: A is allocated (no B yet) → A registered → B is allocated (no A yet) → B registered → B's field injection: A (already in the registry, half-constructed) → B is fully constructed → A's field injection: B (fully constructed) → A is fully constructed.

This works for singleton + field/setter injection. Spring caches the early reference (the half-constructed bean) in `singletonObjects` (a 3-level cache).

**The 3-level cache:** 1. `singletonObjects` — fully constructed singletons. 2. `earlySingletonObjects` — half-constructed (early references) for cycle resolution. 3. `singletonFactories` — ObjectFactory that produces the early reference (deferred, supports AOP proxies).

**Why constructor injection fails:** the constructor needs the dependency **at construction time**. Spring can't allocate A without B (its constructor requires it), can't allocate B without A. No half-constructed instances to register.

**Spring Boot 2.6+ disables circular references by default:** `spring.main.allow-circular-references=false` (default). Throws on cycle. To enable (not recommended), set true.

**Fixes (preferred order):** 1. **Refactor** — extract the shared logic into a third bean (C) that both A and B depend on. 2. **Use events** — A publishes, B listens (no direct dependency). 3. **@Lazy** on one side — Spring injects a proxy; the actual bean is resolved on first method call. 4. **Setter/field injection** — allow the cycle (not recommended; masks design smell). 5. **Self-injection** — A injects itself for the "B" role (rare; usually a design smell).

**Why circular deps are a smell:** - Tight coupling (A and B can't change independently). - Order-dependent initialization (subtle bugs). - Hard to test (must construct both). - Indicates the responsibility split is wrong (one bean should own the shared concern).

**CODE/EXAMPLE SNIPPET (fix with @Lazy):**

```
@Service
public class A {
 private final B b;
 public A(@Lazy B b) { this.b = b; } // proxy injected; resolved on first call
}
```

**KEY TERMS TO MENTION:** - Constructor injection → `BeanCurrentlyInCreationException` (can't allocate either first) - Field/setter: 3-level cache (`singletonObjects`, `earlySingletonObjects`, `singletonFactories`) for early refs - Spring Boot 2.6+: disabled by default (`allow-circular-references=false`) - Fixes: refactor (extract shared), events, `@Lazy` proxy, self-inject - Smell: tight coupling, init order issues, hard to test

**FOLLOW-UP QUESTIONS:** 1. Why does Spring use a 3-level cache and not just 2? 2. How does `@Lazy` actually break the cycle?

**FOLLOW-UP ANSWERS:** 1. The third level ( `singletonFactories` ) holds an `ObjectFactory` that, when called, produces the early reference (and may wrap it in an AOP proxy if the bean needs advice). It's deferred so that AOP proxies are created only if needed (lazy). If only 2 levels: either always create the proxy eagerly (overhead) or never (aspects broken on circular beans). The 3-level design lets Spring create the proxy lazily on first need. 2. `@Lazy` injects a proxy at construction time. The proxy is constructed without needing the target bean. So A's constructor takes the B proxy (no real B yet), B's constructor takes the A proxy (no real A yet). Both fully constructed. The first time a method is called on the proxy, it looks up the real bean (now constructed) from the context. The cycle is broken because the proxy defers the dependency resolution.

**RED FLAGS TO AVOID:** - Enabling circular refs to "fix" a startup error (masks design smell) - Field injection to enable circular refs (deprecated + untestable) - Not refactoring obvious cycles

---

## Q-BE-37: Exception handling in Spring (@ControllerAdvice)

**DIFFICULTY:** Medium **CATEGORY TAG:** Backend / Spring

**ANSWER:** Centralized exception handling via `@ControllerAdvice` + `@ExceptionHandler`. Avoids try/catch in every controller; consistent error responses.

**CODE/EXAMPLE SNIPPET:**

```
@RestControllerAdvice
public class GlobalExceptionHandler {

 @ExceptionHandler(MethodArgumentNotValidException.class) // @Valid fail
 public ResponseEntity<Error> handleValidation(MethodArgumentNotValidException e) {
 List<String> errors = e.getFieldErrors().stream()
 .map(f -> f.getField() + ": " + f.getDefaultMessage()).toList();
 return ResponseEntity.badRequest().body(new Error("VALIDATION", errors));
 }

 @ExceptionHandler(NotFoundException.class)
 public ResponseEntity<Error> handleNotFound(NotFoundException e) {
 return ResponseEntity.status(404).body(new Error("NOT_FOUND", e.getMessage()));
 }

 @ExceptionHandler(OptimisticLockException.class)
 public ResponseEntity<Error> handleConflict(OptimisticLockException e) {
 return ResponseEntity.status(409).body(new Error("CONFLICT", "Try again"));
 }

 @ExceptionHandler(Exception.class) // catch-all (last resort)
 public ResponseEntity<Error> handleAll(Exception e) {
 log.error("unhandled", e);
 return ResponseEntity.status(500).body(new Error("INTERNAL", "Try again later"));
 }
}

record Error(String code, Object details) {}
```

**Best practices:** - Specific handlers before generic (Spring picks the most specific). - Use a consistent error envelope (code, message, traceId, fields). - Use correct HTTP status (400/401/403/404/409/422/429/500). - Log on the catch-all (don't swallow). - Don't leak stack traces to clients (security + UX). - Add a `traceId` for log correlation (MDC + log it in the catch-all). - For Spring Security exceptions (`AccessDeniedException`, `AuthenticationException`), use `AuthenticationEntryPoint` + `AccessDeniedHandler` (security filter chain), not `@ControllerAdvice` (the security filter runs before the controller).

**ProblemDetails (RFC 7807):** Spring 6 has built-in support (`ProblemDetail` class):

```
return ResponseEntity.status(404).body(ProblemDetail.forStatusAndDetail(NOT_FOUND,
 "user not found"));
```

Standardizes error format across APIs ( `type` , `title` , `status` , `detail` , `instance` ).

**Validation:** `@Valid` on the request body triggers `MethodArgumentNotValidException` if validation annotations ( `@NotBlank` , `@Min` , etc.) fail. Handle in `@ControllerAdvice` → 400 with field errors.

**KEY TERMS TO MENTION:** - `@RestControllerAdvice` + `@ExceptionHandler` for centralized handling - Specific handlers before generic (most specific wins) - Consistent error envelope (code, message, traceId, fields) - Correct HTTP status; don't leak stack traces - ProblemDetails (RFC 7807) — Spring 6 `ProblemDetail` - Security exceptions via `SecurityFilterChain` (not `ControllerAdvice`) - `@Valid` → `MethodArgumentNotValidException` → 400

**FOLLOW-UP QUESTIONS:** 1. What's the difference between `@ControllerAdvice` and `@RestControllerAdvice`? 2. How do you log the traceId in every error?

**FOLLOW-UP ANSWERS:** 1. `@ControllerAdvice` returns view names (resolved by a `ViewResolver`) or `ResponseEntity`. `@RestControllerAdvice` is `@ControllerAdvice` + `@ResponseBody` — returns are serialized to JSON, no view resolution. Use `@RestControllerAdvice` for JSON APIs (the common case); `@ControllerAdvice` if you also handle view-rendering errors (Thymeleaf, JSP). 2. Use MDC (Mapped Diagnostic Context). A filter sets `MDC.put("traceId", id)` per request. Logback pattern includes `%X{traceId}`. The catch-all handler reads `MDC.get("traceId")` and includes it in the error response. For distributed tracing, use Spring Cloud Sleuth / Micrometer Tracing — they auto-propagate traceId across services via headers + MDC.

**RED FLAGS TO AVOID:** - Catch-all returning 200 OK + error body (broken status) - Leaking stack traces (security) - Swallowing exceptions silently (no log)

---

## Q-BE-38: Async in Spring — @Async and @Scheduled

**DIFFICULTY:** Medium **CATEGORY TAG:** Backend / Spring Async

**ANSWER:** `@Async`: runs a method on a separate thread (returns `void` or `CompletableFuture`). Requires `@EnableAsync`. Spring wraps the bean in a proxy that submits to a `TaskExecutor`.

```
@EnableAsync
@Configuration
public class AsyncConfig {
 @Bean("notifyExecutor")
 public Executor notifyExecutor() {
 ThreadPoolTaskExecutor e = new ThreadPoolTaskExecutor();
 e.setCorePoolSize(4); e.setMaxPoolSize(16);
 e.setQueueCapacity(500);
 e.setThreadNamePrefix("notify-");
 e.setRejectedExecutionHandler(new ThreadPoolExecutor.CallerRunsPolicy());
 e.initialize();
 return e;
 }
}

@Service
public class NotifyService {
 @Async("notifyExecutor") // specific executor
 public CompletableFuture<Void> sendEmail(String to) {
 mailer.send(to);
 return CompletableFuture.completedFuture(null);
 }
}
```

**Gotchas:** 1. **Self-invocation bypass:** calling `@Async` method from within the same class doesn't go through the proxy → runs synchronously. Same as `@Transactional`. Fix: extract into another bean or self-inject. 2. **Default executor:** without a name, uses the default (`SimpleAsyncTaskExecutor` in Spring Boot 2 — creates a new thread per task! Spring Boot 3 default is configurable). Always specify an executor or override the default. 3. **Exception handling:** `@Async` methods can't throw to the caller. Use `AsyncUncaughtExceptionHandler` for void methods, or return `CompletableFuture` and complete exceptionally. 4. **Virtual threads:** Spring Boot 3.2+ — `spring.threads.virtual.enabled=true` makes `@Async` use virtual threads automatically. Great for IO-bound async.

**@Scheduled:** runs a method on a schedule. Requires `@EnableScheduling`. Cron, fixed rate, or fixed delay.

```

@Scheduled(cron = "0 0 2 * * ?") // 2 AM daily
public void nightlyReport() { ... }

@Scheduled(fixedDelay = 60_000, initialDelay = 5_000) // every 60s after completion
public void healthCheck() { ... }

```

**Scheduled gotchas:** 1. **Single-threaded by default:** all `@Scheduled` methods share one scheduler thread. A long method delays others. Use `@EnableScheduling` + a `TaskScheduler` bean with a pool, or `@Async` on the method (with an executor). 2. **Clustered scheduling:** in a multi-instance deployment, every instance runs the schedule → duplicate execution. Use `ShedLock` (DB-based distributed lock) or a scheduler service (Quartz with clustering, Spring Cloud `ScheduledTaskScheduler`). 3. **Fixed rate vs fixed delay:** rate = every N ms regardless of execution time (may overlap if tasks run longer than rate); delay = N ms after completion (no overlap). 4. **Time zones:** `zone = "UTC"` to avoid DST issues. Use UTC for server-side; convert at the UI.

**Spring Boot 3.2+:** virtual threads make `@Async` cheap; `@Scheduled` can run on a virtual-thread scheduler.

**KEY TERMS TO MENTION:** - `@Async` + `@EnableAsync`; specify executor (don't use default `SimpleAsync`) - `@Async` self-invocation bypasses proxy (runs sync) - `@Async` exceptions: `AsyncUncaughtExceptionHandler` or `CompletableFuture` - Virtual threads (Spring 3.2): `spring.threads.virtual.enabled=true` - `@Scheduled` + `@EnableScheduling`; single-threaded by default (use pool) - Clustered: `ShedLock` or Quartz cluster (avoid duplicate runs) - cron/fixedRate/fixedDelay; timezones in UTC

**FOLLOW-UP QUESTIONS:** 1. How do you prevent duplicate scheduled runs in a cluster? 2. What happens if a `@Scheduled` method throws?

**FOLLOW-UP ANSWERS:** 1. `ShedLock` — annotates `@Scheduled` methods; uses a DB table to lock (each instance tries to acquire the lock; only one wins per period). Alternative: Quartz with JDBC clustering (every instance coordinates via DB; jobs distributed). For complex schedules, use a dedicated scheduler (Chronos, Airflow) that triggers the service via HTTP. Don't rely on "only one instance runs" (defeats HA). 2. The exception is caught by the scheduler's error handler (`ScheduledTaskRegistrar.setErrorHandler`). Default: logs the exception. The next scheduled run still fires (one failure doesn't stop the schedule). Customize the handler for alerting. For critical jobs, wrap the body in try/catch + retry, then re-throw to the handler for logging.

**RED FLAGS TO AVOID:** - `@Async` without named executor (default creates unbounded threads) - `@Scheduled` in a cluster without `ShedLock` (duplicate runs) - Self-invocation (proxy bypass)

## Q-BE-39: Spring Boot 3 — Jakarta EE migration, virtual threads, GraalVM

**DIFFICULTY:** Medium **CATEGORY TAG:** Backend / Spring Boot 3

**ANSWER:** Spring Boot 3 (Nov 2022) — Java 17 baseline, Jakarta EE 9+ (javax → jakarta), first-class virtual threads, GraalVM native image support, observability improvements.

**Jakarta EE migration (javax → jakarta):** - All `javax.servlet.*`, `javax.persistence.*`, `javax.validation.*` → `jakarta.*`. - Breaking change — must update imports + dependencies. - Migration: `spring-boot-migrator` or manual. Most major libs (Hibernate, Jetty, Tomcat) shipped Jakarta versions by 2023. - Why: Oracle gave up the Java EE trademark; EE moved to Eclipse Foundation as “Jakarta EE” — required namespace change for future development.

**Virtual threads (Spring Boot 3.2+):**

```
spring.threads.virtual.enabled: true
```

- Tomcat uses virtual threads per request (instead of fixed thread pool).
- `@Async` executors use virtual threads.
- `@Scheduled` runs on virtual threads.
- Massive concurrency for IO-bound apps without reactive programming.
- Caveats: pinning with `synchronized` (use `ReentrantLock`), `ThreadLocal` memory, don't pool.

**GraalVM native image:** - Spring AOT (Ahead-of-Time) compiler analyzes the app at build time, generating native code. - Spring Boot 3 + GraalVM → native binary: starts in ms, ~50MB RAM (vs JVM's 500MB-2GB). - Trade-offs: longer build time, no runtime reflection (must declare), no classpath scanning at runtime (AOT resolves), small heap tuning constraints, slower debugging. - Use for: CLI tools, serverless (AWS Lambda — fast cold start), microservices with many replicas (memory savings). - Don't use for: long-running heavy apps (JIT eventually beats AOT), apps with dynamic class loading, dev (slower dev cycle).

```
./gradlew nativeCompile # produces build/native-image/app
```

**Observability (Spring Boot 3):** - Micrometer Observation API (unifies metrics + traces). - Auto-tracing with Micrometer Tracing (Zipkin, Tempo, Jaeger). - `@Observed` annotation for declarative instrumentation.

**Other 3.x changes:** - `@ConstructorBinding` for `@ConfigurationProperties` (immutable records). - HTTP interface clients (declarative, like Feign but Spring-native). - `ProblemDetails` (RFC 7807) support. - WebFlux improvements (`SseEmitter`, multipart).

**CODE/EXAMPLE SNIPPET (HTTP interface client):**

```

interface OrderClient {
 @GetExchange("/orders/{id}")
 Order getOrder(@PathVariable long id);
}

@Bean
OrderClient orderClient(WebClient.Builder b) {
 HttpServiceProxyFactory f = HttpServiceProxyFactory
 .builderFor(WebClientAdapter.forClient(b.baseUrl("http://
 orders").build())).build();
 return f.createClient(OrderClient.class);
}

```

**KEY TERMS TO MENTION:** - Jakarta EE: javax → jakarta namespace (breaking) - Virtual threads: `spring.threads.virtual.enabled=true` (Tomcat, `@Async`, `@Scheduled`) - GraalVM native: AOT, ms startup, 50MB RAM, no runtime reflection - Use native for: serverless, CLI, many-replica microservices - Don't use native for: long-running heavy apps, dynamic class loading, dev - Micrometer Observation (unified metrics + traces), `@Observed` - HTTP interface clients (declarative, like Feign)

**FOLLOW-UP QUESTIONS:** 1. What breaks when migrating to native image? 2. When are virtual threads worse than platform threads?

**FOLLOW-UP ANSWERS:** 1. Runtime reflection (Spring's bread + butter) breaks — AOT analysis must know all reflective access at build time. Hibernate proxies, Jackson serialization (mostly handled by Spring AOT), third-party libs using reflection — must declare via `@RegisterReflection` or `reflect-config.json`. Dynamic proxies (CGLib), `Class.forName()`, `ServiceLoader`, `ResourceBundle` need hints. Spring AOT generates most hints; you add what it misses. Some libs (older Lombok, ASM-based tools) don't support native — drop or replace. 2. CPU-bound work (virtual threads don't add parallelism — limited by cores). Pinning situations (`synchronized` blocking — use `ReentrantLock`). Heavy `ThreadLocal` use (memory: 100K VTs × large TL = a lot). When you need real OS-level thread priority/preemption (rare). Virtual threads shine for IO-bound concurrency; platform threads for CPU-bound.

**RED FLAGS TO AVOID:** - Expecting native image to be a “free speedup” (build complexity, reflection limits) - Virtual threads with `synchronized` (pinning) - Not updating javax imports on Spring Boot 3 upgrade (compile fails)

## Q-BE-40: JVM tuning flags — what to set in production

**DIFFICULTY:** Medium **CATEGORY TAG:** Backend / JVM

**ANSWER:** Start with sensible defaults; tune based on metrics, not guesses.

**Heap:** - `-Xms<val>` `-Xmx<val>` — set equal to avoid resize jitter. E.g. `-Xms4g` `-Xmx4g`. - `-XX:MaxRAMPercentage=75` — for containers (Java 10+, better than fixed Xmx in k8s). - Don't set both `-Xmx` and `MaxRAMPercentage` (one or the other).

**GC:** - `-XX:+UseG1GC` (default since Java 9) — general purpose. - `-XX:MaxGCPauseMillis=200` — G1 target pause. - `-XX:+UseZGC` for large heaps (32GB+) with low-latency needs. - `-XX:+UseParallelGC` for throughput-oriented batch.

**OOM handling:** - `-XX:+HeapDumpOnOutOfMemoryError` — auto heap dump on OOM. - `-XX:HeapDumpPath=/var/log/app/heap-%p.hprof` — path with PID. - `-XX:OnOutOfMemoryError="kill -9 %p"` — kill the JVM (let k8s restart it).

### Diagnostics:

- `-Xlog:gc*:file=/var/log/app/gc.log:time,level,tags:filecount=10,filesize=10M` — GC log rotation. - `-XX:+UnlockDiagnosticVMOptions` `-XX:+PrintInlining` (dev only). - `-XX:ErrorFile=/var/log/app/hs_err_pid%p.log` — fatal error log.

**Container awareness:** - Java 10+ detects cgroup limits (memory + CPU). - `-XX:MaxRAMPercentage=75` — leaves room for off-heap (Metaspace, direct buffers, threads). - `-XX:InitialRAMPercentage=75` — initial heap.

**Class metadata:** - `-XX:MaxMetaspaceSize=256m` — cap (prevent leak). - `-XX:+UseCompressedOops` (auto on for heaps <32GB) — pointer compression.

**Thread stack:** - `-Xss512k` — smaller stacks (default 1MB; reduce for many threads).

**Direct memory:** - `-XX:MaxDirectMemorySize=1g` — for Netty/gRPC apps.

**Networking:** - `-Djava.net.preferIPv4Stack=true` — avoid IPv6 issues in some networks. - `-Dnetworkaddress.cache.ttl=30` — JDK DNS cache (default 30s for security manager; infinite otherwise).

### Common Spring Boot prod flags:

```
-Xms4g -Xmx4g
-XX:+UseG1GC -XX:MaxGCPauseMillis=200 -XX:+ParallelRefProcEnabled
-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/var/log/app/heap-%p.hprof
-Xlog:gc*:file=/var/log/app/gc.log:time,level,tags:filecount=10,filesize=10M
-XX:ErrorFile=/var/log/app/hs_err_pid%p.log
-Djava.security.egd=file:/dev/./urandom # faster SecureRandom startup
```

**Don't tune blindly:** measure first (`jstat`, GC log, APM). Tune one thing at a time. Most services run fine with defaults + G1 + heap-dump-on-OOM.

**KEY TERMS TO MENTION:** - `-Xms = -Xmx` (avoid resize), `MaxRAMPercentage` (containers) - G1 default, ZGC for large heaps - `HeapDumpOnOutOfMemoryError` + `HeapDumpPath` - `-Xlog:gc*` (rotated GC log) - Container: `MaxRAMPercentage=75` (room for off-heap) - `MaxMetaspaceSize` (cap class metadata) - `-Xss` (smaller stacks for many threads) - Measure first, tune one thing at a time

**FOLLOW-UP QUESTIONS:** 1. Why set `-Xms = -Xmx`? 2. What's `MaxRAMPercentage` vs `-Xmx`?

**FOLLOW-UP ANSWERS:** 1. If `-Xms < -Xmx`, the heap grows under load — but growing requires GC + page faulting + heap resize (a stop-the-world event). Under load, this causes jitter (regular latency degrades when heap grows). Equal sizes avoid resize entirely. The trade-off: you commit the full heap upfront (memory reservation), even if you don't use it. In containers, this is fine (you're paying for the container anyway). 2. `-Xmx` is an absolute value (e.g. 4g). `MaxRAMPercentage` is a fraction of the container's memory limit (e.g. 75% of an 8g container = 6g heap). Use `MaxRAMPercentage` in containers — if you resize the container, the heap auto-scales. Use `-Xmx` when you want a fixed heap regardless of container size. Don't set both — one wins (depends on version).

**RED FLAGS TO AVOID:** - Tuning flags blindly without measuring - `-Xms < -Xmx` in production (resize jitter) - `MaxRAMPercentage=100` (no room for off-heap → OOM-killed by k8s)

---

## Q-BE-41: Class loading + classloader hierarchy

**DIFFICULTY:** Hard **CATEGORY TAG:** Backend / JVM

**ANSWER:** Java uses classloaders to load classes on demand. Each classloader has a parent; the delegation model asks the parent first.

**Hierarchy (Java 9+ module system):** 1. Bootstrap classloader — JVM-internal (C++); loads `java.lang.*`, `java.util.*` from `rt.jar`/modules. No Java representation (null parent). 2. Platform classloader (was Extension) — loads `java.sql`, `java.xml` modules. 3. Application (System) classloader — loads your app's classpath. 4. Custom classloaders — for plugins, dynamic loading, app servers (each webapp has its own).

**Delegation (parent-first):** when asked to load `com.foo.Bar`, a classloader asks its parent first. If the parent can't load, the child tries. This ensures core classes (`java.lang.String`) are loaded once by Bootstrap (not duplicated by app classloader).

**Why multiple classloaders:** - Web apps (Tomcat): each webapp has its own classloader → isolation (webapp A's lib v1 doesn't conflict with webapp B's v2). - Plugins: each plugin loads in its own classloader → unload by dropping the classloader. - Hot-deploy: load a new version of a class in a new classloader. - OSGi: explicit module-level classloader with imports/exports.

**Class identity:** a class is identified by (classloader, fully-qualified name). Two `Foo` loaded by different classloaders are **different classes** — `instanceof` fails, `ClassCastException`. Common in app servers.

**Common issues:** - **ClassNotFoundException:** a class isn't on any classloader's path. - **NoClassDefFoundError:** class was present at compile time, not at runtime (different from CNE — it's a previously-loaded class now missing a dep). - **LinkageError:** two classloaders loaded the same class (different instances) → conflicts. - **Jar hell:** multiple versions of the same JAR on the classpath; only one loads (unspecified which). - **Leaked classloaders:** a classloader holds a class that holds a static field that holds a classloader → the classloader never GCs → all its classes never unload (Metaspace leak). Webapp redeploy without this fix = OOM.

**CODE/EXAMPLE SNIPPET (custom classloader):**

```
URLClassLoader pluginLoader = new URLClassLoader(
 new URL[]{new File("plugin.jar").toURI().toURL()},
 Thread.currentThread().getContextClassLoader() // parent
);
Class<?> pluginClass = pluginLoader.loadClass("com.foo.Plugin");
Plugin plugin = (Plugin) pluginClass.getDeclaredConstructor().newInstance();
// Later, unload: pluginLoader = null; plugin = null; // GC'd when no refs
```

**Thread context classloader:** each thread has a context classloader (default = the creator's). Some frameworks (JNDI, ServiceLoader) use it to load classes when the current

classloader is the wrong one (e.g. Bootstrap needs to load an app class). Set it carefully in async tasks.

**KEY TERMS TO MENTION:** - Hierarchy: Bootstrap → Platform → Application → Custom - Parent-first delegation (core classes loaded once) - Class identity = (classloader, FQN); different CLs = different classes - Web apps: per-webapp classloader for isolation; plugins: unload by dropping CL - Issues: ClassNotFoundException, NoClassDefFound, LinkageError, jar hell, Metaspace leak - Thread context classloader for framework-loading-app-class cases

**FOLLOW-UP QUESTIONS:** 1. Why do webapp classloaders invert delegation? 2. How does Spring Boot's executable jar loading work?

**FOLLOW-UP ANSWERS:** 1. Tomcat webapp classloaders are **child-first** (servlet spec recommendation): they try to load from the webapp's WEB-INF/lib first, then the parent. This prevents the parent (Tomcat's libs) from overriding the webapp's libs. Trade-off: if the webapp uses a different version of a class the parent also has (e.g. Servlet API), the webapp gets the wrong one — so the servlet API is loaded by the parent (parent-first for those). In practice, Tomcat's classloader has rules about which classes go to parent-first (JVM + servlet API) vs child-first (everything else). 2. Spring Boot's fat jar packages your app + all dependencies inside one jar. The jar's **META-INF/MANIFEST.MF** points to **JarLauncher** as the main class. The launcher creates a custom classloader that loads classes from **BOOT-INF/classes** and **BOOT-INF/lib/\*.jar** (nested JARs). Standard URLClassLoader can't read nested JARs, so Spring Boot has a custom **JarFile** implementation. The classloader is parent-first to the app classloader (which loaded the launcher). This is why Spring Boot jars "just work" without a classpath.

**RED FLAGS TO AVOID:** - Expecting Class.forName in two classloaders to return the same Class - Leaking classloaders (Metaspace leak) on redeploy - Mixing two versions of the same JAR (jar hell)

---

## Q-BE-42: Stream API — when to use, common pitfalls

**DIFFICULTY:** Medium **CATEGORY TAG:** Backend / Java

**ANSWER:** Stream API (Java 8+): declarative operations on collections. Concise + parallelizable, but with pitfalls.

**When to use:** - Map/filter/reduce pipelines. - Collect into a different shape (group, partition, join). - Aggregation (count, sum, max). - Avoiding explicit loops for readability.

**When NOT to use:** - Single-pass iteration with side effects (a for-loop is clearer). - Performance-critical tight loops (Stream has overhead: object alloc, lambda capture). - Stateful intermediate ops ( `sorted` , `distinct` ) — costly.

**CODE/EXAMPLE SNIPPET:**

```
// Good: declarative
Map<Status, Long> counts = orders.stream()
 .filter(o -> o.total().compareTo(BigDecimal.ZERO) > 0)
 .collect(Collectors.groupingBy(Order::status, Collectors.counting()));

// Good: reduce
BigDecimal total = orders.stream().map(Order::total).reduce(BigDecimal.ZERO,
 BigDecimal::add);

// Bad: side effects in forEach (use for-loop instead)
orders.stream().forEach(o -> { counter++; emails.add(o.email()); }); // mutable
state
```

**Pitfalls:** 1. **Side effects in lambdas:** `forEach(o -> list.add(o))` — if `list` isn't thread-safe, parallel streams corrupt. Use `collect(toList())` instead. 2. **Infinite streams:** `Stream.iterate(0, i -> i+1).filter(...).findFirst()` — never terminates if the filter never matches. Use `limit()` or `takeWhile()`. 3. **Order-dependent ops:** `unordered().distinct()` may skip in a different order than expected; don't rely on order after `unordered`. 4. **Reusing streams:** a stream is single-use; second terminal op throws `IllegalStateException`. Re-create from the source. 5. **parallelStream gotcha:** uses the common ForkJoinPool. Blocking IO inside blocks the whole JVM's async. Don't use for IO; use a dedicated pool. 6. **Auto-boxing:** `IntStream` is unboxed (no autoboxing); `Stream<Integer>` boxes. Use `IntStream` for primitives. 7. `collect(Collectors.toList())` **mutability:** the returned list is mutable but type-unspecified (JDK 16+ has `Stream.toList()` returning an `unmodifiable` list — different!). 8. **Exception in lambda:** checked exceptions don't compile. Wrap with try/catch or extract to a method.

**Stream vs for-loop performance:** - Small collections (<1000): for-loop faster (Stream overhead dominates). - Large collections, simple ops: comparable. - Large collections, complex ops: Stream + parallel can win (with careful tuning). - Most app code: Stream is fine — readability matters more.

**KEY TERMS TO MENTION:** - Use: map/filter/reduce/collect pipelines, declarative - Don't: side effects in forEach, performance-critical tight loops - Pitfalls: side effects, infinite streams, single-use, parallelStream blocks common pool - Auto-boxing (use IntStream for primitives) - collect(toList()) is mutable; Stream.toList() (JDK 16) is unmodifiable - Checked exceptions in lambdas (wrap or extract) - Small collections: for-loop faster; large: Stream fine

**FOLLOW-UP QUESTIONS:** 1. Why is parallelStream risky for IO? 2. What's the difference between map and flatMap?

**FOLLOW-UP ANSWERS:** 1. `parallelStream` uses the common ForkJoinPool (parallelism = cores-1). If your lambda does blocking IO (HTTP, DB), it blocks the pool workers → all other parallelStream + CompletableFuture (without explicit executor) in the JVM slow down. Use a dedicated pool: `pool.submit(() -> list.parallelStream()...)`. Or use `CompletableFuture.supplyAsync(task, ioExecutor)` for IO. 2. `map` applies a function to each element (1:1 — T → R). `flatMap` applies a function that returns a Stream, and flattens (1:N — T → Stream → flat). Use `map` for 1:1 transformations; `flatMap` when each input produces multiple outputs (e.g. `orders.flatMap(o -> o.items().stream())` flattens all items across orders into one stream).

**RED FLAGS TO AVOID:** - Side effects in forEach (mutable state) - parallelStream for IO (blocks common pool) - Reusing streams (single-use)

---

## Q-BE-43: Records and sealed classes (Java 17+)

**DIFFICULTY:** Easy **CATEGORY TAG:** Backend / Java

**ANSWER: Records (Java 14+, stable 16):** concise immutable data carriers. Auto-generates constructor, accessors, equals, hashCode, toString.

```
public record Order(Long id, BigDecimal total, Status status) {}
// Equivalent to: private final fields, canonical constructor, accessors id()/
// total()/status(),
// equals by all fields, hashCode, toString
```

**Why use records:** - Less boilerplate (vs Lombok @Data or hand-written). - Immutable by construction (no setters). - Perfect for DTOs, value objects, event payloads, return types. - Pattern matching friendly.

**Compact constructor** for validation:

```
public record Email(String value) {
 public Email {
 if (!value.contains("@")) throw new IllegalArgumentException("invalid email");
 value = value.toLowerCase().trim(); // can normalize
 }
}
```

**Sealed classes (Java 17):** restrict which classes can extend/implement. Closed hierarchies — exhaustive pattern matching.

```
public sealed interface Shape permits Circle, Square, Triangle {}
record Circle(double r) implements Shape {}
record Square(double side) implements Shape {}
record Triangle(double a, double b, double c) implements Shape {}
// Only these 3 can implement Shape — no others.
```

**Why sealed:** - Domain modeling (closed set of cases — like enums but with data). - Pattern matching exhaustive checks (the compiler knows all subtypes). - Security: prevents unauthorized extensions.

**Pattern matching (Java 21):** with sealed + records, `switch` is exhaustive (no `default` needed).

```
double area(Shape s) {
 return switch (s) {
 case Circle c -> Math.PI * c.r() * c.r();
 case Square s -> s.side() * s.side();
 case Triangle t -> /* ... */;
 }; // exhaustive - compiler knows all cases
}
```

**Records caveats:** - Mutable components: if a component is a mutable type (List, Map), the record isn't truly immutable. Defensive copy or use immutable collections. - Can't extend (records are implicitly final). Can implement interfaces. - Can't define additional instance fields (only the components). - Reflection + serialization: records have special handling (no need for externalizable).

**Use records for:** DTOs, value objects, event payloads, API responses, multi-return (replacing `Pair<>`). **Don't use for:** JPA entities (need mutable + no-arg constructor + proxies) — Java 21 supports records as JPA `@Embeddable` but not as `@Entity`. Beans needing inheritance.

**KEY TERMS TO MENTION:** - Records: immutable data carriers; auto constructor/accessors/equals/hashCode/toString - Compact constructor for validation/normalization - Sealed: restricts subtypes (permits); closed hierarchies for exhaustive pattern matching - Pattern matching (Java 21): switch over sealed is exhaustive (no default needed) - Records: DTOs, value objects, events. Don't: JPA entities (mutable + no-arg ctor needed) - Caveat: mutable component = not truly immutable (defensive copy)

**FOLLOW-UP QUESTIONS:** 1. Can a record have additional methods? 2. Why can't JPA entities be records?

**FOLLOW-UP ANSWERS:** 1. Yes — you can add methods, static fields, implement interfaces, override accessors. You can't add instance fields beyond the components (the canonical constructor's signature is the record's "shape"). Common: add a static factory (`of`), a derived method (`isValid()`), or override an accessor. 2. JPA requires: no-arg constructor (Hibernate instantiates via reflection), non-final class (Hibernate proxies via subclassing), mutable fields (dirty tracking). Records are implicitly final + only have the canonical constructor + immutable. So Hibernate can't instantiate or proxy them. Java 21+ relaxed some rules (records can be `@Embeddable`), but `@Entity` records still don't work for the proxying + dirty-tracking reasons. Use a class for entities; records for DTOs.

**RED FLAGS TO AVOID:** - Mutable components in records (not immutable) - Records as JPA entities (proxy/dirty-tracking breaks) - Sealed without pattern matching (underusing the feature)

## Q-BE-44: Pattern matching + switch expressions (Java 17+)

**DIFFICULTY:** Medium **CATEGORY TAG:** Backend / Java

**ANSWER:** Java has been adding pattern matching gradually — `instanceof` patterns (16), switch expressions (14), switch patterns (21), record patterns (21).

### Switch expressions (Java 14):

```
String label = switch (status) {
 case ACTIVE -> "Active";
 case INACTIVE, SUSPENDED -> "Inactive";
 case PENDING -> throw new IllegalStateException();
 // no default needed if exhaustive (enum)
};
```

- Yields a value (no break/fallthrough bugs).
- Exhaustive for enums (compile error if a case missing — except with `default`).
- Multiple labels per case.
- `yield` for blocks.

### Pattern matching for instanceof (Java 16):

```
// Old
if (obj instanceof String) {
 String s = (String) obj;
 return s.length();
}

// New
if (obj instanceof String s) {
 return s.length(); // s is bound, no cast
}
```

### Switch patterns (Java 21):

```
String desc(Object o) {
 return switch (o) {
 case null -> "null"; // null case (no NPE)
 case Integer i -> "int " + i;
 case String s -> "str " + s;
 case int[] arr -> "array len " + arr.length;
 case Order o -> "order " + o.id();
 default -> "other";
 };
}
```

- Type patterns + binding.
- `case null` (no NPE for null — switch historically NPE'd on null).
- Exhaustive with sealed types (no `default` needed).

**Record patterns (Java 21):** destructure records in patterns.

```
double area(Shape s) {
 return switch (s) {
 case Circle(double r) -> Math.PI * r * r;
 case Square(double side) -> side * side;
 case Triangle(double a, double b, double c) -> /* ... */;
 };
}
```

**Guarded patterns (Java 21):**

```
return switch (o) {
 case String s when s.length() > 100 -> "long string";
 case String s -> "short string";
 default -> "other";
};
```

**Why it matters:** - Concise + safe (no cast boilerplate). - Exhaustive checks catch missing cases at compile time. - Algebraic data types (sealed + records + patterns) for domain modeling.

**KEY TERMS TO MENTION:** - Switch expressions (Java 14): yield value, no fallthrough, multiple labels, enum-exhaustive - instanceof patterns (16): bind without cast - Switch patterns (21): type + null case, exhaustive with sealed - Record patterns (21): destructure - Guarded patterns: `when` predicate - ADTs: sealed + records + patterns for domain modeling

**FOLLOW-UP QUESTIONS:** 1. What's the difference between switch statement and switch expression? 2. How does exhaustive switch help with sealed hierarchies?

**FOLLOW-UP ANSWERS:** 1. Statement: doesn't return a value, uses `break` to exit (fallthrough by default). Expression: returns a value (assigned to a variable), uses `->` (no fallthrough), must yield for blocks. Expressions are safer (no forgotten break). Both still exist; use expressions for value-returning switches, statements for side-effecting. 2. If `Shape` is

`sealed` permits `Circle`, `Square`, the compiler knows all subtypes. A `switch` over `Shape` without `default` must cover both → compile error if one is missing. When you add a new subtype ( `Triangle` ), every existing switch over `Shape` fails to compile until you add the case. Forces you to handle the new case everywhere — catches missing logic at compile time (vs runtime `default` falling through). Big win for domain evolution safety.

**RED FLAGS TO AVOID:** - Using old switch with fallthrough when expression is clearer - Adding `default` to a sealed switch (defeats exhaustive checking) - Forgetting `case null` (NPE on null)

---

## Q-BE-45: ThreadLocal — use cases and leaks

**DIFFICULTY:** Medium **CATEGORY TAG:** Backend / Java Concurrency

**ANSWER:** `ThreadLocal` holds a per-thread value. Each thread sees its own copy; no synchronization needed (the value isn't shared).

**Use cases:** - **Per-request context** — correlation ID, user ID, locale. Set in a filter, read in business code, cleared at request end. - **Per-thread singletons** — non-thread-safe objects reused (SimpleDateFormat, JDBC Connection before pools). Each thread has its own. -

**Storing the “current” thing** — current transaction, current security context (Spring's `SecurityContextHolder` uses `ThreadLocal`).

**CODE/EXAMPLE SNIPPET:**

```
public class CorrelationId {
 private static final ThreadLocal<String> ID = new ThreadLocal<>();
 public static void set(String id) { ID.set(id); }
 public static String get() { return ID.get(); }
 public static void clear() { ID.remove(); }
}
// In a filter
try {
 CorrelationId.set(UUID.randomUUID().toString());
 chain.doFilter(req, res);
} finally {
 CorrelationId.clear(); // MUST clear to avoid leak
}
```

**Leaks:** - Thread pools reuse threads. If you set a `ThreadLocal` but don't remove it, the next task on that thread sees the old value (cross-request contamination). - The value holds references → can't be GC'd → memory leak. - Common leak: setting `ThreadLocal` in async tasks (the thread is from a pool, reused).

**MDC (Mapped Diagnostic Context):** SLF4J's per-thread map of context values, used to add to every log line ( `traceId`, `userId` ). Backed by `ThreadLocal`. Must `MDC.clear()` at request end (most logging frameworks do this in their filter).

**InheritableThreadLocal:** child threads inherit the parent's value at creation time. Snapshotted — later parent changes don't propagate. Use sparingly (most use cases want explicit propagation, not inheritance).

**Virtual threads caveat:** virtual threads are cheap → you can have 100K → 100K `ThreadLocals` × large values = lots of memory. Use `ScopedValue` (Java 21 preview) — immutable, bounded scope, less memory.

**ThreadLocal vs context propagation:** - `ThreadLocal` works only within one thread. - Across async (`CompletableFuture`, `ExecutorService`): `ThreadLocal` is lost when the task

switches threads. Use a context-propagation library (Micrometer Context Propagation) that snapshots + restores. - Across services: HTTP headers (traceId, Baggage).

**KEY TERMS TO MENTION:** - ThreadLocal: per-thread value, no sync needed - Use cases: per-request context (corr ID, user ID), per-thread singletons (SimpleDateFormat), current-thing (SecurityContext) - Leaks: thread pools reuse threads → stale value; MUST remove() in finally - MDC: SLF4J's per-thread log context (traceId) - Virtual threads: many VTs × TL = memory; use ScopedValue (Java 21) - Context propagation across async: Micrometer Context Propagation

**FOLLOW-UP QUESTIONS:** 1. Why does ThreadLocal leak in a thread pool? 2. How do you propagate a ThreadLocal across async boundaries?

**FOLLOW-UP ANSWERS:** 1. Thread pools reuse threads. A task sets a ThreadLocal but doesn't remove it. The next task on the same thread sees the old value (or, if the value is a heavy object, holds a reference preventing GC). Over time, every pooled thread holds the cumulative set of leaked ThreadLocals → memory grows. Fix: always `remove()` in a finally block. Detect: track thread pool size + ThreadLocal set count; if set count >> pool size × request count, you're leaking. 2. Use a context-propagation library. Micrometer Context Propagation: registers `ThreadLocalAccessor`s for your context types; Reactor + Spring `@Async` + `CompletableFuture` (with context-aware executors) auto-snapshot on thread switch + restore. Without it, the ThreadLocal value stays on the originating thread; the new thread sees null. Pattern: snapshot manually ( `String id = CorrelationId.get();` ), pass to the async task, set on entry, clear in finally. Or use Reactor's `Context` (immutable, propagates through the reactive chain).

**RED FLAGS TO AVOID:** - Forgetting remove() in finally (leak across requests) - ThreadLocal for shared mutable state (defeats isolation) - Many ThreadLocals on virtual threads (memory)

---

## Q-BE-46: Java IO — BIO, NIO, AIO/async

**DIFFICULTY:** Hard **CATEGORY TAG:** Backend / Java IO

**ANSWER:** Three IO models in Java:

**BIO (Blocking IO):** `java.io` — `InputStream/OutputStream/Reader/Writer`. One thread per connection. The thread blocks on `read()` until data arrives. Doesn't scale beyond ~1000 connections per JVM (thread cost).

**NIO (New IO, Java 1.4):** `java.nio` — channels, buffers, selectors. Non-blocking: a channel can be set non-blocking; `read()` returns immediately (0 bytes if nothing). A selector watches many channels for events (read-ready, write-ready). One thread can handle thousands of connections (event loop). Used by Netty, gRPC, high-performance servers.

**AIO (Asynchronous IO, Java 7 NIO.2):** true async — initiate read, get a `Future` or `CompletionHandler` callback when done. OS notifies on completion. Theoretical best for IO concurrency. In practice, Linux's `epoll+eventfd` support is good but the Java AIO API has quirks; Netty sticks with NIO (more mature + tunable).

**Buffers:** NIO uses `ByteBuffer` (direct or heap). Direct = off-heap (no GC), faster for IO; allocate is expensive (use a pool). Heap = easier, GC'd.

**Selector event loop (Netty-style):**

```
Selector sel = Selector.open();
ServerSocketChannel ssc = ServerSocketChannel.open();
ssc.configureBlocking(false);
ssc.register(sel, SelectionKey.OP_ACCEPT);
while (true) {
 sel.select(); // block until event
 for (var key : sel.selectedKeys()) {
 if (key.isAcceptable()) accept(key); // new conn
 if (key.isReadable()) read(key); // data available
 }
}
```

**Why Netty wins:** it abstracts NIO into a clean API (Channels, EventLoop, ChannelPipeline), handles edge cases (half-close, SSL, thread-bound binding), and adds features (`epoll` native transport on Linux). Most Java async servers (gRPC, Spring WebFlux on Netty, Cassandra driver) build on Netty.

**Choosing:** - Traditional request-response with low concurrency → BIO + servlet container (Tomcat default). - High concurrency, many connections → NIO (Netty, WebFlux). - Truly async, OS callback-driven → AIO (rare; Netty's NIO is more practical). - Java 21 virtual threads + BIO → cheap thread per connection again (best of both: simple code, high concurrency). WebFlux loses its main advantage.

**KEY TERMS TO MENTION:** - BIO: `java.io`, blocking, one thread per conn, doesn't scale beyond ~1000 - NIO: `java.nio`, channels + buffers + selectors, event loop, scales to

thousands - AIO (NIO.2): true async, Future/CompletionHandler callback, less used in practice (Netty uses NIO) - Buffers: direct (off-heap, fast) vs heap (GC'd) - Netty: NIO abstraction with EventLoop + ChannelPipeline + epoll native - Java 21: virtual threads + BIO  $\approx$  simple code + high concurrency (WebFlux loses edge)

**FOLLOW-UP QUESTIONS:** 1. Why does NIO scale better than BIO? 2. When would you still use BIO?

**FOLLOW-UP ANSWERS:** 1. BIO blocks one thread per connection. 10K connections = 10K threads = 10GB stacks + kernel scheduling overhead. NIO uses a selector — one (or few) threads watch many channels via OS events (epoll/kqueue). The OS notifies when a channel has data; the thread reads then moves on. 10K connections = 4 threads. The bottleneck shifts from threads to OS scheduling (which scales). 2. (a) Low concurrency (a few hundred connections) — BIO's simplicity wins, no event loop complexity. (b) CPU-bound work — the IO model doesn't matter. (c) With virtual threads (Java 21) — you write BIO-style code (one virtual thread per connection) but the JVM unmounts on IO block, so you get NIO-like scalability. The virtual-thread + BIO combination is the new default for IO-bound request handling.

**RED FLAGS TO AVOID:** - BIO for high concurrency (thread explosion) - Direct ByteBuffer without pooling (alloc is expensive) - AIO expecting better perf than NIO (Netty benchmarks show NIO wins)

---

## Q-BE-47: Heap dump analysis with Eclipse MAT

**DIFFICULTY:** Hard **CATEGORY TAG:** Backend / JVM Diagnostics

**ANSWER:** A heap dump ( `.hprof` ) is a snapshot of all live objects + their references. Eclipse MAT (Memory Analyzer Tool) is the gold standard for analysis.

**Capture:** - `-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/tmp/heap-%p.hprof` (auto on OOM). - `jcmd <pid> GC.heap_dump /tmp/heap.hprof` (live). - `jmap -dump:format=b,file=heap.hprof <pid>` (legacy).

**Open in MAT:** MAT parses, indexes, shows overview (size, # objects, biggest classes).

**Key reports:** 1. **Leak Suspects Report** — auto-finds suspiciously large retainers. Quick triage. Often pinpoints the leak in 1 paragraph. 2. **Dominator Tree** — objects sorted by retained size (memory that would be freed if this object were GC'd). Top entries = biggest retainers. 3. **Histogram** — class → instance count + size. Sort by shallow/retained size. Watch for unexpected classes (e.g. `byte[]` huge → some cache holds large blobs). 4. **Top Consumers** — biggest objects. 5. **Duplicate Classes** — same class loaded by multiple classloaders (Metaspace leak).

**Shallow vs retained size:** - Shallow = the object's own memory (header + fields). - Retained = shallow + memory freed when the object is GC'd (everything only reachable through this object). Retained is what matters for leak analysis.

**Path to GC roots:** - Right-click an object → "Path To GC Roots → exclude weak/soft references". Shows the chain of strong refs keeping it alive. - The chain reveals the cause (e.g. "HashMap in static field of Config class → 100K entries").

**OQL (Object Query Language):** SQL-like queries on the heap:

```
SELECT * FROM com.example.Order WHERE total > 1000
SELECT s, s.toString() FROM java.lang.String s WHERE s.@retainedHeapSize > 10000
```

**Common leak patterns MAT reveals:** - `HashMap$Node` with huge count → unbounded cache. - `byte[]` retained by `String` → holding file contents / payloads. - `Thread` objects → thread leak (unclosed executor). - Class objects from one classloader → redeploy leak.

**Performance tip:** large heaps (8GB+) take minutes to parse + GBs of MAT's own heap. Open with `-Xmx` large enough; use MAT's "leak suspects" report (fastest) before deep dives.

**KEY TERMS TO MENTION:** - Capture: `HeapDumpOnOutOfMemoryError`, `jcmd` `GC.heap_dump` - MAT reports: Leak Suspects (auto), Dominator Tree (retained size), Histogram (per class), Path to GC Roots - Shallow (own memory) vs retained (memory freed if GC'd) — retained matters for leaks - OQL: SQL-like queries on heap - Patterns: `HashMap$Node` huge (cache), `byte[]` (payloads), `Thread` (leaked executor), `Class` (redeploy leak) - Large heap → MAT needs big `-Xmx` too

**FOLLOW-UP QUESTIONS:** 1. How do you find which thread created an object in MAT? 2. What's the difference between a shallow and retained heap?

**FOLLOW-UP ANSWERS:** 1. Not directly — heap dumps don't record creating threads. But you can: (a) inspect the object's fields (often reveals the context — e.g. `Thread.currentThread` field, request object); (b) "incoming references" shows who holds it; (c) for ThreadLocal leaks, the Thread object shows its ThreadLocalMap (per-thread ThreadLocals); (d) trace from the Thread's stack (if the dump captured stacks, which `jcmd` can). For diagnosis, the "who holds it" matters more than "who created it." 2. Shallow = the object's own bytes (header + fields, e.g. an ArrayList's shallow is ~40 bytes regardless of contents). Retained = shallow + everything that would be GC'd if this object were (the ArrayList's retained includes the backing array + all elements only reachable via it). For leak hunting, retained is what matters — a 10MB ArrayList with 1M String elements has shallow ~40 bytes but retained ~50MB. Sort Dominator Tree by retained.

**RED FLAGS TO AVOID:** - Opening a 16GB heap dump with default MAT -Xmx (MAT will OOM) - Looking at shallow size (misleading) - Not excluding soft/weak refs in "Path to GC Roots" (noise)

---

## Q-BE-48: Diagnosing a CPU spike in production

**DIFFICULTY:** Hard **CATEGORY TAG:** Backend / JVM Diagnostics

**ANSWER:** A CPU spike playbook:

**Step 1: Identify the process + thread.**

```
top -H -p <pid> # top threads (H shows threads)
Note the high-CPU thread ID (e.g. 12345)
printf "%x\n" 12345 # convert to hex (e.g. 3039) for jstack
jstack <pid> > dump.txt
grep "nid=0x3039" dump.txt -A 30 # find that thread's stack
```

**Step 2: Look at the stack.** Common causes: - Infinite loop ( `while(true)` without break). - Regex catastrophic backtracking ( `(a+)+b` against `aaaaaaaaaaaaaaaaa!` ). - Hash collision attack (all keys hash to one bucket in HashMap →  $O(n^2)$  lookup; ConcurrentHashMap →  $O(n)$  on a list-bin). - Hot lock contention (threads spinning on a synchronized block). - Busy-wait (loop with `sleep(0)`). - GC running wild (frequent full GCs → CPU-bound). Check GC log + `jstat -gcutil`.

**Step 3: Profile.** - Async-profiler — flame graphs. `./profiler.sh -d 30 -f flame.html <pid>` — samples CPU, no overhead. Best for prod. - JFR (Java Flight Recorder) — built-in, low overhead. `jcmd <pid> JFR.start duration=60s filename=recording.jfr` then open in JDK Mission Control. - VisualVM (dev).

**Step 4: Fix based on cause.** - Infinite loop → fix code. - Regex → fix the pattern (avoid nested quantifiers, use possessive quantifiers `++`, or use RE2J (no backtracking)). - Hash attack → use a strong hash (String.hashCode is weak; use SipHash with random seed, or a Map with tree fallback on collisions). - Lock contention → reduce critical section, use lock-free structures, partition the lock. - GC → fix the leak (heap dump) or tune.

**Other diagnostic commands:**

```
vmstat 1 # CPU (us/sy/id), context switches, IO wait
iostat -x 1 # disk IO
pidstat -p <pid> 1 # per-process CPU
jstat -gcutil <pid> 1s # GC frequency
jcmd <pid> Thread.print # alternative jstack
```

**Watch for false positives:** - A thread at 100% CPU for 30 seconds is normal (GC, batch). Sustained high CPU is the issue. - “CPU” includes user + system + iowait. iowait is the CPU idle waiting on disk — not real CPU usage. High “load avg” with low CPU often = iowait.

**KEY TERMS TO MENTION:** - top -H to find high-CPU thread; jstack + nid (hex) to find stack - Common causes: infinite loop, regex backtracking, hash attack, lock contention, busy-wait, GC churn - Async-profiler (flame graphs, no overhead, prod-safe) - JFR (Java Flight Recorder,

built-in, JDK Mission Control) - vmstat/iostat/pidstat for OS-level - Fix per cause: code, regex, hash, locks, GC

**FOLLOW-UP QUESTIONS:** 1. How do you profile without overhead in production? 2. How do you tell CPU-bound vs IO-bound?

**FOLLOW-UP ANSWERS:** 1. Async-profiler — uses Linux perf\_events (or macOS's stackshot), samples at N Hz (default 10ms), the JVM isn't aware (no JVMTI overhead). CPU + heap + lock profiling, all low-overhead. Or JFR (Java Flight Recorder) — built into the JDK, designed for always-on prod profiling (<1% overhead). Both produce flame graphs (async-profiler via `flame.html`, JFR via JMC or `jfr2flame`). Avoid JProfiler/YourKit in prod (significant overhead); use them in dev only. 2. CPU-bound: CPU% high (user), iowait low, throughput scales with cores. The threads are in `RUNNABLE` state, doing computation (no IO in stack). Fix: optimize code, parallelize. IO-bound: CPU% low, iowait high (or wait time high), throughput scales with concurrency (not cores). Threads in `WAITING` / `TIMED_WAITING` (waiting on IO). Fix: increase concurrency (more threads, async IO, virtual threads), cache, faster storage. A “high CPU” complaint that turns out to be IO-bound → adding CPU doesn't help; reduce IO (caching, batching) or add concurrency.

**RED FLAGS TO AVOID:** - Assuming high CPU = bad code (could be GC, could be legitimate load) - Profiling with heavy tools in prod (overhead skews results) - Not checking iowait vs user CPU (different problems)

---

## Q-BE-49: Microservices communication patterns — saga, outbox

**DIFFICULTY:** Expert **CATEGORY TAG:** Backend / Microservices

**ANSWER:** Microservices need to coordinate state changes across services without a distributed transaction (which doesn't scale).

**Two-phase commit (2PC):** a coordinator asks all participants to prepare, then commit. Slow, blocking, single point of failure. Rarely used in microservices.

**Saga pattern:** a sequence of local transactions, each with a compensating action. If step N fails, run compensations for steps N-1 down to 1. - **Choreography:** services publish events; each listens + reacts. No central coordinator. Decentralized but harder to follow the flow. - **Orchestration:** a saga orchestrator sends commands to services, tracks state, calls compensations on failure. Centralized, easier to visualize, but the orchestrator is a single point of failure (HA via DB/cluster).

**CODE/EXAMPLE (orchestrated saga — order + payment + inventory):**

```
class OrderSaga {
 void create(Order o) {
 try {
 payment.charge(o); // T1
 inventory.reserve(o); // T2
 shipping.schedule(o); // T3
 } catch (Exception e) {
 compensate(o); // rollback in reverse
 }
 }
 void compensate(Order o) {
 if (shippingScheduled) shipping.cancel(o);
 if (inventoryReserved) inventory.release(o);
 if (paymentCharged) payment.refund(o);
 }
}
```

**Transactional outbox pattern:** solve the dual-write problem (DB write + message publish aren't atomic). - Problem: `db.save(order); kafka.send(event);` — if Kafka send fails, the order is saved but no event; if DB commits but the producer crashes, no event. Either way, inconsistent. - Solution: write the order + an `outbox` row in the `same DB transaction`. A separate process (CDC) reads the outbox table + publishes to Kafka. The DB write is atomic; the publish is best-effort (retries on failure; the outbox row marked "published" when done). - CDC tools: Debezium (PG/MySQL → Kafka), custom poller (simpler but less efficient).

**Outbox schema:**

```
CREATE TABLE outbox (
 id BIGINT PK,
 aggregate_type VARCHAR, -- "Order"
 aggregate_id VARCHAR, -- order ID
 event_type VARCHAR, -- "OrderCreated"
 payload JSONB, -- the event body
 created_at TIMESTAMP,
 published_at TIMESTAMP NULL -- null until published
);
```

**Idempotency:** consumers dedupe by event ID (outbox row id as event id). Outbox id is unique → no duplicate publishes; consumer stores processed IDs.

**Other patterns:** - **API composition** (aggregator): one service calls N others + aggregates. Synchronous (downstream failures affect you). - **CQRS:** separate write model (commands) from read model (projections). Events update the read model. - **Event sourcing:** store events as the source of truth; state derived by replay. Heavy; use when audit + replay matter. - **Strangler fig:** gradually replace a legacy service — new code intercepts calls, handles them, falls back to legacy for the rest.

**Real-world:** Uber (saga for trip flow), Airbnb (outbox for their data pipeline), Netflix (orchestrated sagas for billing).

**KEY TERMS TO MENTION:** - 2PC: slow, blocking, single point of failure — not for microservices - Saga: sequence of local tx + compensations; choreography (events) vs orchestration (central) - Outbox: solve dual-write (DB + Kafka non-atomic) — write order + outbox row in one tx, CDC publishes - Debezium CDC; outbox row id = event id for idempotency - Other: API composition, CQRS, event sourcing, strangler fig - Real: Uber trip saga, Airbnb outbox, Netflix billing saga

**FOLLOW-UP QUESTIONS:** 1. Choreography vs orchestration — when which? 2. How does Debezium avoid losing events?

**FOLLOW-UP ANSWERS:** 1. Choreography: simpler to add a new service (just subscribe to events), no central coordinator to maintain. Works when the flow is simple (3-4 steps) + you don't need a global view. Breaks down at 5+ steps (the flow becomes hard to follow, debug, change). Orchestration: easier to follow the flow (one place), easier to add compensations, supports long-running sagas (persisted state). The orchestrator is a single point of failure (HA via DB-backed state + leader election). Use orchestration for complex flows + when you need observability. 2. Debezium reads the DB's WAL (PostgreSQL's logical replication, MySQL's binlog) — every committed change is in the log. The connector tracks its position (LSN for PG, GTID/binlog position for MySQL) in a Kafka topic (`dbhistory`); on restart, it resumes from the last position. The WAL is durable (committed = on disk) → no loss. Even if Debezium crashes, the DB keeps the WAL until Debezium catches up (with retention). The at-least-once guarantee: if Debezium publishes an event but the position update fails, it re-publishes on restart (idempotent consumers handle duplicates).

**RED FLAGS TO AVOID:** - Distributed transactions (2PC) in microservices (doesn't scale) - Dual-write without outbox (lost or duplicated events) - Saga without idempotent compensations (rollback failures)

---

## Q-BE-50: Outbox pattern + idempotency keys in detail

**DIFFICULTY:** Expert **CATEGORY TAG:** Backend / Distributed Systems

**ANSWER:** The outbox pattern solves “I want to update my DB and publish an event atomically” — without 2PC.

**The problem:**

```
@Transactional
public void placeOrder(Order o) {
 repo.save(o);
 kafka.send(new OrderCreated(o.id())); // can fail: crash, network, timeout
}
```

If `kafka.send` throws: the tx rolls back → no order saved (good). If `kafka.send` “succeeds” but the broker didn’t ack (network blip): the tx commits, no event → downstream never sees the order. If `kafka.send` succeeds + the tx rolls back (DB fails after send): downstream sees an event for an order that doesn’t exist. Any combination of failures can break consistency.

**Outbox solution:**

```
@Transactional
public void placeOrder(Order o) {
 repo.save(o);
 outboxRepo.save(new OutboxRow("Order", o.id(), "OrderCreated", toJson(o))); //
 same tx
}
```

The DB write + outbox write are atomic. The Kafka publish is decoupled.

**Publisher (separate process):** - CDC (Debezium) reads the outbox table → publishes to Kafka → updates a “published\_at” column. - Or a poller: `SELECT * FROM outbox WHERE published_at IS NULL ORDER BY created_at LIMIT 100`, publishes, marks published. Simpler, less efficient than CDC. - Either way: at-least-once (retries on failure). Idempotent consumers dedupe.

**Outbox table:**

```
CREATE TABLE outbox (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 aggregate_type VARCHAR NOT NULL,
 aggregate_id VARCHAR NOT NULL,
 event_type VARCHAR NOT NULL,
 payload JSONB NOT NULL,
 headers JSONB, -- traceId, userId, etc.
 created_at TIMESTAMP NOT NULL DEFAULT now(),
 published_at TIMESTAMP,
 INDEX (published_at, created_at) -- for the poller
);
-- Cleanup job: DELETE FROM outbox WHERE published_at < now() - INTERVAL '7 days';
```

**Idempotency key:** the outbox row ID (UUID) is the event ID. Consumers store processed IDs (Redis or DB) → skip duplicates.

```
@KafkaListener(topics = "orders")
public void handle(OrderEvent e) {
 if (processedStore.contains(e.id())) return; // dedupe
 orderService.process(e); // idempotent in itself
 processedStore.mark(e.id(), Duration.ofDays(7));
}
```

**Ordering:** within an aggregate, events must be processed in order. Options: - Same aggregate ID → same partition (key = aggregate ID). Kafka guarantees per-partition order. - Consumer applies events in order; skip duplicates by version (event N+1 only if N processed).

**Outbox + event sourcing:** if you want to replay state, the outbox IS the event log. Append-only (no `published_at` mutation — use a separate `publish_state` table). Re-derive state by replaying events.

**Real-world implementation tips:** - Outbox row size: keep payload small (KB, not MB). Large payloads → publish a reference + store the body elsewhere (S3). - Backpressure: if Kafka is down, outbox grows. Monitor + alert. The poller should back off (not hammer). - Schema evolution: the payload is JSON — design for backward compat (add fields, don't remove). - Multiple publishers: if multiple consumers want the same event, publish to Kafka + each consumer subscribes (Kafka fan-out). Don't have multiple outbox tables for one event.

**Twist — distributed outbox with sticky ordering:** if events must be ordered across partitions (rare), use a single partition (kills parallelism) or a sequence number that the consumer tracks + buffers out-of-order events.

**KEY TERMS TO MENTION:** - Outbox: DB write + outbox row in one tx, separate publisher (Debezium CDC or poller) - Atomic DB + event (solves dual-write) - At-least-once from publisher; idempotent consumers (dedupe by event ID = outbox row UUID) - Per-aggregate ordering: same aggregate ID → same partition (Kafka key) - Cleanup old published rows;

keep payload small (KB); backpressure when Kafka down - Event sourcing: outbox IS the event log (append-only) for replay

**FOLLOW-UP QUESTIONS:** 1. Why not just publish to Kafka after the DB commit (without outbox)? 2. How do you handle the case where the publisher crashes mid-publish?

**FOLLOW-UP ANSWERS:** 1. The race window: DB commit succeeds, the JVM crashes before `kafka.send`. Or `kafka.send` succeeds locally but the broker didn't ack (network). In either, the event is lost. The outbox removes this window: the event is durable in the DB (committed with the business write), the publisher retries until success. The cost: an extra table + a publisher process + idempotency. The benefit: no lost events. For events that matter (financial, audit, downstream state), it's worth it. For low-stakes events (telemetry), fire-and-forget without outbox is fine. 2. At-least-once. The publisher tracks its position (Debezium: LSN/binlog; poller: `published_at`). If it crashes after publishing but before updating the position, on restart it re-publishes the same events. Idempotent consumers dedupe by event ID. So a crash mid-publish just means some duplicates (handled). The invariant: every committed outbox row is published at least once. The publisher never loses a row (DB is the source of truth until marked published). It may re-publish (deduped by consumer).

**RED FLAGS TO AVOID:** - Dual-write without outbox (lost events) - Poller without back-off (hammers DB + Kafka when down) - Large payloads in outbox (table bloat, slow publishes)

---

## PART 11 — BONUS: ADVANCED & TWISTED QUESTIONS (Uber / Meta / Amazon Drill-Downs)

**Purpose of this appendix.** The sections above are the [base reference](#). The questions below are the **twisted, end-to-end, “what if the world breaks”** variants that Uber / Meta / Amazon interviewers use to separate mid-level from senior. Each includes (where relevant) a code snippet, an ASCII diagram, or a schema. Treat these as the final stretch.

### Q-BNS-1: Design a distributed rate limiter that works across a cluster of 50 nodes with a global limit of 1000 req/s, preserving per-client fairness. Walk through the algorithm and failure modes.

- **Difficulty:** Expert
- **Category:** System Design / Distributed Systems
- **Tags:** rate-limit, token-bucket, redis, lua, fairness

**ANSWER:** A single-node token bucket is trivial; the hard part is a [global](#) limit across 50 nodes. Three designs, escalating in correctness:

**Design A — Shared Redis counter (naive):** every node does `INCR client:counter` and checks `if > 1000 → 429`. Problem: INCR is atomic but the check-then-reject is not — 50 nodes can all read 999 and all increment to 1049 before any rejects. Over-admits under contention.

**Design B — Redis Lua atomic token bucket (correct):** a Lua script does `INCR + EXPIRE-if-new + compare` atomically in Redis. No race. The bucket lives in Redis (single source of truth). Every request round-trips to Redis (~1ms). Works, but every request now has a Redis dependency.

**Design C — Token-bucket with local leases (best for 50 nodes):** each node periodically [leases](#) a slice of the global rate from Redis — e.g., node N asks “give me 20 tokens for the next 100ms.” Redis atomically decrements the global bucket and grants the lease. The node serves 20 requests locally (no Redis round-trip per request), then renews. Global limit preserved (sum of leases  $\leq$  1000), per-request latency is local (~0.1ms), Redis load is 50 leases/100ms = 500/s (not 1000/s). Failure mode: if a node crashes holding a lease, those tokens are “lost” until expiry (the lease has a TTL); the global limit drops by that node’s share for the TTL window — bounded waste.

**Code (Design C, lease-based):**

```
// Redis Lua: atomically grant a lease from the global bucket
private static final String LEASE_LUA = """
 local key = KEYS[1]
 local now = tonumber(ARGV[1])
 local lease = tonumber(ARGV[2]) -- tokens requested
 local ttl = tonumber(ARGV[3]) -- lease validity (ms)
 local cap = tonumber(ARGV[4]) -- global capacity
 local cur = tonumber(redis.call('GET', key) or cap)
 if cur < lease then return 0 end -- not enough global tokens
 redis.call('DECRBY', key, lease)
 redis.call('PEXPIRE', key, ttl) -- refilled by a separate
 refiller? No – see below
 return lease
 """;

// Local: serve from the local lease until empty, then re-lease
public boolean allow(String client) {
 long now = System.currentTimeMillis();
 LeaseBucket b = buckets.computeIfAbsent(client, k -> new LeaseBucket());
 synchronized (b) {
 if (b.tokens > 0 && now < b.expiresAt) { b.tokens--; return true; }
 // Renew
 int granted = redis.eval(LEASE_LUA, List.of("rl:"+client), now, 20, 100, 1000);
 if (granted == 0) return false;
 b.tokens = granted; b.expiresAt = now + 100;
 b.tokens--; return true;
 }
}
}
```

The **refill** of the global bucket: a token bucket refills at **rate** tokens/sec. In Design C, a separate **@Scheduled** job (or the lease Lua itself) adds **rate \* elapsed** tokens back to the global bucket, capped at **capacity**. The lease is a **withdrawal**; the refill is a **deposit**. Both must be atomic (Lua).

**Per-client fairness:** the limit is **per client** (API key / IP), not global-aggregate. Each client has its own bucket in Redis. A noisy client can't starve others — they each have their own 1000/s. A global aggregate limit (1000/s total) is a different design (a single bucket shared by all clients) — used for backend protection, not per-client SLAs.

**KEY TERMS TO MENTION:** - Redis Lua atomic check-and-decrement (no race) - Lease-based local serving (Redis round-trip amortised, not per-request) - Lease TTL — bounded token loss on node crash - Refill job (rate \* elapsed, capped) — separate from leases - Per-client bucket for fairness; global aggregate for backend protection

**FOLLOW-UP QUESTIONS:** 1. Redis is down. Do you fail open or closed? 2. The lease is 20 tokens / 100ms. What if a single client bursts 50 req in 10ms? 3. How do you handle a 50-node cluster where 10 nodes are hot and 40 are idle?

**FOLLOW-UP ANSWERS:** 1. Fail-open (serve locally, best-effort) for read endpoints, fail-closed for write/spend endpoints. With leases, a Redis outage means nodes can't renew — they exhaust their lease in 100ms and then must decide. We fail-open on reads (over-allow briefly, alert), fail-closed on writes (return 503, **Retry-After: 1**). The trade-off is availability

vs correctness; a 1-minute Redis outage failing-closed on writes is better than unlimited writes. 2. The local bucket only has 20 tokens; the 21st request in the 10ms window gets 429, **even though the global bucket has capacity**. This is the “local over-restriction” cost of the lease design — you under-admit bursts. Fix: a larger lease (50 tokens/100ms) reduces false rejects but increases the “lost tokens on crash” waste. Tune to the burstiness profile. Alternatively, a request that finds the local lease empty can do a synchronous “borrow” Lua call (slow path, but admits the burst). 3. Hot nodes renew leases faster than idle ones — the Redis lease call is per-bucket, and a hot node has more active buckets. Idle nodes hold leases they’re not using (waste). A “lease return” on idle (the node returns unused tokens to the global bucket when its lease is half-used) recovers them. Or a per-node lease cap ( `max_lease_per_node` ) so one hot node can’t hoard the global budget — it gets a slice, others get theirs. The exact tuning is workload-specific; measure and adjust.

**RED FLAGS TO AVOID:** - Naive INCR + check in application code — the check-then-reject race over-admits. - A global counter without per-client buckets — one client’s burst starves others. - Lease with no TTL — a crashed node holds tokens forever.

## Q-BNS-2: Implement consistent hashing for a distributed cache of 100 nodes. Walk through the ring, virtual nodes, and the re-distribution math on add/remove.

---

- **Difficulty:** Hard
- **Category:** System Design / Distributed Systems / Algorithms
- **Tags:** consistent-hashing, ring, virtual-nodes, cache

**ANSWER:** Consistent hashing distributes keys across nodes such that adding or removing a node only moves  $\sim 1/N$  of the keys (vs. `mod N` which moves nearly all keys on any change). The keys and nodes are placed on a ring (a hash space  $0..2^{32}-1$ ); a key maps to the **next** node clockwise. Virtual nodes (VNodes) — each physical node appears at K positions on the ring — fix the load-imbalance problem (with plain consistent hashing, one node can get 2-3x its fair share).

**Algorithm:** 1. Hash each node ID (e.g., `node-1`) to a position on the ring, K times with different salts ( `node-1#0` , `node-1#1` , ..., `node-1#199` ). 2. Sort the positions; store in a sorted map (position  $\rightarrow$  physical node). 3. For a key: hash it to a position; binary-search the ring for the next position clockwise; return the node.

**Code (Java, 100 nodes, 200 vnodes each = 20000 positions):**

```

public class ConsistentHash {
 private static final int VNODES = 200;
 private final TreeMap<Long, String> ring = new TreeMap<>(); // hash → nodeId
 private final HashFunction hash = Hashing.murmur3_128();

 public void addNode(String nodeId) {
 for (int i = 0; i < VNODES; i++) {
 long h = hash.hashString(nodeId + "#" + i).asLong();
 ring.put(h, nodeId);
 }
 }

 public void removeNode(String nodeId) {
 for (int i = 0; i < VNODES; i++) {
 ring.remove(hash.hashString(nodeId + "#" + i).asLong());
 }
 }

 public String nodeFor(String key) {
 if (ring.isEmpty()) throw new IllegalStateException("no nodes");
 long h = hash.hashString(key).asLong();
 SortedMap<Long, String> tail = ring.tailMap(h);
 long target = tail.isEmpty() ? ring.firstKey() : tail.firstKey();
 return ring.get(target);
 }
}

```

**Re-distribution math:** with N nodes and K vnodes, the ring has NK positions. Adding a node adds K random positions; each new position “steals” the keys that were previously assigned to the next-old-position clockwise. The expected fraction of keys moved =  $K / (NK) = 1/N$ . So adding the 101st node moves ~1% of keys — only those keys need re-fetching from the backing store. Compare to  $\text{hash}(\text{key}) \bmod 100 \rightarrow \text{hash}(\text{key}) \bmod 101$ : ~99% of keys move.

### Diagram:

```

ring 0 ----- 2^32
 0x12(node-3#5) 0x40(node-1#12) 0x88(key-A) 0xC0(node-2#7)
 | | | |
key-A (0x88) → next clockwise = node-2 (0xC0) → cached on node-2.
Add node-4 at 0x90: keys in (0x88, 0x90] now go to node-4; key-A still → node-2.
Only keys in node-4's new "slice" move (~1% of all keys).

```

**VNodes fix skew:** without vnodes, with 100 nodes the most-loaded node has ~1.5-3x the average (hash placement is uneven). With K=200 vnodes, the most-loaded is within ~10% of average (concentration of measure). The cost is more ring memory (20k entries) and slower lookups (binary search on 20k = ~15 comparisons, still sub-microsecond).

**KEY TERMS TO MENTION:** - Ring (hash space  $0..2^k-1$ ), clockwise assignment - Virtual nodes (VNodes) for load balance (K=100-300 typical) - Re-distribution cost =  $1/N$  on add/remove (vs ~all on mod N) - `TreeMap.tailMap` for  $O(\log N)$  lookup (binary search on sorted positions) - Murmur3 hash (fast, well-distributed; not cryptographic)

**FOLLOW-UP QUESTIONS:** 1. A node is “slow but up” — requests time out. How do you shed it without removing it? 2. Two keys that must co-locate (e.g., user + their cart) — how? 3. The ring has 100 nodes; you add 10 more at once. How do you avoid a thundering-herd of cache misses?

**FOLLOW-UP ANSWERS:** 1. Mark the node “quarantined” in the ring (a flag), and the `nodeFor` method skips quarantined nodes (jumps to the next). Quarantine is time-bounded (e.g., 60s) with a circuit-breaker driving it; if the node recovers, un-quarantine. Don’t physically remove it (removal triggers re-distribution, expensive); just route around it temporarily. This is the “governor” pattern. 2. Use a **compound key** with the same hash prefix — hash `user:123` and `user:123:cart` both → same node because the router hashes the prefix `user:123`. Or use a “affinity key” field in the key that the router hashes (the part before `:`), so all keys with the same affinity co-locate. DynamoDB’s “partition key” works this way. 3. Add them one at a time with a delay (e.g., 30s apart). Each addition moves 1% of keys; 10 additions move 10% — but spread over 5 minutes, the cache-miss storm is amortised. Also: when a new node boots, it pre-warms by fetching the top-K keys it will own (compute via the ring before going live) from the backing store in the background, so when traffic shifts, the cache is warm. This is “warm-up on join.”

**RED FLAGS TO AVOID:** - No vnodes — load imbalance of 2-3x is typical, and one node OOMs while others are idle. - MD5/SHA hash for the ring — slow; consistent hashing doesn’t need cryptographic strength. - Physically removing a node on a transient failure — the re-distribution is permanent and expensive; quarantine instead.

**Q-BNS-3: You have two data centres, active-active. A user in DC-A updates their profile; the same user (via a flaky connection) is also updating in DC-B. How do you converge without losing either write?**

- **Difficulty:** Expert
- **Category:** Distributed Systems / Databases
- **Tags:** active-active, crdt, vector-clock, last-write-wins, conflict-resolution

**ANSWER:** Active-active multi-region is the hardest consistency problem. Three strategies, escalating in sophistication:

**A — Last-Write-Wins (LWW) with clock skew bound:** each write carries a timestamp; on conflict, the later timestamp wins. Problem: clock skew between DCs can be 50-200ms; the “later” write may actually have happened first. We bound skew with NTP + a bound check (reject writes with a clock > local+skew). This loses one of the two writes — acceptable for low-stakes fields (display name), unacceptable for high-stakes (balance).

**B — Vector clocks + application merge:** each replica maintains a vector clock `{DC-A: 5, DC-B: 3}`; on conflict, the resolver sees both clocks are concurrent (neither dominates) and calls an application-specific merge. For a “profile” (a JSON object), merge field-by-field ( `name` from A, `avatar` from B). For a counter, use a CRDT (e.g., G-counter: each DC has its own sub-counter; the value = sum of all sub-counters; conflicts merge by taking max per-DC). This never loses data but requires CRDT-aware data types.

**C — CRDTs (Conflict-free Replicated Data Types):** data structures that merge automatically — G-Counter, PN-Counter, OR-Set, LWW-Register, etc. Each replica converges to the same value given the same set of updates regardless of order. DynamoDB, Riak, and Redis CRDTs (Redis Enterprise Active-Active) use this. The cost: CRDTs are type-restrictive (you can't have an arbitrary JSON merge without defining it).

For AcquisitionOS's profile field, **LWW with bounded skew** is fine — a 50ms-old display name is acceptable. For the credit balance, we'd use a **PN-Counter CRDT** (each DC has `p_a`, `n_a` and `p_b`, `n_b`;  $\text{value} = (p_a + p_b) - (n_a + n_b)$ ; credits spent in DC-A increment `n_a`, credits spent in DC-B increment `n_b`; they converge). But we don't actually run active-active for credits — we run active-passive with the credit DB in a single region (strong consistency), because the complexity of CRDT credits isn't worth the availability gain for a billing field.

**The honest answer for an interview:** "For a B2B SaaS, I would not run active-active on the strong-consistency data (billing, leads). The cost of CRDTs or 2PC is too high. We run active-passive with async replication (RPO ~1s) and a manual promotion runbook. Active-active is reserved for read-heavy caches (a CDN or Redis Active-Active) where staleness is acceptable. If the business truly needs active-active multi-region writes, we'd use a CRDT-aware store (DynamoDB Global Tables, CockroachDB, or YugaByte) and design the data types to be CRDT-friendly from day one."

**Code (LWW register with vector-clock awareness — pseudo):**

```
record LwwRegister<T>(T value, long timestampNanos, String writerDc) {
 LwwRegister<T> merge(LwwRegister<T> other) {
 if (this.timestampNanos > other.timestampNanos) return this;
 if (this.timestampNanos < other.timestampNanos) return other;
 // Tie-break by DC id (deterministic)
 return this.writerDc.compareTo(other.writerDc) > 0 ? this : other;
 }
}
```

**KEY TERMS TO MENTION:** - LWW with bounded clock skew (NTP, reject > local+skew) - Vector clocks for detecting concurrent writes - CRDTs (G-Counter, PN-Counter, OR-Set, LWW-Register) for automatic convergence - Active-active vs active-passive trade-off (CRDT cost vs 2PC cost vs RPO) - DynamoDB Global Tables / CockroachDB as CRDT-aware stores

**FOLLOW-UP QUESTIONS:** 1. Why not just use 2PC (two-phase commit) across the two DCs? 2. The PN-Counter — how does a refund (a negative credit) work? 3. The user sees a stale profile after their update. How do you fix the read path?

**FOLLOW-UP ANSWERS:** 1. 2PC blocks both writes until both DCs ack; if DC-B is slow or partitioned, DC-A's write stalls (or the coordinator holds locks). 2PC is a CP choice (consistency over availability) — under a partition, you can't write at all. For active-active, we want AP (availability + eventual consistency), which 2PC doesn't provide. 2PC is fine for same-DC transactions; across DCs with real latency, it kills throughput. 2. A PN-Counter is a (P-counter, N-counter) pair. Increments go to P (per-DC sub-counters); decrements go to N.  $\text{Value} = \text{sum}(P) - \text{sum}(N)$ . A refund is an increment to P (a +credit), not a decrement to N.

Merges take per-DC max of each sub-counter. So a refund in DC-A and a spend in DC-B converge:  $P_a += \text{refund}$ ,  $N_b += \text{spend}$ ;  $\text{value} = (P_a + P_b) - (N_a + N_b) = \text{correct}$ . The catch: PN-counters can't go below zero safely (the value is just  $P-N$ ), so a "no negative balance" rule must be enforced at the write side (a DC rejects a decrement that would make its **local**  $N$  exceed the global  $P$  — but it doesn't know the global  $P$  without sync). This is why we don't use CRDTs for billing. 3. After a write, the read should go to the same DC that just wrote (sticky), with a "read-your-writes" guarantee via a session token. The token carries the user's last-write timestamp; the read replica refuses to serve if its replication lag is older than that timestamp (it waits or returns a "retry in 100ms" 309). This is "read-your-writes consistency" (a session guarantee). Without it, an active-active read can show stale data right after a write, which users notice immediately.

**RED FLAGS TO AVOID:** - "We use LWW for everything" — LWW loses data on concurrent writes; for billing it's a bug. - "We use 2PC across regions" — 2PC kills availability under partitions; not active-active. - No clock-skew bound — unbounded skew means LWW can pick the wrong winner.

### Q-BNS-4: Design a distributed lock with a fencing token that survives a GC pause. Walk through the bug and the fix.

- **Difficulty:** Expert
- **Category:** Distributed Systems / Concurrency
- **Tags:** distributed-lock, fencing-token, gc-pause, martin-kleppmann

**ANSWER:** This is the classic Martin Kleppmann critique of Redis-based distributed locks. The bug: client A acquires a lock (Redis `SET NX EX 10`). A then hits a 60s GC pause. During the pause, the lock TTL (10s) expires; Redis auto-releases. Client B acquires the lock. Both A and B now believe they hold it. When A's GC ends, A writes — clobbering B's write. The lock was supposed to serialise; it didn't.

The fix is a **fencing token**: the lock acquisition returns a monotonically increasing number (a global counter). A gets token 5, B gets token 6. Both write to a downstream store (DB, file) that checks the token — the DB only accepts writes with a token  $\geq$  the last-seen token for that resource. A's late write (token 5) is rejected because the DB has already seen 6. B's write (token 6) succeeds. The lock didn't prevent A from **thinking** it held the lock, but the **downstream** rejected A's stale write.

#### Without fencing (the bug):

```
T0 A: SET lock NX EX 10 → OK (A holds)
T1 A: starts long GC pause (60s)
T10 lock TTL expires → Redis releases
T11 B: SET lock NX EX 10 → OK (B holds)
T12 B: WRITE file="B's value" → file is now B's
T60 A: GC ends, A thinks it holds → WRITE file="A's value" → file is now A's (B lost!)
```

#### With fencing (the fix):

```

T0 A: SET lock NX EX 10 → token 5 (A holds)
T1 A: GC pause
T10 lock expires
T11 B: SET lock NX EX 10 → token 6 (B holds)
T12 B: WRITE file, fence_token=6 → file.token=6, file.value="B"
T60 A: GC ends → WRITE file, fence_token=5 → DB checks 5 < 6 → REJECT. B's write survives.

```

### Implementation (Redis + monotonic counter):

```

-- Acquire (Lua, atomic):
local cur = redis.call('GET', KEYS[1])
if cur == false then
 local tok = redis.call('INCR', 'fence:counter')
 redis.call('SET', KEYS[1], tok, 'NX', 'EX', ARGV[1])
 return tok -- 0 = not acquired; >0 = fencing token
else return 0 end

-- Release (Lua, compare-and-delete so you don't delete another's lock):
if redis.call('GET', KEYS[1]) == ARGV[1] then
 return redis.call('DEL', KEYS[1])
else return 0 end

```

### Downstream check (the critical part):

```

-- The protected write must include the fencing token; the DB rejects stale tokens.
UPDATE invoices SET status='PAID', fence_token = :newTok
WHERE id = :id AND (fence_token IS NULL OR fence_token < :newTok);
-- Returns 1 row if accepted (A's stale token=5 → 0 rows → A knows it lost; retry or abort).

```

**Why this works:** the fencing token turns “the lock lied to A” into “the downstream tells A no.” The lock is a [hint](#); the downstream is the [truth](#). The counter is monotonic (Redis `INCR` is atomic), so no two holders ever get the same token.

**KEY TERMS TO MENTION:** - GC pause invalidates TTL-based locks (Kleppmann’s critique) - Fencing token (monotonic counter) returned at acquire - Downstream rejects stale tokens (conditional write) - Compare-and-delete on release (don’t delete another’s lock) - Redis `INCR` for the monotonic counter (atomic) - The lock is a hint; the store is the truth

**FOLLOW-UP QUESTIONS:** 1. The downstream store has no fencing-token column (e.g., an external API). What then? 2. Why does `INCR fence:counter` not need to be in the same Redis as the lock? 3. The lock holder finished in 5s, well before the 10s TTL. It releases. Does the token get re-used?

**FOLLOW-UP ANSWERS:** 1. If the downstream can’t check a token, the fencing-token fix doesn’t apply. Options: (a) make the downstream idempotent on the operation (e.g., include a request-id; the downstream dedupes); (b) reduce the GC-pause risk (use a low-pause GC like ZGC, or no stop-the-world) — but you can’t eliminate it; (c) use a consensus-based lock (etcd/Zookeeper) with a lease that the client must actively renew and that is revoked on GC

pause detection — still imperfect. The honest answer: if the downstream can't fence, don't use a Redis lock for correctness; use a database transaction or a consensus lock. 2. They can be different Redis instances, but the counter must be in the **same** Redis as something atomic and durable. If the lock is on Redis-A and the counter on Redis-B, a failure of Redis-B doesn't affect the lock but means new acquires can't get tokens (degraded). For simplicity, run both in the same Redis (or a Redis cluster with the counter on a known shard). 3. No — **INCR** is monotonic and never reuses a value. The released token (say, 5) is gone; the next acquire gets 6. Even if the same client re-acquires, it gets a new token. The "release" doesn't reset the counter; the counter grows for the lifetime of the system. (64-bit counter at 1M acquires/sec = 584,000 years — overflow is not a concern.)

**RED FLAGS TO AVOID:** - "Redis **SETNX** is enough" — without a fencing token, a GC pause defeats it. - "We use short TTLs so the GC pause can't happen" — GC pauses can be 100ms-60s; you can't bound them below your TTL. - Releasing the lock by **DEL** without a compare — you delete the next holder's lock.

### Q-BNS-5: A Kafka topic has a hot partition — one key gets 80% of the traffic, the others are idle. The consumer for that partition is overloaded. Diagnose and fix.

- **Difficulty:** Hard
- **Category:** Distributed Systems / Messaging
- **Tags:** kafka, hot-partition, keying, parallelism

**ANSWER:** A hot partition means the partitioner (default: `hash(key) % numPartitions`) maps one high-traffic key to one partition; that partition's consumer is saturated while others are idle. Three fixes:

**A — Re-key with a sub-partition:** if the hot key is `customer_id=42`, re-key to `customer_id=42#0`, `#1`, ..., `#9` (10 sub-keys). The producer picks a sub-key round-robin (or by a secondary attribute). The 10 sub-keys hash to (ideally) 10 different partitions, spreading the load 10x. The consumer must handle that messages for `customer_id=42` can now arrive on 10 partitions (order within a sub-key is preserved, not across sub-keys). If you need global order for the customer, this breaks it; if order within a sub-key is enough, this works.

**B — Increase partitions (if the topic allows):** more partitions = more consumers in the group = more parallelism. But the hot key still maps to ONE partition (the hash is deterministic). Increasing partitions doesn't fix a single-key hotspot; it only helps if the traffic is spread across many keys but the partition count is the bottleneck. So B is the wrong fix for a true single-key hotspot (but the right fix for "many keys, too few partitions").

**C — Producer-side fan-out + consumer-side aggregation:** the producer splits the hot key's messages across N partitions with a synthetic suffix ( `customerId:42:shard0` ), and the consumer that needs to act on "all messages for customer 42" reads from all N partitions and re-aggregates (a windowed operator). This is the Kafka Streams `groupBy` pattern — the repartition step spreads the load.

**Diagnosis:** look at per-partition consumer lag ( `kafka-consumer-groups --describe` ). If one partition's lag is 100k and others are 0, it's a hot partition. Look at the message key distribution ( `kafka-console-consumer | jq -r .key | sort | uniq -c | sort -rn` ) — the top key has 80% of the messages. That's the culprit.

**Fix (code: producer-side sub-keying):**

```
// Before: every message for customer 42 → partition = hash(42) % N (always the same partition)
producer.send(new ProducerRecord<>("events", "42", event));

// After: spread customer 42 across 10 sub-partitions by adding a shard suffix
String shard = "shard" + (counter.getAndIncrement() % 10);
String compositeKey = "42:" + shard; // different key → different partition
producer.send(new ProducerRecord<>("events", compositeKey, event));
// Consumer sees messages for customer 42 across 10 partitions; aggregates by customer_id field in the payload.
```

**The order trade-off:** before, all events for customer 42 were in one partition, in order. After, they're in 10 partitions, in order **within** a shard but not across. If the consumer needs total order (e.g., a balance update sequence), this is a regression — you'd need a different design (a per-customer queue, or a single partition and accept the bottleneck). If the consumer is event-aggregating (counting, summing, windowing), out-of-order across shards is fine (the aggregation is commutative).

**KEY TERMS TO MENTION:** - `hash(key) % numPartitions` — the default partitioner; deterministic - Per-partition consumer lag as the diagnostic ( `kafka-consumer-groups --describe` ) - Sub-keying / composite key to spread one hot key across partitions - Order trade-off: within-shard preserved, across-shard lost - Kafka Streams `groupBy` repartition as the framework-level fix

**FOLLOW-UP QUESTIONS:** 1. The hot key is the customer ID for a “login” event stream. We need per-customer order for security (detect impossible-travel). Sub-keying breaks order. What now? 2. The producer can't be changed (third-party). Can we fix it on the consumer side? 3. Adding partitions to a live topic — does it break existing consumers?

**FOLLOW-UP ANSWERS:** 1. Don't sub-key; instead, accept the hotspot and scale the one partition's consumer vertically (more CPU). Or move that one customer to a dedicated topic with one partition and a dedicated consumer. Or use a tiered design: the hot customer writes to a dedicated high-throughput topic, others to the shared topic. The cost is operational complexity; the benefit is correct order. The honest answer is “a single hot key with a strict order requirement is a fundamentally serial problem — you can't parallelise it without losing order, so scale vertically or isolate it.” 2. Limited options on the consumer: you can't re-partition a topic without the producer changing keys (consumers read what's there). You can add more partitions (but the existing keys remap unpredictably — see #3) and hope the hot key spreads (it won't, because `hash(42) % new_N` is still one partition). The real fix is a new topic with a new partitioner and a producer migration; consumer-side alone can't fix a hot key. 3. Adding partitions to a live topic: existing messages stay in their original partitions (Kafka doesn't rehash existing data). New messages are hashed with the new partition

count, so a key that mapped to partition 3 may now map to partition 12. This breaks order for that key (its old messages are on partition 3, new ones on partition 12). Consumers reading “the whole topic” see both; consumers relying on per-key order break. The safe path: create a new topic with the desired partitions, dual-publish during migration, switch consumers, drain the old topic, delete it. Never `kafka-topics --alter --partitions` on a topic where per-key order matters.

**RED FLAGS TO AVOID:** - “Just add more consumers” — they’ll be idle; the bottleneck is the single partition. - Re-keying without considering the order requirement — breaks security/invariants. - `--alter --partitions` on a live topic with order-sensitive consumers — silent order breaks.

## Q-BNS-6: Design an idempotent payment API end-to-end. The user can retry on network failure. The payment must happen exactly once.

---

- **Difficulty:** Hard
- **Category:** System Design / REST API / Database
- **Tags:** idempotency, payment, unique-constraint, outbox

**ANSWER:** Idempotency = the same input produces the same effect as one call. For payments, “effect” = one charge, one record, one receipt. Three layers:

**Layer 1 — Client idempotency key:** the client generates a UUID per “Pay” button arming. Every retry sends the same UUID in the `Idempotency-Key` header. The server stores a row keyed by this UUID; a retry hits the unique constraint and returns the original result.

**Layer 2 — Server-side idempotency store:** an `idempotency_record` table with `(idempotency_key, request_hash, response_status, response_body, created_at, expires_at)`. The `request_hash` is a hash of the request body — if a second call uses the same key but a different body, we reject (409) because the client is misusing the key. The unique index on `idempotency_key` is the concurrency primitive.

**Layer 3 — Downstream idempotency (the payment provider):** Stripe/Adyen accept an idempotency key on their API call; we forward the same UUID. If we crash after Stripe charges but before we record, the retry re-calls Stripe with the same key — Stripe returns the original charge (no double charge). We then record it. Eventually consistent.

**Flow:**

```

Client → POST /payments {amount=500, currency=USD} Idempotency-Key: abc-123
Server:
 1. BEGIN TX
 2. INSERT INTO idempotency_record (key=abc-123, hash=hash(body), status=PENDING)
 → unique violation? → SELECT existing → return cached response (COMPLETED or
PENDING)
 3. Call Stripe with idempotencyKey=abc-123 (Stripe dedupes)
 4. UPDATE idempotency_record SET status=COMPLETED, response_body=...
 5. INSERT INTO payment (id, amount, stripe_charge_id) VALUES (...)
 6. COMMIT
 7. Return 201 {payment_id, status=COMPLETED}
Retry (client thinks first failed):
 → INSERT violates unique → SELECT existing → return 201 (same payment_id). No
second charge.

```

**The crash-recovery window:** if the server crashes between step 3 (Stripe charged) and step 4 (record updated), the `idempotency_record` is `PENDING`. On retry, the server sees `PENDING` — it must NOT re-call Stripe blindly (it might double-charge if Stripe lost its idempotency). Instead, it queries Stripe by the idempotency key (`/v1/charges?idempotency_key=abc-123` — Stripe exposes this); if a charge exists, it records `COMPLETED`; if not, it re-calls. This is the “reconciliation on `PENDING`” pattern.

**Code (server):**

```

@PostMapping("/payments")
public ResponseEntity<PaymentResponse> pay(@Valid @RequestBody PayRequest req,
 @RequestHeader("Idempotency-Key") String key) {
 // 1. Dedupe via unique index
 IdempotencyRecord rec;
 try {
 rec = idemRepo.insert(new IdempotencyRecord(key, hash(req), "PENDING"));
 } catch (DataIntegrityViolationException dup) {
 IdempotencyRecord existing = idemRepo.findByKey(key);
 if (!existing.requestHash().equals(hash(req))) {
 throw new IdempotencyKeyReuseException(key); // 409 - client misused the key
 }
 if (existing.status().equals("COMPLETED")) return ok(existing.toResponse());
 // PENDING - crash-recovery path
 return reconcilePending(existing, req);
 }
 // 2. New request - call Stripe with the same key
 try {
 Charge ch = stripe.charges.create(ChargeCreateParams.builder()
 .setAmount(req.amount()).setCurrency(req.currency())
 .setIdempotencyKey(key).build());
 rec.markCompleted(ch.getId(), toJson(responseOf(ch)));
 idemRepo.save(rec);
 paymentRepo.save(new Payment(ch.getId(), req.amount()));
 return created(new PaymentResponse(ch.getId(), "COMPLETED"));
 } catch (StripeException e) {
 rec.markFailed(e.getCode());
 idemRepo.save(rec);
 throw new PaymentFailedException(e); // client retries with a NEW key
 }
}

```

**KEY TERMS TO MENTION:** - Client-generated **Idempotency-Key** (UUID per button arming) - Server **idempotency\_record** table with unique index on key - **request\_hash** to detect key misuse (same key, different body → 409) - Provider-side idempotency (Stripe **Idempotency-Key** header) as the outer layer - PENDING crash-recovery: query the provider by the key before re-calling - 409 on key reuse with a different body (catches client bugs)

**FOLLOW-UP QUESTIONS:** 1. The **idempotency\_record** table grows forever. How do you prune? 2. Why store the response body — isn't the payment ID enough? 3. The user opens a new tab and clicks Pay again (different idempotency key). They get charged twice. Is that a bug?

**FOLLOW-UP ANSWERS:** 1. TTL the records (24h or 7d, depending on the client retry window). After TTL, a replayed key is treated as fresh — acceptable because the client won't retry a 24h-old request. A nightly job `DELETE FROM idempotency_record WHERE created_at < now() - INTERVAL '7 days'`. We also keep a 90-day cold copy in S3 for financial audit, but the live table stays small. Stripe's own idempotency keys expire after 24h (their default), so storing ours for 7d is the outer bound. 2. The response body lets us return the **exact** original response on a retry — same status, same body, same **payment\_id**. If we only stored the ID, a retry's response would have to be

reconstructed; subtle differences (a timestamp, a status field) would make the client think the payment changed. Storing the body is the “exact same response” guarantee. It’s a few KB per record; cheap. 3. No — that’s the user’s action, not a bug. Idempotency keys are per-“operation arming”; a new tab is a new arming with a new key. The system correctly charges twice because the user clicked Pay twice with intent. If we wanted to prevent it, we’d need a higher-level “one pending payment per cart” business rule (the cart can hold a lock that a second Pay attempt rejects with 409 “payment already in progress”). That’s a business rule, not idempotency. Idempotency protects retries; business rules protect user intent.

**RED FLAGS TO AVOID:** - No `request_hash` check — a client reusing a key with a different body gets the first response (silent bug). - Re-calling Stripe on a PENDING record without first querying — risks a double charge if Stripe’s own idempotency expired. - Storing only `payment_id` (not the full response) — retries get subtly different responses, confusing the client.

## Q-BNS-7: Your service’s p99 latency is 50ms but p99.9 is 2s. Walk through the long-tail diagnosis.

- **Difficulty:** Hard
- **Category:** Performance / Observability
- **Tags:** long-tail, p99, latency, gc, tracing

**ANSWER:** A p99.9 that’s 40x p99 means 1 in 1000 requests hits a pathological case. The tail is where users suffer; the average hides it. Diagnosis order:

**1. GC pauses:** a 2s tail is suspicious for a stop-the-world GC. Check JVM GC logs ( `-Xlog:gc*` ) for “Pause Full” or “Pause Young” > 500ms. A 1-2s GC pause on a 4GB heap with G1 is rare but possible (humongous allocation, evacuation failure). Fix: tune G1 ( `-XX:MaxGCPauseMillis=200` , `-XX:G1HeapRegionSize=16m` ) or switch to ZGC (sub-10ms pauses, at a throughput cost).

**2. Connection-pool exhaustion:** HikariCP `connectionTimeout` defaults to 30s, but if the pool is saturated, a request waits for a connection. Look at `hikari_connections_pending` and `hikari_connections_active` . If active = max and pending > 0, the pool is the bottleneck. Fix: increase `maximumPoolSize` (carefully — too high overloads Postgres), or fix the slow query holding connections.

**3. Lock contention:** a `synchronized` block or a DB row lock held by a slow path. The 999th-percentile request waits behind the lock. Thread dump ( `jstack` ) shows many threads in `BLOCKED` on the same monitor. Fix: shrink the critical section, switch to lock-free (ConcurrentHashMap, LongAdder), or partition the lock.

**4. A slow dependency:** one Redis call, one Stripe call, one Postgres query that occasionally takes 2s (network blip, cold cache, a slow query plan). Distributed tracing surfaces this — a span that’s usually 2ms is occasionally 2000ms. Fix: add a timeout ( `WebClient.timeout(2s)` ), a circuit breaker, or fix the slow query.

**5. Compaction / vacuum:** Postgres autovacuum on a large table can block (not lock, but slow) reads. Check `pg_stat_activity` for `autovacuum` and the table's `n_dead_tup`. Fix: tune autovacuum thresholds, or run manual `VACUUM` off-peak.

**6. Tenant hot-spot:** one tenant's request touches a much larger dataset (1M leads vs. the average 1k). The query is the same but the work is 1000x. Fix: per-tenant query limits, a tenant-specific index, or a separate "large tenant" code path.

**The tracing investigation:** for the specific 2s request, find its `trace_id` (we return it in `X-Trace-Id` for support). In Tempo, filter by that trace; the span tree shows exactly where the 2s went. The span with a 2s duration is the culprit; its attributes (SQL, Redis command, HTTP URL) tell you the cause. Without traces, you're guessing; with traces, you see it.

**Histogram-based alerting:** don't alert on p99 (it's fine); alert on p99.9 or p99.99. We use a Micrometer histogram (`service.id=...`) with buckets `[5ms, 10ms, 25ms, 50ms, 100ms, 250ms, 500ms, 1s, 2s, 5s]`; a Prometheus query `histogram_quantile(0.999, rate(http_request_duration_seconds_bucket[5m]))` and an alert if `> 500ms` for 10m. The tail is what we SLO on.

**KEY TERMS TO MENTION:** - p99 vs p99.9 (the tail where users suffer) - GC logs (`-Xlog:gc*`), G1/ZGC tuning - HikariCP saturation (`connections_pending`, `connections_active`) - Distributed tracing (the span with 2s duration is the answer) - Micrometer histogram buckets + `histogram_quantile(0.999, ...)` - Tenant hot-spot (one tenant's 1M-row query vs the 1k average)

**FOLLOW-UP QUESTIONS:** 1. The trace shows 2s in Postgres but `EXPLAIN ANALYZE` is 5ms. Where's the time? 2. You fix the GC (switch to ZGC), but p99.9 is still 1s. Next? 3. How do you set an SLO for p99.9 — what's the right target?

**FOLLOW-UP ANSWERS:** 1. Connection acquisition (Hikari wait), lock wait (`pg_locks` shows `granted=false`), or network round-trip (rare in same-AZ). The JDBC span in OTel separates "acquire" from "execute"; if acquire is 1.99s and execute is 5ms, the pool is saturated. If execute is 1.99s but `EXPLAIN` is 5ms, the query plan changed (check `pg_stat_statements` for the actual plan vs. the prepared plan; a parameter sniffing issue). Run `EXPLAIN (ANALYZE, BUFFERS)` with the `exact` parameters from the slow trace. 2. Next long-tail culprit after GC is usually lock contention or a slow dependency. Repeat the trace investigation on the new p99.9 (the traces that are still 1s). Likely a Redis call with no timeout (occasionally 1s on a Redis GC), or a `synchronized` block on a hot path. Add a 200ms timeout to every external call; add a thread-dump-based alert if `BLOCKED` threads spike. 3. SLO is a business decision, not a technical one. The questions: what does the user perceive as "instant" vs "slow" vs "broken"? For an API that a user waits on (checkout), p99.9 < 1s is a reasonable SLO (1 in 1000 checkout flows taking >1s is tolerable). For a background API (analytics), p99 < 5s is fine. The error budget: if SLO is 99.9% < 1s, you allow 43 min/month of >1s requests; beyond that, you stop shipping features and fix reliability. The target is set by what the business can afford to lose (a slow checkout → cart abandonment), not by what's technically achievable.

**RED FLAGS TO AVOID:** - Only measuring p50/p99 — the tail is where churn happens. - “It’s GC” without checking GC logs — guessing; the GC log is 5 minutes to read. - No timeouts on external calls — one slow Redis call becomes a 2s p99.9.

## Q-BNS-8: Design a Bloom filter to pre-check “has this user seen this lead” before hitting the DB, for 100M leads and 50M users. Walk through sizing and false-positive management.

- **Difficulty:** Medium
- **Category:** Algorithms / Data Structures / Performance
- **Tags:** bloom-filter, probabilistic, false-positive, cardinality

**ANSWER:** A Bloom filter is a probabilistic set-membership structure: it can say “definitely not in the set” or “possibly in the set” (with a false-positive rate). For “has user X seen lead Y”, the DB query is the source of truth; the Bloom filter is a fast pre-check that avoids the DB for the ~95% of “no” answers. The trade-off: a few % of “no” answers are wrong (false positives → a redundant DB query, not a wrong answer).

**Sizing:** for  $n=100\text{M}$  elements (lead-user pairs) and a target false-positive rate  $p=1\%$ , the optimal bit-array size  $m = -(n * \ln(p)) / (\ln(2)^2) \approx 958\text{M bits} \approx 114\text{ MB}$ . The optimal number of hash functions  $k = (m/n) * \ln(2) \approx 7$ . So: 114 MB filter, 7 hashes per insert/lookup. Memory is the cost; 114 MB per filter (we’d have one per “seen” type, or a single filter with a composite key `userId:leadId`).

**The catch — no deletion:** a standard Bloom filter can’t delete (you can’t unset a bit — other keys may have set it). For “seen” which is monotonic (a user never “unsees” a lead), this is fine. For non-monotonic data, use a counting Bloom filter (4 bits per slot, decrement on delete) or a Cuckoo filter (supports deletion, similar FPR).

### Code (Guava BloomFilter):

```
BloomFilter<String> seen = BloomFilter.create(Funnels.stringFunnel(Charsets.UTF_8),
 100_000_000, 0.01);
// Insert
seen.put(userId + ":" + leadId);
// Lookup
if (seen.mightContain(userId + ":" + leadId)) {
 // Possibly seen → query DB to confirm
 return db.hasSeen(userId, leadId);
} else {
 // Definitely not seen → skip DB, return false
 return false;
}
```

**False-positive management:** the 1% FPR means 1% of “no” answers are wrong → 1% of the queries the filter tried to skip still hit the DB. For 100M lookups/day, that’s 1M redundant DB queries — acceptable (it’s 1% of traffic, and the filter saved 99M queries). If you want 0.1% FPR, the filter grows to ~180 MB. The business trade-off: 1% is usually the sweet spot; below 0.1% the memory cost rises faster than the DB savings.

**Scale beyond memory:** if  $114 \text{ MB/filter} \times \text{many filters}$  exceeds RAM, use a [partitioned](#) Bloom filter (shard by user ID hash; each shard is a smaller filter in its own Redis key) — lookups touch one shard. Or use RedisBloom (a Redis module) which handles distributed Bloom filters natively.

**The reset problem:** a Bloom filter for “seen in the last 30 days” can’t be built with a standard Bloom (no TTL). Use a [rotating](#) filter: keep two filters, “current” and “previous”; rotate every 30 days. A “seen” check queries both; an “insert” goes only to “current”. After 30 days, “current” becomes “previous” and a fresh “current” starts. This gives an approximate 30-day window with a doubling of memory and lookup cost.

**KEY TERMS TO MENTION:** - Optimal  $m = -(n \ln p) / (\ln 2)^2$ , optimal  $k = (m/n) \ln 2$  - 1% FPR  $\approx 9.6 \text{ bits/element} \approx 114 \text{ MB}$  for 100M - No deletion (standard Bloom) — monotonic data only - Counting Bloom / Cuckoo filter for deletable - Rotating (double-buffered) filter for TTL windows

**FOLLOW-UP QUESTIONS:** 1. The filter is rebuilt on restart (in-memory). How do you avoid a cold-start storm? 2. A false positive sends the user a “you’ve already seen this” when they haven’t. Is that acceptable? 3. The filter says “not seen” but the DB says “seen” — how?

**FOLLOW-UP ANSWERS:** 1. Persist the filter to disk periodically (a snapshot every hour) and on graceful shutdown. On restart, load the snapshot (seconds) and re-apply the inserts since the snapshot (from the DB’s `seen_log` table with `created_at > snapshot_time`). The cold-start storm is the gap; for a 1h snapshot + 5m replay, the filter is consistent within minutes of restart. For zero-gap, use a RedisBloom module (the filter lives in Redis, not the app memory). 2. A Bloom filter never false-negatives — if it says “seen,” the DB check either confirms (true positive) or finds “not seen” (false positive). The user never sees “already seen” for a lead they haven’t seen; the only effect of a false positive is a redundant DB query (transparent to the user). So no, false positives don’t cause user-visible errors; they cause DB load. The DB is the truth; the filter is a hint. 3. A Bloom filter never false-negatives by construction (every bit set by the key is checked; if any is 0, the key was never inserted). So “filter says no, DB says yes” is impossible [for the same filter state](#). It can happen if the filter is stale (rebuilt from an old snapshot and the DB has newer inserts not yet in the filter) — fix by replaying the inserts since the snapshot. Or if the filter’s hash function changed between insert and lookup (a deploy bug) — fix by versioning the filter. The invariant: a correctly-maintained Bloom filter never returns “no” for a key it has.

**RED FLAGS TO AVOID:** - Sizing the filter too small for the cardinality — FPR explodes (10x more elements than designed  $\rightarrow$  FPR  $\sim 50\%$ ). - Using a Bloom for data with deletions — the filter never forgets; “deleted” items still show as “possibly in set.” - Treating “possibly” as “yes” — Bloom is a [pre-filter](#); the DB is the truth for positives.

## Q-BNS-9: Implement a Snowflake-style distributed unique ID generator. Walk through the 64-bit layout, the clock-skew problem, and the worker-bootstrapping.

- **Difficulty:** Medium
- **Category:** Algorithms / Distributed Systems

- **Tags:** snowflake, unique-id, clock-skew, twitter

**ANSWER:** Snowflake (Twitter 2010) generates 64-bit IDs: `[1 sign bit][41 timestamp ms][10 worker][12 sequence]`. The 41-bit timestamp gives ~69 years from the epoch; the 10-bit worker allows 1024 nodes; the 12-bit sequence allows 4096 IDs per ms per node. The total is 64 bits — fits in a `long`, sortable by time (the timestamp is the high bits), and collision-free (each node has a unique worker ID, and the sequence handles same-ms bursts).

### Layout:

```

1 bit 41 bits 10 bits 12 bits
[unused][milliseconds since epoch][worker_id][sequence]

```

**The clock-skew problem:** Snowflake assumes a monotonic clock. If NTP moves the clock backwards (rare but possible), a node could generate an ID `older` than one it already generated — breaking sort order and risking collisions (a future call with the same ms + sequence). The fix: on detecting a backward clock move, the generator `refuses to generate` until the clock catches up (it throws or waits). Twitter’s Snowflake blocks for up to the skew duration; we throw (fail-closed) and alert — a backward clock is a bug worth knowing about.

**Worker bootstrapping:** each node needs a unique 10-bit worker ID. Options: (a) ZooKeeper-allocated (a node gets a worker ID from ZK, releases it on shutdown); (b) config-file per node (manual, error-prone at scale); (c) derived from the IP (last octet, modulo 1024 — collisions possible across subnets); (d) a database sequence (a `worker_id` table, the node inserts and gets an auto-increment; releases on shutdown). For 1024 nodes, (a) or (d) is robust.

### Code (simplified):

```

public class Snowflake {
 private static final long EPOCH = 1_288_834_947_000L; // Twitter's epoch (Nov 4,
 2010)
 private static final long WORKER_BITS = 10, SEQ_BITS = 12;
 private static final long MAX_WORKER = (1L << WORKER_BITS) - 1; // 1023
 private static final long MAX_SEQ = (1L << SEQ_BITS) - 1; // 4095
 private final long workerId;
 private long lastTs = -1, seq = 0;

 public Snowflake(long workerId) {
 if (workerId < 0 || workerId > MAX_WORKER) throw new IllegalArgumentException();
 this.workerId = workerId;
 }

 public synchronized long next() {
 long now = System.currentTimeMillis();
 if (now < lastTs) throw new ClockMovedBackwardsException(lastTs - now); // fail-
 closed
 if (now == lastTs) {
 seq = (seq + 1) & MAX_SEQ;
 if (seq == 0) now = tilNextMillis(now); // sequence exhausted in this ms →
 wait
 } else {
 seq = 0;
 }
 lastTs = now;
 return ((now - EPOCH) << (WORKER_BITS + SEQ_BITS))
 | (workerId << SEQ_BITS)
 | seq;
 }

 private long tilNextMillis(long now) {
 long t = System.currentTimeMillis();
 while (t <= now) t = System.currentTimeMillis();
 return t;
 }
}

```

**Properties:** sortable (timestamp is the high bits), unique (worker + sequence disambiguate within a ms), 64-bit (long), ~4M IDs/sec/node, 1024 nodes = ~4B IDs/sec cluster-wide. The IDs are **roughly** time-ordered (within a node, fully ordered; across nodes, ordered to the ms, not the sub-ms).

**Variants:** (a) Sony's Sonyflake — uses a 10-bit sequence and a 16-bit worker (more workers, fewer per-ms); (b) Boundary's flake — adds a datacenter ID; (c) UUID v7 — a standardized time-ordered UUID (128-bit, Sortable prefix).

**KEY TERMS TO MENTION:** - 64-bit layout: 1+41+10+12 (timestamp, worker, sequence) - Monotonic clock assumption; fail-closed on backward clock - `tilNextMillis` when sequence exhausts in a ms - Worker ID allocation (ZK, DB sequence, or config) - Sortable (timestamp is high bits), fits in a long - Variants: Sonyflake, UUID v7

**FOLLOW-UP QUESTIONS:** 1. Two nodes have the same worker ID (misconfiguration). What happens? 2. 69 years from epoch, the timestamp overflows. What then? 3. Why not just use a UUID?

**FOLLOW-UP ANSWERS:** 1. They can collide — both can generate the same (ts, worker, seq) at the same instant. This is a silent data-corruption bug (two rows with the same ID, or one overwrites the other). Mitigations: worker-ID allocation via a centralised store (ZK/DB) prevents duplicates by construction; a startup check that the worker ID isn't already registered in the cluster (a heartbeat to a coordinator); monitoring that alerts if two nodes report the same worker ID. For us, worker IDs come from a DB sequence with a **UNIQUE** constraint and a **leased\_until** TTL — a node can't boot with an in-use ID. 2. After 69 years, the 41-bit ms-timestamp wraps. The IDs become unordered (the wrap flips the high bit). The fix: re-base the epoch (pick a new EPOCH closer to "now") and migrate to a new ID scheme. 69 years is a long horizon; most systems will have been rewritten. The honest answer: "this is a known 69-year limitation; we will re-epoch or migrate to a 128-bit scheme (UUID v7) before it's a problem, and it's far enough out that it's not a current concern." 3. UUIDs (v4, random) are 128-bit, not sortable, and index-unfriendly (random insertion into a B-tree fragments). Snowflake's 64-bit time-ordered IDs sort well (append to a B-tree = fast insert), are smaller (8 bytes vs 16), and are easier to debug (you can read the timestamp from the ID). UUID v7 (time-ordered) fixes the sortability and is the modern choice for systems that don't need a centralised generator; Snowflake is the choice when you want compact 64-bit sortable IDs. For a DB primary key, Snowflake is better than UUID v4 for index performance.

**RED FLAGS TO AVOID:** - No clock-skew handling — a backward clock silently produces out-of-order or colliding IDs. - Worker IDs from config without a uniqueness check — a misconfiguration causes collisions. - Using UUID v4 for a high-write PK — B-tree fragmentation kills insert throughput.

## Q-BNS-10: Design a feature-flag system with gradual rollout, kill switches, and per-user targeting. What's the data model and the evaluation path?

---

- **Difficulty:** Medium
- **Category:** System Design / Spring Boot
- **Tags:** feature-flags, rollout, kill-switch, launchdarkly

**ANSWER:** A feature flag is a runtime decision: "for this user, in this context, is feature X on?" The system stores flag definitions, evaluates them per-request, and lets ops toggle without a deploy. Three flag types: (a) boolean (on/off), (b) percentage rollout (X% of users), (c) targeted (specific users/orgs/attributes).

**Data model:**

```

flag:
 key: "new_pipeline_ui" -- the identifier in code
 type: PERCENTAGE | BOOLEAN | TARGETED
 enabled: true -- master kill switch
 rollout_percentage: 25 -- for PERCENTAGE
 rules: -- for TARGETED
 - attribute: org_id
 op: IN
 values: [uuid-of-test-orgs...]
 - attribute: plan
 op: EQUALS
 value: "PRO"
 default: false -- if evaluation fails, return this
 updated_at, updated_by, version -- audit

```

### Evaluation path (per request):

```

public boolean evaluate(String flagKey, UserContext user) {
 Flag f = flagCache.get(flagKey); // local cache, 30s TTL
 if (f == null || !f.enabled) return f.default_; // missing or killed → default
 switch (f.type) {
 case BOOLEAN: return f.value;
 case PERCENTAGE:
 int bucket = stableHash(user.id, flagKey) % 100; // 0-99, stable per (user, flag)
 return bucket < f.rolloutPercentage;
 case TARGETED:
 return f.rules.stream().allMatch(r -> r.evaluate(user)); // AND of all rules
 }
}

```

**Critical properties:**

- 1. Stable bucketing:** `stableHash(user.id, flagKey) % 100` means user 42 is *always* in bucket 17 for flag X — bumping the rollout from 25% to 30% adds buckets 25-29, not re-shuffles everyone. A user who saw the feature at 25% keeps seeing it at 30%. This is the “sticky rollout” — without it, users flicker between on/off on every percentage change.
- 2. Local cache:** flags are read on every request; a DB read per request kills latency. We cache flags in-memory (Caffeine, 30s TTL) and push updates via a pub/sub (Redis or Kafka) for near-real-time kill-switch. A kill switch should take effect in <30s (the cache TTL); a cache-invalidation pub/sub makes it <1s.
- 3. Kill switch:** `enabled=false` overrides everything; the evaluation returns `default` immediately. This is the “turn it off NOW” button. It must be fast (cache + pub/sub) and reliable (no evaluation path can skip the enabled check).
- 4. Audit:** every flag change is logged (who, when, from-version, to-version). A botched rollout can be reverted to a previous version.
- 5. Test environment:** flags have separate values per environment (dev/staging/prod); a flag is “on in staging, off in prod” until proven.

### Code (Spring Boot integration):

```

@Component @RequiredArgsConstructor
public class FeatureFlags {
 private final FlagCache cache; // Caffeine + Redis pub-sub invalidation
 public boolean isOn(String key, UserContext ctx) {
 return cache.get(key).map(f -> evaluate(f, ctx)).orElse(false);
 }
}

// Usage in a controller:
@GetMapping("/pipeline")
public String pipeline(@AuthUser User u) {
 if (flags.isOn("new_pipeline_ui", u.toContext())) return "pipeline-v2";
 return "pipeline-v1";
}

```

**Build vs buy:** LaunchDarkly, Flagsmith, Unleash are SaaS/self-host options. For a small team, a 200-line home-rolled system (DB + cache + pub/sub) is fine and avoids vendor lock-in. For a large team with many flags and A/B-test analytics, LaunchDarkly’s analytics + experimentation are worth the cost.

**KEY TERMS TO MENTION:** - Stable bucketing ( `hash(userId, flagKey) % 100` ) — sticky rollout - Local cache + pub/sub invalidation (Caffeine + Redis) - Kill switch (enabled=false, overrides all) - Audit log (version, who, when) - Per-environment values (dev/staging/prod separate)

**FOLLOW-UP QUESTIONS:** 1. The flag cache is 30s. A kill switch needs to take effect in <1s. How? 2. A percentage rollout at 25% — how do you guarantee exactly 25% of users see it (not “25% of traffic”)? 3. How do you A/B test with feature flags?

**FOLLOW-UP ANSWERS:** 1. The cache TTL is the **worst-case** propagation (if pub/sub is down). The pub/sub is the **fast** path — when a flag changes, we publish `invalidate:flagKey`; every node’s cache evicts that key; the next request re-reads from DB. So normal case <1s (pub/sub), worst case 30s (TTL). The kill switch’s reliability depends on the pub-sub; we use Redis PubSub (in our existing Redis cluster) so it shares the Redis availability. For an even-faster kill, a dedicated “kill” endpoint that the cache polls every 1s (a separate, tiny, high-frequency poll just for the kill-switch list) — cheaper than pub/sub for the rare event. 2. Percentage rollout is over **users** (hash by user ID), not traffic. Bucket = `hash(userId, flagKey) % 100`; if bucket < 25, the user is in. This means exactly 25% of **distinct users** see the feature, regardless of how many requests each makes. A power user with 1000 requests sees the feature in all 1000 (or none) — sticky. If you rolled out by traffic (every request, 25% chance), a power user would see it 25% of the time — flicker. User-bucketing is correct. 3. A/B test = a percentage rollout + measurement. Roll out at 50% (buckets 0-49 = control, 50-99 = treatment). Track a metric (conversion, retention) per bucket. Compare with statistical significance (a t-test or sequence-based). The flag system provides the bucketing; the analytics system provides the measurement. LaunchDarkly’s experimentation module does this; for home-rolled, you log `flag_key + bucket + outcome` to your analytics warehouse and query.

**RED FLAGS TO AVOID:** - Hash by request, not by user — flicker; users see the feature on/off randomly. - No kill switch (only percentage = 0) — a percentage bump from a buggy flag is slow to revert. - Cache with no invalidation — a flag change takes 30s (the TTL) to propagate; too slow for an emergency.

## Q-BNS-11: You deploy a new version and 5xx errors spike. The rollback command is run. Walk through what happens and the pitfalls.

- **Difficulty:** Medium
- **Category:** DevOps / Deployment
- **Tags:** rollback, deployment, blue-green, state

**ANSWER:** Rollback = revert the running code to the previous version. With blue/green, it's a traffic shift (seconds); with a DB schema change, it's a migration reversal (harder). The pitfalls are in the state, not the code.

### What happens (blue/green):

1. Monitoring detects 5xx > threshold (CloudWatch alarm).
2. On-call runs `./rollback.sh v1.4` (or auto-rollback triggers).`
3. ALB weighted target groups: v1.5 (blue) 100% → 0%, v1.4 (green) 0% → 100%. Takes ~10s.
4. v1.5 tasks drain (existing requests finish, 5min timeout), then stop.
5. v1.4 serves 100% of traffic. Errors drop.

### Pitfalls:

1. **DB schema forward-compatibility:** v1.5 ran an “expand” migration (added a column). v1.4 doesn't know the column exists — fine, it ignores it. But if v1.5 ran a “contract” (dropped a column), v1.4 expects the column and 500s. Rule: never auto-roll back across a contract migration; the contract is irreversible (the column is gone). Roll forward a fix instead.
2. **In-flight requests on v1.5:** a request that started on v1.5 and is mid-transaction when traffic shifts. The ALB's connection-draining lets it finish (up to 5min); if it commits a v1.5-format write, v1.4 reading it might misinterpret. Mitigation: v1.5 writes should be backward-compatible with v1.4 readers (the expand/contract discipline).
3. **Schema migrations applied by v1.5 that v1.4 can't read:** e.g., v1.5 changed a column type ( `VARCHAR` → `JSONB` ). v1.4 reads it as a string, gets garbage. Mitigation: type changes go through a two-release expand/contract (add new column, dual-write, migrate, read new, drop old).
4. **Cached data in Redis with v1.5 format:** v1.4 reads a v1.5-serialised object, deserialises wrong. Mitigation: version the cache keys ( `user:v1.4:42` vs `user:v1.5:42` ) so a rollback doesn't read the other version's cache. Or flush the cache on rollback (acceptable if the cache is warm-able).

5. **Background jobs started by v1.5 still running:** a Spring Batch job launched by v1.5 is mid-chunk when v1.4 takes over; v1.4's job-launcher doesn't know about it and may start a duplicate. Mitigation: distributed lock on job ID (Q-Bonus-8 of the project doc); a rollback should drain the job or wait for it.
6. **Stateful sessions:** if sessions are in-memory (not Redis), v1.4 doesn't have v1.5's sessions — users get logged out. Mitigation: stateless sessions (JWT) or Redis-backed sessions (shared).
7. **The bug that caused 5xx might still be there in v1.4:** if v1.4 has the same bug (rare, but if the 5xx is from a [data](#) issue v1.5 created, v1.4 reading that data also 500s). Mitigation: also roll back the data (point-in-time recovery on RDS to before v1.5's bad writes) — drastic, used only if the data is corrupt.

**The right answer:** rollbacks are a [code](#) revert; data reverts are a separate, harder decision. Auto-rollback on code 5xx is safe; auto-rollback on data corruption is dangerous (you might lose recent valid writes). For data, prefer a forward-fix (a migration that corrects the bad rows) over a DB revert. The runbook should distinguish.

**KEY TERMS TO MENTION:** - Blue/green rollback = traffic shift (seconds) - Expand/contract: rollback safe across expand, unsafe across contract - Connection draining (in-flight requests finish) - Versioned cache keys ( [user:v1.4:42](#) ) to avoid format mismatch - Distributed lock on background jobs (no duplicate on rollback) - Forward-fix over DB-revert for data corruption

**FOLLOW-UP QUESTIONS:** 1. Auto-rollback: what triggers it, and what's the risk? 2. The rollback succeeded but 5xx continue. What's the next step? 3. How do you test the rollback path before you need it?

**FOLLOW-UP ANSWERS:** 1. Trigger: SLO burn-rate (e.g., 2% of monthly error budget in 1h) or a hard 5xx rate (>1% for 5 min). Risk: auto-rollback can fire on a transient blip (a Redis restart that causes 30s of 5xx, then recovers) — you've rolled back unnecessarily. Mitigate with a confirmation window (the alarm must be sustained for 5 min) and a "auto-rollback for code-deploy alarms only" filter (don't auto-rollback on infrastructure alarms that aren't caused by the deploy). Always notify on-call; auto-rollback is a tool, not a substitute. 2. If 5xx persist after rollback to v1.4, the cause is [not](#) v1.5. Likely: (a) v1.5 wrote bad data that v1.4 reads and 500s on (forward-fix the data); (b) a shared dependency (RDS, Redis) is degraded (check their health); (c) v1.4 had a latent bug exposed by v1.5's traffic pattern (load test v1.4 to repro). The next step is NOT another rollback (there's no older version to roll to that's safer); it's diagnose + forward-fix. Disable the failing feature via a feature flag (kill switch) to stop the 5xx while you fix. 3. Game days: a monthly exercise where we deploy v1.5-staging to staging, inject a fault (kill a dependency, corrupt a row), trigger auto-rollback, and verify it works. We also have a CI step that runs the rollback script in a canary environment after every deploy — if the rollback script is broken (a Terraform change broke the ALB weighting), CI fails before prod. The rollback path is code; it must be tested like code.

**RED FLAGS TO AVOID:** - Rolling back across a contract migration — the data shape doesn't match the old code. - In-flight v1.5 writes that v1.4 can't read — backward-compat breaks. - Trusting auto-rollback without a game-day test — the rollback path rots.

## Q-BNS-12: Back-of-the-envelope: design a URL shortener for 100M URLs, 1B reads/day. What's the storage, the read QPS, and the cache design?

- **Difficulty:** Medium
- **Category:** System Design / Estimation
- **Tags:** capacity, estimation, url-shortener, cache

**ANSWER: Traffic estimates:** - 100M URLs total; 1B reads/day = ~12k reads/sec average, ~36k/sec peak (3x). - Writes: assume 100M URLs created over 5 years = ~50k writes/day = ~0.6 writes/sec. Writes are negligible; reads dominate 20,000:1.

**Storage:** - Per URL: short code (7 chars = 7 bytes) + long URL (avg 500 bytes) + metadata (created\_at, user\_id, click count) = ~600 bytes/row. - 100M URLs × 600 bytes = 60 GB. Fits on a single Postgres instance with room to grow. No sharding needed yet. - Index: a B-tree on `short_code` (7 bytes) is ~1 GB; the index fits in RAM (good for lookups).

**Read QPS:** - 36k reads/sec peak. A single Postgres instance with the index in RAM does ~10k simple `SELECT long_url FROM urls WHERE short_code=?` per sec. We need ~4x that. Options: - Read replicas (3-4 slaves, read from them): cheap, works. - Redis cache (cache-aside): 36k reads/sec from Redis is trivial (Redis does 100k/sec on one instance). The DB is only hit on cache miss (~5% if the working set is hot URLs). - Both: Redis as L1, Postgres replicas as L2. The 95% hit rate means ~1.8k/sec hits the DB; one replica handles it.

**Cache design:** - Working set: the 100M URLs aren't equally hot. A Zipf distribution: the top 1% of URLs get 80% of traffic. So ~1M hot URLs × 600 bytes = 600 MB hot working set — fits in Redis (a 4 GB instance) with room. - TTL: 24h. A URL that hasn't been read in 24h is unlikely to be read again; evict it. The DB lookup on miss is cheap. - Eviction: Redis `allkeys-lru` — evict the least-recently-used when full. The 600 MB hot set in a 4 GB instance means we never evict the hot set. - Negative caching: a 404 (short code doesn't exist) is cached for 1 min (avoid hammering the DB for a deleted/never-existed code). A short TTL on negatives prevents a “scan all codes” DoS.

**Short code generation:** - 7 chars, base62 (a-zA-Z0-9) =  $62^7 \approx 3.5$  trillion combinations. Plenty for 100M URLs. - Generation: a Snowflake-style ID (Q-BNS-9) → encode in base62. The ID is unique (Snowflake guarantees), the encoding is deterministic, no collisions, no DB lookup to check.

**Write path (rare):** - `POST /shorten {long_url}` → generate Snowflake ID → encode → `INSERT INTO urls (code, long_url, ...)`. Return `https://s.io/{code}`. - The insert is ~1ms; no caching on write (the next read will populate the cache).

**Read path (common):**

```
GET /{code} → Redis GET url:{code}
├─ hit? 301 redirect to long_url (with click tracking via async event)
└─ miss? SELECT long_url FROM urls WHERE code=? → cache (TTL 24h) → redirect
 └─ not found? cache negative (TTL 60s) → 404
```

**Click tracking:** on each redirect, increment a counter. Don't do a synchronous DB write (it'd slow the redirect). Publish to Kafka ( `clicks` topic); a consumer aggregates per-URL per-hour and writes to a `clicks` table. The counter is eventually consistent (a few seconds lag) — acceptable for analytics.

**KEY TERMS TO MENTION:** - Zipf distribution (1% of URLs = 80% of traffic) - Cache-aside + negative caching (TTL on 404) - Snowflake ID → base62 encode for the short code (collision-free, no DB lookup) - Async click tracking via Kafka (no sync DB write on the read path) - 60 GB fits on one Postgres; 600 MB hot set fits in one Redis

**FOLLOW-UP QUESTIONS:** 1. A URL is shortened 1M times (same long URL). Do you create 1M short codes? 2. The 100M grows to 10B. When do you shard, and how? 3. A user wants a custom short code ( `s.io/mylaunch` ). How?

**FOLLOW-UP ANSWERS:** 1. Dedup: hash the long URL (SHA-256) and check `SELECT code FROM url_dedup WHERE url_hash=?`. If exists, return the existing code. If not, create. This makes “shorten the same URL twice” idempotent. Trade-off: two users sharing a short code see each other's click stats (privacy/analytics issue). Mitigate: dedup per-user (hash includes `user_id`), so the same URL shortened by two users gets two codes. Dedup only within a user. The business rule decides. 2. Shard when: (a) the DB is too big to fit on one machine (10B × 600 bytes = 6 TB — needs sharding or a bigger instance); (b) the write QPS exceeds one instance (we're nowhere near); (c) the working set doesn't fit in the cache (a 10B-URL Zipf has a larger hot set). Shard by `short_code` hash (consistent hashing across N shards). The lookup router hashes the code, routes to the shard. Cross-shard queries (analytics) go to a separate warehouse (Redshift/BigQuery) populated by CDC. 3. Custom codes: a separate `custom_code` table with a unique constraint on `code`. On `POST /shorten {long_url, custom_code="mylaunch"}`, try `INSERT INTO custom_code (code, url_id)`. If unique violation, return 409 “code taken”. The lookup checks `custom_code` first, then `urls`. Custom codes are a premium feature (paying users); the namespace is global (no per-user custom codes — first-come, first-served, like Twitter handles).

**RED FLAGS TO AVOID:** - Generating a random short code and checking the DB for collision — O(n) for a popular service; Snowflake avoids the check. - Synchronous click-count writes on every redirect — kills the read-path latency. - One giant Redis for everything — at 10B URLs the cache gets big; shard Redis too (Redis Cluster).

---

## Common Interview Mistakes (Senior-Level)

1. **Being vague on numbers** — “we had a lot of users” instead of “3,200 paying orgs, ~50K active users.” Senior means you know your metrics cold.
2. **Naming a tech without the why** — “we used Kafka” without “because we needed to decouple the lead-discovery pipeline from the outreach pipeline at >1k events/sec, with replay.” The why is the senior part.
3. **Hand-waving on failure modes** — “we handle Redis outages” without “fail-open for reads, fail-closed for writes, with a Caffeine fallback that halves the rate.” Failure handling is where mid and senior diverge.
4. **Not knowing your own scale** — p50/p95/p99, uptime, QPS, table sizes, partition counts. If you don’t know, the interviewer thinks you didn’t really run it.
5. **Saying “we used microservices” without the trade-off** — the trade-off (operational complexity, network failure modes, distributed transactions) is the point; “we used them because they’re modern” is a red flag.
6. **Not asking clarifying questions in system design** — a senior asks “what’s the read:write ratio? what’s the consistency requirement? what’s the latency target?” before designing. Jumping straight to a solution is a mid-level tell.
7. **Not drawing on the whiteboard** — system design is a visual exercise; verbal-only answers lose points. Draw the boxes, name the data flows.
8. **Forgetting the data** — designing the API and service without the schema. The schema is half the design; skipping it shows you don’t think end-to-end.
9. **Over-engineering** — “we’d use Kafka + Flink + Cassandra + Kubernetes” for a 1k QPS service. Senior = right-sized, not maximalist.
10. **Not having stories** — “tell me about a production incident” with no specific story is a fail. Have 3 STAR stories ready (incident, performance fix, hard trade-off).

## Questions to Ask the Interviewer (Senior-Level, Project-Agnostic)

### About their architecture

---

- “What’s the biggest consistency-vs-availability trade-off you’ve made recently, and would you make it again?”
- “Where does your system’s data get stale, and how do you communicate that to users?”
- “What’s the most expensive query in your system, and have you considered alternatives?”
- “How do you handle a schema change on a 1TB+ table?”
- “What’s your event-driven vs request-driven split, and why?”

### About their scale + operations

---

- “What’s your on-call structure, and what’s the alert-to-incident ratio?”
- “What’s the largest outage in the last year, and what changed because of it?”
- “How do you measure developer velocity, and what’s the biggest blocker to it?”
- “What’s your deploy frequency and lead time, and what’s the next improvement?”
- “How do you test in prod — shadow traffic, canaries, feature flags, all of the above?”

### About their engineering culture

---

- “How are technical decisions made — RFC, consensus, lead-engineer-decides?”
- “What’s the path from SDE-2 to senior, and what does the rubric look like?”
- “How do you handle tech debt — dedicated time, embedded in sprints, neither?”
- “What’s the code-review culture, and who approves what?”
- “What’s a recent decision the team got wrong, and how did you course-correct?”

### About the role

---

- “What does success look like in the first 6 months?”
- “What’s the technical direction for the next year, and how does this role shape it?”
- “What’s the hardest problem the person in this role will own?”
- “How does this team’s work tie to the company’s revenue or growth?”

## Salary Negotiation Tips (Product-Based Companies)

### India (4-year backend, product companies)

- **Uber India (Bangalore/Hyderabad):** SDE-2 (L4 equivalent) base ₹40-60L, RSU \$40-80k vesting over 4 years, joining bonus ₹5-15L. Total ~₹55-90L.
- **Meta (India — Menlo Park/remote):** E4 base ₹50-80L (India office), or US E4 ~\$185-220k base + \$150-250k RSU/year. India remote for US: rare.
- **NetApp (Bangalore):** MTS-4 base ₹35-55L, RSU ₹15-30L over 4 years, bonus 10-15%. Total ~₹45-70L.
- **Amazon (Bangalore/Hyderabad):** SDE-2 base ₹35-55L, RSU \$80-140k over 4 years (back-loaded), sign-on ₹10-25L. Total ~₹50-80L. SDE-3 if you negotiate hard.
- **Google (Bangalore/Hyderabad):** L4 base ₹40-60L, RSU \$80-150k over 4 years, bonus 15%, GSU. Total ~₹60-100L.

**Negotiation tactics:** 1. **Always have a competing offer** — product companies move 10-20% on a counter. 2. **Negotiate RSU, not just base** — base is capped per band; RSU is flexible and compounds. 3. **Don't disclose current comp first** — if asked, say "I'd like to understand the role and band first." 4. **The signing bonus is a one-time lever** — if base and RSU are maxed, ask for a higher sign-on. 5. **Leverage the vesting cliff** — Amazon's 5-15-40-40 vesting means year 1 is light; negotiate a sign-on to cover year 1.

### US (for reference)

- Uber L5a: \$190-230k base + \$150-250k RSU/year. Total ~\$400-480k.
- Meta E4: \$185-220k base + \$150-250k RSU/year. Total ~\$390-470k.
- Google L4: \$185-220k base + \$140-220k RSU/year + 15% bonus. Total ~\$400-470k.

## How to Handle “Do You Have Any Questions for Us?”

The answer is always **yes**. The questions you ask are evaluated — they signal curiosity, depth, and cultural fit.

**Structure:** ask 2-3 questions, mixing technical and team/culture. Don't ask 5 (you'll run out of time); don't ask 0 (you signal disinterest).

**Strong closing questions:** - “Based on this conversation, what's a concern you have about me for this role that I could address before we wrap up?” — surfaces a hidden objection and gives you a chance to rebut. - “What's a project the person in this role would own in the first 90 days, and what does success on it look like?” — signals you're already thinking about delivering. - “What's the technical decision your team made in the last year that you'd most like to revisit, and why?” — signals you think about trade-offs and long-term consequences.

**Weak questions to avoid:** - “What's the work-life balance like?” — signals you're optimizing for ease, not impact. - “What's the salary for this role?” — premature; save for the recruiter call. - “What does your company do?” — you should know this cold before the interview. - Anything answered on the careers page.

## PART 12 — BONUS 2: MORE ADVANCED & TWISTED QUESTIONS (Concurrency, DB Internals, JVM, Networking, Algorithms)

**Statement for the LLM / candidate:** The questions in this section (and the entire document) are [sample reference questions](#) — a starting template designed to seed your preparation. They are NOT exhaustive. To be truly interview-ready for Uber / Meta / Amazon / NetApp at the 4+ year level, extend each one into harder, twisted variants — add edge cases, multi-step follow-ups, “what if X fails” branches, and code-level “show me the exact API / SQL / config” drills. Treat every question here as the [easiest](#) version of itself. Code snippets, ASCII diagrams, and DB schemas below are the depth to reach for every topic.

### QG-Bonus2-1: Implement a thread-safe bounded LRU cache in Java without using `LinkedHashMap`. Walk me through the data structure and concurrency.

- **Difficulty:** Hard
- **Category:** Concurrency / Data Structures
- **Tags:** lru-cache, readwrite-lock, concurrent, hashmap, doubly-linked-list

#### ANSWER:

An LRU cache needs  $O(1)$  get and put with eviction of the least-recently-used entry when the capacity is exceeded. The data structure: a **doubly linked list** (for  $O(1)$  move-to-front and remove-tail) + a **hash map** (key → list node, for  $O(1)$  lookup). On `get(k)`, look up the node, move it to the front (most-recently-used). On `put(k,v)`, if exists, update value + move-to-front; else insert at front, evict tail if over capacity.

For concurrency, the simplest correct approach: a `ReentrantReadWriteLock` around the whole structure — reads take the read lock, writes (put + moveToFront on get) take the write lock. But `get` mutates the list (move-to-front), so it must take the `write` lock, defeating the read-write split. So `get` is a write operation in LRU; the read lock is only useful for `peek` (which doesn't update recency).

**Better: segment the cache (like `ConcurrentHashMap`'s stripes):**  $N$  stripes, each with its own lock + map + list. `get(k)` hashes  $k$  to a stripe, locks only that stripe. Concurrency =  $N$  (the number of stripes). Capacity is split evenly across stripes; eviction is per-stripe.

**Code — segmented LRU:**

```

public class ConcurrentLruCache<K, V> {
 private static class Node<K,V> { K k; V v; Node<K,V> prev, next; }
 private static class Stripe<K,V> {
 final ReentrantLock lock = new ReentrantLock();
 final Map<K, Node<K,V>> map = new HashMap<>();
 Node<K,V> head, tail; // sentinels
 int capacity, size;
 // get/put/peek methods, each `lock.lock(); try { ... } finally
 // { lock.unlock(); }`
 }
 private final Stripe<K,V>[] stripes;
 private final int stripeCount;

 public ConcurrentLruCache(int totalCapacity, int stripeCount) {
 this.stripeCount = stripeCount;
 this.stripes = (Stripe<K,V>[]) new Stripe[stripeCount];
 int per = (totalCapacity + stripeCount - 1) / stripeCount;
 for (int i = 0; i < stripeCount; i++) stripes[i] = new Stripe<>(per);
 }

 public V get(K key) {
 return stripeFor(key).get(key); // hashes key to a stripe
 }
 public void put(K key, V val) {
 stripeFor(key).put(key, val);
 }
 private Stripe<K,V> stripeFor(K key) {
 return stripes[Math.floorMod(key.hashCode(), stripeCount)];
 }
}

```

**The trade-off:** Segmentation gives concurrency but uneven eviction — a hot key in stripe 3 evicts a cold key in stripe 3, while stripe 5 has space. The global LRU is approximate (within a stripe it's exact; across stripes it's not strictly LRU). For most caches (HTTP response cache, session cache), this approximation is fine. For strictly-LRU, use a single lock (slow) or a lock-free LRU (research-level, e.g., the LIRS or W-TinyLFU algorithms used by Caffeine).

**Caffeine (production choice):** In a real interview, mention **Caffeine** — it's the de-facto Java cache (uses W-TinyLFU, near-optimal hit rate, fully concurrent, async refresh, expiry by age/access/write). "We use Caffeine for our response cache; the LRU/W-TinyLFU question came up when sizing it. The segmented-LRU above is the from-scratch answer if you ask me to implement it."

**KEY TERMS TO MENTION:** - Doubly linked list + hash map ( $O(1)$  get/put/evict) - `ReentrantReadWriteLock` (but LRU's get mutates → write lock) - Segmentation (stripes, per-stripe lock + map + list) - W-TinyLFU (Caffeine's algorithm, frequency + recency) - Caffeine (the production choice — don't reimplement)

**FOLLOW-UP QUESTIONS:** 1. The segmented cache has uneven eviction. Stripe 3 is full, stripe 5 has space. Can you balance globally? 2. Why not just `Collections.synchronizedMap(new LinkedHashMap(...))`? 3. The cache is hit 1M times/sec. Lock contention is the bottleneck. Next optimization?

**FOLLOW-UP ANSWERS:** 1. Global balancing requires a global structure (one list, one lock), which kills concurrency. The middle ground: a “victim” stripe — when a stripe is full and needs to evict, it first tries to steal capacity from a less-full stripe (lock both, move a fraction of capacity). Complex and rarely worth it. The simpler fix: size each stripe for the **peak** load, accept some wasted capacity in cold stripes. For a 1000-entry cache with 16 stripes, 62 per stripe; if load is uneven, increase total to 1500 (94 per stripe) — the wasted 500 is cheap memory. Production caches rarely care about exact capacity; they care about hit rate. 2. `synchronizedMap` uses one lock for the whole map — fully serializes all get/put (no concurrency). For an LRU with `accessOrder=true`, `LinkedHashMap`’s `get` mutates the list, so it’s a write under the `synchronizedMap`’s lock — all reads block each other. Throughput is 1/N of a segmented cache (N=stripes). Useful only for low-throughput caches (<10K ops/sec). 3. Move to **lock-free reads**: an optimistic read that retries on conflict. `get` reads the map without a lock, checks the node’s “version” (a volatile int incremented on every mutation of that node); if unchanged during the read, use the value; if changed, retry. This is the `StampedLock` optimistic read pattern. For move-to-front on `get`, defer it — batch the recency updates (a queue of “recently accessed keys”); a background thread flushes them to the list every 1ms. The reads are fully lock-free; the writes (recency batch) are amortized. This is how Caffeine achieves 10M+ ops/sec. If you mention this in an interview, you’re showing senior concurrency depth.

**RED FLAGS TO AVOID:** - One lock for the whole cache (`synchronized` or single `ReentrantLock`) — serializes all ops. - `LinkedHashMap` with `accessOrder=true` + no synchronization — race condition, the list corrupts. - Ignoring Caffeine (“I’d write my own”) — production code uses Caffeine; reimplementing is a red flag.

## QG-Bonus2-2: Two threads call `transfer(from, to, amount)` on the same bank account concurrently. The balance must not go negative. Walk through 4 ways to make this safe, and their trade-offs.

- **Difficulty:** Hard
- **Category:** Concurrency / Transactions
- **Tags:** `synchronized`, lock-ordering, stm, cas, optimistic, pessimistic

### ANSWER:

The classic concurrent transfer problem. Four approaches, increasing sophistication:

#### (1) Coarse-grained lock (one lock for all accounts):

```
synchronized (globalLock) {
 if (from.balance >= amount) { from.balance -= amount; to.balance += amount; }
 else throw new InsufficientFunds();
}
```

Safe, simple, but serializes ALL transfers globally — throughput = 1.

#### (2) Fine-grained lock per account, with lock ordering (to avoid deadlock):

```
public void transfer(Account from, Account to, long amount) {
 Account first = from.id < to.id ? from : to; // consistent ordering by id
 Account second = from.id < to.id ? to : from;
 synchronized (first) {
 synchronized (second) {
 if (from.balance >= amount) { from.balance -= amount; to.balance += amount; }
 else throw new InsufficientFunds();
 }
 }
}
```

Locks both accounts, but always in the same order (by `id`) — no deadlock (a cycle requires inconsistent ordering). Throughput =  $N$  (number of accounts); two transfers on disjoint accounts run in parallel.

### (3) Optimistic with CAS (AtomicLong balance):

```
public void transfer(Account from, Account to, long amount) {
 while (true) {
 long fromBal = from.balance.get();
 if (fromBal < amount) throw new InsufficientFunds();
 long toBal = to.balance.get();
 if (from.balance.compareAndSet(fromBal, fromBal - amount)) {
 // committed the debit; now credit (could fail if to.balance changed)
 while (!to.balance.compareAndSet(toBal, toBal + amount)) toBal =
 to.balance.get();
 return;
 }
 // CAS failed (someone else changed from.balance), retry
 }
}
```

Wait-free reads, retry on conflict. But the “from” CAS commits the debit *before* the “to” credit — if the thread dies between, the money vanishes (not atomic). To fix, you’d need a multi-variable CAS (which Java doesn’t have) or a single “ledger” row that both sides reference — defeats the per-account design. Optimistic transfer is hard to make atomic without a coordinator.

### (4) Database transaction (the real answer):

```
@Transactional
public void transfer(long fromId, long toId, long amount) {
 Account from = repo.findById(fromId); // SELECT FOR UPDATE
 Account to = repo.findById(toId);
 if (from.balance >= amount) { from.balance -= amount; to.balance += amount; }
 else throw new InsufficientFunds();
}
```

The DB handles locking (row locks on both accounts, ordered by some invariant to avoid deadlock), atomicity (commit/rollback), and isolation. For a banking app, this is the production answer; the in-JVM approaches (1-3) are interview concepts.

**The deadlock-avoidance rule (lock ordering):** If thread A locks account 1 then account 2, and thread B locks account 2 then account 1, deadlock. The fix: always lock in a canonical order (by account `id`). Then any two threads acquiring the same pair lock in the same order; no cycle. This is the universal pattern for multi-lock acquisition — define a total order on lockables.

### ASCII — the deadlock and the fix:

```
Without ordering (DEADLOCK):
 T1: lock(1) —————> lock(2) ... waiting (T2 holds 2)
 T2: lock(2) —————> lock(1) ... waiting (T1 holds 1)
 => both wait forever

With ordering by id (SAFE):
 T1: transfer(1, 2) -> lock(1) then lock(2) [first=1, second=2]
 T2: transfer(2, 1) -> lock(1) then lock(2) [first=1, second=2, REORDERED!]
 => both acquire 1 first; the second waits; no cycle
```

**KEY TERMS TO MENTION:** - Coarse-grained vs fine-grained locking (throughput trade-off) - Lock ordering (canonical order on lockables; avoids deadlock) - CAS (compare-and-swap, `AtomicLong.compareAndSet`) — optimistic, retry-on-conflict - DB transaction with `SELECT FOR UPDATE` (the production answer) - Deadlock (cycle in the wait-for graph)

**FOLLOW-UP QUESTIONS:** 1. The DB has 1000 transfers/sec on the same hot account (a “settlement” account everyone pays into). Lock contention. How do you scale? 2. The optimistic CAS approach (3) isn’t atomic. How would you make it atomic without a DB? 3. Lock ordering by `id` — what if `id` is a UUID (no natural ordering)?

**FOLLOW-UP ANSWERS:** 1. Two patterns: (a) **Batching** — accumulate transfers in a per-tenant buffer, flush to the settlement account every 1s with a single update (`UPDATE account SET balance = balance + SUM(...) WHERE id = ?`). Reduces 1000 lock acquisitions to 1. (b) **Account splitting** — split the settlement account into N sub-accounts (`settlement_1...settlement_N`); each transfer goes to a random sub-account; contention is N× lower. A nightly job consolidates. This is how high-throughput ledgers (e.g., payment processors) handle the hot-account problem. 2. With a single coordinator: a “transaction manager” thread that holds the list of pending ops and applies them atomically. The transfer enqueues `(from, to, amount)` to the manager’s queue; the manager dequeues and applies (debit + credit) under its own single lock. This is essentially a single-threaded actor model (Akka-style) for the account domain. Throughput is bounded by the manager’s thread (1 thread = ~1M ops/sec for in-memory), but it’s atomic. Or, use an STM (Software Transactional Memory) library like Clojure’s refs or Multiverse for Java — the STM tracks read/write sets and retries the whole transaction on conflict. STM is academic-elegant but rare in production Java. 3. UUIDs have no natural order. Use a deterministic derived order: convert to `BigInteger` (UUIDs are 128-bit ints; sort numerically), or hash to an int and order by hash, or compare the two UUIDs’ most-significant bits first. The rule: any total order works; pick

one and apply it consistently. The trap: comparing the UUIDs' `toString()` (lexicographic on the hex string) is fine as long as it's consistent — the order doesn't need to be “meaningful”, just total and stable. If both threads use the same comparison, no deadlock.

**RED FLAGS TO AVOID:** - “Just `synchronized` on the transfer method” — locks the wrong thing (the method, not the accounts); serializes all transfers globally. - Lock in the order the args are passed ( `synchronized(from) { synchronized(to) {...}}` ) — deadlock when args are swapped. - Optimistic CAS without atomicity across the two accounts — money vanishes on mid-failure. - “I'd use a database” without explaining the lock-ordering that the DB does internally.

## QG-Bonus2-3: PostgreSQL B-tree index internals — how does a `SELECT` with `WHERE id = 42` use the index? What's in a leaf page? What's a heap-only scan?

- **Difficulty:** Hard
- **Category:** PostgreSQL / Indexing
- **Tags:** btree, leaf-page, heap-fetch, index-only-scan, visibility-map

### ANSWER:

A Postgres B-tree index is a balanced tree of `pages` (8KB each). Internal pages store (key, pointer) pairs to child pages; leaf pages store (key, TID) pairs where TID is the Tuple ID (page number + offset in the heap). The heap is the table itself (where the row data lives). Postgres is *not* a clustered-index engine (unlike MySQL InnoDB's primary key index, where the leaf *is* the row); Postgres's index leaf stores only the key + a pointer; the actual row is fetched from the heap.

`SELECT * FROM lead WHERE id = 42`: 1. Start at the root page; binary-search for 42; follow the pointer to a child page. 2. Descend to the leaf page (log N pages, ~3-4 for a 10M-row table). 3. In the leaf, find the entry with key=42; get the TID (e.g., page 17, offset 3). 4. Fetch the heap page (page 17), read the row at offset 3. 5. Check the row's visibility (xmin/xmax) — is it visible to our transaction (not deleted, not by a future tx)? 6. Return the row.

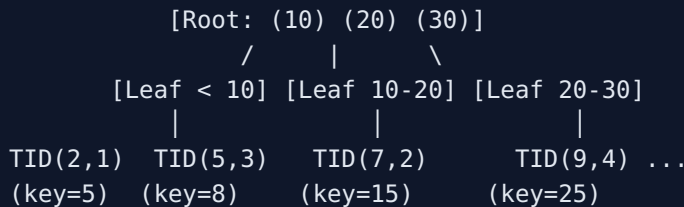
**The heap fetch (step 4) is the cost.** The index points to the heap; fetching the heap page is a random IO (unless it's cached). For a query that returns many rows, this is N random IOs — slow.

**Index-only scan:** If the query is `SELECT id FROM lead WHERE id = 42`, all columns are in the index (the key `id`). Postgres can skip the heap fetch — return the key directly from the index. But Postgres needs to verify the row's *visibility* (is this version visible to my tx?), which lives in the heap... unless the heap page is “all-visible” (the visibility map says so). If the page is all-visible (no pending deletes, vacuum ran), the index-only scan skips the heap fetch. This is why `VACUUM` matters for index-only scans — it sets the visibility-map bits.

**Covering index ( `INCLUDE` clause):**  
`CREATE INDEX lead_stage_owner ON lead (stage_id) INCLUDE (owner_id);` — the index stores `stage_id` (the key) + `owner_id` (a payload, not a key, but stored in the leaf). A query

`SELECT owner_id FROM lead WHERE stage_id = 5` does an index-only scan (both columns are in the leaf, and if the page is all-visible, no heap fetch). The `INCLUDE` column isn't used for ordering (only the key is), so it's smaller than a full composite index, and it doesn't help `ORDER BY owner_id`. But for point lookups + coverage, it's perfect.

### ASCII — the B-tree + heap:



For `SELECT * FROM lead WHERE id = 15`:

1. Root:  $15 \in [10, 20) \rightarrow$  child "Leaf 10-20"
2. Leaf: find `key=15`  $\rightarrow$  `TID(7, 2)`
3. Heap page 7, offset 2  $\rightarrow$  fetch row  $\rightarrow$  check `xmin/xmax` visible
4. Return row

For `SELECT id FROM lead WHERE id = 15` (index-only scan, if all-visible):

1. Root  $\rightarrow$  Leaf  $\rightarrow$  find `key=15`
2. (skip heap fetch)  $\rightarrow$  return `id=15` ✓ no random IO

**KEY TERMS TO MENTION:** - B-tree pages (8KB each; root, internal, leaf) - TID (Tuple ID; heap page + offset) - Heap (the table; the row data lives here, not in the index) - Visibility map (per-page bit; "all-visible" = no pending deletes) - Index-only scan (skip heap fetch if all-visible + all columns in index) - Covering index ( `INCLUDE` for non-key payload columns) - `VACUUM` sets visibility-map bits (enables index-only scans)

**FOLLOW-UP QUESTIONS:** 1. The query is `SELECT id, name FROM lead WHERE id = 42`. Is it index-only? 2. The visibility map says "all-visible" but the row was deleted by a concurrent tx (uncommitted). What happens? 3. Why not store the row in the index leaf (clustered index, like MySQL)?

**FOLLOW-UP ANSWERS:** 1. Only if `name` is in the index. `id` is the key; `name` isn't in the index leaf, so the heap fetch is needed to get `name`. To make it index-only, add a covering index: `CREATE INDEX lead_id_name ON lead (id) INCLUDE (name);` Now the leaf has both `id` (key) and `name` (payload); index-only scan works. The trade-off: the index is larger (stores `name` per row), slower to update (every `name` change updates the index), but the query is faster (no heap fetch). For hot queries, covering indexes are the single biggest SQL optimization. 2. The visibility map is conservative — "all-visible" means "no deletes committed as of the last vacuum." If a delete is in-progress (uncommitted) by another tx, the visibility map might still say all-visible (the delete hasn't committed). Our query reads the row from the index (index-only), but the row's `xmin/xmax` in the heap would say "deleted by tx X"; if X isn't committed, the row is still visible to us. Wait — index-only scan [skips the heap fetch](#), so it can't check `xmin/xmax`. This is the subtle bit: index-only scan is correct because the visibility map says all-visible, and the row's [key](#) in the index is from a row that was visible when the index entry was made. If a delete commits after the vacuum but before

our query, the visibility map is stale — but Postgres re-checks: an index-only scan fetches the [heap page's visibility map bit](#) (a tiny read, not the whole row) and only skips the heap fetch if the bit is set. So a concurrent delete doesn't break correctness; it just disables the index-only optimization for that page (falls back to heap fetch). Correct but slightly slower. 3. Postgres's design separates index from heap (flexibility: multiple indexes on one table, no reordering needed; the heap is the source of truth). MySQL InnoDB's clustered index (primary key IS the table) is faster for primary-key lookups (no heap fetch) but slower for secondary indexes (secondary index leaves store the PK, requiring a second lookup: index → PK → table). Postgres's choice is better for write-heavy workloads (every update doesn't rewrite the clustered index); MySQL's is better for PK-lookup-heavy workloads. Trade-offs; both are valid. In an interview, mentioning this comparison shows depth.

**RED FLAGS TO AVOID:** - "The index stores the row" — wrong (Postgres); the index stores key + TID, row is in the heap. - "Index-only scan is always faster" — only if visibility-map bit is set; vacuum-dependent. - "Covering index = composite index" — composite indexes key the additional columns (used for ordering); covering indexes only payload them (smaller, can't help ORDER BY).

---

## QG-Bonus2-4: The API returns 500ms p99. The DB query is 5ms. Where's the other 495ms? Walk me through the diagnosis.

---

- **Difficulty:** Hard
- **Category:** Performance / Observability
- **Tags:** flame-graph, async-profiler, jfr, gc, network, latency-breakdown

### ANSWER:

The 500ms is the sum of: client→LB, LB→app, app processing (CPU + GC + lock waits), app→DB (network), DB query, DB→app, app serialization, app→LB, LB→client. With DB at 5ms, the other 495ms is somewhere else. Diagnosis in order of cheapest-first:

**Step 1 — Trace spans (the breakdown):** OpenTelemetry trace of a slow request. The spans show where the time goes: - If the trace shows `controller.handle = 495ms` with no children, the time is in the controller (CPU/GC/lock). - If it shows `jdbc.execute = 5ms` + `http.client.post = 480ms`, the time is in an external call (Stripe, LLM). - If it shows `controller.handle = 100ms` + `jdbc = 5ms` + a 395ms `gap` between spans, the gap is GC pause, lock wait, or queue wait (no span — invisible to tracing).

**Step 2 — GC logs ( `-Xlog:gc*` ):** A 200ms GC pause mid-request shows as a gap in the trace. Check GC log for pauses around the request's timestamp. If the pause matches the gap, GC is the culprit. Fix: tune GC (see Q-Bonus2-11 in the AcquisitionOS doc) or reduce allocations.

**Step 3 — JFR (Java Flight Recorder):** A 60s JFR recording during the slow period shows: - CPU flame graph (which methods burn CPU) - Lock contention (which monitors are contended; wait time) - Allocation hotspots (which methods allocate the most) - IO wait

(socket read waiting on DB/external) Open JFR in JDK Mission Control; look at the “Hot Methods” and “Lock Instances” tabs.

**Step 4 — `async-profiler` for CPU flame graphs:** `./async-profiler.sh -d 60 -f flame.html <pid>` — produces an HTML flame graph. The widest bars are the CPU consumers. If a method like `ObjectMapper.writeValueAsString` is 30% of CPU, that’s your bottleneck (serialization). Fix: stream the response (don’t materialize), use a faster serializer (Jackson `Afterburner` / `Blackbird` module), or reduce payload size.

**Step 5 — Network/IO wait:** If JFR shows “Socket Read” as the dominant wait, the app is waiting on a downstream (DB or external). For the DB: check `pg_stat_activity` for slow queries; for external: check the downstream’s latency (Stripe’s API dashboard). A common culprit: the connection pool is exhausted, and the request waits 490ms for a connection (the `connectionTimeout` is 30s, but under load, the `average` wait is 490ms). Check HikariCP metrics: `connections_pending`, `connections_active`. If pending > 0, the pool is the bottleneck.

**Step 6 — Lock contention:** JFR’s “Lock Instances” shows contended monitors. If `synchronized(someMap)` has 200ms avg wait, that’s the culprit. Fix: switch to `ConcurrentHashMap` (stripe locks) or remove the synchronization (immutable data).

**Step 7 — TCP / network:** Use `tcpdump` + `Wireshark` to see the actual packet timing. A 200ms TCP retransmit, a 100ms DNS lookup, a 50ms TLS handshake — these add up. `ss -i` shows per-socket stats. For LB→app, check the LB’s access logs (AWS ALB logs to CloudWatch); for app→DB, check `pg_stat_statements` for query time + the DB’s wait events.

**Common 500ms-breakdown cases I’ve seen:** - 200ms GC pause (g1, untuned) + 200ms connection pool wait + 100ms Jackson serialization of a large DTO. Fix: tune GC, size the pool, stream the response. - 400ms waiting on an external API (Stripe) that the trace shows as a single span. Fix: circuit breaker + cache (Stripe’s idempotent GETs are cacheable). - 500ms `Thread.sleep` in a `@Transactional` method (a bug — the tx holds a DB connection for 500ms; the pool exhausts under load). Fix: extract the sleep outside the tx.

### ASCII — the latency breakdown:

```
Request: 500ms total
├─ 5ms client→LB (TLS, network)
├─ 1ms LB→app (internal)
├─ 200ms app CPU (serialization, ObjectMapper)
├─ 100ms GC pause (G1 young gen)
├─ 5ms app→DB (network, local)
├─ 5ms DB query
├─ 5ms DB→app
├─ 100ms connection pool wait (Hikari pending)
├─ 50ms app→LB (serialization done, response start)
└─ 29ms LB→client (TLS, network)
Total = 500ms
Fix: cache ObjectMapper (kills 200ms CPU), tune GC (kills 100ms), size pool (kills 100ms) → 100ms p99
```

**KEY TERMS TO MENTION:** - OpenTelemetry trace spans (the breakdown; gaps = GC/lock/queue) - GC logs ( `-Xlog:gc*`, pause times) - JFR (Java Flight Recorder, JDK Mission Control, hot methods + lock contention) - `async-profiler` (CPU flame graphs, the de-facto Java profiler) - HikariCP `connections_pending` (pool exhaustion = silent latency) - `pg_stat_statements` (DB query time), `pg_stat_activity` (in-flight queries)

**FOLLOW-UP QUESTIONS:** 1. The trace shows a 400ms gap with no span. What is it? 2. The DB query is 5ms but `pg_stat_activity` shows the query waiting on `Lock waits`. What's the contention? 3. After fixing all of the above, p99 is still 200ms. Where now?

**FOLLOW-UP ANSWERS:** 1. Either GC pause, lock wait, or queue wait. (a) Check GC logs at the request's timestamp — a 400ms pause matches. (b) JFR "Lock Instances" — a contended monitor with 400ms wait matches. (c) Hikari `connections_pending` — the request waited 400ms for a DB connection (no span because the JDBC `getConnection()` is in the auto-instrument's "pre-controller" phase, often not spanned). The fix differs by cause: GC → tune; lock → switch to lock-free; pool → size up or fix long-running txs. The trace gap is the symptom; the tool (GC log / JFR / pool metrics) is the diagnosis. 2. Row-lock contention. Two concurrent updates on the same row (e.g., two requests updating the same lead); one waits for the other's tx to commit. `pg_stat_activity.wait_event = 'Lock' + wait_event_type = 'Lock'` confirms. `pg_locks` join shows which row. Fix: shorten the tx (commit faster), batch the updates (one tx for many rows = one lock acquisition), or use optimistic locking (`@Version`) to fail-fast instead of waiting. For a **hot row** (everyone updates it), consider an append-only design (insert events, compute balance on read) to eliminate the row lock. 3. The long tail (p99) is often queueing theory. If your server has 100 threads and 1000 concurrent requests, 900 are queued; the queue wait time grows with the request rate. p99 = the 99th percentile of (queue wait + service time); even if service time is 5ms, queue wait at p99 can be 200ms. The fix: more servers (horizontal scale, lower per-server queue depth), or load shedding (reject requests beyond capacity, return 503 — better than 200ms latency for all). Or, reduce the service time (the 5ms query → 1ms with a covering index), which shrinks the queue (Little's Law:  $L = \lambda \times W$ ; smaller  $W$  = smaller  $L$  = shorter queue). The "p99 is queueing, not service time" insight is the senior-level diagnosis; most engineers chase the service time when the queue is the problem.

**RED FLAGS TO AVOID:** - "The DB is the bottleneck" without measuring (the query is 5ms; the bottleneck is elsewhere). - "Add more CPU" without finding the actual consumer (a lock wait isn't CPU-bound). - "It's the network" without `tcpdump` evidence. - Tuning GC without checking allocation rate (the root cause is upstream of GC).

## QG-Bonus2-5: TCP — why does a server in `TIME_WAIT` after closing a socket hold the port? How does `SO_REUSEADDR` help, and what's the `TIME_WAIT` assassination risk?

- **Difficulty:** Medium-Hard
- **Category:** Networking / TCP
- **Tags:** tcp, time-wait, so-reuseaddr, fin, rst, sequence-number

**ANSWER:**

When the server closes a TCP connection (sends FIN, client ACKs, client sends FIN, server ACKs), the server's socket enters `TIME_WAIT` for  $2 \times \text{MSL}$  (Maximum Segment Lifetime, typically 60-120s). The reasons: 1. **Late packets:** A delayed segment from the closed connection could arrive at the server after the close; if the server reused the port for a new connection, the late segment would be misinterpreted as data for the new connection. `TIME_WAIT` keeps the port reserved so late segments are dropped (not delivered to a new socket). 2. **Final ACK loss:** The server's final ACK to the client's FIN could be lost; the client would retransmit the FIN; the server in `TIME_WAIT` re-ACKs. If the server moved to `CLOSED` immediately, the retransmitted FIN gets an RST, and the client thinks the connection failed ungracefully.

The  $2 \times \text{MSL}$  window (60-120s) is the maximum time a packet can live in the network (one MSL to the destination, one back). After  $2 \times \text{MSL}$ , no late packet from the old connection can arrive.

**The problem:** A server that opens/closes many short connections (e.g., a proxy, or a load test client) accumulates `TIME_WAIT` sockets (60s each). At 1000 connections/sec, that's 60K `TIME_WAIT` sockets, each consuming ~1.5KB of kernel memory = 90MB; and on the client side, you exhaust ephemeral ports (~28K available) — "Cannot assign requested address."

`SO_REUSEADDR`: Allows a socket to bind to a port that's in `TIME_WAIT`. The new socket accepts connections, and the kernel drops any late segments from the old connection (the new socket's seq numbers differ, so old segments don't match). This lets the server restart immediately after a crash (the port is in `TIME_WAIT` from the old process) without waiting 60s. Critical for fast restarts.

`SO_REUSEPORT` (**Linux 3.9+**): Multiple sockets can bind to the `same` port; the kernel load-balances incoming connections across them. Used by multi-process servers (nginx workers, or JVM with multiple instances on the same port — rare) to avoid the thundering-herd on accept.

`TIME_WAIT` **assassination risk:** If the server in `TIME_WAIT` receives a new SYN (initiating a new connection) with the same 4-tuple (src ip, src port, dst ip, dst port) as the old connection, RFC 1337 says it should NOT reset the `TIME_WAIT` (the SYN could be from a stale packet). Some buggy implementations reset on SYN, causing the old connection's late packets to be misinterpreted. The fix: the `tcp_rfc1337` sysctl (Linux) — set to 1 to prevent the assassination.

**For an HTTP server:** The client (browser) opens connections to the server; the server is the one that should `not` close first (to avoid `TIME_WAIT` on the server side). HTTP keep-alive keeps connections open (no FIN); the server limits the number of idle connections (close after 60s idle). For server-to-server (proxy → backend), use keep-alive aggressively (avoid `TIME_WAIT` on the proxy). For a server that must close (e.g., short-lived test connections), `SO_REUSEADDR` on the socket is the standard mitigation.

**Code — Java socket with `SO_REUSEADDR`:**

```
ServerSocket socket = new ServerSocket();
socket.setReuseAddress(true); // SO_REUSEADDR
socket.bind(new InetSocketAddress(8080));
// ... accept connections ...
```

For a client (or any socket that opens many short connections), `setReuseAddress(true)` before `bind()`. For a server, set before `bind()`. The default in Java is `true` for `ServerSocket` (since Java 1.4), but explicit is safer.

### ASCII — the TCP close handshake:



**KEY TERMS TO MENTION:** - `TIME_WAIT` ( $2 \times \text{MSL} = 60\text{-}120\text{s}$ ; the closer holds the port) - Two purposes: late-packet drop + final-ACK retransmit - `SO_REUSEADDR` (bind to a `TIME_WAIT` port; enables fast restart) - `SO_REUSEPORT` (Linux 3.9+; multiple sockets on the same port; LB across them) - RFC 1337 ( `tcp_rfc1337` sysctl; prevent `TIME_WAIT` assassination) - Ephemeral port exhaustion (client-side;  $\sim 28\text{K}$  ports)

**FOLLOW-UP QUESTIONS:** 1. A load test opens 50K short connections/sec to your server. After 60s, the server can't accept new connections — port exhaustion. How do you fix? 2. The server restarts and can't bind to port 8080 ("Address already in use") — the old process's socket is in `TIME_WAIT`. `SO_REUSEADDR` is set. Why still failing? 3. The client (not the server) is closing connections, accumulating `TIME_WAIT` on the client side, exhausting ephemeral ports. Fix?

**FOLLOW-UP ANSWERS:** 1. The server is the `close initiator` (it sent FIN first), so it holds the `TIME_WAIT` (the client side is `CLOSED`). Fix: have the `client` close first (server stays in `CLOSE_WAIT` → `CLOSED`, no `TIME_WAIT` on the server). For HTTP, the server can use `Connection: close` to indicate the client should close, but typically the client closes. Or, use keep-alive aggressively (no close at all; connections reused). Or, increase the ephemeral port range ( `sysctl net.ipv4.ip_local_port_range = "1024 65535"` ) — 65K ports, but at 50K/sec you exhaust in 1.3s; not enough. The real fix: keep-alive + connection pooling (don't open 50K connections/sec; reuse 1000 long-lived connections). For a load test specifically, the test framework should reuse connections (e.g., `wrk --timeout 60s` with keep-alive). 2. `SO_REUSEADDR` allows binding to a port in `TIME_WAIT`, but if the `old process is still running`

(didn't actually die; zombie or stuck), the port is in `LISTEN`, not `TIME_WAIT`. `SO_REUSEADDR` doesn't let two processes listen on the same port (that's `SO_REUSEPORT`). Check `netstat -tlnp | grep 8080` — if you see `LISTEN` from a live PID, kill that PID first. Or, `SO_REUSEPORT` would allow two processes (rare; usually you don't want this). The "Address already in use" with `SO_REUSEADDR` set is almost always a live process holding the port, not a `TIME_WAIT`. 3. The client's `TIME_WAIT` is unavoidable (the closer holds it). Fixes: (a) **Keep-alive** — don't close; reuse connections (HTTP keep-alive, JDBC connection pool, gRPC multiplexing). (b) **Server-side close** — let the server send FIN first; the server holds the `TIME_WAIT`, the client is free. For HTTP, the server can include `Connection: close` after N requests to rotate connections (server-side `TIME_WAIT` is bounded; client-side is not, because clients come and go). (c) `tcp_tw_reuse=1` (Linux sysctl) — allows the client to reuse a `TIME_WAIT` socket for a new outbound connection [to the same destination](#), if the new connection's initial seq number is greater than the old. This is the standard fix for client-side `TIME_WAIT` exhaustion; it's safe per RFC 1323 (with PAWS — Protection Against Wrapped Sequence numbers). (d) `tcp_tw_recycle=1` (Linux, removed in 4.12) — was faster but caused issues with NAT (multiple clients behind one IP); removed. Don't use it.

**RED FLAGS TO AVOID:** - "Disable `TIME_WAIT`" — no sysctl for it; it's a TCP requirement. - `tcp_tw_recycle=1` (removed in Linux 4.12; breaks NAT). - Server closing first without keep-alive — `TIME_WAIT` accumulates server-side. - Not setting `SO_REUSEADDR` on server sockets — restart blocked for 60s after crash.

## QG-Bonus2-6: You have a stream of 1 billion events. Find the top-100 most frequent event IDs in real-time, $O(N)$ memory bound. Algorithm?

- **Difficulty:** Hard
- **Category:** Algorithms / Streaming
- **Tags:** count-min-sketch, heavy-hitters, top-k, min-heap, misra-gries

### ANSWER:

This is the **Top-K Heavy Hitters** problem on a stream. Exact counting (a `HashMap<id, count>`) needs  $O(\text{unique IDs})$  memory — if there are 100M unique IDs, that's 800MB for the counts alone. The constraint " $O(N)$  memory bound" ( $N$  small, say 1000 entries) rules out exact counting. Two approximate algorithms:

**(1) Misra-Gries (a.k.a. "Frequent Items" / Lossy Counting):** Maintains  $N$  counters, each with an ID + count. On each event: - If the ID is in the map, increment its counter. - If the map has  $< N$  entries, add (ID, 1). - Else, decrement all counters by 1; remove any that hit 0.

At the end, the map contains [candidates](#) for the top-K; any ID with frequency  $> (\text{stream\_size} / N)$  is guaranteed to be in the map. To get the exact top-100, do a second pass over the stream counting only the candidates (small set). Memory:  $O(N)$  counters. Two-pass; the first pass is the filter, the second is the verification.

**(2) Count-Min Sketch + Min-Heap (the streaming answer):** - A Count-Min Sketch (CMS) is a 2D array of  $d$  rows  $\times$   $w$  columns; each row has an independent hash function. To count an ID: for each row  $i$ , increment `table[i][h_i(ID)]`. To estimate the count of an ID: take `min(table[i][h_i(ID)])` across rows. The min is an over-estimate (the sketch over-counts due to collisions), bounded by  $\epsilon \times \text{stream\_size}$  with probability  $1 - \delta$  for  $w = \text{ceil}(e/\epsilon)$ ,  $d = \text{ceil}(\ln(1/\delta))$ . For 1% error,  $w = 271$ ,  $d = 5 \rightarrow$  1355 counters total = 11KB. - A min-heap of size  $K=100$  tracks the current top-K by estimated count. On each event: increment in CMS, get the new estimated count, if it's  $>$  the heap's min (and the ID isn't already in the heap), evict the min and insert. Memory:  $O(K) = 100$  entries.

The CMS uses  $O(d \times w) = 11\text{KB}$  regardless of stream size or unique count. The heap is  $O(K)$ . Total memory:  $\sim 12\text{KB}$ . The catch: the counts are approximate (over-estimates); the top-K is approximately correct (some borderline IDs may be mis-ordered, but the truly heavy hitters — frequency  $\gg$  the rest — are correct).

### Code — Count-Min Sketch + Heap:

```
public class TopKStreamer {
 private final int[][] sketch; // d rows x w cols
 private final int[] hashSeeds;
 private final PriorityQueue<IdCount> heap = new
 PriorityQueue<>(Comparator.comparingLong(IdCount::count));
 private final Set<Long> inHeap = new HashSet<>();
 private final int k;

 public void observe(long id) {
 long est = increment(id); // increment CMS, return new estimate
 if (inHeap.contains(id)) {
 // re-heapify with new count (remove + re-insert)
 heap.removeIf(e -> e.id() == id);
 heap.offer(new IdCount(id, est));
 } else if (heap.size() < k) {
 heap.offer(new IdCount(id, est)); inHeap.add(id);
 } else if (est > heap.peek().count()) {
 IdCount evicted = heap.poll(); inHeap.remove(evicted.id());
 heap.offer(new IdCount(id, est)); inHeap.add(id);
 }
 }

 private long increment(long id) {
 long min = Long.MAX_VALUE;
 for (int i = 0; i < d; i++) {
 int col = hash(id, hashSeeds[i]) % w;
 sketch[i][col]++;
 min = Math.min(min, sketch[i][col]);
 }
 return min;
 }

 public List<IdCount> topK() { return new ArrayList<>(heap); }
}
```

**Trade-off:** CMS over-estimates (collisions add to other IDs' counts). The estimated top-K may include an ID whose true count is lower than another ID's true count, if both are near the K-th boundary. For **heavy** hitters (top ID is  $100\times$  the K-th ID's count), the estimate is exact. For **borderline** IDs (the K-th and (K+1)-th have similar counts), the order may flip. To verify, do a second pass over a recent window (e.g., the last hour's events) with exact counting for only the candidate IDs from the CMS — small exact pass.

**When to use exact (HashMap):** If the number of unique IDs is small (e.g., 10K) and fits in memory, exact is better (no error). CMS shines when unique IDs are huge (millions) and you only care about the heavy hitters.

### ASCII — the CMS:

```
d=3 rows, w=10 cols. Hashes: h1, h2, h3.
```

```
Event ID = 42:
```

```
h1(42) = 3 → sketch[0][3]++
```

```
h2(42) = 7 → sketch[1][7]++
```

```
h3(42) = 1 → sketch[2][1]++
```

```
Estimate count of 42:
```

```
min(sketch[0][3], sketch[1][7], sketch[2][1])
```

```
(collisions over-count; min is the best estimate)
```

```
sketch:
```

```

 0 1 2 3 4 5 6 7 8 9
r0: 0 0 0 [5] 0 0 0 0 0 0 ← h1(42) = 3 → 5
r1: 0 0 0 0 0 0 0 [4] 0 0 ← h2(42) = 7 → 4 (someone else also hashed to
7)
r2: 0 [5] 0 0 0 0 0 0 0 0 ← h3(42) = 1 → 5
Estimate = min(5, 4, 5) = 4 (true count might be 4; the r1 has a collision)
```

**KEY TERMS TO MENTION:** - Top-K Heavy Hitters (streaming problem; exact vs approximate) - Misra-Gries (Lossy Counting; N counters, decrement-on-overflow, candidate set) - Count-Min Sketch ( $d \times w$  array, d hashes, min of counts, over-estimate) - Min-Heap of size K (evict min on insert) - Two-pass (approximate first pass for candidates, exact second pass on candidates)

**FOLLOW-UP QUESTIONS:** 1. The K-th and (K+1)-th IDs have the same true count (a tie). The CMS gives them different estimates. Which is in the top-K? 2. The stream has 1B events but only 100 unique IDs. Use CMS or HashMap? 3. You need exact top-K, no approximation. Memory bound is 1GB. 1B events, 100M unique IDs. Feasible?

**FOLLOW-UP ANSWERS:** 1. The CMS's estimate is the over-count; the heap uses the estimate to order. In a tie, the heap may pick either; the estimate breaks the tie (which is wrong). To handle ties: when two estimates are within  $\epsilon$  (the CMS error bound), treat them as tied and use a tie-breaker (e.g., lexicographic ID). Or, run a second exact pass on the top 200 candidates (twice K) to break ties exactly. The top-K is "approximately K" — at the boundary, ties are inherent to the problem; the CMS gives you a candidate set, the exact pass gives the true top-K. 2. HashMap. 100 unique IDs  $\times$  8 bytes (long ID) + 8 bytes (count)

= 1.6KB. Exact, no error. CMS is overkill (and approximate). The decision rule: if `unique_count × entry_size` fits in memory, use `HashMap`; else, use CMS. The “1B events” doesn’t matter — it’s the unique count that drives memory. CMS shines when unique count is 10M+ and you can’t fit `10M × 16B = 160MB` (you might be able to; CMS is for when you can’t, or for sub-MS increment latency — CMS is also faster than `HashMap` due to no resizing, fixed-size array). 3. 100M unique IDs × 16B (long + count) = 1.6GB — slightly over your 1GB. Tight. Options: (a) Use 4-byte ints for the ID (if IDs are  $< 2^{32}$ ) → 8 bytes per entry → 800MB, fits. (b) Use a compact hash map (e.g., `Trove` `TLongIntHashMap` or `fastutil Long2IntOpenHashMap`) — primitives, no boxing, ~50% the memory of `HashMap<Long, Integer>`. (c) Sort the stream externally (external merge sort,  $O(N/B)$  IOs), then count adjacent duplicates (one pass after sort) — no in-memory map at all; just an open file scan. (d) If you have a cluster, partition the stream across N nodes (by hash of ID), each counts locally, then aggregate top-K from each node. For a single-node, 1GB-bound, 100M uniques: option (a) or (b); option (c) if you can sort cheaply (the stream is already on disk).

**RED FLAGS TO AVOID:** - `HashMap` for 100M unique IDs with a 1GB bound — OOM. - CMS for 100 unique IDs — exact is better; CMS is approximate. - Min-heap of size `stream_size` (not K) —  $O(N)$  memory, misses the point. - Treating CMS estimates as exact — they over-count; ties are inherent.

## QG-Bonus2-7: Design a URL shortener for 100M URLs, 10K new URLs/sec, 100K redirects/sec. Walk through the full system design.

- **Difficulty:** Hard
- **Category:** System Design
- **Tags:** url-shortener, base62, hash, database, cache, cdn

### ANSWER:

The classic system design question. Components + sizing first, then dive.

**Sizing:** - 100M URLs × 500 bytes (long URL + metadata) = 50GB. Fits on one machine’s disk, but we want HA + scale → sharded. - 10K new URLs/sec × 86,400 s/day = 864M URLs/day → wait, that’s 864M, contradicting 100M. Let me reconcile: 100M is the **total** stored; 10K/sec is the **write rate**. If we keep URLs forever, 10K/sec × 30 days = 26B URLs/month. Let’s say 100M URLs is the **hot** set (recently created, frequently redirected); older URLs go to cold storage. Or, the prompt’s 100M is the steady-state; the write rate is lower (10K/sec is the peak, average 1K/sec → 86M/day, ~matches 100M total after 1 day). I’ll assume 100M URLs active, 10K/sec peak writes, 100K/sec peak redirects (10:1 read:write ratio, typical for shorteners).

- Short code: 7 chars of base62 ( $62^7 = 3.5T$  combinations) — plenty for 100M URLs ( $62^6 = 56B$  is enough, but 7 chars gives headroom). 7 chars × 100M = 700MB for the codes alone (small).

### Architecture:

Client → CDN (Cloudflare/CloudFront) → API Gateway → App servers → Cache (Redis) → DB (Postgres sharded)

**Components:** 1. **API Gateway:** TLS termination, rate limiting (per-IP, per-API-key), auth (for POST /shorten; GET redirects are public). 2. **App servers (stateless, N=10):** Spring Boot; each handles POST /shorten and GET /:code. Scaled behind a LB. 3. **Cache (Redis, sharded):** short\_code → long\_url, TTL = 30 days (the hot 100M URLs fit in  $100M \times 100$  bytes = 10GB Redis). 100K reads/sec, all from cache (after warmup). Misses fall through to DB. 4. **DB (Postgres, sharded by short\_code hash):** 100M URLs, 50GB, sharded across 5 nodes (10GB each). Writes: 10K/sec, distributed across 5 shards = 2K/sec/shard (well within Postgres's 10K write/sec capability). Reads: only on cache miss (~1% of 100K/sec = 1K/sec) — well within Postgres's read capacity. 5. **CDN:** Caches the 301 redirect responses for hot URLs (high-traffic shorteners' redirects are highly skewed; the top 1% of URLs get 50% of the traffic). CDN absorbs 80% of reads; origin sees 20K/sec instead of 100K/sec.

**Short code generation:** - **Option A — Hash (MD5/SHA) of the long URL, take first 7 chars of base62:** Deterministic (same URL → same code), enables dedup. But collisions (two URLs hash to the same 7 chars) — handle with a unique constraint + retry with a longer code (8 chars) on collision. The hash is **shortening** (the same URL always shortens to the same code), which is a feature (analytics dedup) or a privacy leak (you can enumerate all shortened URLs by hashing common URLs). For a public shortener, a hash is fine. - **Option B — Auto-increment ID, base62-encoded:** `id = 1234567 → base62(1234567) = "5bH1"`. No collisions, no hash, but predictable (sequential IDs are enumerable; a scraper can enumerate all URLs by walking 1, 2, 3...). For a public shortener, predictable IDs leak private URLs. Fix: encrypt the ID (AES with a server secret) before base62 — looks random but is reversible by the server. - **Option C — Random 7-char base62:** `crypto.random(7 base62 chars)`, check uniqueness against DB, retry on collision.  $62^7 = 3.5T$ ; with 100M used, the collision probability on a new random is  $100M / 3.5T = 0.003\%$  — about 1 in 35K. So 1 in 35K writes retries; acceptable. Random codes are not enumerable; best for privacy.

We use **C (random) with a twist:** generate the code, INSERT with `ON CONFLICT (code) DO NOTHING` (Postgres unique index), if 0 rows inserted (collision), regenerate. Idempotent, no locking.

**The redirect flow (GET /:code):**

```

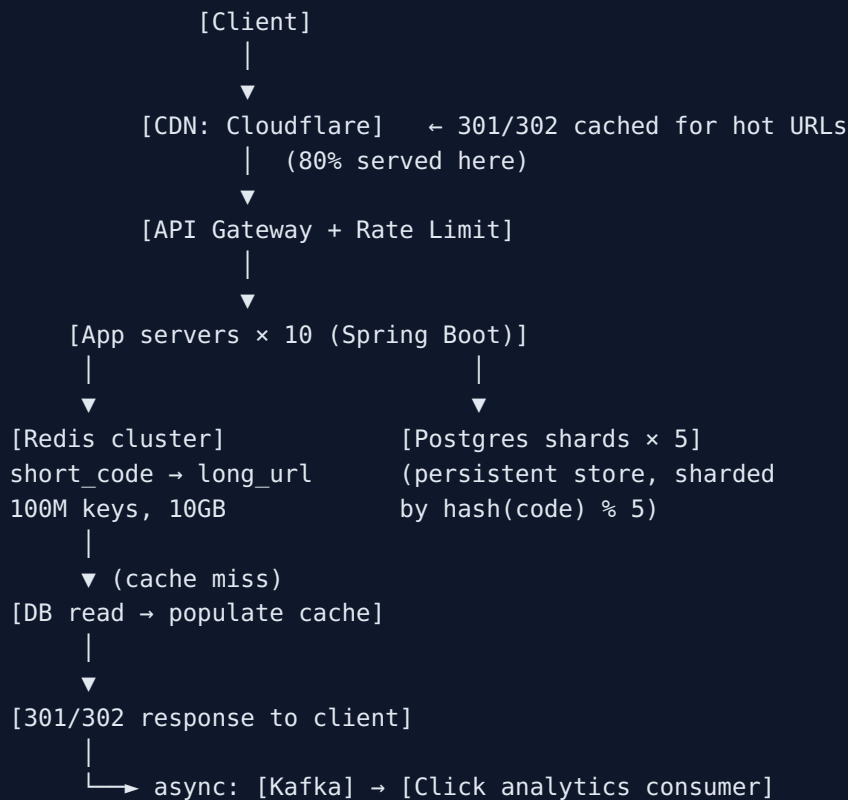
@GetMapping("/{code}")
public ResponseEntity<Void> redirect(@PathVariable String code) {
 // 1. Check cache
 String longUrl = redis.get("url:" + code);
 if (longUrl != null) {
 return ResponseEntity.status(301).location(URI.create(longUrl)).build();
 }
 // 2. Cache miss → DB
 Optional<UrlMapping> row = urlRepo.findByCode(code);
 if (row.isEmpty()) return ResponseEntity.status(404).build();
 longUrl = row.get().getLongUrl();
 // 3. Populate cache (read-through)
 redis.set("url:" + code, longUrl, Duration.ofDays(30));
 // 4. Async analytics (don't block the redirect)
 analytics.record(code, request); // async, fire-and-forget
 return ResponseEntity.status(301).location(URI.create(longUrl)).build();
}

```

**The 301 vs 302 choice:** 301 (permanent) is cached by browsers + CDNs — subsequent visits don't hit our server (faster, less load). 302 (temporary) is not cached — every visit hits us (slower, but analytics sees every click). For a marketing shortener (where click analytics is the product), use 302. For a pure shortener (no analytics needed), use 301. We use 302 for the first N clicks (analytics), then 301 (caching) for the long tail — a hybrid.

**Analytics:** Don't block the redirect. Async publish to Kafka: `click_event(code, ip, ua, ts)`. A consumer updates the click count + pushes to analytics dashboards. The redirect's latency = cache lookup (1ms) + 302 response; analytics is fire-and-forget.

**ASCII — the architecture:**



**KEY TERMS TO MENTION:** - Sizing (URLs × size, write rate, read rate, 10:1 R:W) - Base62 encoding ( $62^7 = 3.5T$  combinations) - Code generation strategies (hash, auto-increment+encrypt, random) - 301 (cached, browser+CDN) vs 302 (no cache, analytics) - Read-through cache (miss → DB → populate → return) - Async analytics (fire-and-forget to Kafka, don't block the redirect) - CDN for hot-URL caching (80% offload)

**FOLLOW-UP QUESTIONS:** 1. The DB shard for `code = X` goes down. The cache has `X → long_url`. The cache TTL expires. Now what? 2. The same long URL is shortened twice. Do you return the same short code or a new one? 3. A URL is shortened, then the user wants to delete it. Existing redirects in the wild?

**FOLLOW-UP ANSWERS:** 1. The cache still serves `X` for up to TTL (30 days). After TTL, the cache misses, and the DB shard is down → 404 (we can't resolve). Mitigations: (a) DB HA — each shard is a Postgres primary + 2 read replicas; the primary's failover to a replica is <30s. (b) Cache the 404 too (negative caching, short TTL = 60s) to avoid hammering the down shard. (c) Cross-shard replication for hot URLs (the top 1% of URLs are replicated to all shards; if one shard is down, others serve the hot URLs). (d) For the brief outage, accept 404s for that shard's URLs — the SLA is "99.9% of redirects succeed", and a single-shard outage is 20% of URLs (1 of 5 shards), so we're at 80% success — below SLA. The fix is DB HA (failover <30s); a 30s outage is acceptable. 2. Depends on the policy. (a) **Idempotent (same code):** hash the long URL, INSERT with `ON CONFLICT (long_url) DO UPDATE SET code = EXCLUDED.code RETURNING code` — returns the existing code. Pros: dedup (same URL → same code, easier analytics). Cons: a user can't have two short codes for the same long URL (e.g., one for Twitter campaign, one for email campaign — to track separately). (b) **Per-user (different codes):** each shortening creates a new code, even for the same long URL. Pros: per-campaign analytics. Cons: storage bloat

(many codes for one URL). We choose (b) for analytics (each short code is a “campaign”), with a dedup option for the API ( `?dedup=true` returns the existing code). 3. Mark the URL as deleted (soft delete: `deleted_at = now()`). The redirect checks `deleted_at IS NULL`; if deleted, return 404 or a “this link has been removed” page. Existing redirects in the wild (printed on billboards, etc.) now 404. For a “redirect to a different URL” (re-targeting), update `long_url` (the code stays, the destination changes) — useful for marketing campaigns. For analytics, the click count before/after the deletion is preserved. Hard delete (DROP the row) is rare; we soft delete for audit and analytics.

**RED FLAGS TO AVOID:** - Hash with first 7 chars of MD5 without collision handling — collisions break the shortener. - 301 without analytics consideration — you lose click data to CDN caching. - DB single instance (no HA) — a 30s outage breaks redirects. - Synchronous analytics in the redirect path — adds latency to every redirect.

---

## QG-Bonus2-8: Implement `Promise.all` and `Promise.race` from scratch in JavaScript. Explain the semantics and edge cases.

---

- **Difficulty:** Medium-Hard
- **Category:** Frontend / Async
- **Tags:** promise, async, javascript, all, race, allsettled

### ANSWER:

These are foundational frontend interview questions for senior roles. The semantics: - `Promise.all([p1, p2, ...])`: Returns a promise that resolves to an array of results when **all** input promises resolve, in order. Rejects **immediately** if any input rejects (with the rejection reason; other results are discarded). - `Promise.race([p1, p2, ...])`: Returns a promise that resolves or rejects with the **first** settled input (whichever settles first; the rest are ignored). - `Promise.allSettled([p1, p2, ...])`: Returns a promise that resolves to an array of `{status: 'fulfilled', value} | {status: 'rejected', reason}` when **all** settle. Never rejects. - `Promise.any([p1, p2, ...])`: Resolves with the first **fulfilled** value. Rejects with `AggregateError` if all reject.

### `Promise.all` implementation:

```
function promiseAll(promises) {
 return new Promise((resolve, reject) => {
 const results = new Array(promises.length);
 let remaining = promises.length;
 if (remaining === 0) { resolve([]); return; } // edge: empty array
 promises.forEach((p, i) => {
 Promise.resolve(p).then(
 (val) => {
 results[i] = val; // preserve order
 if (--remaining === 0) resolve(results);
 },
 (err) => reject(err) // reject on first rejection
);
 });
 });
}
```

Edge cases: - Empty array → resolves with `[]` immediately. - Non-promise inputs → wrapped via `Promise.resolve()`. - Order preserved (`results[i]` = i-th promise's result, regardless of resolution order).

#### **Promise.race implementation:**

```
function promiseRace(promises) {
 return new Promise((resolve, reject) => {
 promises.forEach(p => {
 Promise.resolve(p).then(resolve, reject); // first to settle wins
 });
 });
}
```

Edge cases: - Empty array → pending forever (never settles). This is a footgun; ES6 specified it this way. - Non-promise inputs settle synchronously (via `Promise.resolve`).

#### **Promise.allSettled implementation:**

```
function promiseAllSettled(promises) {
 return new Promise((resolve) => {
 const results = new Array(promises.length);
 let remaining = promises.length;
 if (remaining === 0) { resolve([]); return; }
 promises.forEach((p, i) => {
 Promise.resolve(p).then(
 (val) => { results[i] = { status: 'fulfilled', value: val }; if (--remaining === 0) resolve(results); },
 (err) => { results[i] = { status: 'rejected', reason: err }; if (--remaining === 0) resolve(results); }
);
 });
 });
}
```

**Promise.any implementation:**

```
function promiseAny(promises) {
 return new Promise((resolve, reject) => {
 const errors = new Array(promises.length);
 let remaining = promises.length;
 if (remaining === 0) { reject(new AggregateError([], 'All promises were rejected')); return; }
 promises.forEach((p, i) => {
 Promise.resolve(p).then(
 resolve, // first fulfilled wins
 (err) => {
 errors[i] = err;
 if (--remaining === 0) reject(new AggregateError(errors, 'All promises were rejected'));
 }
);
 });
 });
}
```

**The four-way comparison:**

Function	Resolves when	Rejects when
<code>all</code>	All fulfill	Any rejects (immediately)
<code>race</code>	First settles (fulfilled)	First settles (rejected)
<code>allSettled</code>	All settle (never rejects)	(never)
<code>any</code>	First fulfills	All reject (AggregateError)

**Common use cases:** - `Promise.all`: Parallel fetch of independent resources; fail-fast if any fails. “Load user + posts + comments; render when all done.” - `Promise.race`: Timeout — `Promise.race([fetch(url), timeoutAfter(5s)])`. Or “first available mirror responds.” - `Promise.allSettled`: Best-effort fan-out — “send to all 3 analytics providers; don’t fail if one is down.” - `Promise.any`: Redundancy — “try 3 mirrors; first success wins; fail only if all down.”

**ASCII — Promise.all semantics:**

```
p1: —————> resolve(1) —————> results[0] = 1
p2: —————> resolve(2) —————> results[1] = 2
p3: —————> reject(err) —————> Promise.all rejects with err; results discarded
```

**KEY TERMS TO MENTION:** - The four combinators (`all`, `race`, `allSettled`, `any`) and their semantics - `Promise.resolve(p)` wrapping (handles non-promises + thenables) - Order preservation (`results[i]` = *i*-th promise, regardless of settle order) - `AggregateError` (for `any` when all reject) - Empty array behavior (`all` / `allSettled` resolve `[]`; `race` / `any` pending forever or reject)

**FOLLOW-UP QUESTIONS:** 1. In `Promise.all`, one promise rejects. The other promises are still pending. Are they cancelled? 2. Implement a `Promise.all` with a concurrency limit (max 5 in flight at a time). 3. `Promise.race` with a timeout — implement `withTimeout(promise, ms)`.

**FOLLOW-UP ANSWERS:** 1. No — JavaScript promises are not cancellable (the spec has no cancellation). The other promises continue to completion; their results are just discarded (the `Promise.all`'s resolve callback never fires after a rejection). The side effects of those promises (network requests, file writes) **still happen**. If you need cancellation, use `AbortController` (fetch supports it; pass an `AbortSignal` to the fetch, call `controller.abort()` to cancel). The `Promise.all` rejection is a signal to your code to abort the others via the controller. This is the modern (post-2020) way to handle cancellation in JS. 2. Use a worker pool pattern: maintain a queue and a count of in-flight; pull from the queue when one completes. Code:

```
async function promiseAllLimit(promises, limit) {
 const results = new Array(promises.length);
 let next = 0;
 async function worker() {
 while (next < promises.length) {
 const i = next++;
 results[i] = await promises[i];
 }
 }
 await Promise.all(Array.from({length: limit}, worker));
 return results;
}
```

This spawns `limit` workers; each pulls the next index, awaits, repeats. Preserves order (results[i] = i-th promise). Handles rejections (the `Promise.all` of workers rejects, propagating the first error; remaining workers are abandoned). For 100 promises with limit=5, only 5 are in flight at any time. 3.

`Promise.race([promise, new Promise((_, reject) => setTimeout(() => reject(new Error('timeout')), ms))`

The timeout promise rejects after `ms`; `race` rejects with the timeout error if the original hasn't settled. The original promise continues (no cancellation) — its side effects still happen, but the caller has already moved on. To actually cancel the original, use `AbortController`:

```
function withTimeout(promiseFn, ms) { // promiseFn takes an AbortSignal
 const controller = new AbortController();
 const timeout = setTimeout(() => controller.abort(), ms);
 return promiseFn(controller.signal).finally(() => clearTimeout(timeout));
}
```

The `promiseFn` (e.g., `fetch(url, {signal})`) will throw an `AbortError` when aborted, and the underlying network request is cancelled (not just the promise). This is the production pattern for fetch-with-timeout.

**RED FLAGS TO AVOID:** - “Promises are cancellable” — they’re not; `AbortController` is the mechanism. - `Promise.race` on empty array — hangs forever. - `Promise.all` with order bug (push to results array instead of `results[i]`) — order breaks under async resolution. - `Promise.any` rejecting with the last error instead of `AggregateError` — wrong type.

## QG-Bonus2-9: A React component re-renders on every parent render even though its props are unchanged. Why, and how do you fix it (3 ways)?

- **Difficulty:** Medium
- **Category:** Frontend / React
- **Tags:** react, memo, usememo, usecallback, referential-equality

### ANSWER:

The default React behavior: when a parent re-renders, all children re-render regardless of prop changes. React doesn’t deep-compare props (expensive); it shallow-compares by default only if the child is wrapped in `React.memo`. Without `memo`, every parent render → every child render. The “every prop unchanged but child still re-renders” bug is one of:

**(1) Inline object/array/function props:** `<Child onClick={() => ...} style={{color: 'red'}} />` — each render creates a **new** object/function, breaking referential equality. Even `React.memo`’s shallow comparison sees a “new” prop. Fix: `useMemo` for objects/arrays, `useCallback` for functions, or hoist constants outside the component.

**(2) Children as props (the “hidden re-render”):** `<Layout>{<Child />}</Layout>` — the `<Child />` element is created in the parent’s render, so it re-renders even if `Layout` is memoized. Fix: pass the `element` via `useMemo`, or render the child inside `Layout` (children as a function: `<Layout>{() => <Child />}</Layout>`).

**(3) Context consumer re-renders:** A `useContext(MyContext)` re-renders on every context value change, even if the consumed slice is unchanged. Fix: split the context (one for the changing value, one for the stable), or use a selector (`use-context-selector` library), or memoize the context value.

### Three fixes, in order of preference:

#### Fix A — `React.memo` (component-level memoization):

```
const Child = React.memo(function Child({ name, onClick }) {
 return <div onClick={onClick}>{name}</div>;
});
```

`React.memo` shallow-compares props; if all are referentially equal, skips re-render. But: inline objects/functions (Fix A above) still break this. Combine with B.

#### Fix B — `useMemo` / `useCallback` (stabilize props):

```
function Parent() {
 const [count, setCount] = useState(0);
 const name = useMemo(() => ({ first: 'Alice', last: 'Smith' }), []); // stable
 const onClick = useCallback(() => console.log('click'), []); // stable
 return <Child name={name} onClick={onClick} />;
}
```

Now `React.memo`'s shallow compare succeeds; `Child` doesn't re-render when `count` changes.

**Fix C — React Compiler (the modern fix, 2024+):** React Compiler (formerly React Forget) auto-memoizes — no need for `useMemo` / `useCallback`. Currently opt-in (beta in React 19). The future: write idiomatic code (inline objects, inline functions), and the compiler memoizes under the hood. Eliminates the entire class of bugs.

**Diagnosing which fix:** Use React DevTools' "Highlight updates when components render" — components that flash on every render are the ones to investigate. Use the "why-did-you-render" library (or React 19's built-in profiler) to log the changed props. For a parent re-rendering the child unnecessarily, the diff will show "props: onClick changed (function identity)" → it's a function reference issue → `useCallback`. For "children prop changed" → it's the children-as-prop issue → restructure.

**The decision tree:**

```
Child re-renders on every parent render?
├─ Is the child wrapped in React.memo?
│ ├── No → wrap it (Fix A). Re-test.
│ └─ Yes → check props for inline objects/functions (Fix B).
│ ├── Found inline → useMemo/useCallback (Fix B). Re-test.
│ └─ No inline → check children prop (Fix B'). Re-test.
├─ Still re-renders?
│ └─ Context consumer (Fix C): split context or use selector.
└─ Using React 19? Consider React Compiler (auto-memoization, Fix C').
```

**KEY TERMS TO MENTION:** - `React.memo` (component-level shallow compare) - `useMemo` / `useCallback` (stabilize objects/functions between renders) - Referential equality (the cause of unnecessary re-renders) - Context consumer re-renders (split context or selector) - React Compiler / React Forget (auto-memoization, the future) - React DevTools "Highlight updates" (diagnostic)

**FOLLOW-UP QUESTIONS:** 1. `React.memo` with a prop that's an array. The array's contents are unchanged but the parent re-creates the array. `useMemo([])` vs `useMemo([deps])`? 2. `useCallback(fn, [])` — the callback is stable, but it captures state from the closure. The state changes; the callback is stale. Fix? 3. React Compiler memoizes everything. What's the downside?

**FOLLOW-UP ANSWERS:** 1. `useMemo(() => arr, [])` — the array is created once (empty deps), stable across renders. `useMemo(() => arr, [deps])` — recreated when deps change. The empty-deps version is what you want for a `constant` array. But: if the array's `contents` depend on props/state, you need them in deps

(`useMemo(() => computeArr(prop1, prop2), [prop1, prop2])`). The trap: forgetting deps (`[]`) when the array depends on `prop1`) → stale array. Use the `eslint-plugin-react-hooks` `exhaustive-deps` rule to catch this. 2. The classic stale-closure bug. `useCallback(() => setCount(count + 1), [])` — the `count` captured is the initial value; the callback is stable but always sets `count` to 1 (the initial +1). Fix: use the `functional` update form: `useCallback(() => setCount(c => c + 1), [])` — `c` is the current state, no closure capture. For multiple state values, use a ref: `const countRef = useRef(count); countRef.current = count; useCallback(() => countRef.current + 1, [])`. Or, include `count` in deps (`useCallback(() => setCount(count + 1), [count])`) — but then the callback changes when `count` changes, defeating the memoization. The functional update is the cleanest. 3. Memory + build time. The compiler caches memoized values; for a large app, this is more memory than necessary (some values are cheap to recompute). The compiler's analysis is compile-time, adding to build time (typically +20-50% build time). And it can have false positives (memoizing something that genuinely should recompute). The current recommendation: enable in beta, measure, disable selectively for hot paths. It's not a silver bullet; it eliminates the `common case` (forgot to memoize) but doesn't beat a senior engineer who knows exactly what to memoize. For most apps, the compiler is a net win; for performance-critical apps, manual `useMemo` with precise deps is better.

**RED FLAGS TO AVOID:** - "Wrap everything in `React.memo`" — useless without `useMemo` / `useCallback` (inline props break it). - `useCallback(fn, [])` with stale closures — the bug above. - `useMemo` for primitive values (string, number) — primitives are referentially equal by value; no need to memo. - Ignoring React 19's React Compiler — it's the future; mention it.

## QG-Bonus2-10: Postgres `EXPLAIN ANALYZE` shows a `Seq Scan` on the `lead` table (50M rows) for `WHERE created_at > now() - '7 days'`. You have an index on `created_at`. Why isn't it used?

- **Difficulty:** Hard
- **Category:** PostgreSQL / Query Optimization
- **Tags:** explain, seq-scan, index-scan, bitmap-scan, selectivity, cost-estimator

### ANSWER:

Postgres's planner chose `Seq Scan` (full table scan) over `Index Scan` because the `cost estimate` for the seq scan is lower. The cost estimate depends on `selectivity` (what fraction of rows match) and the `cost model` (`random_page_cost` vs `seq_page_cost`).

**The likely cause: low selectivity.** `WHERE created_at > now() - '7 days'` on a 50M-row table where the data spans, say, 1 year — 7 days = ~2% of rows = 1M rows. Postgres's planner estimates: an `Index Scan` would do 1M random IOs (one heap fetch per matching row) = 1M × 4ms (`random_page_cost`, default 4.0) = 4000ms. A `Seq Scan` reads the whole table sequentially = 50M rows × 0.1ms (`seq_page_cost`, default 1.0) = 5000ms... but with a hot cache (the table is mostly in memory), the seq scan is faster. The planner sees the seq scan as cheaper.

**Why an index scan is slow for low-selectivity:** The index tells us [which](#) heap tuples match, but fetching each is a random IO. For 1M matches, that's 1M random reads, each ~4ms on cold storage, vs a sequential scan that reads the pages in order (~0.1ms each). The crossover is at ~5-10% selectivity: below, index wins; above, seq scan wins.

**Diagnosis:** `EXPLAIN (ANALYZE, BUFFERS) :`

```
Seq Scan on lead (cost=0.00..5000.00 rows=1000000 width=...)
 Filter: (created_at > (now() - '7 days'::interval))
 Rows Removed by Filter: 49000000
 Buffers: shared hit=20000 read=30000 ← 50K pages read (the whole table)
```

If `rows=1000000` (the estimate matches the actual), the planner is right (1M matches, seq scan is faster). If `rows=1000` (the estimate is way off), the planner is wrong — the [statistics](#) are stale (`ANALYZE` hasn't run; the histogram on `created_at` is from 30 days ago when the data was different). Fix: `ANALYZE lead;` to refresh stats.

**The fixes, by cause:**

**(A) Low selectivity (the planner is right):** Either accept the seq scan (it's faster), or: - Use `partial index:` `CREATE INDEX lead_recent ON lead (created_at) WHERE created_at > now() - '30 days';` — only indexes recent rows; much smaller; the planner uses it because the index is small and the random IOs are fewer. - Use a [covering index](#) with `INCLUDE` for the columns you `SELECT`: avoids the heap fetch entirely for an index-only scan. - Use a [partitioned table](#) by `created_at` (monthly partitions): the query only scans the recent partitions; old partitions are untouched. This is the right long-term fix for time-series data.

**(B) Stale statistics:** `ANALYZE lead;` (or `VACUUM ANALYZE lead`) refreshes the histogram. The planner sees the actual data distribution and picks the index scan. Schedule autovacuum (it runs `ANALYZE` too) to keep stats fresh.

**(C) Wrong cost model:** If your storage is SSD (random IO is cheap, ~0.1ms), the default `random_page_cost=4.0` (tuned for HDD) over-estimates random IO cost. Lower it: `ALTER SYSTEM SET random_page_cost = 1.1;` (SSD-appropriate). The planner now sees index scans as cheaper. This is a global setting; per-database for finer control.

**(D) Type mismatch (the rare cause):** If `created_at` is `TIMESTAMP` and you compare to `TIMESTAMPTZ` (or vice versa), the planner might not use the index because of an implicit cast. Use `created_at::timestamptz > ...` or `created_at > (now() - '7 days'::interval)::timestamp` to match types.

**The “force index” anti-pattern:** Postgres doesn't have a hint to force an index (`SET enable_seqscan = off;` is a debug tool, not production — it makes seq scans look infinitely expensive). Don't use it; the planner is smarter than you about cost. Fix the root cause (stats, cost model, partial index, partitioning), don't override the planner.

**ASCII — the planner's decision:**

```
Query: SELECT * FROM lead WHERE created_at > now() - '7 days'
```

Planner's estimate:

Index Scan: 1M matches × random\_page\_cost(4.0) = 4,000,000 cost

Seq Scan: 50M rows × seq\_page\_cost(1.0) = 50,000,000 cost?

NO — with caching, the seq scan reads from cache (hit, not read):

Seq Scan: 50M rows × 0.0 (cached) + 1000 actual reads × seq\_page\_cost(1.0) = 1,000 cost

Index Scan: 1M × random\_page\_cost(4.0) = 4,000,000 cost (mostly cold)

Planner picks Seq Scan (1K < 4M).  Correct for this hot-cache scenario.

Fix (if seq scan is too slow in practice): partial index on recent rows

```
CREATE INDEX lead_recent ON lead (created_at) WHERE created_at > now() - '30 days';
→ Index Scan uses the small partial index; few heap fetches; faster.
```

**KEY TERMS TO MENTION:** - `EXPLAIN (ANALYZE, BUFFERS)` — the diagnostic - Selectivity (fraction of rows matching; the planner's key input) - `random_page_cost` (default 4.0, HDD-tuned; lower for SSD, ~1.1) - `seq_page_cost` (default 1.0) - Partial index ( `WHERE` clause on the index; small, focused) - Stale statistics ( `ANALYZE` refreshes the histogram) - Partitioning (time-based; the long-term fix for time-series)

**FOLLOW-UP QUESTIONS:** 1. The query is `WHERE created_at > '2026-09-01'` (a specific date, not a relative `now()`). Same issue? 2. `random_page_cost = 1.1` — global or per-query? 3. After `ANALYZE`, the planner still picks seq scan. The actual run is 30s. Now what?

**FOLLOW-UP ANSWERS:** 1. Different — `'2026-09-01'` is a constant; the planner can use the histogram precisely (it knows exactly how many rows are after this date). The `now() - '7 days'` is a `volatile` expression; the planner estimates conservatively (assumes it returns ~10% of rows by default for `>`, unless stats say otherwise). The fix for `now() - '7 days'`: a `prepared statement` with a bind parameter ( `WHERE created_at > $1`, bind to `now() - '7 days'`) lets the planner use the histogram on the `value`. Or, a partial index ( `WHERE created_at > now() - '30 days'`) — but `now()` in an index predicate is not allowed (the index would be stale). Use a fixed cutoff: `WHERE created_at > '2026-08-01'` (rebuild monthly) — inelegant but works. 2. Global ( `ALTER SYSTEM`) or per-database ( `ALTER DATABASE ... SET`) or per-session ( `SET random_page_cost = 1.1`) or per-query (no direct way; you can `SET` in a transaction). The right level: per-database for SSD (set once); per-session for experimentation (don't pollute global). In a managed Postgres (RDS, CloudSQL), the global setting is exposed as a parameter group; per-database `ALTER DATABASE` works. Per-query isn't supported (no hints). 3. The seq scan is genuinely slow — the table doesn't fit in cache, and the seq scan reads from disk. The index scan would be faster. Why didn't the planner pick it? (a) Stats are still wrong — `ANALYZE` didn't fix (the histogram is too coarse; bump `default_statistics_target = 1000` for this column, re-`ANALYZE`). (b) The index is bloated (autovacuum didn't keep up; the index is large and slow to scan). `REINDEX INDEX lead_created_at_idx`. (c) The query selects many columns (no index-only; heap fetches dominate) — add a covering index with `INCLUDE`. (d) Last resort: partition the table by month — the seq scan becomes a scan of just the recent partitions; old partitions

untouched. For a 50M-row time-series table, partitioning is the **right** long-term fix; the planner routes the query to the relevant partitions automatically.

**RED FLAGS TO AVOID:** - `SET enable_seqscan = off` in production — a debug tool; the planner’s “alternative” is often worse. - “Just add an index” without checking selectivity — the planner may ignore it (correctly). - `random_page_cost = 4.0` on SSD — over-estimates random IO; the planner avoids index scans. - Not partitioning a 50M-row time-series table — the seq-scan problem will recur as the table grows.

## QG-Bonus2-11: Design a distributed counter (e.g., “total likes on a post”) that handles 1M increments/sec across 1000 servers. Strong consistency not required; approximate is OK.

- **Difficulty:** Hard
- **Category:** Distributed Systems / Counters
- **Tags:** counter, crdt, hyperloglog, gossip, batch, redis-cluster

### ANSWER:

The naive approach — a single DB row `UPDATE counter SET count = count + 1 WHERE post_id = 42` — serializes all increments on one row, ~1K writes/sec max (the row lock). For 1M/sec, we need to **shard** the counter and **batch**.

**Approach 1: Shard + Aggregate (Redis cluster):** - N Redis shards; each increment hashes `post_id` to a shard, INCRs `counter:{post_id}` on that shard. - 1M increments/sec / N shards = 10K INCR/sec per shard (well within Redis’s 100K ops/sec). - Reads: GET `counter:{post_id}` from the shard. - Trade-off: Redis is in-memory; on crash, recent increments (since the last BGSAVE) are lost. For “likes” (approximate), acceptable. For exact, use Redis AOF (every write logged, slower) or a DB-backed counter.

**Approach 2: Per-server buffer + periodic flush (the high-throughput answer):** - Each app server maintains an in-memory `Map<postId, count>`; increments go to memory (1M/sec trivial — no IO). - Every 1s, each server flushes its buffer: for each `postId` in the buffer, send `(postId, delta)` to a Kafka topic. - A consumer aggregates the Kafka stream: maintains a `Map<postId, total>`; on each `(postId, delta)`, adds to total; periodically writes to DB. - Reads: read from DB (eventually consistent, lag = flush interval + consumer lag = ~2s) or from a Redis cache (the consumer updates Redis on each aggregate). - 1M/sec → 1000 servers × 1000 increments/sec/server (in-memory) → batched into 1000/sec/server = 1M increments/sec to Kafka in 1000 batches (1 batch/sec/server). Kafka handles 1M msgs/sec easily. The consumer aggregates (sum) and writes to DB at a much lower rate (1 write/sec/post for active posts).

**Approach 3: CRDT (counter CRDT — PN-Counter):** - A PN-Counter (Positive-Negative) is a CRDT: each server has a **grow-only** counter for increments and one for decrements; the total = sum(increments) - sum(decrements). Merge = element-wise max (per server’s increment counter). - Each server increments **only its own** increment sub-counter (no coordination). Reads sum across all servers (gossip the sub-counters periodically). - Eventual

consistency: if a server is partitioned, its increments are not counted until it rejoins and gossips. - For “likes” (no decrements), use a G-Counter (grow-only); simpler. - This is the [state-based CRDT](#); the operation-based version (CvRDT) sends only the increment op. - The trade-off:  $O(N)$  memory per counter ( $N$  = servers); for 1000 servers  $\times$  1M posts = 1B entries — too much. Use only for low-cardinality counters (e.g., total likes on a single popular post; not for per-post counters).

**The choice for “1M increments/sec across 1000 posts” (the prompt):** - Approach 2 (per-server buffer + Kafka) is the production answer. Scales to any rate; memory bounded by the buffer size; latency  $\sim 2s$  (acceptable for “likes”). Backed by Kafka (durable)  $\rightarrow$  consumer  $\rightarrow$  DB (source of truth). - For ultra-low-latency reads (display the count immediately after a like), use Redis (Approach 1) as the read cache, with the Kafka pipeline as the durable backing. The Redis counter is eventually consistent with the Kafka aggregate (the consumer updates Redis on each aggregate). Reads from Redis (1ms), writes to local buffer (1ms), aggregate writes Redis + DB (async).

### ASCII — Approach 2 (the chosen design):

```
1M likes/sec, 1000 servers, 1000 active posts

Server 1: Map<postId, count> (in-memory, 1K inc/sec in)
 └ flush every 1s → Kafka topic "likes"
Server 2: Map<postId, count> (in-memory)
 └ flush every 1s → Kafka
...
Server 1000: same

Kafka topic "likes":
 (post=42, delta=980)
 (post=42, delta=1010) ← from server 2
 (post=42, delta=995)
 ...
 (post=43, delta=...)

Consumer (parallelized by post_id):
 Map<postId, total> in memory
 On (post=42, delta=980): total[42] += 980
 Periodic: write to DB (every 10s per post)
 Update Redis cache on each aggregate (for low-latency reads)

Reads:
 GET /api/posts/42/likes → Redis (1ms) → 1,995,023
 (Redis is updated by the consumer; lag = flush interval + consumer lag = ~2s)
```

**KEY TERMS TO MENTION:** - Single-row counter bottleneck (row lock,  $\sim 1K$  writes/sec) - Shard + Redis cluster (per-post counter, no row lock) - Per-server buffer + Kafka flush (the high-throughput pattern) - PN-Counter / G-Counter CRDT (for low-cardinality, no-coordination) - Eventual consistency (lag = flush + consumer + cache propagation) - Read-through cache (Redis, updated by the consumer)

**FOLLOW-UP QUESTIONS:** 1. A post goes viral: 1M likes in 1 minute on a single post. The Kafka partition for that `post_id` is hot. How do you handle the hot partition? 2. A server crashes with 1000 unflushed likes in its buffer. They're lost. Acceptable? 3. You need exact counts (financial transactions, not likes). Same architecture?

**FOLLOW-UP ANSWERS:** 1. The hot partition: all increments for `post_id = 42` go to the same Kafka partition (keyed by `post_id` for ordering). 1M msgs/min on one partition = 17K msgs/sec; Kafka handles ~100K msgs/sec per partition, so 17K is fine. The `consumer` for that partition is the bottleneck — one consumer per partition. If the consumer can't keep up (17K aggregates/sec), the consumer lags. Fixes: (a) Smaller batches in the consumer (process each message, don't batch) — reduces per-message latency. (b) Multiple consumers on the same partition (not supported by Kafka — one consumer per partition in a consumer group). (c) Don't key by `post_id` — distribute across partitions uniformly (lose ordering per post, but the aggregate is commutative — order doesn't matter for sum). Use `(post_id, server_id)` as the key, or just `server_id` (round-robin), or a random key. This is the right fix: since sum is commutative, we don't need per-post ordering. Hash by `server_id` or random; all consumers process uniformly; no hot partition. 2. Acceptable for likes (approximate). For exact: the buffer should be backed by a write-ahead log (WAL) on the server's disk; on crash, replay the WAL. Or, write the buffer entries to Kafka `synchronously` (every increment, not every 1s) — but that defeats the batching (1M Kafka writes/sec/server, too much). The compromise: write to Kafka every 100ms (10 flushes/sec/server, ~10K msgs/sec/server); on crash, lose up to 100ms of increments (~100 per server, ~100K total — acceptable for likes). For financial transactions, use the DB directly (no buffer, no loss) and accept the throughput limit (the DB can't do 1M/sec, so you scale the DB or partition the counter). 3. No — the architecture is for approximate counters (likes, views). For exact (financial), use a different design: a single DB row with optimistic locking (for low-write counters), or a sharded DB counter (each shard a row, sum on read; row locks per shard), or an event-sourced design (every increment is an `INSERT` into an events table; sum on read, materialized periodically). The buffer-Kafka approach loses data on crash; financial counters cannot tolerate this. The exactness-vs-throughput trade-off is fundamental: exact + high-throughput is hard (you need either a single coordinator or a 2PC, both of which limit throughput). For financial, scale the DB (vertical then horizontal partitioning); accept the limit.

**RED FLAGS TO AVOID:** - Single DB row for a hot counter — row lock serializes, ~1K writes/sec. - In-memory buffer with no flush — lost on crash, no durability. - Per-post Kafka key on a viral post — hot partition, consumer lag. - CRDT for high-cardinality counters (1M posts) —  $O(N \times M)$  memory, infeasible.

---

## QG-Bonus2-12: You inherit a 5-year-old Java codebase with no tests, 1000 classes, and a tangled dependency graph. The team wants to add tests. Where do you start?

---

- **Difficulty:** Medium
- **Category:** Testing / Legacy Code
- **Tags:** legacy-code, characterization-tests, seam, dependency-injection, strangler-fig

**ANSWER:**

The “test a legacy codebase” problem (Michael Feathers’ [Working Effectively with Legacy Code](#)). The trap: try to write unit tests for every class — you’ll spend months untangling dependencies, the team gives up, no tests. The right approach: prioritize, use [characterization tests](#) (capture current behavior, not desired), and use the [strangler fig](#) pattern for incremental refactoring.

**Step 1 — Triage by risk + churn:** - Get the git churn data: which files change most often? `git log --format= --name-only | sort | uniq -c | sort -rn | head -50`. The top 50 files are the [hot](#) ones — bugs happen here, tests are most valuable here. - Get the bug history: which files have had the most bug-fix commits? (grep the commit messages for “fix”/“bug”.) These are the [fragile](#) files. - The intersection (hot + fragile) is the top 10-20 files. Start here.

**Step 2 — Characterization tests (capture current behavior):** For each hot file, write tests that [execute the current code](#) and assert the [current output](#). You’re not testing “does it do the right thing?” (you don’t know); you’re pinning “it does whatever it does now, so a refactor doesn’t change it.” These are safety nets for refactoring.

```
@Test void testCreateLead_currentBehavior() {
 // Setup with the current (legacy) dependencies – even the bad ones
 LegacyLeadService svc = new LegacyLeadService(legacyDb, legacyEmail, ...);
 Lead result = svc.create(input);
 assertEquals(42, result.getId()); // pin the current behavior
 assertEquals("legacy-template", result.getEmailTemplate()); // yes, even the
 wrong template
}
```

These tests look weird (asserting “wrong” behavior), but they pin the behavior so refactoring is safe. As you refactor, you update the assertions to the new (correct) behavior.

**Step 3 — Find seams (where to inject test doubles):** Legacy code has hard-coded dependencies ( `new LegacyDb()` inside the method) — no way to inject a mock. The seam is a place where you can substitute. Options: - **Extract method + override in test:** Move the `new LegacyDb()` to a protected method `createDb()`; in tests, override to return a mock. No production change, no new dependencies. - **Constructor injection (the long-term fix):** Add a constructor that takes the dependency; the old constructor delegates with `this(defaultDb)`. Slowly migrate callers to the new constructor. - **Static method → instance method:** Static methods are hard to mock; convert to instance (extract to a class, inject). Larger change but enables testing.

**Step 4 — Strangler fig pattern (incremental replacement):** Don’t rewrite the legacy module. Add a [new](#) module with tests; route new features through it; over time, migrate old features. The legacy and new coexist; new code is tested, legacy is characterization-tested (pinned). Eventually, the legacy shrinks to nothing. - New endpoint: write in the new style (Spring, DI, tests). - Existing endpoint that needs a change: extract the change to the new module; the legacy endpoint delegates the new logic to it. - Old endpoint with no changes: leave it (characterization tests protect it).

**Step 5 — Integration tests over unit tests (initially):** Unit tests need seams (mocking); legacy has no seams. Integration tests (Spring Boot Test, real DB via Testcontainers) test the [whole](#) flow without mocking — they work on legacy code as-is. Slower than unit tests, but they're the entry point. As you refactor (add seams), you add unit tests for the newly-decoupled pieces.

**The pragmatic 30-day plan for a team of 3:** - Week 1: Triage (top 20 files). Setup Testcontainers for DB integration tests. Write characterization tests for the top 3 files (the most-touched). - Week 2: Refactor one file (add a seam — constructor injection); add unit tests for the extracted dependency. The characterization tests pass before and after (proving no behavior change). - Week 3: Apply the same pattern to the next 5 files. Each refactor: extract seam + unit test. - Week 4: Add a CI gate: new code must have tests (coverage gate on changed files, not whole codebase). Legacy is exempt; new isn't. - Month 2+: Continue the strangler fig; aim for 50% coverage on hot files by month 3; 80% by month 6. Don't chase 100% — diminishing returns; cover the hot + fragile paths.

### ASCII — the strangler fig:

Legacy monolith (no tests, tangled):

Endpoint A (legacy, no test)	← characterization test (pin current)
Endpoint B (legacy, no test)	← characterization test
Endpoint C (legacy, no test)	
— new feature D —	← unit + integration tests (new code, fully tested)
Endpoint E (legacy, modified)	← modified part extracted to new module (tested),
legacy delegates	
Endpoint F (legacy, no test)	

Over time: legacy shrinks (endpoints refactored to new modules with tests), new code grows. Eventually the legacy is 0.

**KEY TERMS TO MENTION:** - Triage by churn + bug history (focus on hot + fragile) - Characterization tests (pin current behavior, not desired) - Seams (extract method, constructor injection — the testing entry points) - Strangler fig pattern (new alongside legacy; incrementally replace) - Integration tests first (Testcontainers), unit tests as seams emerge - Michael Feathers' [Working Effectively with Legacy Code](#) (the canonical reference)

**FOLLOW-UP QUESTIONS:** 1. The team pushes back: "We don't have time for tests; we have features to ship." How do you respond? 2. A characterization test pins a [buggy](#) behavior. You fix the bug; the test fails. Update the test? 3. Coverage is 30% after 3 months. Management wants 80%. How do you reconcile?

**FOLLOW-UP ANSWERS:** 1. With data. Show the bug rate (bugs/month) and the time-to-fix (hours). Tests reduce bug rate by ~40% (industry data) and time-to-fix by ~60% (the test localizes the bug). The investment pays back in 3-6 months. Pair this with a [gradual](#) approach: don't ask for "test everything"; ask for "test the hot files as we touch them" — the cost is per-feature (one test per modified file), not a separate testing project. The CI gate (new code must have tests) is the lever — it makes testing part of feature work, not a tax.

The team's pushback is usually about **time**; the response is "the test takes 30 min to write, saves 2 hours of debugging per bug, and we ship fewer bugs — net faster." 2. Yes, update the test. The characterization test's purpose was to **pin** behavior so refactoring is safe; now you're **intentionally changing** behavior (the bug fix), so the pin must update to the new behavior. The pattern: (a) write a new test asserting the **correct** behavior (it fails — the bug exists). (b) Fix the bug. (c) The new test passes; the old characterization test fails. (d) Update the old test to assert the new behavior (or delete it if the new test covers it). The key: the characterization test protected you from **accidental** change; the bug fix is **intentional** change, and the test is updated **with** the fix, documenting the change. 3. 80% coverage of the **whole** codebase is the wrong target — most of the code is cold (rarely touched), and testing it has low ROI. Reframe: 80% coverage of the **hot** files (the top 20 by churn), which is the 20% of code that produces 80% of the bugs (Pareto). The whole-codebase 80% is a vanity metric; the hot-files 80% is the impact metric. Show management: "We have 80% on the files that change weekly; the legacy files (rarely touched) are characterization-tested. Bug rate is down 50%." That's the real win. If management insists on 80% globally, point out the cost (months of effort on cold code) and the alternative (focus on hot files, ship the same bug reduction in weeks). Most reasonable managers accept the focused approach; the rest are pursuing a metric, not impact — escalate or accept the metric (write low-value tests to hit the number; not ideal but sometimes political reality).

**RED FLAGS TO AVOID:** - "Rewrite it all with tests" — never works; the rewrite is buggy and the team abandons. - "Mock every dependency in legacy code" — there are no seams; mocking is impossible without refactoring. - "100% coverage" — vanity metric; chase hot-file coverage, not global. - "Test before refactoring" without characterization tests — you'll change behavior without knowing.

---

This document is a **cross-section** of what a 4-year backend engineer should know for product-company interviews. The depth in any one area (Kafka, Postgres, distributed systems) is enough to start a conversation; the **breadth** across all areas is what gets you through the 5-round loop. Practice:

1. **One system design a day** for the week before — URL shortener, Twitter, rate limiter, Top-K, messaging, news feed. Draw on paper, talk aloud, time to 30 min.
2. **Two coding problems a day** — LeetCode medium/hard, focusing on the patterns (sliding window, two heaps, graph BFS/DFS, DP). For backend roles, prioritise concurrency + graph + tree.
3. **One behavioural story a day** — STAR, 2 min spoken, one of: incident, conflict, ambiguity, delivery, learning.
4. **Mock interview** — at least one with a friend or a service; record yourself and watch the playback. The filler words, the "um," the rushing — these are what you fix last.
5. **Sleep** — the loop is 5 hours of hard thinking. A tired candidate is a worse candidate.

You know more than you think you do. Walk in humble, curious, and ready to draw. Good luck. — Prepared for Uber, NetApp, Meta interviews, September 2026.